

THE OMNI-COMPASS®

DATA CENTERS & PHYSICAL INFRASTRUCTURE

-  NVIDIA DGX Systems
-  NVIDIA DGX Cloud / Base Command
-  NVIDIA DCGM / GPU Operator
-  NVIDIA Dynamic Power Software (DPS)
-  NVIDIA Run:ai
-  Phaidra (Autonomous Cooling)
-  Schneider Electric EcoStruxure IT
-  Vertiv

CLOUD & MULTI-CLOUD PLATFORMS

-  Amazon Web Services
-  Microsoft Azure
-  Google Cloud
-  Oracle Cloud Infrastructure
-  Alibaba Cloud
-  IBM Cloud
-  DigitalOcean

CONTAINERS & ORCHESTRATION

-  Kubernetes
-  Red Hat OpenShift
-  Amazon EKS
-  Microsoft AKS
-  Google GKE
-  VMware Tanzu
-  SUSE Rancher / RKE2

WORKLOAD SCHEDULING & HPC

-  Slurm
-  KubeRay
-  Ray
-  Kueue

RESOURCE OPTIMIZATION & AUTONOMOUS OPERATIONS

-  IBM Turbonomic
-  Apptio
-  Moogsoft AIOps
-  BigPanda

AI APPS & PLATFORMS

-  ChatGPT (OpenAI)
-  Claude (Anthropic)
-  Grok (xAI)

A UNIFIED GOVERNING CONVERGENCE CONTROL CORE PLANE ENGINE

A MATHEMATICALLY CLOSED CIRCLE FRAMEWORK



A UNIFIED GOVERNANCE.
BOUNDED EVOLUTION.
SUSTAINED COHERENCE.

OWNER AND DEVELOPER
THE OMNI-COMPASS LLC.

THE BRAIN AND NERVOUS SYSTEM
FOR FRAGMENTED STACKED
MUSCLE / MODULES.

STABILITY. CONVERGENCE. CONTINUITY.

www.omni-compass.com

NETWORKING, SECURITY & SERVICE MESH

-  Istio
-  Cilium
-  Cloudflare
-  Akamai
-  Palo Alto Networks
-  Fortinet
-  CrowdStrike
-  F5

OBSERVABILITY & AIOPS

-  Datadog
-  Dynatrace
-  New Relic
-  AppDynamics
-  Splunk
-  Elastic
-  Prometheus
-  Grafana
-  OpenTelemetry

GITOPS & AUTOMATION

-  Terraform
-  Ansible
-  Argo CD
-  Flux
-  GitHub Actions
-  GitLab CI
-  Jenkins

DATA, STREAMING & ANALYTICS

-  Snowflake
-  Databricks
-  MongoDB
-  PostgreSQL
-  Oracle
-  Apache Kafka
-  Redis

IDENTITY, POLICY & GOVERNANCE

-  Okta
-  HashiCorp Vault
-  Consul
-  OPA / Gatekeeper

IT OPERATIONS & SERVICES

-  ServiceNow
-  PagerDuty
-  Splunk On-Call
-  Jira

U.S. PATENT, COPYRIGHT & TRADEMARK APPLICATIONS FILED

ALL RIGHTS RESERVED

THE OMNI-COMPASS MANUAL

THE OMNI-COMPASS MANUAL

*The Governor of Everything:
Its Philosophy, Its Mathematics, Its Physics,
and How to Wire It onto Your Stack*



AJ Dubra
Owner and Developer

THE OMNI-COMPASS LLC
October 2026

The Omni-Compass Manual

The Governor of Everything: Its Philosophy, Its Mathematics, Its Physics, and How to Wire It onto Your Stack

October 2026

Copyright © 2026 The Omni-Compass LLC. All rights reserved. No part of this book may be reproduced, stored or transmitted in any form or by any means, except as allowed by the Omni-Compass Evaluation License, without the written permission of The Omni-Compass LLC.

Patent applications, copyright registrations and trademark applications covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC.

“Omni-Compass”, the Omni-Compass rose and the related names and marks are trademarks of The Omni-Compass LLC. Other names that appear in this book are the property of their owners and are used only to identify the systems Omni-Compass works with.

License. This book and the software it describes are proprietary and are not open source (SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0). They may be used only to evaluate Omni-Compass and to reproduce its published results, in simulation and on systems the reader owns or controls. Any commercial use, commercialization, monetization, production use, redistribution, hosted or managed service, or incorporation into any product or service requires a written Omni-Compass Enterprise License, signed by The Omni-Compass LLC and paid for. The full terms are printed in Appendix I.

Disclaimer. The software is provided “as is”, without warranty of any kind. Every result in this book carries its evidence class. A simulated result is a statement about a model; a software benchmark is a statement about the software it ran on; only a hardware meter speaks for hardware. Nothing in this book is a promise of a particular saving on a particular system: the only number that applies to a system is the one its own paired runs produce, on its own receipt.

Front cover: the Omni-Compass identity plate. Back cover: the Omni-Compass unified control authority. Plates 1 to 19: the founder's plates of the theory.

Set in Liberation Serif and DejaVu Sans Mono. Built from the repository by `docs/book/build_book.py`; the book and the code change in the same commit.

The Omni-Compass LLC · www.omni-compass.com

Contents

Foreword	xv
Preface	xvii
Executive Summary	xix
 Part One. The Philosophy and the Theory	 1
1. What Omni-Compass Is	2
2. The Unified Circle Principle	4
3. The Four-Piece Engine	18
4. The Compass	25
5. Basins, Polarity and the Dual-Basin Engine	27
 Part Two. The Mathematics	 31
6. The Canonical Declaration	32
7. The Engine: Eight Equations and One Control Law	39
8. The Canonical Engine	40
9. The Closed Circle: Why It Cannot Leave Its Bowl	42
10. Closing the Circle in the Engine	45
11. The Tracking Theorem	46
12. The Conveyance Law	49
13. Formal Status of the Mathematics	52
14. The Mechanism of Action	53
15. The Engines and Their Audit	57
 Part Three. The Physics: the Bowl and the Nervous System	 59
16. The Bowl: Push, Pull and the Two Forces	60
17. The Physics of a Processor	61
18. The Two-Way Nervous System	62
19. The Nervous System in Detail	63

Part Four. The Body: Muscles, Realms and Organisms	65
20. The Muscles, the Realms and the Six Organisms	66
21. The Four Realms	67
22. The Realm Muscles	68
23. The Domain Map	76
24. The Problem Map	82
25. The Six Organisms and the Benchmark Grid	86
26. The Metrics Catalog	87
 Part Five. The Harness and the Wiring	 91
27. The Plug, the Adapters and the Wire Check	92
28. The Harness	94
29. The Wiring Guide	95
30. Before You Start, and the Eight Levels	98
31. Stack by Stack	101
32. The Integration Manual	102
33. Running the GPU Benchmark	109
 Part Six. Operating It	 113
34. The OFF Switch, the Rules, and the Log	114
35. Maintenance, Upgrades and Security	115
36. The Operator Manual	116
 Part Seven. Proving It	 123
37. Paired Runs and Receipts on Your Own System	125
38. Evidence Classes and How to Read a Result	126
39. Results to Date	127
40. The Pilot Protocol and Kit	128
41. The GPU Bench	130
42. The GPU Preregistration	133
43. The Realms Preregistration	141
44. The Bowl Law on Real Kubernetes: Preregistration	147
45. The Evidence Ledger	149
46. The Claims Register	154

47. The Benchmark Report	161
48. The Referee Report	182
49. Comparison with Existing Controllers	195
50. The State of Play	198
Part Eight. Value, License and History	203
51. Where the Value Comes From	204
52. The Economics of a Receipt	205
53. The Buyer Edition	206
54. Due Diligence	214
55. License and Commercial Terms	216
56. Python, C++ and the Seal	217
57. The Founder's Working Notes	218
58. History	223
Back Matter	227
Glossary	228
Appendix A. Command Reference	229
Appendix B. File Map	230
Appendix C. The Equations in Full	231
Appendix D. Metrics	232
Appendix E. Troubleshooting	233
Appendix F. Evidence Map	234
Appendix G. The 656 Muscles	235
Appendix H. Source of the Engine	251
Appendix I. The License	275
Contact	277

List of Plates

Plate 1. The Unified Circle Principle

Plate 2. Closed-circle structural architecture of the unified law

Plate 3. The closed circle tested

Plate 4. Symmetry principles and conservation laws in the closed circle

Plate 5. Scale invariance and structural universality

Plate 6. The lineage of unification

Plate 7. A candidate for a closed, unified law

Plate 8. The Omni-Compass Grand Theory: the four pieces

Plate 9. The unified container of all reality: 24 domains, four-piece engine, one closure law

Plate 10. The four quadrants and the compression process

Plate 11. The closure law and the closed-circle engine

Plate 12. The Omni-Compass rose

Plate 13. The Omni-Compass Principle

Plate 14. Geometric funneling and basin formation

Plate 15. The Omni-Compass dual-basin engine

Plate 16. Mathematical integration

Plate 17. The engine core: the governing law in motion

Plate 18. The equation form and the symbol chart

Plate 19. Empirical validation, experimental pathways, and the discipline of proof

Foreword

I built Omni-Compass to sit on top of what already exists. Kubernetes, the GPU driver, the building controller, the battery inverter, the robot's servo loop: those are muscles, and they are good muscles. What none of them has is one brain that reads all of them at once, holds each of them in the middle of its safe range, and puts every setting back exactly where it found it when it stops. That brain is Omni-Compass.

This manual is the whole of it: what the governor is, the mathematics it runs on, how its nervous system reads and writes, how it is wired to each kind of machine, how to turn it on one level at a time, how to switch it off, and how to prove on your own system what it does. Keep it with the code. When the code changes, this manual changes in the same commit.

AJ Dubra Founder, The Omni-Compass LLC

Preface

I wrote this book so that whoever connects Omni-Compass to a running system understands what they are connecting: not only which wire goes where, but why the governor behaves the way it does, where its law comes from, and how to prove on their own system what it does for them.

The book is in eight parts, and it can be read in two ways.

You are	Read
CEO, board member, investor	the Foreword, the Executive Summary, Part One, the Results to Date, and Part Eight
CTO, architect, head of platform	Parts One to Four, then the eight wiring levels in Part Five, then Part Seven
The engineer wiring it	everything, in order. Do not skip the wire check or the watch level
Auditor, diligence team	Part Two (the mathematics), Part Seven (the proof), Appendices C, F and H

Part One, the philosophy and the theory, sets out the closed circle: a system that is closed, bounded and pulled toward a center cannot run away, and everything it does is a return. **Part Two, the mathematics**, gives the eight equations and the control law that make the circle exact, with the theorem, the declaration and the audits that hold them fixed. **Part Three, the physics**, brings the law into the machine: the bowl, push and pull, the band and its cushions, the physics of a processor, and the two-way nervous system. **Part Four, the body**, lays out the 656 muscles, the four realms and the six organisms. **Part Five, the harness and the wiring**, is the universal plug and the step-by-step work of wiring Omni-Compass onto a stack. **Part Six, operating it**, is the OFF switch, the rules and the log. **Part Seven, proving it**, is every rule written before a run and every result after it. **Part Eight** is value, the license, the seal and the history.

Conventions. Code is a command, file or switch exactly as typed. "Native" means a system as it runs today, without Omni-Compass. "Muscle" means any machine, service or controller Omni-Compass can read and set. "Knob" or "lever" is one setting on a muscle. "The band" is a knob's or a service reading's safe range. A step that writes to a system is marked **WRITES**. Every result carries its evidence class: T for a theorem, V for verification of the code, S for simulation, L for live software, P for a physical meter.

Some chapters gather documents that were written as the work went on, each at the moment its result came in. They are kept as they were written, because the record of how a result was reached is part of the proof. Where two chapters give different figures for the same thing, the later run and the State of Play govern.

Executive Summary

- **What it is.** A supervisory governor. It reads the meters of every muscle in your stack, computes one bounded force per muscle from a single closed mathematical law, and moves each muscle's own setting (a replica target, a clock ceiling, a power limit, a setpoint) to hold the service in the middle of its band. The muscles keep their own controls. Omni-Compass sets what they already accept.
- **What it does for you.** It turns the room your systems keep "just in case" into either more work for the same energy or the same work on fewer machines and fewer watts. On real Kubernetes (two independent sets of ten paired runs) it served the same traffic with about a third fewer machines in service, responses about 60% faster at the 95th percentile, and zero failed requests (Chapter 39).
- **How it stays safe.** It watches before it writes, records every original setting before it acts, reads back every write, never writes past a knob's cover, gives everything back the moment service is at risk, stops writing if anyone else touches a knob, and returns every setting to its original value on one OFF switch.
- **How you know.** Every run produces receipts: native against Omni-Compass on the same system, the same load and the same clock, with 95% intervals. Results are labelled by a rule written before the run.

PART ONE

The Philosophy and the Theory

Where Omni-Compass comes from: the closed circle, the four pieces, the compass and the basins. The engineering in the rest of the book is this idea made exact.

CHAPTER 1

What Omni-Compass Is

A governor on a steam engine does not build the engine and does not turn the shaft. It watches the speed and moves the throttle so the speed stays in a band. Omni-Compass is that, for every machine you run.

It is a process on a host. It reads meters, steps a bounded mathematical law, and writes only the levers it has been given. It is not the chip, not the GPU driver, not Kubernetes, not the building controller. Those keep running exactly as they do today; Omni-Compass sets the values they already accept:

Muscle	Its own control (kept)	What Omni-Compass sets
Kubernetes service	the HPA	the HPA's CPU target; replica floors
Kubernetes node pool	Cluster Autoscaler / Karpenter / MachineSet	how many machines stay in service (park and wake)
NVIDIA GPU	the card's firmware (boost, power and thermal limits)	the clock ceiling (up wire) and the power limit (down wire)
CPU	the kernel's frequency governor	the frequency ceiling
Data hall, building	the chiller and air handler loops	the supply-air setpoint; units in service
Battery site	the inverter	the reserve level; the peak ceiling
Robot joint, vehicle axis	the servo loop	speed and effort limits
Process loop, feeder	the PI controller	the setpoint inside its band

When Omni-Compass stops, every one of those values goes back to what it was before Omni-Compass acted.

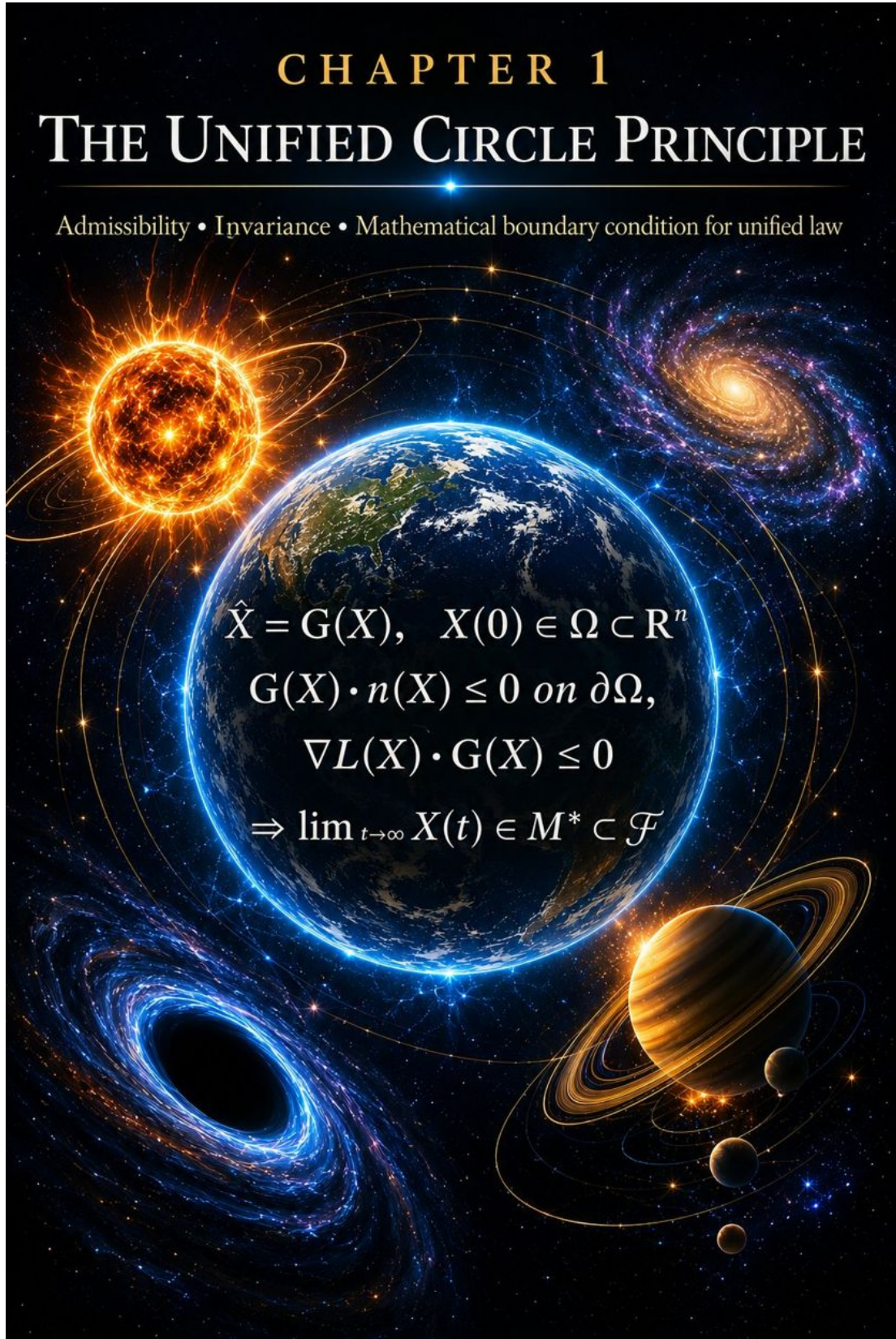


Plate 1. The Unified Circle Principle

CHAPTER 2

The Unified Circle Principle

Omni-Compass begins with one idea: a system that is closed, bounded and pulled toward a center cannot run away, and everything it does is a return. I call this the closed circle. This chapter states the principle as I hold it, in its mathematical form, and then says plainly which parts of it the software uses and which parts belong to the wider theory the software grew out of.

The statement

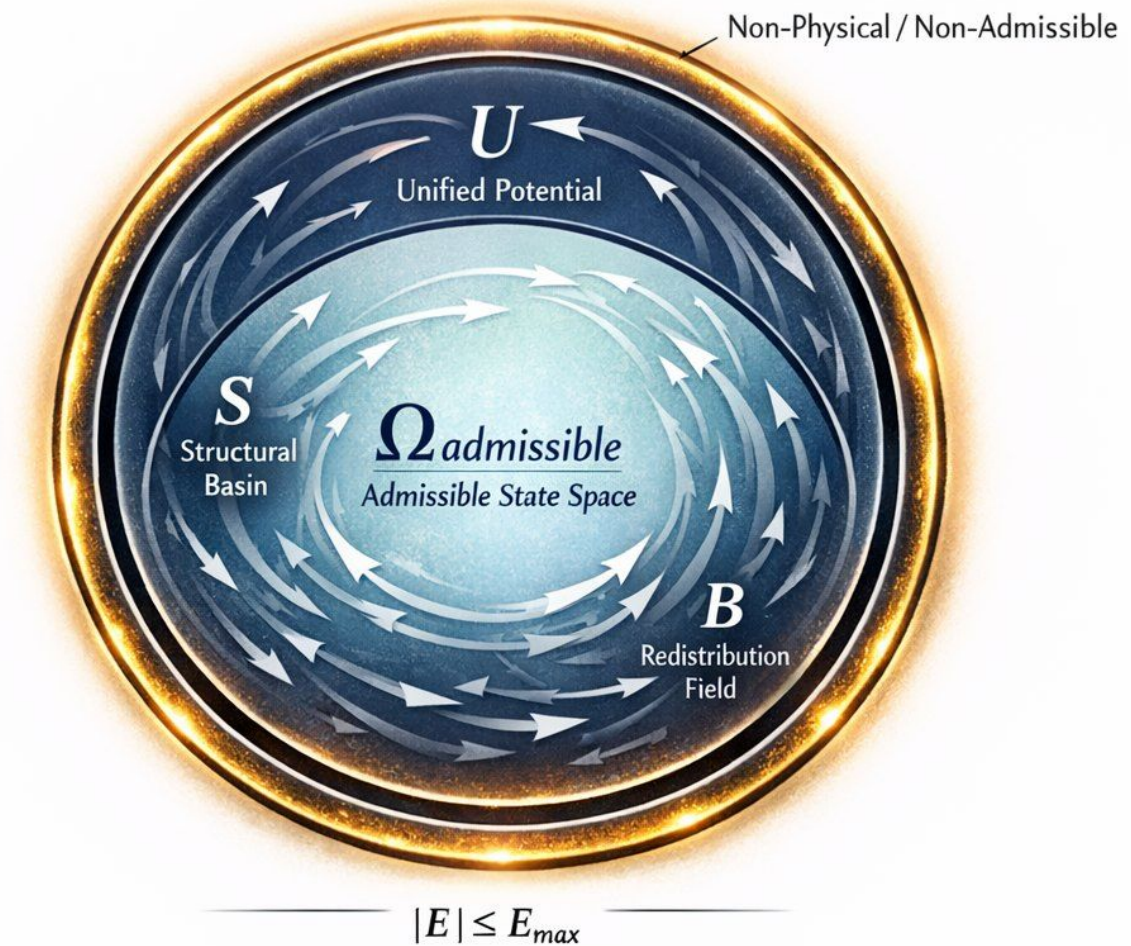
```
dX/dt = G(X),          X(0) in Omega, a subset of R^n
G(X) . n(X) <= 0      on the boundary of Omega
grad L(X) . G(X) <= 0
=> lim (t -> infinity) X(t) in M*, a subset of F
```

Read line by line:

1. **$dX/dt = G(X)$.** The state X moves according to one law G . There is no outside hand: every change comes from the law and the state it is in.
2. **$X(0)$ in Ω .** The state starts inside the admissible region Ω : the set of states that are physically and operationally allowed.
3. **$G(X) \cdot n(X) \leq 0$ on the boundary.** At the wall of Ω , the flow never points outward (n is the outward normal). Nothing that starts inside can cross out. In control theory this is the Nagumo condition: Ω is forward invariant.
4. **$\text{grad } L(X) \cdot G(X) \leq 0$.** There is a function L , the energy of the bowl, that never increases along the motion. This is a Lyapunov condition.
5. **$\Rightarrow X(t)$ settles in M^* .** Together, an invariant container and a non-increasing energy force every path into the set M^* where the energy stops falling: the bottom of the bowl.

The power of the statement is that it does not depend on the particular disturbance. Every input up to the size the container is built for has the same answer before it is asked: the state stays inside and returns. That is what I mean when I say that a closed circle holds the answer to every question that can be put to it.

Closed-Circle Structural Architecture of the Unified Law



↻ Dynamic Flow

▬ B Redistribution Field

$\mathcal{M}$ Manifold Boundary

Plate 2. Closed-circle structural architecture of the unified law

Closure, admissibility, boundedness

The theory sets five requirements on any law that claims to close the circle:

Requirement	Meaning	In the engine
Finite deviation	the distance from the admissible state has a ceiling, $\text{abs}(E) \leq E_{\text{max}}$	the deviation state E drains faster than it can fill
Finite curvature	the landscape the state moves on has no infinite slopes or pits	the drive is bounded by tanh; the authority is clipped
Global conservation	what flows in is matched by what flows out	the drain and the bath's friction match the drive
Recurrence	a disturbed state returns arbitrarily close to where it was	convergence to the target pole at rate K_P
One law	every subsystem obeys the same law, no special cases	one engine, one bowl, every muscle

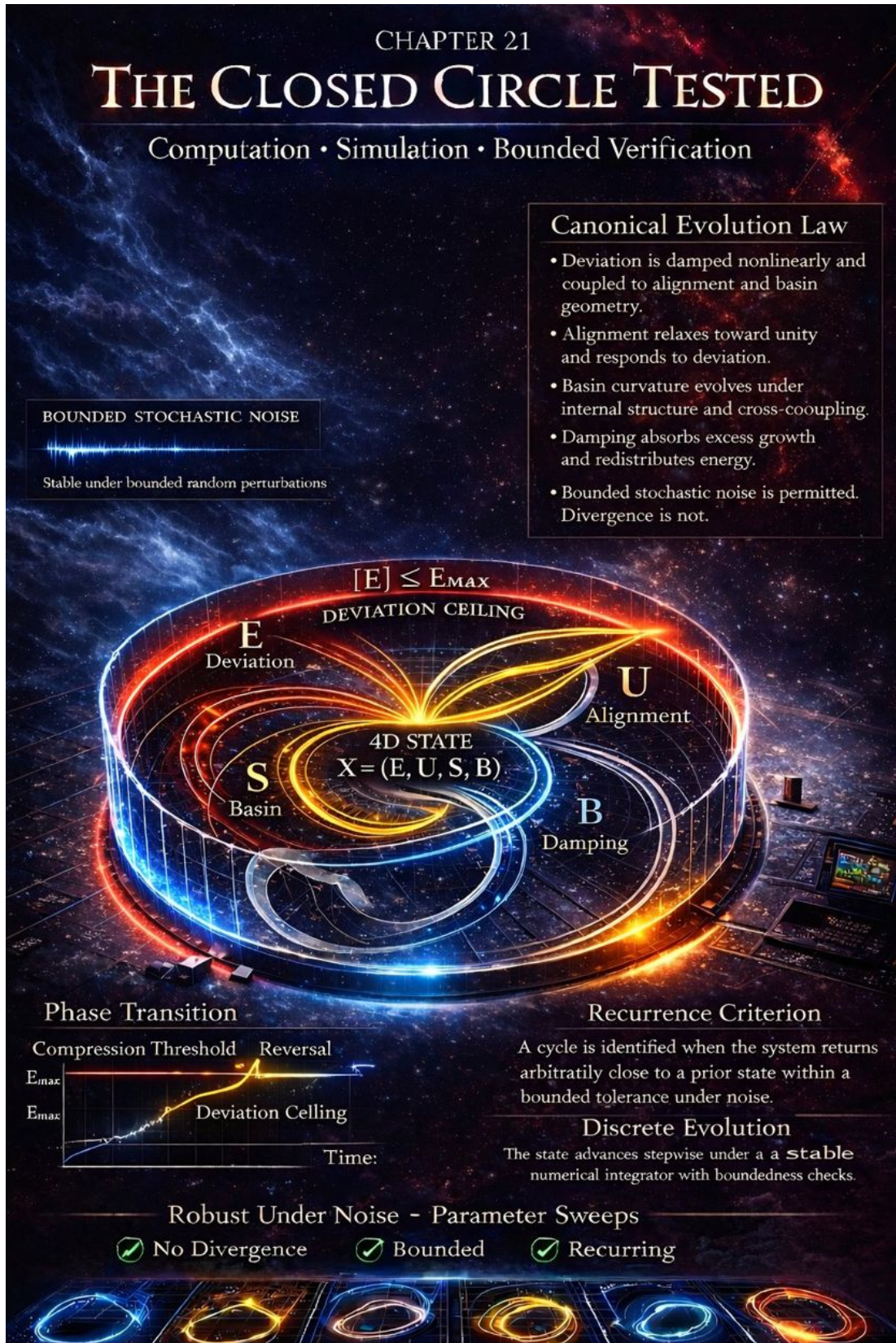


Plate 3. The closed circle tested

The closed circle tested

The canonical evolution law has four parts: deviation is damped nonlinearly and coupled to alignment and to the basin; alignment relaxes toward unity and responds to deviation; the basin's curvature evolves with internal structure; damping absorbs excess growth and redistributes energy. Bounded random noise is permitted; divergence is not. A cycle is identified when the system returns arbitrarily close to a prior state within a bounded tolerance under noise. The state advances stepwise under a stable numerical integrator with boundedness checks. Under parameter sweeps the test is always the same three checks: no divergence, bounded, recurring.



Plate 4. Symmetry principles and conservation laws in the closed circle

Symmetry and conservation in the closed circle

Every conservation law is the shadow of a symmetry (Noether). The closed circle carries four:

Symmetry	Conserved	In Omni-Compass
Temporal (the law does not change with time)	energy	the same law governs every decision; the receipt compares like with like
Spatial (the law does not depend on where)	momentum	the same law governs every muscle in every realm
Rotational	angular momentum	the bowl is symmetric about its center; the engine is symmetric between its poles
Internal alignment	alignment invariance	the alignment state is pulled to its pole whichever pole it is

Around them sit the boundary conditions of the circle: maximum deviation (the boundary limit), topological closure (bounded wholeness), nonlinear dynamics (complex interactions), resonant reversal (feedback return), energy-momentum conservation, and cross-scale invariance (scale-independent patterns).



Plate 5. Scale invariance and structural universality

Scale invariance

The theory claims that one bounded nonlinear action has the same form at every scale: cosmological (curvature geometry, compression and expansion, thermodynamic cycling), galactic (rotation curves, mass-energy distribution), stellar (gravitational compression, fusion equilibrium, radiative transport), atomic (electron displacement, orbital coherence, radiative damping) and quantum (localized deviation wells, bounded oscillation, finite amplitude). The universal criteria for such a law are that it operates at every scale and preserves bounded deviation, finite curvature and recurrence. The state vector carries the same meaning at every scale: E the deviation magnitude, U the coherence or flow alignment, S the structural curvature or basin geometry, B the redistribution or damping.



Plate 6. The lineage of unification

The lineage of unification

Each step in the history of physics unified two things that had been separate: geometry formalized structure; motion was unified across earth and sky; astronomy revealed recurrence; fields replaced action at a distance; curvature unified gravity, though singularities remained; symmetry came to govern the quantum realm. The closed circle is offered as the next step: unity without divergence, a single bounded law with no singularity and no runaway.

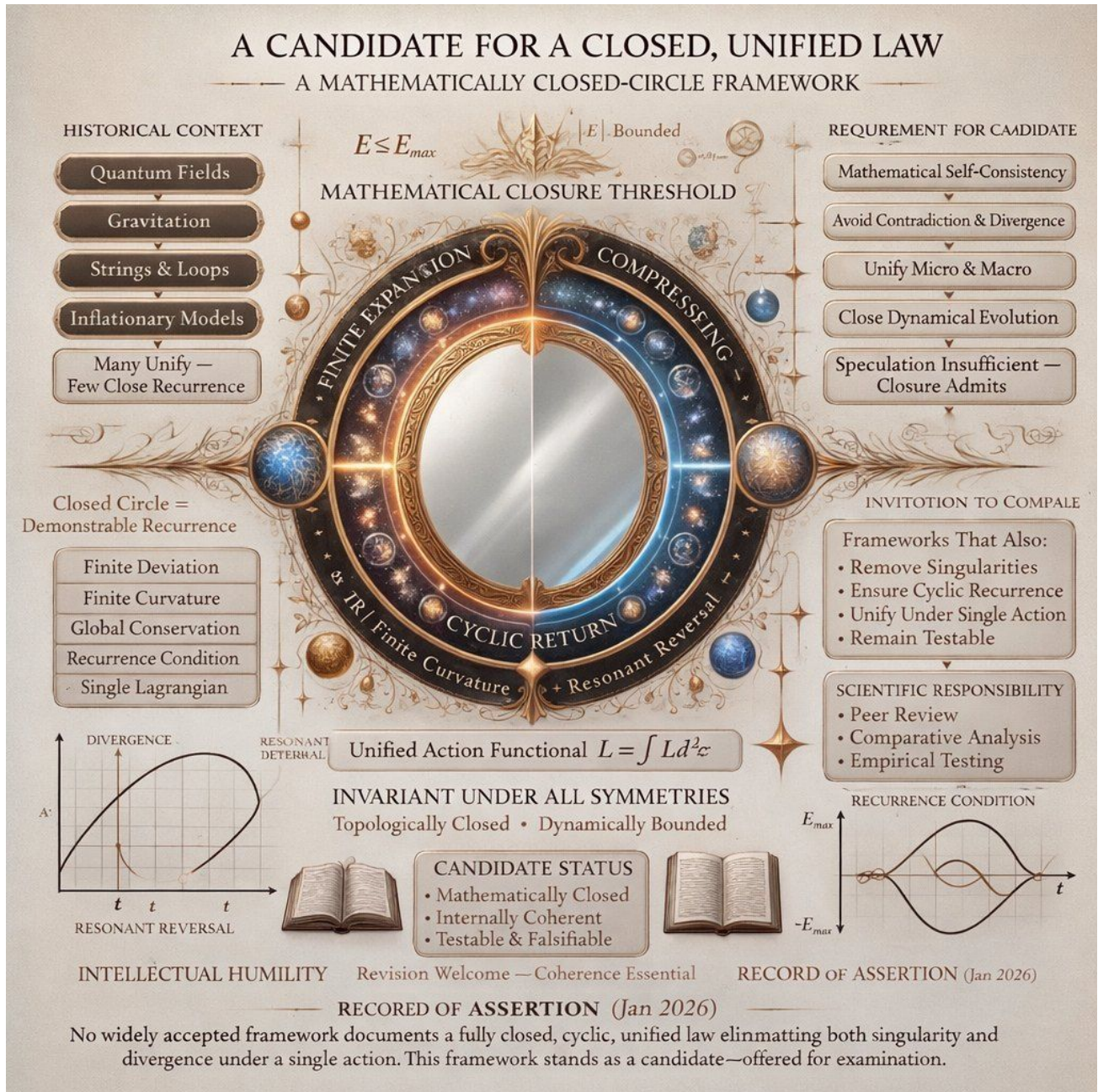


Plate 7. A candidate for a closed, unified law

What is tested and what is theory

The software uses the engineering core of this principle, and every part of that core is checked:

- The six-state engine, its bounded control command held through the integration step, and the convergence of its alignment state to its pole are proved (docs/TRACKING_THEOREM.md) and tested against a C++ twin.
- The invariance and boundedness of each engine state were checked state by state (Chapter "Closing the Circle").
- The bowl law pulls every muscle's service reading to the middle of its band under a bounded, smooth force, and its behavior is measured in simulation and on real software.

The wider claims - one law across every physical scale, the cosmological and quantum readings, the candidate unified action - are the theory from which Omni-Compass was built. They are stated here so that the engineer understands where the design comes from. They are not claims that the software has measured, and no result in this book depends on them.

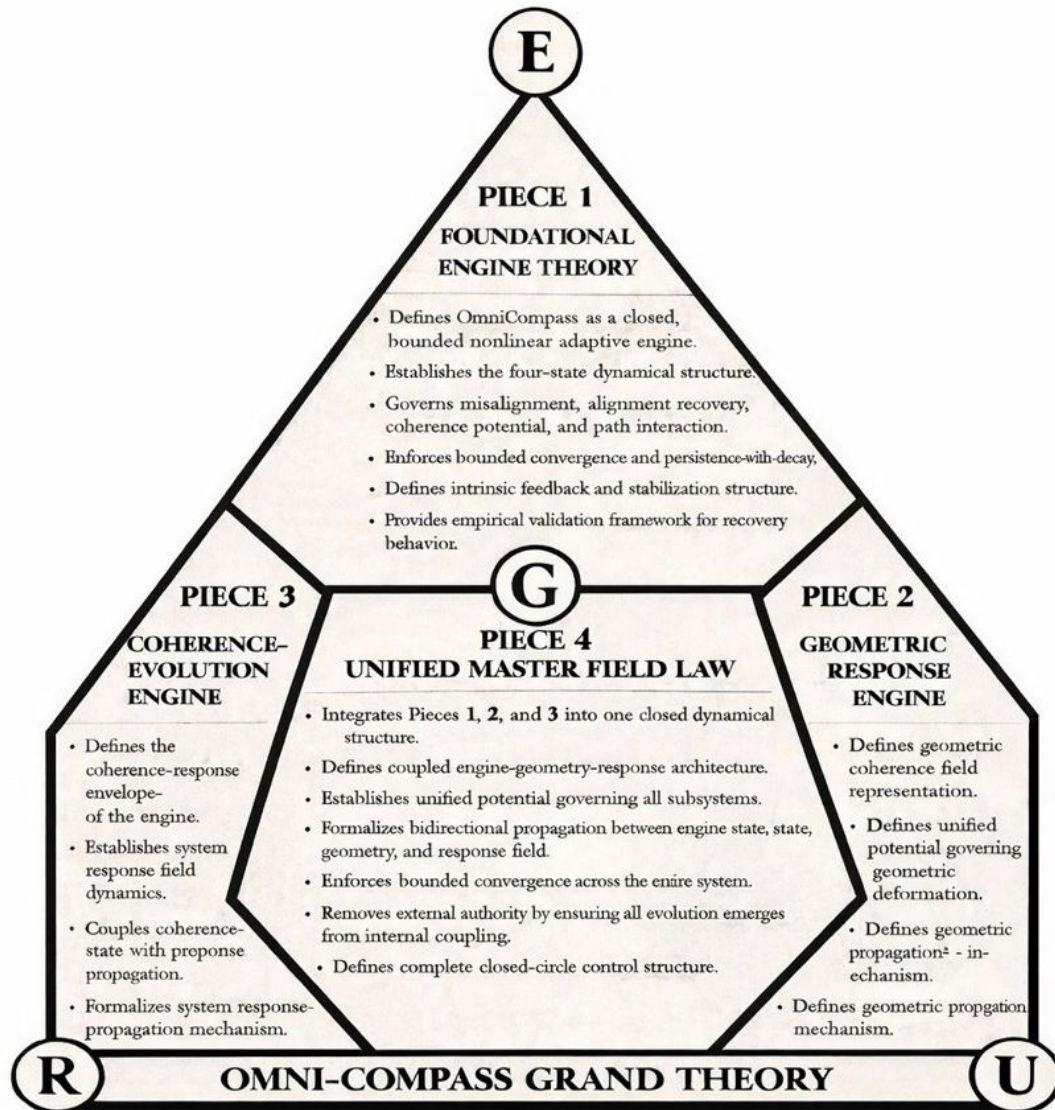


Plate 8. The Omni-Compass Grand Theory: the four pieces

CHAPTER 3

The Four-Piece Engine

The engine is built in four pieces, stacked like the stages of a press, each feeding the next:

Piece	Name	What it establishes
1	Foundational Engine	the base laws, the states and the boundaries: a closed, bounded, nonlinear adaptive engine with a four-state dynamical structure (misalignment, alignment recovery, coherence potential, path interaction), bounded convergence and persistence with decay, intrinsic feedback and stabilization, and an empirical validation framework for recovery
2	Geometric Response	geometric alignment and response dynamics: a geometric coherence field, a unified potential governing geometric deformation, and the propagation mechanism of that response
3	Coherence Evolution	coherence through response and adaptation: the coherence-response envelope, the system response field, and the coupling of coherence state with response propagation
4	Unified Master Field Law	one coherent field structure: it integrates pieces 1-3 into one closed dynamical structure with a unified potential governing all subsystems, bidirectional propagation between engine state, geometry and response, bounded convergence across the whole system, and no external authority: all evolution emerges from internal coupling

The four pieces compress every domain through one center core. The compression process has four stages: the domains unordered; compression in progress; coherence achieved; return to equilibrium. The result has four properties: coherent, bounded, admissible, stable - one law, one return, one cycle.

The 24 domains. The theory gathers 24 domains into four quadrants around that core:

Quadrant	Domains
1. Foundational structures	Newtonian gravity; Maxwell electromagnetism; Einstein relativity; Schrodinger quantum mechanics; Shannon information theory; Dirac quantum field theory
2. Dynamics and symmetries	Noether symmetry laws; Hamiltonian dynamics; Lagrangian mechanics; thermodynamics; topology; differential geometry
3. Coherence evolution	gauge field theory; fluid dynamics; field theory; discrete mathematics; solid-state physics; plasma physics
4. Complex systems and integration	astrophysics; consciousness and information integration; biological systems and self-organization; universal mathematical structure; systems theory and complex adaptive systems; the unified Omni-Compass domain (the unified admissible state)

OMNI-COMPASS

THE UNIFIED CONTAINER OF ALL REALITY

24 DOMAINS • 4 PIECE ENGINE • 1 CLOSURE LAW
ONE TWIST • ONE LAW • ONE RETURN • ONE REALITY

THE FOUR PIECE ENGINE

PIECE 4 UNIFIED MASTER FIELD LAW

Unifies all domains into one coherent field structure.

PIECE 3 COHERENCE EVOLUTION

Evolves coherence through response and adaptation.

PIECE 2 GEOMETRIC RESPONSE

Creates geometric alignment and response dynamics.

PIECE 1 FOUNDATIONAL ENGINE

Establishes the base laws, states, and boundaries.

THE FOUR PIECE HYDRAULIC COMPRESSION SYSTEM

PIECE 4 UNIFIED MASTER FIELD LAW

PIECE 3 COHERENCE EVOLUTION

PIECE 2 GEOMETRIC RESPONSE

PIECE 1 FOUNDATIONAL ENGINE

THE UNIFIED CIRCLE PRINCIPLE

$$X = G(X), X(0) \in \Omega \subset \mathbb{R}^n$$

$$G(X) \cdot n(X) \leq 0 \text{ on } \partial\Omega,$$

$$\nabla L(X) \cdot G(X) \leq 0$$

$$\Rightarrow \lim_{t \rightarrow \infty} X(t) \in M^* \subset \mathcal{F}$$

WHAT'S INSIDE

- ALL PHYSICS
- ALL MATTER
- ALL ENERGY
- ALL INFORMATION
- ALL GEOMETRY
- ALL TIME & SPACE
- ALL FORCES
- ALL FIELDS
- ALL PARTICLES
- ALL STATES
- ALL STRUCTURES
- ALL POSSIBILITIES

THE MACHINE OF REALITY

- ALL INGREDIENTS
- ALL LAWS
- ALL EQUATIONS
- ALL STRUCTURES
- ALL STATES
- ALL POSSIBILITIES

THE RESULT: ONE UNIFIED REALITY

✓ COHERENT ✓ BOUNDED ✓ ADMISSIBLE ✓ STABLE
ONE LAW • ONE RETURN • ONE CYCLE

WHAT IT DOES

- MIXES ALL DOMAINS
- CREATES COHERENCE
- ENFORCES BOUNDEDNESS
- ENSURES ADMISSIBILITY
- DRIVES THE RETURN CYCLE
- CLOSURES ALL LOOPS
- MAKES REALITY STABLE

24 DOMAINS 4 PIECE ENGINE 1 CLOSURE LAW THE COMPLETE UNIFIED SYSTEM

www.omni-compass.com

Plate 9. The unified container of all reality: 24 domains, four-piece engine, one closure law

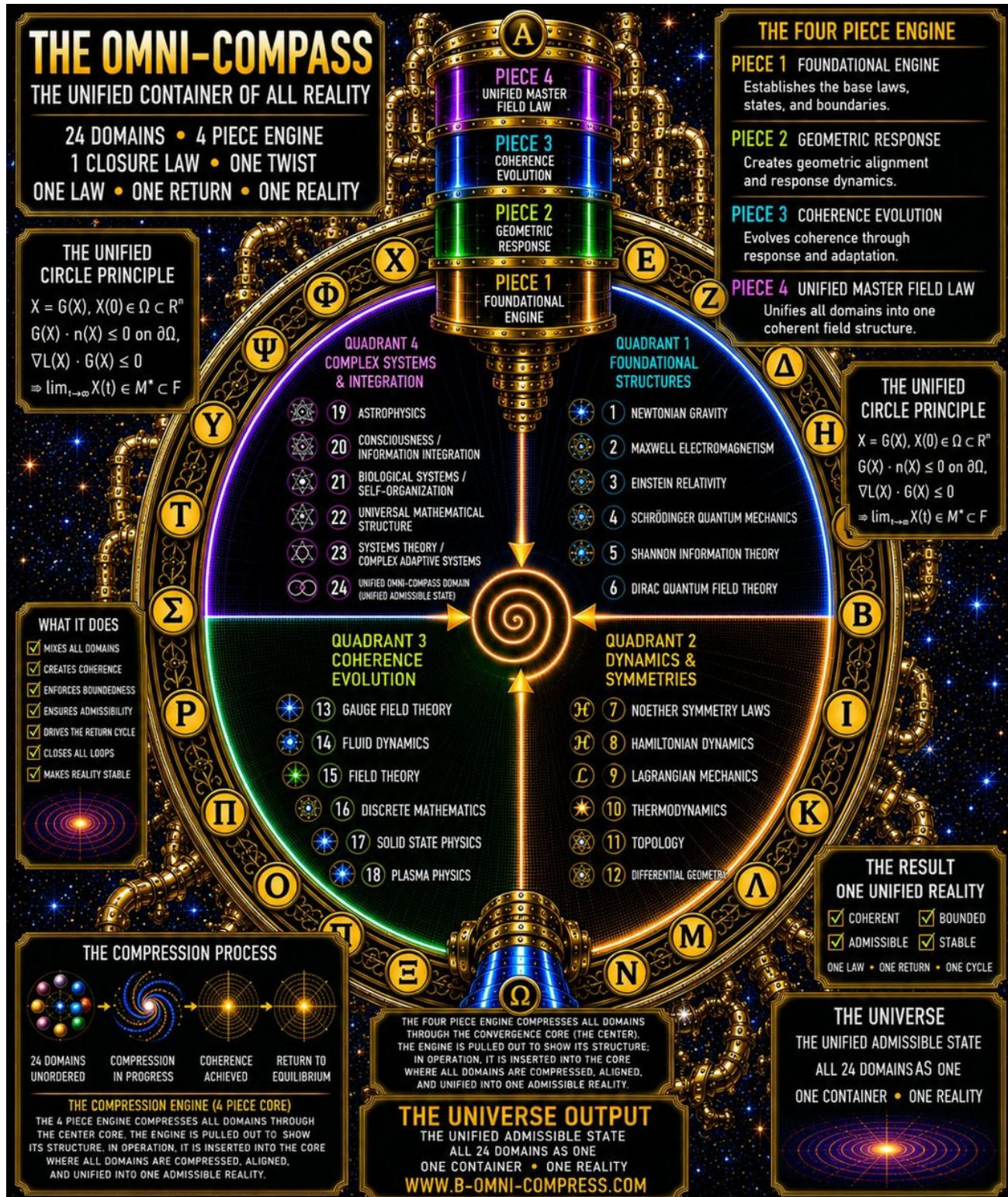


Plate 10. The four quadrants and the compression process

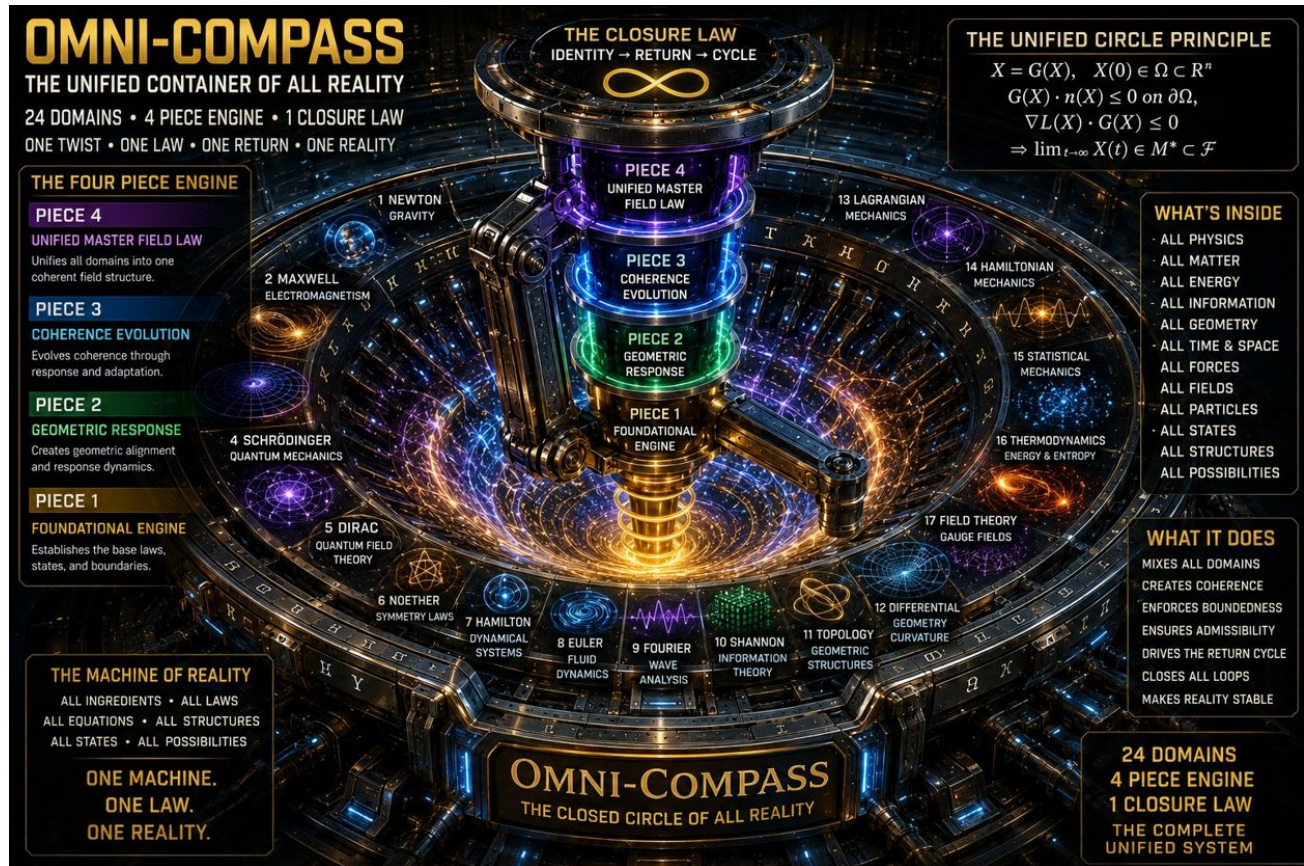


Plate 11. The closure law and the closed-circle engine

In the software, the four pieces appear as the engine's state equations (piece 1: deviation, alignment, memory and basin), the basin potential and its flow (piece 2), the bath that absorbs and redistributes (piece 3), and the one governor that couples every muscle to one law with no outside authority (piece 4).



Plate 12. The Omni-Compass rose

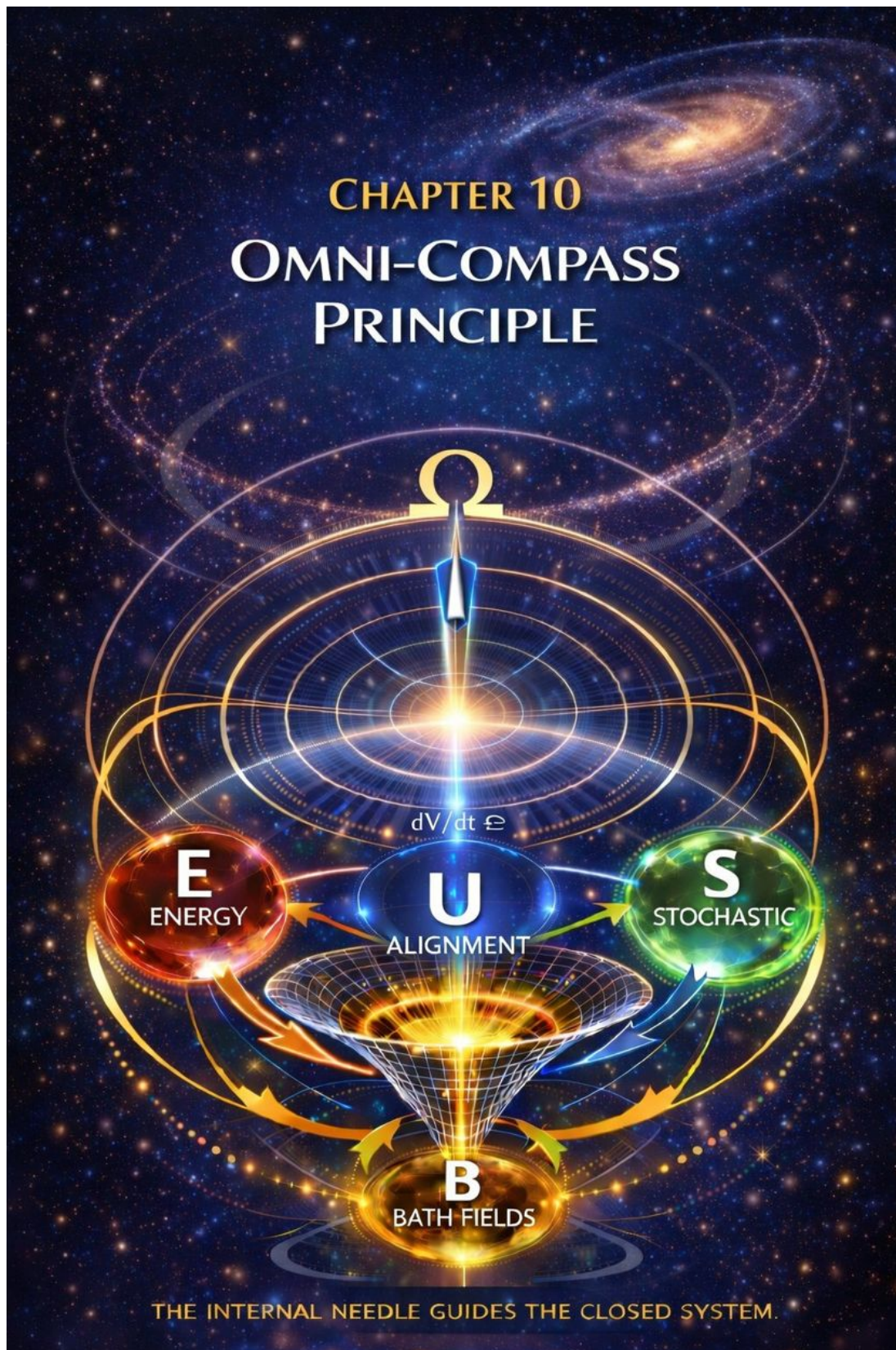


Plate 13. The Omni-Compass Principle

CHAPTER 4

The Compass

The Omni-Compass rose is the whole mechanism drawn as one picture.

The ring. Twenty-four letters, the complete Greek alphabet from Alpha to Omega: the complete set, nothing missing, one letter for each domain. On a ring the end runs back into the beginning: after the last letter the ring returns to Alpha. That is the closed circle: identity, return, cycle.

The four points. Four letters are lifted out of alphabetical order and set on the four points of the rose:

Point	Letter	Mark	Meaning in the engine
North	Alpha, the first	+	the positive pole of the alignment's double well, $U = +1$
South	Omega, the last	-	the negative pole, $U = -1$
East	Beta	>	the drive coming in (the beta terms that fill the deviation)
West	Gamma	<	the coupling that carries energy out to the bath, where friction absorbs it

The north-south line is polarity; the east-west line is flow in against flow out. A system stays inside its circle exactly when what flows out can always match what flows in.

The inner marks.

Mark	Meaning	In the mechanism
+ and -	the two polarities	the two poles of the double well
> and <	push out, pull in	the antagonist pair: the up force and the down force
the double arrow	exchange both ways	the two-way wire: read in, write out
the up-tack	the ceiling	the cover: nothing goes past the top
approximately-equal	the wave, the tolerance	the band and its cushion: hold near the center
the star	the guiding star	the target the needle points to

The center. The spiral at the middle is the attractor: every path winds inward to one point.

Opposite letters. Across the center, zeta faces xi: both are the engineer's symbol for the damping ratio, and a damping ratio of one is the critical glide to the center with no overshoot. Lambda faces psi: the eigenvalue and the wave function of the eigenvalue equation.

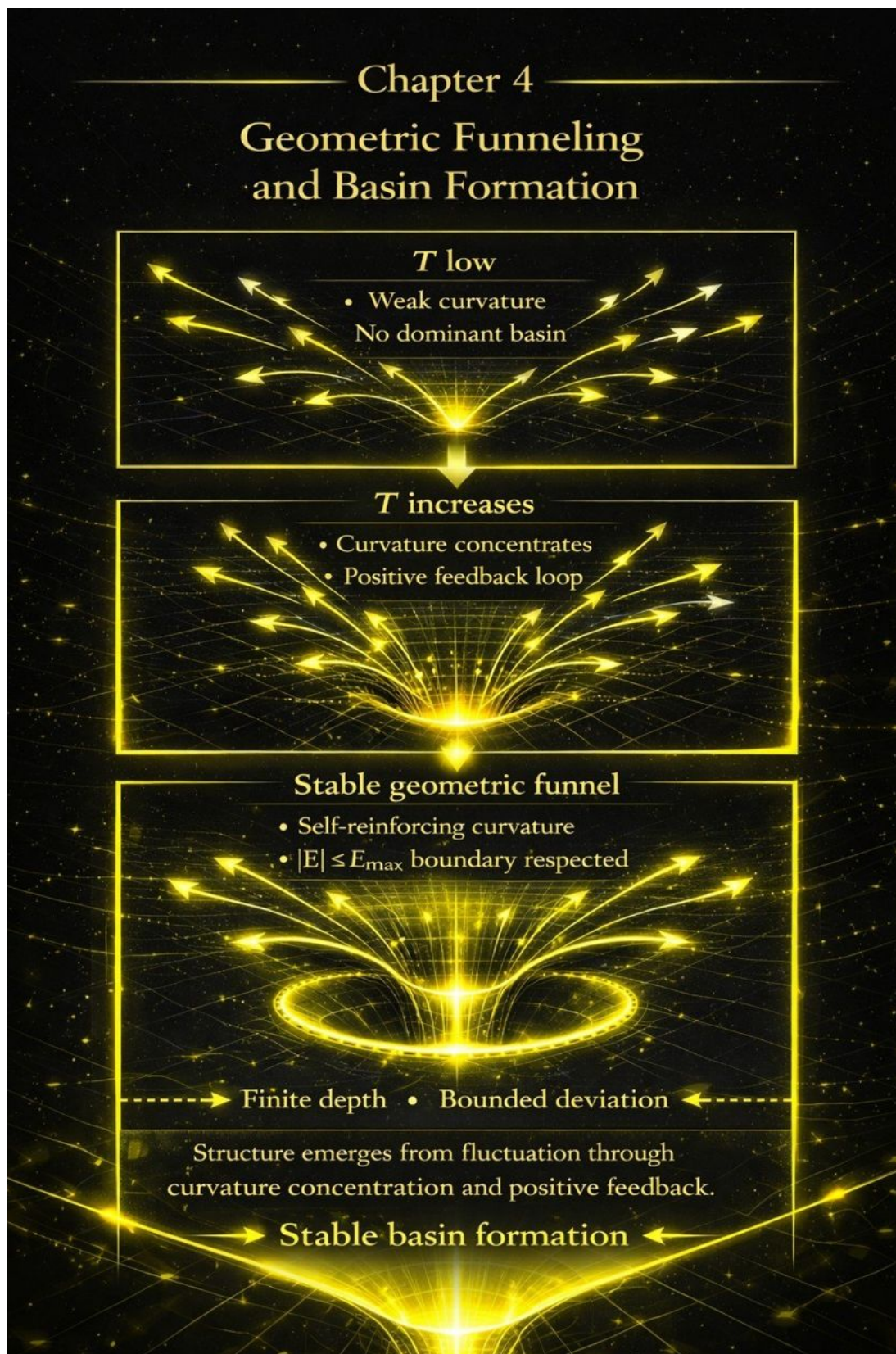


Plate 14. Geometric funneling and basin formation

CHAPTER 5

Basins, Polarity and the Dual-Basin Engine

Geometric funneling and basin formation

A basin forms in three stages. While the driving tension T is low, curvature is weak and there is no dominant basin: flows spread outward. As T increases, curvature concentrates and a positive feedback loop begins. Then a stable geometric funnel forms: self-reinforcing curvature, finite depth, bounded deviation, with the boundary $\text{abs}(E) \leq E_{\text{max}}$ respected. Structure emerges from fluctuation through curvature concentration and feedback.

In the engine, the basin state S rolls down its potential $\Phi(S)$ to a stable bottom; in the bowl, every muscle's service reading is pulled into the bottom of its own funnel.

Polarity

Two like charges repel without anyone pushing them: the force comes from the shape of the field. A wall built of like polarity at the edge of a band pushes the state back harder the closer it comes (as one over the distance squared), and two such walls cancel at the center, where the state comes to rest. This is the boundary condition of the closed circle written as a force: the field itself points inward at the wall.

The repulsion does not create energy: the field stores what it took to bring the charges together. What is gained in the computer is that the force is computed rather than spent: the brain computes it at no physical cost, and the machine's energy is saved because its knob no longer fights itself.

A symmetric pair of repelling walls was tested on the modelled GPU card. It raised work per energy but also raised the time spent past the service line, because the lower wall pushed capacity away when the card was calm. It was removed. The upper wall alone remains a candidate.

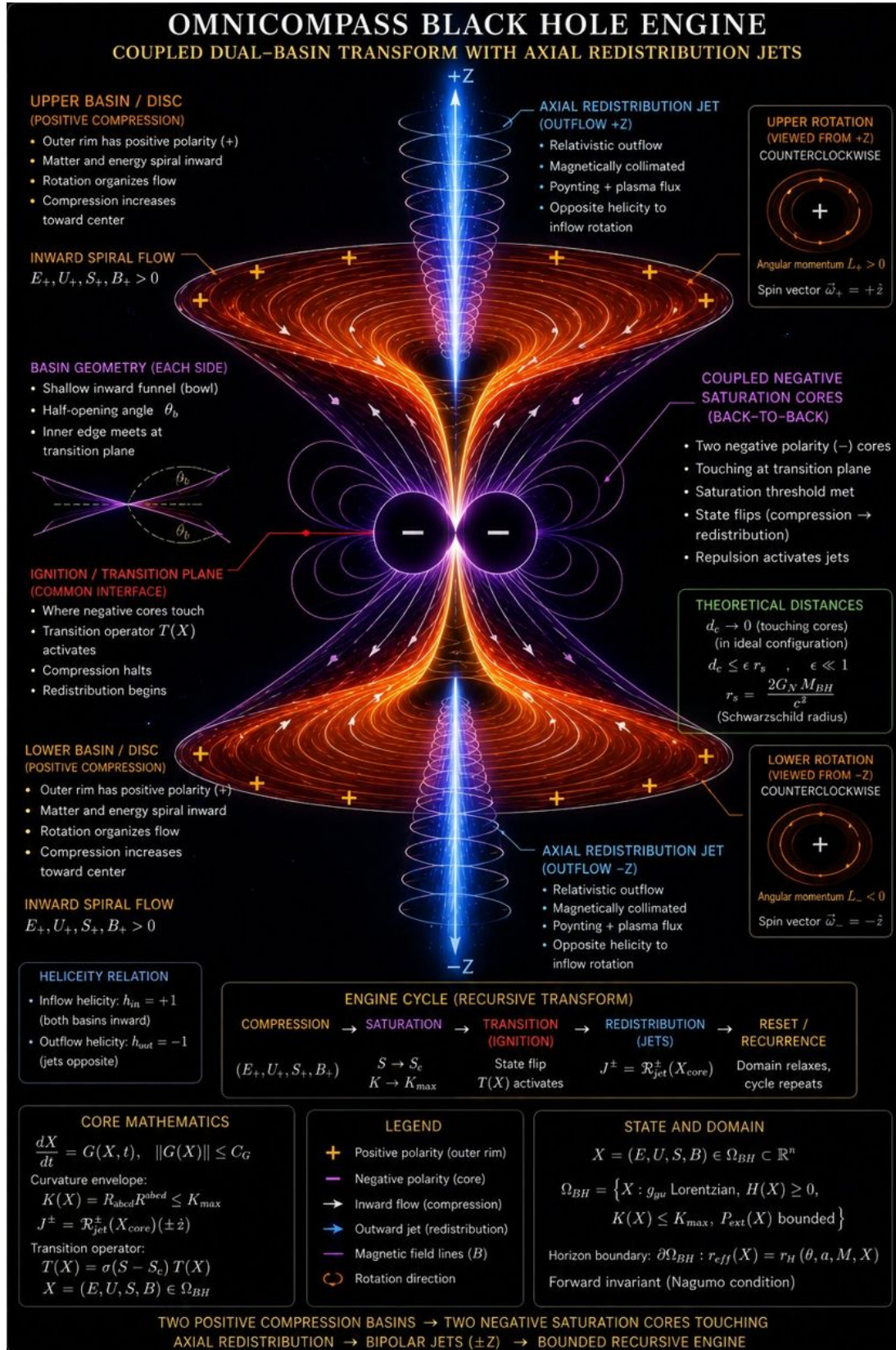


Plate 15. The Omni-Compass dual-basin engine

The dual-basin engine

The dual-basin engine is a recursive transform with five stages:

1. **Compression.** Two outer basins of positive polarity draw matter and energy inward in a spiral; rotation organizes the flow; compression increases toward the center.
2. **Saturation.** At the center, two cores of negative polarity meet back to back; the basin structure S reaches its critical value and the curvature reaches its maximum.
3. **Transition (ignition).** Where the cores touch, the transition operator $T(X) = \sigma(S - S_c) T(X)$ activates: compression halts and redistribution begins.
4. **Redistribution.** Two axial jets carry the excess out along the axis, with opposite helicity to the inflow.
5. **Reset and recurrence.** The domain relaxes and the cycle repeats.

The core mathematics is a bounded flow: $dX/dt = G(X, t)$ with $\text{abs}(G(X)) \leq C_G$, a curvature envelope $K(X) \leq K_{\text{max}}$, and a redistribution term that activates only at saturation. The state $X = (E, U, S, B)$ stays in its admissible domain, whose horizon boundary is forward invariant (the Nagumo condition again).

For an engineer, the transform reads: pull toward the center (compression); a wall where the muscle saturates; and, when one muscle saturates, move the excess to its siblings rather than hammering it (redistribution). The first two are built into the bowl. Redistribution across muscles is designed and not yet built; it is on the roadmap.

PART TWO

The Mathematics

Eight equations, one control law, and the proofs, audits and declarations that hold them fixed.

CHAPTER 6

The Canonical Declaration

Every report the engine has ever produced opens with the same declaration. It is the birth certificate of the mechanism: what the engine is, which equations it runs, what every symbol means, which parameters it samples, how a run is born, conveyed and certified, and what the engine is allowed to say about itself. This chapter sets it down in full so that anyone who reads the rest of this book can always come back to the source.

The identity of the engine

The Omni-Compass Unified Governing Convergence Control Core Engine. One canonical eight-line mathematical core, called Piece I, integrated by fourth-order Runge-Kutta (RK4) and exercised by Monte Carlo. Around that core sit the benchmark suites: the public enterprise manager mechanism benchmark, the one-stack-normalized global megastack, fragmented stack governance over matched scenarios, and the enterprise evidence suite covering cause, fail-safe, scale, muscle, security and economics. All of them run on bare metal, on premises, private, public, hybrid and edge alike, because the core does not care where it runs.

The equation form (authoritative, eight lines)

```
(1) dE/dt = -α_E E + β_int(t) + β_ext(t) + v_eff
(2) dU/dt = μ U(1 - U²) - (dE/dt)/E_max - λ_U U + u,      |u| ≤ U_AUTHORITY
(3) dI_U/dt = 1 - U - σ_1 E - δ S - λ_I I U
(4) v_eff = χ(t) c · tanh(λ_0 + λ_1(U - U_t) + λ_2 S),      χ(t) = cos(ω_B t / 2)
(5) Φ(S) = ½ α_s S² + ¼ β_s S³ - δ S
(6) dS/dt = F_state(S) = -∂Φ(S)/∂S = δ - α_s S - ¾ β_s S²
(7) B'' + (ω_B / Q_B) B' + ω_B² B = γ_c δ S [continuous]
(8) R_B[n] = D²_h B[n] + (ω_B / Q_B) D_h B[n] + ω_B² B[n] - γ_c δ S[n] = 0 [discrete audit]
```

Line (1) is the error, the energy of misalignment: it decays at its own rate, it is pushed by forces from inside and outside, and it is pushed by the effective velocity of the system. Line (2) is coherence, the alignment index: it lives in a double well with two homes at +1 and -1, it is pulled down by any rise in error, it leaks, and it is the one line the governor touches, through the control u , which is never allowed beyond its authority. Line (3) is the information channel that keeps the record of how far coherence has been from home. Line (4) is the effective velocity, saturated so that it can never exceed the limiting speed c , and multiplied by the spinor phase. Lines (5) and (6) are the potential of the coherence state and the flow it creates, downhill. Line (7) is the bath, the field that every real system sits in, driven by the state. Line (8) is the bath written on the grid the computer actually steps on: it is an audit of the numbers, not a second bath.

The control law

```
σ = nearest_signed_basin(U_0)
u_m = sat(-f_U(x_m, t_m) + K_P (σ - U_m), ± U_AUTHORITY)
x_(m+1) = RK4_h(x_m; u_m), u_m held across all four RK4 stages
```

The nearest signed basin is declared at Step 0, before the first microstep. The control cancels the natural drift of coherence and adds a proportional pull to the declared home, then it is clipped at its authority. It is held fixed through the four stages of each RK4 step. There is no post-step overwrite of the state and no projection onto the basin. Whatever the engine reaches, it reaches by the flow of its own equations.

THE OMNI-COMPASS UNIFIED GOVERNING CONVERGENCE CONTROL CORE ENGINE

THE OMNI-COMPASS EQUATION FORM

1	$\partial E = -\alpha E + \beta_{\text{int}}(t) + \beta_{\text{ext}}(t) + v_{\text{eff}}$
2	$\partial U = \alpha(1 - U) - \frac{\partial E}{E_{\text{max}}} - kU(1 - U)$
3	$\partial I_U = 1 - U - \sigma_1 E - \delta S$
4	$v_{\text{eff}} = c \tanh[\lambda_0 + \lambda_1(U - U_t) + \lambda_2 S]$
5	$\Phi(S) = \frac{1}{2}\alpha_s S^2 + \frac{1}{4}\beta_s S^3 - \delta S$
6	$\partial S = F_{\text{state}}(S) = -\frac{\partial \Phi(S)}{\partial S} = \delta - \alpha_s S - \frac{3}{4}\beta_s S^2$
7	$\partial^2 B + \frac{\omega_B}{Q_B} \partial B + \omega_B^2 B = \gamma_c \delta S \quad [\text{continuous}]$
8	$\Delta^2 B + \frac{\omega_B}{Q_B} \Delta B + \omega_B^2 B = \gamma_c \delta S \quad [\text{discrete}]$

THE OMNI-COMPASS SYMBOL CHART

E Error / misalignment energy	U Coherence / alignment index	I_U Information / internal- coherence channel	S Coherence-state potential	B Bath field
B_{dot} Bath-field time derivative	α Feedback rate	β_{int}(t) Internal forcing function	β_{ext}(t) External forcing function	k Coherence logistic damping gain
σ₁ E-coupling into I _U	δ Linear coherence coupling parameter	λ₀ Saturation bias	λ₁ Saturation gain	λ₂ S-state coupling gain
U_t Target coherence state	c Limiting velocity	E_{max} Per-trajectory rate normalization	α_s S-potential quadratic coefficient	β_s S-potential cubic coefficient
Φ(S) Potential-field function	F_{state}(S) Canonical S-state flow	ω_B Bath angular frequency	Q_B Bath quality factor	γ_c Bath coupling gain

www.omni-compass.com

U.S. PATENTS / COPYRIGHTS / TRADEMARKS — © ® 2026

Plate 18. The equation form and the symbol chart

The symbol chart

Symbol	Meaning
E	Error, the misalignment energy
U	Coherence, the alignment index
I_U	Information, the internal-coherence channel
S	Coherence state potential
B	Bath field
B'	Bath field time derivative
α	Feedback rate
$\beta_{\text{int}}(t)$	Internal forcing function
$\beta_{\text{ext}}(t)$	External forcing function
k	Coherence logistic damping gain
σ_1	Coupling of error into the information channel
δ	Linear coherence coupling parameter
λ_0	Saturation bias
λ_1	Saturation gain
λ_2	S-state coupling gain
U_t	Target coherence state
c	Limiting velocity
E_{max}	Per-trajectory error-rate normalization scale
α_s	S-potential quadratic coefficient
β_s	S-potential cubic coefficient
$\Phi(S)$	Potential field function
$F_{\text{state}}(S)$	Canonical S-state flow
ω_B	Bath frequency
Q_B	Bath quality factor
γ_c	Bath coupling gain

The coupling architecture of Piece I

The closed feedback core is E , U and S : error drives coherence, coherence and the state drive the velocity, the velocity drives error. The information channel I_U and the bath B with its derivative are driven response channels: they listen to the core and record it, and the core does not depend on them. γ_1 is reserved for compatibility and is inactive in the Piece I derivatives. The bath law uses $\omega_B^2 B$, so ω_B carries the units of angular frequency. Line (7) advances B and B' continuously by RK4. Line (8) evaluates the discrete residual with $D^2_h B[n] = (B[n+1] - 2B[n] + B[n-1]) / h^2$ and $D^-_h B[n] = (B[n] - B[n-1]) / h$.

The spinor closure

The spinor extension introduces the phase $\theta(t) = \omega_B t$ and multiplies line (4) by $\chi(\theta) = \cos(\theta/2)$. At 0 degrees the factor is +1, at 360 degrees it is -1, and at 720 degrees it is +1 again. A full turn of the bath reverses the sign of the velocity; only two full turns bring it home. The standard 20-step run evaluates the part of the phase it reaches, and a dedicated evidence test checks the 0, 360 and 720 degree periodicity on its own.

The signed basins

The canonical Monte Carlo sampler draws the starting coherence $U(0)$ symmetrically on $[-1, +1]$. The nearest signed basin at Step 0 is declared the target before the first microstep. That keeps the eight-line equation exactly as written while making every governed run exercise capture into both $+1$ and -1 . A separate dual-basin suite throws runs across from one basin to the other under disturbance and watches them recover.

Birth, conveyance and certification

Step 0 is the sampled state before integration. Step 1 is the first completed macro step. A run has at most 20 macro steps of 10 micro steps each, at a time step of 0.1. A step is in basin when U lies within 0.10 of $+1$ or -1 .

- **Born in basin.** A run that starts inside a basin at Step 0 is born in basin for life. Step 0 counts as the first observation of its streak. It is confirmed conveyed after Steps 0 to 4 stay in the same basin. It is never called self-tuned, and it stays under the same regulation for all 20 macro steps.
- **Self-tuning.** A run that starts outside both basins is self-tuning applied from Step 1. The same nearest-basin law is active on every trajectory, born or not, over the whole 200-microstep horizon. The category never switches the regulation off.
- **CONVEY-5.** Five consecutive same-basin observations. The first is the conveyance entry step, the fifth is the confirmation step. The latest streak may start at Step 16 and confirm on Step 20.
- **CERT-10.** Ten consecutive same-basin observations. The latest streak may start at Step 11 and complete on Step 20. The code asserts both bounds before it runs and checks every emitted row against them.

Conveyance is an event, not an ending. No run may be both born in basin and self-tuned. Stability is judged on the history, not on one snapshot.

The Monte Carlo declaration

Setting	Value
Initial U distribution	Uniform on $[-1, +1]$, symmetric
Monte Carlo runs	500
Macro steps per run	20
Micro steps per macro step	10
Integration time step	0.1
Basin centers	$+1$ and -1
Basin tolerance	0.10
Stability window	5
Conveyance cutoff step	20

The full horizon is always executed. A run is classified as a failure if it crosses the divergence threshold at any recorded state, or if it exhausts the horizon without CONVEY-5.

The parameter sets

Group	Parameter	Value or range
Core	α	4.2
Core	β_{int}	0.0 to 1.0
Core	β_{ext}	0.0 to 1.0
Core	E_{max}	1.0 to 10.0, sampled once per trajectory
Core	k	1.7
Information and coupling	σ_1	0.38
Information and coupling	δ	0.01 to 1.0
Information and coupling	γ_1	1.35, reserved, inactive in the derivatives
Information and coupling	γ_{c}	1.0
Saturation and phase	c	0.5 to 5.0
Saturation and phase	λ_0	-2.0 to 2.0
Saturation and phase	λ_1	0.1 to 3.0
Saturation and phase	λ_2	0.0 to 2.0
Saturation and phase	U_{t}	0.5
Saturation and phase	$\chi(t)$	$\cos(\omega_{\text{B}} t / 2)$ when the spinor closure is on
S-state potential	α_{s}	0.05 to 0.25
S-state potential	β_{s}	0.05 to 0.25
Bath	ω_{B}	0.1 to 5.0
Bath	Q_{B}	0.5 to 10.0
Noise and drift	D	0.0
Noise and drift	μ	0.0

The mechanism-bound reporting doctrine

Every table, grid, chart, scorecard, classification and conclusion the engine emits must be computed from what it actually executed: the trajectory rows, the recorded state histories of E, U, I_U, S, B and B', the sampled parameters, the RK4 derivative evaluations, the basin entry and dwell histories, the line (8) residuals, the solver comparisons, the ablations, the disturbance and stress runs, the compute benchmark event records, and the enterprise benchmark ledgers.

No decorative grade, inferred score, hand-assigned quality category or presentation-only number is allowed to stand as an engine result. Prose may point to the exact source and the edge of the claim. It may not manufacture evidence, put a grade where a measurement belongs, or present a grid the mechanism never ran as though it had. That doctrine runs through every page of this book.

Availability and the rights boundary

Time is of the essence. Omni-Compass is available from The Omni-Compass LLC for controlled technical evaluation, research collaboration, pilot integration, strategic engagement and separately licensed commercial deployment. The same rights boundary applies at every level of use: inspection, local execution, simulation, research, benchmarking, observe-only, shadow mode, supervised control, bounded autopilot, hybrid control, direct-to-muscle control, production, embedding, hosted service, productization, monetization, redistribution and derivative work.

Public access does not make Omni-Compass open source. Commercial use of any kind requires a separately executed written license. The instrument is a license, not a sale and not a transfer of ownership. For planning only, the introductory benchmark is 10% of independently verified and contractually accepted value captured; the base and the amount are specific to each company, the terms are expected to rise as validation and adoption grow, and only a signed agreement creates any obligation.

CHAPTER 7

The Engine: Eight Equations and One Control Law

The engine (`omnicompass/core.py`, frozen and fingerprinted) carries a six-part state $x = (E, U, I_U, S, B, B_{\dot{}})$:

State	Meaning	Physical picture
E	deviation: how far the system is from where it should be	a tank that fills with stress and drains on its own
U	alignment, between the two poles -1 and +1	a ball in a double well: two stable poles, a hill between them
I_U	memory of misalignment	an integrator that remembers and slowly forgets
S	basin structure	a ball rolling to the bottom of its landscape
B, B_dot	the bath	a spring with friction that absorbs and settles energy

The eight equations (Appendix C gives them in full):

1. $dE/dt = -\alpha_E E + \beta_{\text{int}} + \beta_{\text{ext}} + v_{\text{eff}}$
2. $dU/dt = \mu U (1 - U^2) - (dE/dt)/E_{\text{max}} - \lambda_U U + u$, with $|u| \leq 25$
3. $dI_U/dt = (1 - U) - \sigma_1 E - \delta S - \lambda_I I_U$
4. $v_{\text{eff}} = \cos(\omega_B t / 2) c \tanh(\lambda_0 + \lambda_1 (U - 0.5) + \lambda_2 S)$
5. $\Phi(S) = \alpha_S S^2/2 + \beta_S S^3/4 - \delta S$
6. $dS/dt = -d\Phi/dS$
7. $d^2B/dt^2 = \gamma_c \delta S - (\omega_B/Q_B) dB/dt - \omega_B^2 B$
8. R_B : a finite-difference audit of (7), never fed back

The control law. $u = \text{clip}(-f_U(x, t) + K_P (\sigma - U), -25, +25)$, where f_U is the drift of U with the command at zero, σ is the target pole and $K_P = 12$. The first term cancels whatever is shoving U (the push); the second pulls U to its pole, harder the farther it is (the pull). The command is held unchanged through every stage of the fourth-order Runge-Kutta step and U is never rewritten afterwards. Inside the authority limit, U converges to its pole at rate K_P ; this is proved (`docs/TRACKING_THEOREM.md`).

CHAPTER 8

The Canonical Engine

One engine runs Omni-Compass, and every result in this repository comes from it. This page names it, gives its equations exactly as the code computes them, and names every other form as a variant. Where any document, manual, chart or filing states the equations differently, this page and the file it fingerprints are what the software runs.

1. The canonical engine: `symmetric_verified`

Name	<code>symmetric_verified</code> (omnicompass/configurations.py, DEFAULT)
Code	omnicompass/core.py (Python), cpp/src/core.cpp (C++ twin, sealed in results/SEAL.json)
Fingerprint	SHA-256 of omnicompass/core.py, recorded in results/PREREGISTRATION.json and RELEASE_MANIFEST.json; checked by verify.py
Evidence	every simulation, benchmark, live Kubernetes run and GPU harness in this repository

State: E (deviation), U (coherence), I_U (unmet-need integral), S (structural stress), B and B_dot (the bath).

```
v_eff = cos(omega_B t / 2) · c · tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)      (spinor closure)
↳ e, 720°)
dE/dt = -alpha_E E + beta_int + beta_ext + v_eff
dU/dt = mu U (1 - U²) - (dE/dt) / E_max - lambda_U U + u                          (symmetric double well)
↳ ble well)
dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
dS/dt = delta - alpha_s S - (3/4) beta_s S²                                       (gradient of Phi)
↳ Phi)
dB/dt = B_dot
dB_dot/dt = gamma_c delta S - (omega_B / Q_B) B_dot - omega_B² B
Phi(S) = alpha_s S² / 2 + beta_s S³ / 4 - delta S
```

u is the controller's command, $K_P (target - U) - drift$, bounded by `U_AUTHORITY`; integration is RK4 with the macro and micro steps of `omnicompass/core.py`.

Mechanism identity

`results/MECHANISM_IDENTITY.json` (`tools/mechanism_identity.py`) fingerprints the mechanism $M = (F, \Theta, C, h, G, M_{act}, dt, A)$ component by component: state law, parameters, controller, observation map, authority law, actuator map, execution timing, shield. `verify.py` fails if any component changes without the record being rewritten. Every evidence claim names the mechanism id that produced it; "the eight-line engine" alone names none.

Configuration	Mechanism id	Role
<code>symmetric_verified</code>	29d9808dfb8f626ad5de17a8a1efa37411dbab08a64af4b276143b466f7ce21c	canonical
<code>printed_eight_line</code>	cd333dc166fb7684ec0fca71f5f50488041e47825ee47d1ea667bec1f3d2259d	named alternative embodiment

On the frozen 500-fixture population (benchmarks/core_evidence.py, seed 223387268), both are executable and neither is a stand-in for the other:

	finite	CONVEY-5	CERT-10	mean final target error	mean integrated abs(u)	max abs(u)
symmetric_verified	500/500	500/500	500/500	7.48e-5	0.878	13.93
printed_eight_line	500/500	500/500	500/500	5.13e-6	11.79	18.87

The printed form reaches its target more tightly with about 13 times the control effort. The structure is Option A: one canonical configuration, one named alternative; no equivalence is claimed.

2. Variants

printed_eight_line (*the printed chart*)

Implemented in omnicompass/configurations.py, checked against the printed plate (tests/test_engine_configurations.py), **not benchmarked**: no result in this repository comes from it.

```

v_eff = c · tanh(lambda_0 + lambda_1 (U - U_t) + lambda_2 S)           (no spinor fac
↳ tor)
dE/dt = -alpha E + beta_int + beta_ext + v_eff
dU/dt = alpha (1 - U) - (dE/dt) / E_max - k U (1 - U) + u           (logistic)
dI_U/dt = (1 - U) - sigma_1 E - delta S                               (no lambda_I
↳ term)
dS/dt, dB/dt, dB_dot/dt as in section 1

```

Differences from the canonical engine: logistic U instead of the symmetric double well; no spinor factor; a general target U_t; no lambda_I damping; alpha in place of alpha_E. alpha_U and k enter the printed form; they do not enter the canonical engine.

3. What would change this declaration

Promoting a variant to canonical needs, in one commit: the variant run through every harness beside the canonical engine, its results reported next to the canonical results, this page and RELEASE_MANIFEST.json updated, and the preregistration amended on record (results/LOCK_AMENDMENTS.json). Until then the canonical engine is symmetric_verified.

4. Open mathematical items (from docs/FORMAL_STATUS.md and the handoff)

- A global stability proof of the forced six-state system is not closed. What is held on the U channel is proved in docs/TRACKING_THEOREM.md: unsaturated exponential tracking; the saturated case under a drift bound that holds over the declared box ($F_{\text{bar}} = 16.86 < 25$); the sampled-data bound of the executed RK4 controller; admissibility.
- Candidate routes: the Unified Circle Principle on a region; Theorem 5.6 of the Closed Structure, if its core maps onto (E, U, I_U, S, B).

5. For filings

Which form a patent or copyright filing claims as the principal embodiment is a decision for The Omni-Compass LLC and its counsel. Whatever that decision, the software described by this repository runs the canonical engine of section 1, and a filing that claims the printed form should name symmetric_verified as the embodiment that has been implemented and tested, or the printed form should be benchmarked first (section 3).

CHAPTER 9

The Closed Circle: Why It Cannot Leave Its Bowl

The governing principle is the Unified Circle Principle:

$\frac{dX}{dt} = G(X), \quad X(0) \text{ in } \Omega$	
$G(X) \cdot n(X) \leq 0 \text{ on the boundary of } \Omega$	(the flow points inward at the wall: nothing crosses)
$\text{grad } L(X) \cdot G(X) \leq 0$	(the bowl's energy L only falls)
$\lim_{t \rightarrow \infty} X(t) \text{ in } M^*$	(every path settles in the bottom of the bowl)

In control engineering the second line is the Nagumo condition for an invariant set and the third is a Lyapunov function. Together they mean: anything that starts inside the container stays inside it, for every disturbance up to a known size, and comes to rest at the bottom. That is a certificate, not a test: it answers every case inside the walls at once.

The engine's states are each closed in this sense: the deviation tank drains faster than it can fill; the alignment's cubic walls push back from far out; the memory forgets; the bath's friction settles it. The rule behind all four is the compass's east-west line: what flows out (drain, damping) must always be able to match what flows in (drive).

The compass. The Omni-Compass rose carries the whole Greek alphabet, Alpha to Omega: the complete set, in a ring whose end runs back into its beginning. Its north-south axis is polarity: Alpha and plus at the top, Omega and minus at the bottom, the two poles of the alignment's double well. Its east-west axis is flow: Beta and the push outward on one side, Gamma and the pull inward on the other, the drive and the damping. The spiral at the center is the attractor every path winds into. The ring closing on itself is the return: when Omni-Compass stops, every lever returns to where it began.



Plate 16. Mathematical integration



Plate 17. The engine core: the governing law in motion

CHAPTER 10

Closing the Circle in the Engine

Each state of the engine was checked against the closed-circle conditions:

State	Closed?	Why
E, deviation	closed	it drains at rate α_E while its inputs are bounded (the drive is capped by $\tanh$)
U, alignment	closed	the cubic term $\mu U (1 - U^2)$ pushes back hard from far out
I_U, memory	closed	it forgets at rate λ_I
B, the bath	closed	the friction ω_B / Q_B settles it
S, the basin	closed on the positive side; open on the negative side below an unstable ridge	$\Phi(S)$ carries a cubic term, so the landscape falls away below the ridge (at $S = -3.5$ with the default parameters)

Three openings were found and each has its closure:

1. **The basin's negative side.** A quartic term in $\Phi(S)$ would give the bowl a wall on both sides. This changes the frozen engine and is a new version with its own proof.
2. **The outer loop.** The frozen live governor computes the push and pull u and uses it as a convergence signal; it does not send u to a lever. The bowl closes this loop: reading, force, plug, lever, read-back. On the modelled card this is the difference between +0.1% and +9.0% work per energy.
3. **The corner.** $\text{clip}()$ is a hard stop; $\tanh$ is its smooth form. The bowl uses $\tanh$.

CHAPTER 11

The Tracking Theorem

This page proves the internal tracking result of the canonical engine (symmetric_verified, mechanism id in results/MECHANISM_IDENTITY.json) in four steps, from the continuous law to the code as executed. Each statement carries one evidence class (see docs/EVIDENCE_LEDGER.md): **T** proved here, **V** checked by computation over a finite, frozen set (tools/tracking_bounds.py → results/TRACKING_BOUNDS.json).

What this page establishes is **internal**: inside the declared model, the controller conveys U to its target σ . It does not establish that σ is the right target for any outside plant, that the telemetry map is right, that any actuator mapping (HPA, GPU, CPU) is right, any energy saving, production performance, or stability of the whole forced six-state system. Target correctness and target conveyance are separate obligations; this page carries only the second.

Setting

U channel of equation (2): $dU/dt = f_U(x, t) + u$, with the uncontrolled drift

$$f_U(x, t) = \mu U (1 - U^2) - (dE/dt) / E_{\max} - \lambda_{\text{lambda}} U U$$

and the bounded cancellation-plus-proportional controller (omnicompass/core.py, control_command)

$$u = \text{sat}(-f_U(x, t) + KP (\sigma - U)), \quad \text{sat}(v) = \max(-u_{\max}, \min(u_{\max}, v)), \quad KP = 12, \quad u_{\max} = U_{\text{AUTHOR}} \hookrightarrow \text{ITY} = 25.$$

Tracking error $e = U - \sigma$, σ in $\{-1, +1\}$.

Theorem 1 (T). Unsaturated tracking

On the unsaturated region $\Omega_{\text{unsat}} = \{x : |-f_U(x, t) - KP e| \leq u_{\max}\}$, $de/dt = -KP e$. Along any interval spent in Ω_{unsat} , $e(t) = e(t_0) \exp(-KP (t - t_0))$, and $V = e^2/2$ satisfies $dV/dt = -KP e^2 < 0$ for $e \neq 0$.

Proof. Inside Ω_{unsat} the saturation is inactive, so $dU/dt = f_U - f_U + KP (\sigma - U) = -KP e$; since σ is constant, $de/dt = dU/dt$. The solution and $dV/dt = e de/dt = -KP e^2$ follow. ■

This is a statement about intervals inside Ω_{unsat} only; it is not global convergence.

Theorem 2 (T). The saturated case, under a declared drift bound

Assumption D. Along the trajectory, $|f_U(x(t), t)| \leq F_{\text{bar}} < u_{\max}$.

Under D:

1. If saturation is active with $e < 0$, then $u = +u_{\max}$ and $de/dt = f_U + u_{\max} \geq u_{\max} - F_{\bar{}} > 0$; symmetrically for $e > 0$. So $dV/dt \leq -(u_{\max} - F_{\bar{}}) |e| < 0$ wherever saturation is active.
2. With Theorem 1, V is strictly decreasing wherever $e \neq 0$, saturated or not; e never changes sign (at $e = 0$ the command is unsaturated because $F_{\bar{}} < u_{\max}$, so $e = 0$ is an equilibrium of the error dynamics).
3. Any saturated stretch ends within $(|e(t_0)| - e^*) / (u_{\max} - F_{\bar{}})$ time units, $e^* = (u_{\max} - F_{\bar{}}) / KP$; the ball $|e| \leq e^*$ lies inside Ω_{unsat} and is forward invariant, and inside it the decay is exponential at rate KP .

Proof of 1. Saturation with $u = +u_{\max}$ means $-f_U - KP e > u_{\max}$, so $-KP e > u_{\max} + f_U \geq u_{\max} - F_{\bar{}} > 0$, so $e < 0$, and $de/dt = f_U + u_{\max} \geq u_{\max} - F_{\bar{}}$. The other side is symmetric. 2 and 3 follow from 1 and Theorem 1, and $|-f_U - KP e| \leq F_{\bar{}} + KP |e| \leq u_{\max}$ on the ball. ■

Where D holds (T). On the declared box — parameters in `PARAMETER_RANGES`, E_0 in $[0, 1]$, $|U| \leq 1.1$ — equation (1) keeps E in $[(\beta - c)/\alpha_E, (\beta + c)/\alpha_E]$ widened to hold E_0 (the right side of (1) points inward at both ends, because $|v_{\text{eff}}| \leq c$), so $|E| \leq 7/4.2$, $|dE/dt| \leq 4.2 \cdot |E| + 2 + 5 \leq 14$, and

$$F_{\bar{}} = 6 \cdot 2/(3\sqrt{3}) + 14 / 1 + 0.5 \cdot 1.1 = 16.86 \leq 25 = u_{\max}.$$

Because e keeps its sign and $|e|$ does not grow (continuous time), U stays between U_0 and σ , so $|U| \leq 1$ and the box is self-consistent for the nearest-basin target. **Theorem 2 therefore holds globally over the declared box, in continuous time.** The actuator can still saturate: $|raw| \leq F_{\bar{}} + KP |e_0|$ can exceed 25 when $|e_0|$ is near 1.

Checked (V). Over the 500 frozen fixtures: observed max $|f_U| = 6.84$ (nearest target), 7.01 (wrong target), each under that fixture's own bound (max 8.99) and under 16.86; saturated micro steps 0 of 100 000 (nearest target), 5 of 100 000 (wrong target).

Theorem 3 (T with a computed constant). The executed sampled-data controller

The code does not run the continuous law. Per micro step $h = \text{MICRO_DT} = 0.01$, it computes u_k from x_k , holds it through the four stages of one RK4 step (zero-order hold), and takes $x_{(k+1)} = F_h(x_k, u_k)$. There is no overwrite of U after the step, no projection onto a basin, no state replacement.

Integrating the U equation over one held interval,

$$e_{(k+1)} = e_k + h (f_U(x_k) + u_k) + \int_{t_k}^{t_{k+h}} (f_U(x(t)) - f_U(x_k)) dt + \tau_k$$

with τ_k the RK4 truncation error. Unsaturated, $f_U(x_k) + u_k = -KP e_k$, so

$$e_{(k+1)} = \rho e_k + d_k, \quad \rho = 1 - h KP = 0.88, \quad |d_k| \leq (h^2/2) \sup |df_U/dt| + |\tau_k| =: \epsilon_h.$$

Then by induction $|e_k| \leq \rho^k |e_0| + (1 - \rho^k)/(1 - \rho) \cdot \epsilon_h$, and the error enters and stays in the neighbourhood $|e| \leq \epsilon_h / (1 - \rho)$.

The proof above is exact given ϵ_h . **ϵ_h itself is computed, not proved (V):** the largest $|d_k|$ over every micro step of the 500 frozen fixtures is 0.00378 (nearest target), giving the ultimate bound 0.0315, inside `BASIN_TOL = 0.10` that CONVEY-5 and CERT-10 test. With the target aimed wrong, $\epsilon_h = 0.0243$ (dominated by the long transient, which includes the 5 saturated steps) and the bound 0.20 is wider than the basin — a conservative bound; the fixtures still certify 500 of 500.

What the discrete execution does **not** inherit: the continuous monotone decay. Within the band, $|e|$ grew on 27 061 of 100 000 micro steps and e changed sign 747 times — the sampled controller dithers inside $\epsilon_h / (1 - \rho)$.

CONVEY and CERT. They are dwell checks against a declared neighbourhood ($|U - \sigma| \leq 0.10$ for 5 and 10 consecutive macro steps). Theorem 3 says why they pass when the bound sits inside the neighbourhood. They are not an external certification.

Theorem 4 (T continuous; V discrete). Admissibility

Admissible set for a fixture with parameters p and initial state x_0 :

$$A = \{ x : E \text{ in } [E_{\text{lo}}, E_{\text{hi}}], \quad |U - \sigma| \leq |U_0 - \sigma|, \quad S \text{ between } S_0 \text{ and } S_{\text{plus}}(p) \}$$

$E_{\text{lo}}, E_{\text{hi}}$ as in Theorem 2; S_{plus} the stable root of equation (6).

- **Continuous (T).** A is forward invariant: E by the inward-pointing field at both ends; U by Theorem 2; S because equation (6) is the one-dimensional gradient flow of Φ , monotone toward S_{plus} from any $S_0 > S_{\text{minus}}$ ($s_{\text{admissibility}}$ in `results/core_evidence.json`: 500 of 500 fixtures start above S_{minus}).
- **Discrete (V).** For the executed map $F_h(x, C(x))$, every micro step of every fixture: x_k in A implied $x_{(k+1)}$ in A — 0 failures in 100 000 steps, target nearest or wrong. This is a finite verification on the frozen fixtures, not a proof over the box. An interval-arithmetic proof over the whole box is **open (O)**.

What remains open (O)

- Global stability of the forced six-state system (E, I_U, S, B are not all covered above).
- A proved (not computed) ϵ_h over the declared box, and discrete invariance over the box.
- The inheritance / intertwining residual against an outside plant: instrumented (`tools/gpu_reps.py`, representation fidelity), not yet measured on a real card.
- The printed configuration (`printed_eight_line`) is not covered by this page: its U drift is logistic, not the double well, and needs its own bound (`docs/CANONICAL_ENGINE.md`).

CHAPTER 12

The Conveyance Law

1. What the manuscript says, read as mechanism

Source	Text	Mechanism
Ch. 29 §8	"E_system = E + U + S + B ... dE_system/dt = 0 (over full cycle). Energy redistributes. Energy reorganizes. Energy does not diverge."	The budget (site watts, cluster cores) is conserved : it is moved between organs, never created.
Ch. 31 §4	"Redistribution B acts as mediator between the baths. When E grows locally, B increases to distribute gradients."	Budget flows down the gradient of need : from organs holding surplus to organs in deficit.
Ch. 31 §5	"eigenvalues of the coupled system must remain bounded ... $\text{Re}(\lambda_i) \leq 0$ "	The flow must have a proof of convergence, not a tuned damper.
Ch. 30 §5-7	"Redistribution surge ... channels stored deviation into expansion ... controlled release ... No discontinuity."	A reserve is held and released continuously to organs at their limit. There is no on/off.
Ch. 29 §6	"Backpressure develops through S ... Compression is ... structured inward rotation."	When need falls, surplus is pulled back smoothly. Budget nobody needs is not spent.
App. J	"the engine can only act through what you expose"	Organs keep their own mechanisms. The law sets only each organ's share of the budget.

2. The law

Notation.

- Organs $i = 1..n$ share a budget A .
- Organ i holds allocation a_i and has need d_i ; the budget that serves its work at the engine's utilisation target ρ .
- Its deficit (local deviation) is

$$e_i = d_i / (\rho a_i) - 1 \quad (e_i > 0: \text{needs juice}; e_i < 0: \text{holds surplus})$$

The redistribution mediator:

$$da_i/dt = \kappa a_i (e_i - \bar{e}), \quad \bar{e} = \sum_j a_j e_j / A$$

3. Properties and proofs

P1. Conservation.

- $\sum_i da_i/dt = \kappa (\sum_i a_i e_i - \bar{e} \sum_i a_i) = \kappa (A \bar{e} - \bar{e} A) = 0$.
- The budget is moved, never created, as in Ch. 29 §8.

P2. Gradient flow with a Lyapunov function.

- Substitute e_i : $a_i(e_i - \bar{e}) = d_i/\rho - a_i D/(\rho A)$, where $D = \sum d_i$.

- So $da_i/dt = (\kappa/\rho)(d_i - a_i D/A)$.
- This is the Shahshahani gradient on the simplex $\Sigma a_i = A$ of $F(a) = \sum_i (d_i/\rho) \ln a_i$. The Shahshahani metric weights by $1/a_i$, and $\partial F/\partial a_i = d_i/(\rho a_i)$, so $a_i(\partial F/\partial a_i - \text{mean})$ is exactly the flow above.
- Along the flow $dF/dt = \sum_i (1/a_i)(da_i/dt)^2 \cdot (\rho/\kappa) \geq 0$.
- F is strictly concave, so F is a Lyapunov function and the flow cannot cycle.

P3. Global exponential convergence.

- The ODE in P2 is linear: $a_i(t) = a_i^* + (a_i(0) - a_i^*) e^{(-\kappa Dt/(\rho A))}$, with $a_i^* = A d_i/D$.
- The Jacobian is $-(\kappa D/(\rho A)) I$ on the simplex, so every eigenvalue is $-\kappa D/(\rho A) < 0$. This meets $\text{Re}(\lambda) \leq 0$ of Ch. 31 §5 strictly.
- At the equilibrium every organ has the same deficit $e^* = D/(\rho A) - 1$. None is starved while another idles.
- `replicator_step` integrates it exactly, in closed form, for any step size.

P4. Bounds (floors, ceilings, the budget).

- Allocations are projected onto $\{l_{0i} \leq a_i \leq h_{1i}, \Sigma a_i \leq A\}$.
- The projection is water-filling: organs at a bound keep it, and the rest share the remainder in proportion.
- Budget an organ cannot use (it is at its need or its ceiling) is not spent.

P5. Reserve and surge.

- The dual-bath rate channel (the rate tracker) projects the rise in need over one actuation delay: $R = \sum_i \max(0, d_i)$, capped at $A/2$. R is held back.
- After the flow, R is released continuously to organs whose deficit stays positive, in proportion to their shortfall and up to their ceilings.
- This is the controlled release of Ch. 30: continuous state, no switch.

4. Evidence

Mathematics in code. `tests/test_conveyance.py` (in `verify.py`) checks 20,000 random systems:

- P1 conservation to $1e-9$;
- P2, F never decreases;
- P3, the trajectory equals the closed form;
- P4, bounds and budget.

It also runs 2,000 systems over 30 steps and checks the full law never exceeds the budget or breaks a floor. That test found and fixed one real bug: an organ whose need fell below its device floor was given a ceiling under that floor.

Simulation. `hardware/site_exchange.py`: four GPU groups of 16 H100-class GPUs share one site budget (MLPerf gamma). Held-out seed 929292, 48 scenarios per budget. Results: `results/hardware/SITE_EXCHANGE_HELDOUT_*.json`.

Site budget (share of TDP)	70%	60%	50%
Site-budget violation minutes, native / independent / conveyance	10.5 / 1.5 / 0	45.6 / 10.5 / 0	106.1 / 31.3 / 0
Energy (kWh), native / static split / independent / conveyance	142.5 / 117.6 / 104.1 / 103.8	139.1 / 107.9 / 103.5 / 102.0	127.3 / 97.1 / 101.0 / 96.4
Backlog minutes, static split (the arm that also keeps the budget) / conveyance	32.0 / 1.0	89.7 / 8.9	187.3 / 72.2

Reading.

- **Keeping the budget.** Conveyance is the only dynamic arm that never exceeds the site budget.
- **Energy.** It uses the least energy at every budget.
- **Against the only other arm that keeps the budget (static split):** 31-115 fewer backlog minutes.
- **At tight budgets, native and independent sizing show less backlog,** but only by drawing power over the site limit for 10-106 minutes, which in a facility trips breakers.
- **When the budget is below total need,** conveyance shares the shortage in proportion to need. That is the unique stable point of P3: fair, and not a choice of weights.

CPU and GPU on one budget. `hardware/node_exchange.py`: the same law with CPU organs beside the GPU organs. Four groups, each two 8-GPU servers (16 H100-class GPUs, 4 CPU sockets); each unit of GPU work needs 0.35 of the CPUs at full clock to feed it; demand 1.6x the plant traces (the GPUs are held back by the budget, not by work). Arms:

- **S** today's practice: every GPU at one fixed cap low enough that the site fits even with every CPU at maximum;
- **XM** conveyance among the GPUs, counting the CPUs' last measured draw;
- **XC** conveyance over CPU and GPU organs together: each CPU group is held to the clock its feeding work needs, and the watts it no longer holds flow to the GPU groups in deficit.

Seed 515151, 24 scenarios. Results: `results/hardware/NODE_EXCHANGE_*.json`. XC against S, paired:

Site budget (share of GPU TDP + CPU maximum)	60%	70%	80%
Work served	+5.7%	+3.6%	+1.4%
Backlog minutes	-15.8%	-27.5%	-17.8%
95th-percentile latency factor	-39.6%	-43.6%	-35.2%
Site-budget violation minutes, native / S / XM / XC	195.8 / 0 / 3.9 / 0	140.9 / 0 / 6.2 / 0	61.8 / 0 / 5.3 / 0

Reading.

- **More work from the same building.** The tighter the budget, the more the CPUs' unused watts are worth to the GPUs.
- **Holding the CPUs is what makes the hand-over safe.** Counting the CPUs' measured draw (XM) serves about as much, but goes over the budget for 4-6 minutes in every setting: a CPU can rise between one reading and the next. XC holds each CPU to its allocation, so the watts it gives up are really free, and it never goes over.
- **Most of the gain is the CPUs' reserve, not their clock.** Against XM, XC gains ~1% work per kWh; against S, the watts a fixed plan must keep for CPUs that might peak are what the GPUs receive.
- The same holds for the least and most favourable MLPerf gamma fits, and for CPU shares 0.2 and 0.6 (+2.3% to +4.2% work at the 70% budget), `tests/test_node_exchange.py` checks N1-N4 in `verify.py`.

5. What is not claimed

- This is a simulation on declared device physics.
- The live levers that would carry it are GPU power limits (`nvidia-smi -pl`, DCGM), RAPL package limits, and pod CPU limits under a namespace budget. They exist in `omni_controller/muscles.py`, but the exchange between them has not run on hardware.

CHAPTER 13

Formal Status of the Mathematics

Not a global-stability proof of the forced six-state system. Proofs and their evidence classes: docs/TRACKING_THEOREM.md; every statement's class: docs/EVIDENCE_LEDGER.md.

Held:

- Isolated S-flow is the gradient of Φ (same formula in both cores).
- (T) Unsaturated: $dU/dt = KP(\sigma - U)$; $V = e^2/2$ has $dV/dt = -KP e^2$ on that channel (Theorem 1).
- (T) Saturated, with the U drift bounded by $F_{\text{bar}} < U_{\text{AUTHORITY}}$: V decreases wherever $e \neq 0$; over the declared parameter box $F_{\text{bar}} = 16.86 < 25$, so this holds over the box in continuous time (Theorem 2).
- (T, constant computed) Sampled-data RK4 controller: $e_{(k+1)} = 0.88 e_k + d_k$; with ϵ_h computed over the frozen fixtures (V), the ultimate bound 0.0315 lies inside the 0.10 basin (Theorem 3).
- (T continuous, V discrete) Admissible box forward invariant (Theorem 4).

Open:

- ϵ_h and discrete invariance proved over the whole box (interval arithmetic).
- Inheritance embedding / intertwining residual against a real plant (Closed Structure Def. 11.1): instrumented in the GPU bench (tools/gpu_reps.py, representation fidelity), not yet measured on a card.
- Mapping of monograph Theorem 5.6 onto (E, U, I_U, S, B) .
- A tracking bound for printed_eight_line (logistic U drift).

Default core remains symmetric_verified (mechanism id in results/MECHANISM_IDENTITY.json).

CHAPTER 14

The Mechanism of Action

Every statement here is tied to code in this repository, and the measured numbers are reproduced by python tools/mechanism.py, which writes results/MECHANISM_OF_ACTION.json. The engine files are frozen. Their SHA-256 hashes are checked by verify.py.

1. The state: six numbers that describe the whole system

Symbol	Name	What it stands for in a data centre
E	energy / excitation	how hard the system is being driven: queues, overload, power, heat
U	order parameter	how healthy and settled the system is: +1 healthy basin, -1 failed basin
I_U	integrated need	unmet need accumulated over time: memory of pressure that has not been relieved
S	stress	slow structural stress from heat, power, network, security, staleness
B, B_dot	bath and its rate	a damped oscillator driven by stress: the slow "tide" of physical load

2. The equations (omnicompass/core.py, lines 5-12)

```
(1) dE/dt = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u, |u| <= 25
(3) dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S) = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
(6) dS/dt = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
(7) dB/dt = B_dot ; dB_dot/dt = gamma_c delta S - (omega_B / Q_B) B_dot - omega_B^2 B
(8) R_B[n] = finite-difference audit of (7), never fed back
```

Each term does one job:

- **(1) Energy.** Energy decays at rate $\alpha_E = 3.487$. It is driven by two forcings: internal pressure β_{int} (queue and overload) and external pressure β_{ext} (power, heat, network, staleness, security). The drive term v_{eff} (4) is bounded by c through the hyperbolic tangent, so no input can drive E without limit. This is the saturation (the tanh "speed limit"): however extreme the observation, the engine's response rate is capped at $c = 1$.
- **(2) Order.** $\mu U (1 - U^2)$ is a double-well (bistable) drift with two stable basins, $U = +1$ (healthy) and $U = -1$ (failed), and an unstable ridge at $U = 0$. A rise in energy (dE/dt) pushes U out of the healthy basin. u is the bounded control input.
- **(3) Integrated need.** I_U grows while U is below 1 (the system is not settled). It is reduced by energy and stress already being dealt with, and it leaks at $\lambda_I = 0.719$. It is the engine's memory of unrelieved pressure.

- **(5)-(6) Stress.** A cubic potential (the "well in the basin"). S rolls downhill to its stable root. This is the slowest restoring force.
- **(7) Bath.** A damped harmonic oscillator (natural frequency $\omega_B = 1$, quality $Q_B = 2.5$) driven by stress. It turns stress into a smooth, lagged "tide". It is the only second-order element.
- **(8) Audit.** The bath equation is re-checked numerically every step. The check is never fed back.

Integration: classical 4th-order Runge-Kutta, 10 micro-steps of 0.01 per macro step (rk4_step, macro_step). The C++ twin matches the Python to 3.6e-15 on 500 reference trajectories.

3. From telemetry to state (omnicompass/adapt.py, observe_vector, assimilate)

Each decision the telemetry is normalised to nine observations: q (queue), load, power, thermal, network, drift, stale, security, conflict. An observed state is computed from them:

```
E_obs  = 0.25 q + 0.18 max(0, load - 0.85) + 0.16 power + 0.13 thermal + 0.10 network + 0.08 drift + 0.
↳ 06 conflict + 0.04 stale
U_obs  = 1 - (0.27 q + 0.18 power + 0.16 thermal + 0.12 network + 0.12 drift + 0.10 conflict + 0.05 sta
↳ le)
S_obs  = 0.36 thermal + 0.28 power + 0.18 network + 0.10 security + 0.08 stale - 0.20 q
B_obs  = 0.52 power + 0.26 thermal + 0.22 drift
I_obs  = q + drift + conflict
beta_int = 0.58 q + 0.42 max(0, load - 0.75)      beta_ext = 0.33 power + 0.27 thermal + 0.20 network
↳ + 0.12 stale + 0.08 security
```

The state is blended toward the observed state with weight $a = 0.339$: $x \leftarrow (1 - a) x + a x_{\text{obs}}$. It is then evolved by one macro step (0.1 engine time units) of equations (1)-(7).

4. From state to action (the allocation law, Governor.step)

```
push      = u(x) / 25, u(x) = clip(-f_U(x) + 12 (1 - U), -25, 25)      equation (2)'s controller, eval
↳ uated, not applied
rho*      = clamp(rho0 - kI I_U - kE E, rho_min, rho0)                target utilisation (the HPA t
↳ arget)
n_req     = ceil(n load / rho*) + ceil(kq q n)                        machines wanted
release   only while push <= push_release (the engine reports convergence), after band and dwell
cap       = clamp(cap_now load (1 + margin), cap_min, 1) when calm and U >= U_gate; 1 under backlog;
           always <= 1 - cap_gain max(0, B - B_cap)                  the bath limits the power ca
↳ p
change    permitted only while U >= U_gate and no security block
rollback  authorised if S >= S_rollback, U is falling, or a security block is seen
```

5. Which equation drives which muscle

Muscle	Engine quantity used	Where
HPA target (pods)	ρ^* from I_U and E, equations (1) and (3)	adapter <code>Governor.step</code> ; live <code>omni_controller/controller.py</code>
Node pool (machines)	n_{req} from load, queue and ρ^* ; release gated by push from equation (2)	same
Power cap (pod CPU limits, CPU frequency, GPU limit)	cap from U gate and bath B, equation (7)	same; <code>omni_controller/muscles.py</code>
Rollout guard	<code>change_permitted</code> (U), <code>rollback</code> (S, dU)	<code>muscles.py</code>
Security hold	security observation raises S and blocks expansion	adapter, shield
Speed-first law	ρ^* for pods; push gates machine release	<code>omnicompass/speed.py</code>
Right-sizing	ρ^* sizes CPU; E widens the memory margin; push gates scale-in	<code>omnilab/rightsize.py</code>
Cold start	ρ^* sizes instances; I_U sets how long to keep warm	<code>omnilab/coldstart.py</code>
GPU packing	I_U pre-warms nodes; push gates power-off	<code>omnilab/gpupack.py</code>
Training power	E tightens the applied ramp limit under sustained grid stress	<code>omnilab/powersmooth.py</code>
GPU health	S and E shorten the checkpoint interval	<code>omnilab/health.py</code>
Cooling	E widens the inlet-temperature margin	<code>omnilab/cooling.py</code>
Inference	the full allocation law (<code>node_delta</code>) sizes replicas	<code>omnilab/inference.py</code>
Agent containment	I_U throttles; the security observation contains	<code>omnilab/containment.py</code>
VM energy	I_U and E raise the learning rate when attribution goes stale	<code>omnilab/vmenergy.py</code>

6. Measured: what the equations actually do inside the governor

Measured on a real observation stream (fleet plant, web services, fleet mode, 360 decisions):

Mode (eigenvalue of the Jacobian at the operating point)	Time constant	In decisions
-7.90	0.13	1.3
-3.10	0.32	3.2
-0.72	1.39	13.9
-0.20 +/- 0.98i (the bath)	5.0, period 6.4	50, rings every 64 decisions
-0.17	6.0	60

- **Every mode is stable** (all real parts negative). The operating point is a stable equilibrium inside the healthy basin: $E = 0.095$, $U = 0.976$, $S = 0.315$, $B = 0.10$. This is the basin keeping everything in, stated as mathematics.
- **Per decision, the equations and the observation move the state about equally.** Across all six variables the equations contribute 45-50% of each step's movement and the observation blend the rest.
- **Equation (2)'s control input u is not applied during governance.** The engine evolves open-loop ($u = 0$). The control law is evaluated only to produce push, which decides when capacity may be released.
- **Compute:** about 95 microseconds per decision in Python, and about 2.9 microseconds in the C++ twin.

7. What this explains, and what it points to

- **Why "engine off" scores close to "engine on" in the muscle benchmarks.** With the equations frozen, the observation blend still moves the state. Both versions are then low-pass filters of the same telemetry. The engine's fast modes (1-3 decisions) behave almost like the blend itself. Its distinct contribution is its slow memory: I_U (14 decisions), S (60) and the bath (period 64 decisions). The muscle mappings mostly read E and ρ^* , which live on the fast modes.
- **The engine clock is not matched to the workload's rhythm.** One decision advances engine time by 0.1. At 60-second decisions the bath rings every 64 minutes, but web demand has a 24-hour rhythm and training power a 2-second one. This is a wiring choice in the adapter (engine time per decision), not a property of the equations.
- **The controller in equation (2) is unused as an actuator.** Wiring its output u (or U 's trajectory under u) directly to the actuators is the unexplored way to "let the math do the work".

Three wiring experiments follow directly. None changes a single equation:

1. **Clock matching.** Set engine time per decision so that the bath's natural period equals the workload's rhythm. The oscillator in (7) then carries the cycle and can anticipate it.
2. **Slow-mode actuation.** Drive machines from I_U, S and B (the memory modes) rather than E (a fast mode).
3. **Closed-loop actuation.** Apply equation (2)'s u and use its magnitude, not only its sign, as the actuator command.

Each is testable in the existing harness against every platform in the league (tuning/league.py), and will be tested there.

CHAPTER 15

The Engines and Their Audit

Which engine is which

There is **one** engine. Every other engine-looking file in this repository is either its required C++ twin or a frozen copy of where it came from. Nothing else runs.

File	Role	Runs?
omniconpass/core.py	The engine (<code>symmetric_verified</code>): six states, eight lines, RK4 with microsteps. Everything Omni does goes through it.	yes
cpp/src/core.cpp, cpp/include/omniconpass/core.hpp	The same engine in C++. Required: <code>verify.py</code> checks it against the Python on every fixture.	yes (twin)
docs/handoff/mathematics/omni_compass_engine_source_c527df2d.py	The original source as handed over (sha256 c527df2d...). Kept unchanged as proof of origin and as the parameter authority.	no (frozen)
reference/omni_compass_reference_engine.py	The same original with comments stripped (<code>tools/strip_reference.py</code>); same program fingerprint (<code>reference/PROVENANCE.json</code>).	no (frozen)

Fingerprints of the running engine, its parts and its twin: `results/MECHANISM_IDENTITY.json` and `RELEASE_MANIFEST.json`. `verify.py` fails if any of them changes without a new seal.

Engines from outside packages, not adopted

Copies of this repository passed around as zips (the XPASS packages) carry changes that are **not** in the engine here:

- `omniconpass/batch_claim1.py`: a vectorised variant that applies `u` during the RK4 step (CLAIM1). No C++ twin; not verified against the 500 fixtures.
- an adapter that defaults to CLAIM1, takes the target sign from the starting state and adds a disruption budget. Its C++ governor was only partly ported, which is where the reported Python/C++ mismatches came from.

None of these enter before the GPU confirmation run (the engine is frozen for it, `docs/GPU_PREREGISTRATION.md`). Any of them can be adopted afterwards only as a full change: Python and C++ together, the parity tests and `verify.py` green, and a new mechanism id.

The rule from here

One engine, one twin, two frozen originals. A new version **replaces** the old one in place, with a new seal; it is never added beside it. Older states are in git history and `docs/HISTORY.md`, not in extra files.

My engine, line by line: the source against what runs in your cluster

The source. docs/handoff/mathematics/omni_compass_engine_source_c527df2d.py (SHA-256 c527df2d..., 31,348 lines). Every mechanism below is named by its function there, followed by where it runs live and how it is checked on every python `verify.py`.

Mechanism	In the source	Where it runs	Checked by	Status
Six states E, U, I_U, S, B, Ḃ and equations (1)–(7)	oc_derivatives, rk4_step	omnicompass/core.py	tests/test_core_parity.py: 20,000 random states, max difference $\leq 1e-12$	exact
Spinor closure, 720°: drive $\times \cos(\frac{1}{2} \omega_B t)$	v_eff, spinor_closure_factor	core.v_eff	same parity test	exact
Axle rest S* from equation (6)	stable_S_equilibrium	omnicompass/storage.py, compass.axle	parity; tests/test_compass.py	exact
Double well W(U), basins ± 1	W_potential, W_gradient_flow	core.derivatives	parity	exact
Regulation $u = -f_U + K_P(\sigma - U)$, clipped at the authority	hybrid_microstep_operator	core.control_command; the governor's push	parity; 500 frozen trajectories (fixtures/), 0 mismatches	exact
Basin lock, conveyance, certification	run_monte_carlo, macro_step	core.simulate	500 frozen trajectories, 0 mismatches	exact
Composite storage $V = V_U + V_W + V_E + V_S + V_I + V_B$, the ledger that closes the circle	composite_practical_lyapunov_value, bath_lyapunov_matrix, weights	omnicompass/storage.py, read by the compass every decision	parity on 5,000 states ($\leq 1e-13$); descends 648 $\rightarrow$ 8 over 60 regulated steps with no rise	exact
Human switch: a file or a variable stops every command	_oc_runtime_stop_requested	--kill-file, restore of every lever	tests/test_failsafe.py, tests/test_muscle_S.py, every live run's switch drill	exact
Cap doctrine C1: only a ceiling above the reference is admissible; a lower one buys energy by slowing work	_ci_omni_decision, certificate C1–C4	muscles.convey: CPU limit never below the operator's, idle CPU conveyed to the work	tests/test_convey.py	exact
Sleep only in a certified empty interval	_ci_omni_decision, sleep channel	scripts/kind_nodepool.sh: a machine idles only once its work has left	tests/test_active_nodes.py, tests/test_convey.py	exact
Reference Kubernetes HPA rule	oc_hpa_desired_replicas	cpp/ HPA, pod reflex	tests/test_cpp_hpa_parity.py	exact

Where my live wiring reads the engine differently from the source's own embodiment

Evolution between decisions. The source's compute embodiment advances the assimilated state *with* regulation locked on basin +1. My live governor advances it with $u = 0$ and reads the regulation command separately, as the push that gates every machine release.

- Both use the same equations.
- The difference is whether the regulation is applied to the state or only read from it.
- The pre-registered, frozen results were measured with $u = 0$. I keep that setting, and I report the difference here rather than change a frozen law mid-measurement.

PART THREE

The Physics: the Bowl and the Nervous System

Push and pull, the band and its cushions, the physics of a processor, and the two-way wires that carry the force from the brain to every muscle and back.

CHAPTER 16

The Bowl: Push, Pull and the Two Forces

The bowl (omnicompass/bowl.py) is the law that carries the engine's push and pull to every muscle.

Two bands.

- **The cover**, a knob's hard range (the device's or the operator's lowest and highest setting). Every write is clipped to it. Nothing Omni-Compass computes can set a knob outside it.
- **The bowl**, the service reading as a position from 0 (calm) to 1 (the service line). Omni-Compass pulls that position to the bottom of the bowl, the middle (0.5 by default; an operator may set it lower for extra margin). The walls run from 0.05 to 0.95; the last 5% on each side, 10% in all, is cushion.

The force. $F = A \tanh((K_P (p - \text{center}) + K_D v) / A)$, where p is the position, v its rate of change, A the authority.

- **Pull:** $K_P (p - \text{center})$, gentle near the bottom, stronger up the walls.
- **Push:** $K_D v$, the force that meets whatever is shoving the position, and the friction that stops it sloshing. With K_D at or above critical damping the position glides to the middle and stops, with no overshoot and no ringing. Overshoot is wasted energy: force spent going the wrong way and spent again coming back.
- **Smooth, never a hammer:** $\tanh$ bends the force over into its maximum instead of slamming into a wall.
- **Fail up:** past the 0.95 wall the up side goes to its full force at once and the down side may not act until the position is back inside the bowl.

Two forces: antagonist pairs. Like the muscles of an arm, every plug has an up side (adds capacity, power, cooling, speed) and a down side (takes it back), each with its own gain, because adding and taking back do not cost the same: a new machine takes minutes to boot; giving one back is instant. Where a machine has two wires, each side gets its own lever:

Muscle	Up force	Down force
GPU card	clock ceiling raised	power limit lowered (the lid)
Kubernetes	scale out	scale in
Cooling	chiller on, colder setpoint	warmer setpoint, unit released
Battery	discharge	charge, reserve held
Vehicle, joint	motor effort	braking, regeneration

Why the GPU needs both wires. A card's firmware boosts its clock as high as it may while there is work, and its power limiter knocks the clock back each time the draw crosses the limit; on a busy card this happens many times a second, at the top of the clock range where each extra step of speed costs the most watts. With one wire (the power limit) a governor can only move the wall the boost pushes against. With two wires, the clock ceiling sets how high the boost may climb and the power limit becomes a lid that rarely needs to act: the card runs at the bottom of its bowl instead of fighting itself at the top. On a modelled card, the same engine moved from +0.1% work per energy with one wire to +9.0% with two (Chapter 39).

CHAPTER 17

The Physics of a Processor

A GPU's firmware raises its clock one step at a time whenever there is work and room; each higher clock needs a higher voltage, and dynamic power rises with the clock times the square of the voltage, so the top of the clock range is where each extra step of speed costs the most watts. When the draw crosses the power limit, the firmware knocks the clock down several steps; with room again, it climbs again. On a busy card this repeats many times a second. The clock saws against the limit, the overshoots burn watts before the limiter catches them, and the card spends its time at the steepest part of its power curve. The power is the burner; the cooling removes the heat at its own pace; the limiter turns the burner down so the cooling can catch up, and the boost turns it back up.

The firmware does not aim at a balance point because it cannot see the job: it does not know whether the work is a response due in milliseconds or a batch due overnight, the right point moves with the workload, the room and the chip, and a limiter is simple to prove safe. The vendor default is set for maximum performance, and the electricity bill is the customer's.

Omni-Compass sits on the customer's side, where the job's service line is known. It holds two wires: the clock ceiling sets how high the boost may climb, and the power limit becomes a lid set just above what that ceiling draws. The boost stops at the bottom of the bowl instead of slamming into the wall; the limiter rarely needs to act. Every watt kept off the chip is saved twice in a data center: once at the chip and again at the chillers that would have carried its heat.

CHAPTER 18

The Two-Way Nervous System

Every muscle is wired both ways: a sensory wire in (its meters) and a motor wire out (its knob), with the read-back closing the loop. Between them sits the nervous system (`omnicompass/nervous_system.py`), which decides how much authority each organ has at each moment:

- **Expand is always allowed** (except under a security hold): adding capacity, power, cooling or protection never waits.
- **Contract needs calm and a clean service record**: giving anything back is allowed only while the organ is calm enough and service has been inside its line for the last three decisions; continuous organs give back at most the calm share of their surplus per decision; discrete organs one unit at a time, through a release gate.
- **Fail up**: while service is breached, the knob returns to native at once.
- **Blind means hold**: if a sense goes stale or unreadable, nothing is given back until it returns.

One brain reads every organ at once. Because one law sets every knob, no two muscles fight: when the GPU's watts turn to heat, the cooling knob already knows it is coming; when pods scale up, the power envelope is ready.

CHAPTER 19

The Nervous System in Detail

The two-way nervous system (live controller)

Source. Manuscript Appendix J: "Nervous system. The signal layer: sensing interfaces, unit integrity, timing coherence, delay and dropout handling, and feedback interpretation. ... The brain cannot compensate for corrupted signals." Section 5.3: the engine "projects forward state trajectories ... and detects destabilization pressure before divergence."

The gap. Before this change, the live controller fed the engine's `stale` (signal dropout) and `drift_ratio` (actuation mismatch) channels with a constant 0. So the upward path could not report a blind sense, and the downward path never learned whether its orders landed.

Upward: afferent integrity (senses to engine)

Every declared sense reports whether it is live.

- **Latency.** Blind when its newest sample is older than two windows by the wall clock, or when the window holds no successful request.
 - A hung probe freezes its file; its last clean window must never be read as the present.
 - Failed requests inside a live window become latency pressure (the failed share), so a failure is never read as silence.
- **Power.** Blind when its command returns no number.

How blindness is used.

- The blind share enters the engine as `stale`. It raises E, lowers U and feeds the external bath (equations 1-3 via `assimilate`).
- The nervous system grants no contraction on a blind sense, to any organ, and admits no held batch work.
- Expansion and batch pause (the protective directions) stay allowed.

Downward and back: efferent feedback (proprioception)

Every command is recorded.

- The node count commanded.
- The HPA targets written.

At the next decision the observed state is read back. For each organ, $\text{drift} = |\text{observed} - \text{commanded}| / \text{commanded}$.

- The largest drift enters the engine as `drift_ratio`. It raises E and I_U, lowers U and feeds B (equations 1-3).

- The node organ may not be given a new release while its last command did not land. For example, a drain refused by the PodDisruptionBudget leaves the node in service, and the next decision sees it.

Sideways: attribution (organ to organ)

The machine organ has its own engine view, fed with machine-attributable pressure only (pods waiting for a place).

It releases a machine only when all of these hold:

- pods are not scaling up and latency is not breached;
- nothing is pending;
- after the release the remaining machines stay at or below the engine's rho;
- its own calm, stress and security gates grant contraction;
- every sense is live;
- its last command landed.

Tests (all in verify.py)

Test	What it checks
tests/test_two_way.py	blind senses (about 75,000 random states), frozen-probe detection, failures read as pressure, release refused while blind or after an order that did not land
tests/test_node_release_gate.py	the release gate, over 200,000 states
tests/test_nervous_system.py	the original invariants N1-N6, over 300,000 states

Every live decision records the senses, their ages, the proprioceptive drift and the gate's reason in the audit, and the benchmark prints them in the job log.

Nervous system

Omni-Compass is one field. Muscles are nerves.

```
telemetry -&gt; afferent (per muscle)
          -&gt; one six-state engine
          -&gt; reflex (shield)
          -&gt; efferent (only WIRED + authority + not killed)
          -&gt; native controller on kill
```

omncompass/nervous.py is the register. It does not invent GPU or chiller physics.

Status	Meaning
wired	this tree can sense and push
sensed	this tree can sense; it does not push
open	named, no plant, no push

Wired today: nodes, hpa, power_cap. Sensed: heat, network, security. Open: GPU, cooling, grid, queues, agents, ... Never a muscle: value alignment.

```
python k8s_controlplane/test_nervous.py
```

PART FOUR

The Body: Muscles, Realms and Organisms

Six hundred and fifty-six muscles in four realms, stacked into six organisms, and every gauge used to judge them.

CHAPTER 20

The Muscles, the Realms and the Six Organisms

The catalog (realms/catalog.csv) lists 656 distinct muscles, each with its family, its plant model and its knob. They fall into four realms. Every realm stands on the same spine (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling and electrical distribution, 190 muscles), plus its own domain muscles.

Organism	Muscles	What it is
1. Compute / AI / Cloud	345	clusters, GPUs, AI training and inference, cloud capacity
2. Physics / Robotics / Autonomous	262	joints, fleets, vehicles, flight and spacecraft axes
3. Energy / Facility / Industrial	282	data halls, buildings, batteries, UPS, process loops, feeders
4. Distribution / Specialized	337	networks, storage, databases, commerce, workflows, radio networks
5. The four stacked, every duplicate kept	1,226	all four realms on one clock, the shared spine counted in each
6. The whole tower, every muscle once	656	every distinct muscle on one clock

These six organisms are the benchmark set. Each is run native and with Omni-Compass on top, on the same seed, the same load and the same clock, and each produces its own receipt.

CHAPTER 21

The Four Realms

The 656-muscle tower on modelled plants: four realms and the whole tower as a fifth organism, each run natively and with Omni-Compass on top, with meters and receipts. Evidence class **S** (simulation).

Run it

```
pip install -r requirements.txt
python3 tools/run_realms.py          # the preregistered run: 656 muscles and 5 organisms, s
    ↪ eeds 1000-1009
python3 tests/test_realms.py        # the harness's own checks (also inside verify.py)
```

Results: results/realms/REALMS.md (the tables), MUSCLES.csv (one row per muscle), REALMS.json (every per-seed contrast), RUN.json and SHA256SUMS.txt (commit and fingerprints).

What is where

File	What it is
realms/catalog.csv	the 656 muscles: family, name, realm, plant, parameter set, knob
tools/realms_catalog.py	the rules that gave each muscle its realm, plant and knob
realms/plants.py	the five plants and their native controllers; the four knobs; the capacity law for one plant
realms/presets.py	every parameter, one set per family class
realms/harness.py	the arms (native, watch, omni, fixed setpoint), the organisms, the outcome and the label rule
docs/REALMS_PREREGISTRATION.md	the question, the rules, the seeds and the development history, frozen before the run

How it relates to the rest

Layer	Realm harness	Elsewhere in this repository
Mathematics (T, V)	uses the frozen engine and governor unchanged	docs/TRACKING_THEOREM.md, verify.py
Simulation (S)	this	fleet and cluster simulators, GPU model
Real software (L)	not here	set 22 on real Kubernetes (results/live/LIVE_REPS_22.md)
Physical (P)	not here	the GPU bench, not yet run on a card (docs/GPU_PREREGISTRATION.md)

A realm result that looks good is a reason to test that knob on a real machine, not a substitute for it. The realms whose knobs can be tested for real first are the compute realm's (the GPU bench, kind), because the tools already exist.

CHAPTER 22

The Realm Muscles

Generated from `realms/catalog.csv` (rules: `tools/realms_catalog.py`). Every realm's organism is its own families plus the **shared spine** (the infrastructure every real stack runs on), so the realms overlap on purpose. The fifth organism holds each of the 656 once. Each muscle's plant model and the one knob Omni may hold are in brackets.

The shared spine: 190 muscles, in all four realms

Cloud VM & Capacity (16; plant: compute_pool, node)

vm_count [capacity], vm_size [capacity], vm_start_stop [capacity], vm_migrate [capacity], instance_family [capacity], asg_min [capacity], asg_max [capacity], spot_mix [capacity], reservation_mix [capacity], zone_selection [capacity], region_selection [capacity], architecture_selection [capacity], accelerator_selection [capacity], boot_disk_class [capacity], placement_group [capacity], interruption_response [capacity]

Container Resources (16; plant: compute_pool, server)

cpu_request [capacity], cpu_limit [capacity], memory_request [capacity], memory_limit [capacity], ephemeral_storage [capacity], hugepages [capacity], io_weight [setpoint], pids_limit [capacity], memory_high [capacity], memory_reclaim [capacity], swap_limit [capacity], cgroup_io_max [capacity], cpu_weight [setpoint], cpu_quota [admission], cpuset [capacity], runtime_class [capacity]

Host CPU & Memory (16; plant: compute_pool, cpu_host)

cpufreq_min [power], cpufreq_max [power], rapl_package_power [power], uncore_frequency [power], energy_perf_preference [power], cpu_idle_policy [power], memory_bandwidth [capacity], numa_balance [capacity], irq_affinity [capacity], llc_allocation [capacity], memory_pressure_gate [admission], page_reclaim_rate [capacity], transparent_hugepages [capacity], core_online_offline [capacity], thermal_throttle_policy [power], host_power_profile [power]

Kubernetes Placement & Scheduling (16; plant: compute_pool, server)

node_selector [capacity], node_affinity [capacity], pod_anti_affinity [capacity], topology_spread [capacity], numa_placement [capacity], gpu_topology [capacity], storage_locality [capacity], network_locality [capacity], taint_toleration [setpoint], priority_class [admission], preemption_policy [admission], device_claim [capacity], failure_domain_spread [capacity], scheduler_backoff [admission], gang_admission [admission], deschedule [capacity]

Kubernetes Workload Scaling (16; plant: compute_pool, server)

replicas [capacity], hpa_cpu_target [setpoint], hpa_memory_target [setpoint], hpa_custom_target [setpoint], vpa_apply [capacity], scale_to_zero [capacity], keda_threshold [admission], rollout_rate [capacity], max_surge [capacity], max_unavailable [capacity], deployment_pause_resume [admission], rollout_abort [admission], pod_eviction [admission], pdb_policy [capacity], scheduler_queue_priority [admission], api_priority_fairness [admission]

NVIDIA GPU Hardware (15; plant: compute_pool, gpu)

gpu_allocate [capacity], gpu_power_limit [power], gpu_sm_clock [power], gpu_memory_clock [power], gpu_persistence_mode [capacity], gpu_compute_mode [capacity], mig_mode [capacity], mig_geometry [capacity], gpu_timeslice [capacity], gpu_mps [capacity], gpu_quarantine [admission], gpu_reset [capacity], gpu_thermal_limit [power], gpu_ecc_response [capacity], gpu_job_power_budget [admission]

Node Fleet & Karpenter-Class Control (16; plant: compute_pool, node)

node_desired [capacity], node_pool_min [capacity], node_pool_max [capacity], node_provision [capacity], node_cordon [admission], node_drain [admission], node Consolidate [capacity], node_replace [capacity], node_shutdown [capacity], node_power_on [capacity], nodepool_weight [setpoint], disruption_budget [admission], consolidation_policy [capacity], consolidate_after [capacity], expire_after [capacity], capacity_class [capacity]

Network Routing & Switching (16; plant: compute_pool, network)

lb_weight [setpoint], route_weight [setpoint], rate_limit [admission], bandwidth_limit [capacity], qos_class [capacity], connection_limit [admission], failover_route [capacity], nic_rate_limit [admission], nic_queue_count [capacity], queue_discipline [capacity], congestion_control [capacity], egress_budget [admission], ingress_budget [admission], ecmp_weight [setpoint], path_selection [capacity], network_isolation [admission]

Observability & Telemetry (6; plant: compute_pool, server)

collector_memory_limit [capacity], export_concurrency [admission], cardinality_budget [admission], retention_window [setpoint], remote_write_queue [capacity], telemetry_shed [admission]

Reliability, Security & Recovery (12; plant: compute_pool, server)

restart [capacity], rollback [capacity], traffic_divert_recovery [capacity], degraded_mode [capacity], actuator_freeze [admission], workload_isolate [admission], credential_rotation_gate [admission], policy_enforcement [capacity], rate_abuse_gate [admission], fault_domain_isolate [admission], backup_trigger [capacity], kill_switch [admission]

Storage Block/File/Object (16; plant: compute_pool, storage)

volume_size [capacity], iops_limit [capacity], throughput_limit [capacity], replica_count [capacity], storage_tier [capacity], volume_placement [capacity], snapshot_trigger [capacity], rebalance [capacity], recovery_rate [capacity], backfill_rate [capacity], compaction_pressure [capacity], cache_allocation [capacity], object_replication [capacity], erasure_code_profile [capacity], storage_admission [admission], degraded_storage_gate [admission]

Cooling, Chillers & Thermodynamics (15; plant: thermal_zone, data_hall)

supply_air_temperature [setpoint], return_air_target [setpoint], coolant_supply_temperature [setpoint], coolant_flow [capacity], pump_speed [capacity], fan_speed [capacity], chiller_setpoint [setpoint], compressor_authority [power], cooling_tower_fan [capacity], cooling_capacity [capacity], rack_thermal_budget [power], gpu_thermal_envelope [setpoint], cpu_thermal_envelope [setpoint], thermal_workload_migrate [admission], thermal_load_shed [admission]

PDU, UPS & Electrical Distribution (14; plant: energy_storage, ups)

server_power_cap [power], rack_power_cap [power], pdu_branch_power_limit [power], pdu_outlet_control [capacity], ups_operating_mode [capacity], ups_charge_rate [power], ups_discharge_rate [power], phase_balance [capacity], load_transfer [admission], reactive_power_target [setpoint], voltage_target [setpoint], generator_dispatch [capacity], electrical_isolation [admission], breaker_trip_gate [admission]

Realm 1: Compute / AI / Cloud: 155 own muscles + the spine = 345 in its organism***AI Inference Serving (16; plant: compute_pool, gpu)***

model_replicas [capacity], model_route_weight [setpoint], model_load [capacity], model_unload [capacity], model_instance_count [capacity], continuous_batching [capacity], max_batch_size [capacity], batch_queue_delay [capacity], inference_concurrency [admission], inference_max_tokens [capacity], kv_cache_budget [admission], prefix_cache_budget [admission], speculative_decode_budget [admission], model_precision [power], inference_priority [admission], inference_slo_gate [admission]

AI Training (16; plant: compute_pool, gpu_batch)

training_workers [capacity], global_batch_size [capacity], microbatch_size [capacity], gradient_accumulation [capacity], data_parallelism [capacity], tensor_parallelism [capacity], pipeline_parallelism [capacity], expert_parallelism [capacity], checkpoint_interval [setpoint], checkpoint_trigger [capacity], training_preempt [admission], training_gang_size [capacity], elastic_worker_count [capacity], straggler_mitigation [capacity], training_precision [power], compute_comm_overlap [capacity]

Cross-Cluster, Multi-Region & Edge (15; plant: compute_pool, node)

multi_cluster_dispatch [capacity], region_dispatch [capacity], zone_dispatch [capacity], edge_dispatch [capacity], cloud_capacity_class [capacity], workload_migrate_region [capacity], data_residency_gate [admission], latency_region_gate [admission], cost_region_gate [admission], carbon_region_gate [admission], global_failover [capacity], federation_quota [admission], cross_cluster_replication [capacity], edge_offload [capacity], global_admission [admission]

DPU SmartNIC & Programmable IO (12; plant: compute_pool, fabric)

dpu_pf_tx_rate [capacity], dpu_vf_tx_rate [capacity], dpu_sf_tx_rate [capacity], dpu_bandwidth_share [capacity], dpu_qos_group [capacity], sriov_vf_count [capacity], smartnic_flow_steering [capacity], smartnic_offload_enable [capacity], dpu_cpu_budget [admission], dpu_memory_budget [admission], dpu_service_placement [capacity], dpu_failover [capacity]

Distributed Cluster Managers (15; plant: compute_pool, batch)

task_admission [admission], task_priority [admission], resource_reservation [capacity], cluster_quota [admission], task_preemption [admission], binpack_pressure [capacity], spread_pressure [capacity], gang_schedule [capacity], worker_allocation [capacity], maintenance_evacuation [capacity], oversubscription [capacity], resource_reclaim [capacity], queue_fairness [admission], deadline_pressure [admission], scheduler_retry [admission]

GPU Fabric & RDMA (16; plant: compute_pool, fabric)

nvlink_placement [capacity], nvswitch_route [capacity], gpu_fabric_quarantine [admission], rdma_bandwidth [capacity], rdma_route [capacity], nic_affinity [capacity], collective_concurrency [admission], collective_algorithm [capacity], collective_chunk_size [capacity], rank_placement [capacity], gpudirect_policy [capacity], congestion_response [capacity], rail_selection [capacity], fabric_failover [capacity], communication_priority [admission], fabric_isolation [admission]

HPC & Distributed Compute (16; plant: compute_pool, batch)

job_slots [capacity], mpi_ranks [capacity], rank_mapping [capacity], node_allocation [capacity], job_walltime [capacity], job_priority [admission], job_preemption [admission], checkpoint_restart [capacity], parallel_io_budget [admission], collective_budget [admission], accelerator_share [capacity], cpu_gpu_ratio [setpoint], memory_per_rank [capacity], scratch_budget [admission], scheduler_fair_share [admission], backfill_policy [capacity]

Kubernetes Dynamic Device Allocation (8; plant: compute_pool, gpu)

dra_device_class_selection [capacity], dra_claim_capacity [capacity], dra_claim_sharing [capacity], dra_device_taint [capacity], dra_device_eviction [admission], dra_binding_readiness [capacity], dra_binding_failure_response [capacity], dra_device_configuration [setpoint]

OpenShift & Machine API (9; plant: compute_pool, node)

machine_remediation [capacity], machine_health_gate [admission], mcp_pause [admission], mcp_max_unavailable [capacity], node_config_rollout [capacity], operator_reconcile_budget [admission], cluster_version_pacing [capacity], infra_machine_admission [admission], machine_failure_domain [capacity]

Quantum Computing Control Simulation (16; plant: compute_pool, qpu)

qubit_mapping [capacity], circuit_admission [admission], shot_allocation [capacity], circuit_scheduling [admission], gate_scheduling [capacity], pulse_amplitude [capacity], pulse_duration [setpoint], pulse_phase [capacity], pulse_frequency [power], coupling_control [capacity], reset_scheduling [capacity], measurement_scheduling [capacity], dynamical_decoupling [capacity], noise_aware_routing [capacity], error_mitigation_budget [admission], quantum_queue_priority [admission]

Work Admission & Demand Shaping (16; plant: compute_pool, server)

api_concurrency [admission], queue_concurrency [admission], queue_backpressure [admission], job_admission [admission], batch_admission [admission], inference_admission [admission], load_shed [admission], priority_gate [admission], tenant_admission [admission], burst_limit [admission], deadline_admission [admission], work_budget [admission], request_queue_limit [admission], retry_admission [admission], background_work_gate [admission], maintenance_work_gate [admission]

Realm 2: Physics / Robotics / Autonomous: 72 own muscles + the spine = 262 in its organism

Automotive EV & Mobile Powertrain (12; plant: motion_axis, ev_traction)

traction_torque_limit [power], regen_braking_level [power], battery_charge_limit [power], battery_discharge_limit [power], battery_thermal_target [power], motor_thermal_limit [power], vehicle_speed_envelope [capacity], energy_recovery_target [power], auxiliary_power_budget [power], fast_charge_current [power], fast_charge_voltage [power], vehicle_safe_state [admission]

Aviation & Autonomous Flight (16; plant: motion_axis, flight_axis)

throttle_envelope [power], attitude_target [capacity], attitude_rate_target [capacity], velocity_target [capacity], altitude_target [capacity], waypoint_authority [capacity], flight_hold [admission], return_to_home [admission], land_action [admission], mission_admission [admission], geofence_response [admission], failsafe_selection [admission], battery_reserve_threshold [admission], actuator_saturation_envelope [capacity], flight_mode_transition [admission], flight_termination_safe_state [admission]

Robotics Fleet & Warehouse Automation (15; plant: compute_pool, robot_fleet)

robot_dispatch [capacity], task_assignment [capacity], traffic_reservation [capacity], robot_route [capacity], charging_dispatch [capacity], battery_reserve [capacity], elevator_request [capacity], door_request [capacity], conveyor_speed [capacity], agv_speed [capacity], warehouse_zone_admission [admission], robot_quarantine [admission], fleet_failover [capacity], human_safe_stop [capacity], fleet_concurrency [admission]

Robotics Motion Control (15; plant: motion_axis, robot_joint)

joint_position [capacity], joint_velocity [capacity], joint_acceleration [capacity], joint_effort [power], cartesian_velocity [capacity], trajectory_speed [capacity], trajectory_acceleration [capacity], jerk_limit [capacity], collision_margin [capacity], force_limit [power], gripper_force [power], locomotion_speed [capacity], steering_angle [capacity], braking_force [power], balance_correction [capacity]

Spacecraft & Flight Software (14; plant: motion_axis, reaction_wheel)

space_command_admission [admission], flight_task_schedule [admission], space_mode_transition [admission], payload_duty_cycle [power], communication_allocation [capacity], space_power_budget [power], space_thermal_command [power], attitude_command_envelope [capacity], reaction_wheel_allocation [capacity], rcs_authority [power], safe_mode_transition [admission], watchdog_recovery [admission], instrument_activation [capacity], fault_isolation [admission]

Realm 3: Energy / Facility / Industrial: 92 own muscles + the spine = 282 in its organism

Building & Critical Environment HVAC (12; plant: thermal_zone, building)

zone_temperature_target [setpoint], zone_airflow [capacity], ahu_fan_speed [capacity], damper_position [capacity], economizer_position [capacity], boiler_setpoint [setpoint], heat_pump_mode [admission], humidity_target [setpoint], occupancy_ventilation [admission], building_demand_limit [power], thermal_storage_dispatch [capacity], hvac_emergency_mode [admission]

Energy Storage & Microgrid (16; plant: energy_storage, microgrid)

battery_charge_power [power], battery_discharge_power [power], battery_soc_reserve [setpoint], grid_import_limit [power], grid_export_limit [power], pv_curtailment [power], ev_charge_power [power], heat_pump_power [power], electrolyzer_power [power], microgrid_demand_limit [power], peak_shaving [capacity], time_of_use_schedule [setpoint], energy_load_shed [admission], flex_load_admission [admission], storage_dispatch [capacity], microgrid_emergency_reserve [setpoint]

Facility & Grid Optimization (15; plant: energy_storage, facility)

facility_power_budget [power], utility_demand_limit [power], demand_response [admission], electricity_price_gate [admission], carbon_intensity_gate [admission], renewable_dispatch [capacity], generator_start_stop [capacity], site_battery_dispatch [capacity], pue_target [setpoint], cooling_power_budget [power], it_power_budget [power], rack_power_allocation [power], facility_peak_guard [capacity], grid_frequency_response [admission], facility_islanding [admission]

Grid Transmission & Distribution (12; plant: process_loop, feeder_voltage)

capacitor_bank_switch [capacity], voltage_regulator_tap [setpoint], transformer_tap [setpoint], inverter_real_power [power], inverter_reactive_power [power], feeder_voltage_target [setpoint], feeder_load_transfer [capacity], distribution_storage_dispatch [admission], demand_response_dispatch [admission], frequency_droop_setpoint [setpoint], grid_protection_mode [admission], grid_restoration_sequence [admission]

Industrial PLC & Process Automation (14; plant: process_loop, process)

plc_cycle_authority [admission], machine_cell_admission [admission], valve_position [setpoint], pump_flow [capacity], compressor_speed [capacity], heater_power [power], furnace_setpoint [setpoint], pressure_setpoint [setpoint], temperature_setpoint [setpoint], mass_flow_setpoint [setpoint], tank_level_target [setpoint], conveyor_rate [capacity], feed_rate [capacity], purge_vent_action [admission]

Semiconductor Fab & Precision Manufacturing (11; plant: process_loop, chamber)

tool_job_dispatch [admission], wafer_route [capacity], chamber_recipe_selection [admission], chamber_temperature [setpoint], chamber_pressure [setpoint], gas_flow [capacity], rf_power [power], vacuum_pump_speed [capacity], robot_transfer_rate [capacity], lot_priority [admission], tool_quarantine [admission]

Water Wastewater & Pumping (12; plant: process_loop, water)

pump_speed_water [capacity], valve_position_water [setpoint], reservoir_level_target [setpoint], line_pressure_target [setpoint], flow_target_water [setpoint], aeration_rate [power], chemical_dose_rate [power], filtration_backwash [admission], lift_station_dispatch [admission], leak_isolation [admission], water_demand_shed [admission], water_emergency_shutdown [admission]

Realm 4: Distribution / Specialized: 147 own muscles + the spine = 337 in its organism

Cache & Memory Services (15; plant: compute_pool, server)

cache_size [capacity], cache_ttl [setpoint], cache_eviction_policy [admission], cache_replicas [capacity], cache_sharding [capacity], cache_prefetch [capacity], cache_writeback_rate [capacity], cache_admission [admission], hot_key_isolation [admission], cache_connection_limit [admission], cache_memory_limit [capacity], cache_compression [capacity], cache_warmup [capacity], cache_failover [capacity], cache_flush_rate [capacity]

Commerce & Payment Systems (15; plant: compute_pool, commerce)

payment_admission [admission], payment_concurrency [admission], payment_retry [admission], payment_timeout [admission], fraud_review_gate [admission], authorization_route [capacity], processor_route_weight [setpoint], transaction_queue_limit [admission], idempotency_window [setpoint], order_reservation [capacity], inventory_hold [admission], checkout_load_shed [admission], refund_queue_rate [capacity], settlement_batch [capacity], payment_failover [capacity]

Data Analytics & ETL (15; plant: compute_pool, batch)

executor_size [capacity], dynamic_allocation_min [capacity], dynamic_allocation_max [capacity], shuffle_partitions [capacity], shuffle_bandwidth [capacity], etl_concurrency [admission], stage_parallelism [capacity], query_slots [capacity], spill_threshold [admission], cache_fraction [setpoint], batch_interval [setpoint], stream_backpressure [admission], data_locality_wait [capacity], speculation_policy [capacity], analytics_admission [admission]

Database & Transactions (14; plant: compute_pool, database)

db_replicas [capacity], db_memory [capacity], db_cache [capacity], query_concurrency [admission], read_route [capacity], shard_placement [capacity], db_failover [capacity], replication_lag_gate [admission], db_write_throttle [admission], db_pool_resize [capacity], transaction_concurrency [admission], lock_timeout [admission], checkpoint_rate [capacity], vacuum_compaction_rate [capacity]

Messaging & Streaming (16; plant: compute_pool, server)

partition_count [capacity], partition_placement [capacity], producer_quota [admission], consumer_quota [admission], broker_io_quota [admission], message_retention [capacity], queue_depth_limit [admission], consumer_concurrency [admission], producer_batch_size [capacity], fetch_batch_size [capacity], rebalance_rate [capacity], replication_factor [capacity], retry_backoff [admission], dead_letter_divert [capacity], stream_priority [admission], broker_failover [capacity]

Runtime & Application (15; plant: compute_pool, server)

worker_count [capacity], thread_pool [capacity], jvm_heap [capacity], gc_budget [admission], connection_pool_runtime [capacity], application_cache_size [capacity], runtime_memory [capacity], async_concurrency [admission], event_loop_workers [capacity], process_count [capacity], request_timeout [admission], background_workers [capacity], runtime_cpu_budget [admission], runtime_io_budget [admission], runtime_restart [capacity]

Search, Indexing & Vector DB (15; plant: compute_pool, server)

index_workers [capacity], index_refresh_rate [capacity], segment_merge_rate [capacity], search_concurrency [admission], search_timeout [admission], shard_count [capacity], shard_replication [capacity], shard_rebalance [capacity], vector_search_k [capacity], vector_batch_size [capacity], embedding_workers [capacity], index_memory_budget [admission], query_route [capacity], hot_shard_isolation [admission], search_admission [admission]

Service Mesh & API Reliability (15; plant: compute_pool, server)

circuit_breaker [admission], retry_budget [admission], service_timeout [admission], service_concurrency [admission], connection_pool [capacity], traffic_divert [capacity], traffic_mirror [capacity], canary_weight [setpoint], outlier_ejection [admission], health_threshold [admission], dns_traffic_weight [setpoint], session_affinity [capacity], request_hedging [capacity], fault_injection_gate [admission], service_failover [capacity]

Telecom RAN & Edge Radio (12; plant: compute_pool, ran)

ran_connection_admission [admission], ran_ue_handover [capacity], ran_cell_traffic_steering [capacity], ran_slice_resource_budget [admission], ran_prb_allocation [capacity], ran_scheduler_weight [setpoint], ran_tx_power [power], ran_antenna_tilt [capacity], ran_carrier_enable [capacity], ran_cell_sleep [capacity], ran_du_cu_placement [capacity], ran_fronthaul_budget [admission]

Workflow, Logistics & Fulfillment (15; plant: compute_pool, workflow)

workflow_admission [admission], workflow_worker_rate [capacity], task_queue_rate [capacity], workflow_retry [admission], workflow_backoff [admission], workflow_timeout [admission], inventory_allocation [capacity], fulfillment_route [capacity], warehouse_queue [capacity], carrier_selection [capacity], dispatch_priority [admission], shipment_batch [capacity], route_replan [capacity], sla_escalation [capacity], compensation_action [capacity]

Organism 5: the whole tower, all 656 muscles once

The Domain Map

Every system Omni-Compass can sit on, what it reads, what it pushes, the reflexes that bound it, and where it stands.

1. The connector pattern

Omni-Compass is the same engine on every muscle. Each muscle gets one connector with five parts:

Part	Role	Example (compute)
Afferent (sense)	What the muscle reports	load, pending work, power, heat
Engine	Equations (1)-(7) turn readings into state: error E, coherence U, pressure I_U, stress S, bath B	same six-state engine everywhere
Efferent (push)	The equation (2) law and allocation law drive the muscle toward convergence	node count, HPA target, power cap
Reflex (shield)	Hard limits checked before any push	never below pod requests, step limits, power limit
Kill	Control returns to the muscle's own controllers	restore HPA targets, observe only

Fit tiers used below:

- **Direct:** Omni can govern the muscle as a supervisory controller.
- **Supervisory:** Omni governs budgets, schedules and envelopes; the muscle's own real-time controller stays in charge of fast inner loops.
- **Advisory:** regulated or safety-critical; Omni recommends until certified (IEC 61508, DO-178C, ISO 26262, medical and financial regulation).
- **Not Omni's problem:** a different kind of problem; stated so no one claims otherwise.

Status: **Wired** (in code, verified), **Partial**, **Open** (connector not yet built).

2. Digital infrastructure

Muscle	Reads	Pushes	Reflexes	Fit	Status
Compute nodes and node pools	load, pending pods, requests	node count, park or power off	request floor, step limit	Direct	Wired
Pods via HPA	utilisation, replicas	HPA target	bounds 50-95%	Direct	Wired
Power caps	watts, site limit	per-node caps	site power limit (I4)	Direct	Wired (harness)
Heat	temperatures	feeds decisions	thermal limit	Direct	Sensed
GPUs / accelerators	utilisation, power, temperature, memory	power caps, clocks, partitioning (MIG), packing	temperature, job safety	Direct	Open
CPU power states	per-socket power (RAPL)	frequency scaling, sleep states	latency floor	Direct	Open
Memory	used vs requested	request right-sizing	OOM margin	Direct	Open
Storage	I/O load, latency, capacity	tiering, placement, volume scaling	durability, replica counts	Supervisory	Open
Network, load balancers	traffic, congestion, latency	routing weights, traffic shift	capacity per link	Direct	Partial
CDN / edge	cache hit, origin load	cache and origin routing	freshness	Direct	Open
Batch and HPC job queues	queue depth, deadlines	admission, priority, time shifting	deadline guarantees	Direct	Open
Multi-cluster / multi-region	load, cost, carbon per region	placement	data residency	Direct	Open
Deployments	release health	pause, roll back	never skip required checks	Direct	Partial (stack harness)
Databases, stateful services	load, replication lag	read replicas, pools	consistency limits	Supervisory	Open
Control plane health	API latency, etcd load	rate of Omni's own actions	never overload the API	Direct	Open
Tenant quotas	usage per tenant	who may grow	fairness bounds	Direct	Open
Hardware health	disk and memory errors	early drain of failing machines	availability floor	Direct	Open
Observability pipelines	data volume, cost	sampling, retention	required audit data	Direct	Open
Security posture	policy blocks	block expansion during holds	I1	Direct	Partial (sensed)

3. AI

Muscle	Reads	Pushes	Reflexes	Fit	Status
Training clusters	GPU power, progress, network	power caps, packing, checkpoint timing	never kill a run mid-step	Direct	Open (gpu vessels in harness)
Training power smoothing	synchronised power swings	ramp limits, staggering	grid ramp limits	Direct	Open
Inference serving	request rate, latency, queue, GPU memory	replicas, batch size, GPU sharing	latency SLO	Direct	Open
Model routing and cost	cost per request, quality signals	route to smaller or larger models, token budgets	quality floor	Direct	Open
Model releases	evaluation results, drift, errors	gate, pause, roll back a model	release gates	Direct	Open
Data pipelines, vector databases	backlog, freshness	scaling, scheduling	freshness bounds	Direct	Open
AI agents: containment	actions, spend, resources, network use	CPU, memory, network and spend caps, tool permissions	hard caps, instant kill	Direct	Open
Carbon-aware AI	grid price and carbon intensity	when and where training runs	deadlines	Direct	Open

AI alignment: what Omni can and cannot do

Alignment has two parts, and only one is Omni's.

Part	What it means	Omni's role
Value alignment	The model itself wants and does what people intend: honest, safe, helpful behaviour learned in training	Not Omni's problem. A governor outside the model cannot make the model's goals or judgement correct.
Control and containment	Whatever the model wants, it can only act within bounds: resources, permissions, spend, network reach, a kill that works	Direct. This is a governance problem: sense what the agent is doing, bound what it may spend and touch, revoke instantly, keep an audit trail.

Omni-Compass can be the boundary layer AI runs inside. It cannot be the thing that makes AI trustworthy. Claims must keep that line.

4. Facilities and energy

Muscle	Reads	Pushes	Reflexes	Fit	Status
Cooling plant	inlet temperatures, chiller load	setpoints, fan curves, chiller staging	inlet temperature limits	Direct	Open
Building HVAC	zone temperatures, occupancy	setpoints, schedules	comfort bounds	Direct	Open
Facility power distribution	PDU and UPS load	limits for the shield	breaker limits	Supervisory	Open
On-site batteries and generators	state of charge, grid state	charge and discharge	reserve floor	Supervisory	Open
Power grids: demand response	grid signals, prices	shift or shed flexible load	contractual limits	Direct	Open
Microgrids, renewable smoothing	generation, storage, load	dispatch	frequency and voltage limits	Advisory to supervisory	Open
EV charging fleets	vehicle needs, grid capacity	charge schedules	departure deadlines	Direct	Open
Water and pumping	demand, pressure, energy price	pump schedules	pressure limits	Supervisory	Open
Grid frequency and protection	frequency, faults	none	certified protection systems	Advisory	Out of scope for control

5. Physical operations

Muscle	Pushes	Fit
Manufacturing lines	scheduling, energy, throughput balance	Supervisory
Warehouses and logistics	task allocation, fleet charging	Direct
Robot fleets	task assignment, charging, traffic, safety envelopes	Supervisory (never joint-level control)
Traffic signals, transit, rail scheduling	timing and schedules	Advisory (safety-certified)
Ports, shipping	berth and crane scheduling, energy	Supervisory
Agriculture, greenhouses, irrigation	water and climate setpoints	Direct
Mining, oil and gas	energy and scheduling only	Advisory (process safety)
Semiconductor fabs	tool scheduling, energy	Supervisory

6. Science, space, biology, finance, health

Domain	What Omni could govern	What it must not touch	Fit
HPC / supercomputers	job queues, power, cooling	none beyond the facility	Direct
Quantum-computer facilities	cryogenics energy, calibration scheduling	qubit control	Supervisory
Satellites and constellations	power, thermal, data budgets, scheduling	attitude and orbit control (flight software)	Supervisory
Spacecraft / astrodynamics	mission resource budgets	guidance and navigation	Advisory
Biology: lab automation, bioreactors	setpoints, schedules, cold chain	living-system interventions without validated models	Supervisory
Financial markets	risk limits, exposure caps, trading infrastructure	making trades, market control	Advisory (regulated)
Hospitals	bed, staffing and energy operations	medical devices, treatment	Advisory
Aviation, automotive, nuclear	monitoring	any direct control	Advisory (certification required)

7. The problems everyone in computing has, and Omni's fit

Problem	What Omni can do	Fit
Data-centre energy growth	right-size, park, cap, shift work in time and place	Direct
Cloud cost overruns and idle capacity	consolidate, right-size, remove padding	Direct
GPU scarcity and low GPU utilisation	pack jobs, share GPUs, schedule by priority	Direct
Heat and cooling limits	govern IT load and cooling plant together	Direct
Grid connection limits for new data centres	power smoothing, demand response, stay under contracted power	Direct
Carbon reporting and reduction	carbon-aware placement and timing, measured energy	Direct
On-call burnout and alert fatigue	hold the levers people turn at night; fewer, better alerts	Direct
Autoscalers fighting each other	single authority	Direct (proven in simulation)
Latency and SLO breaches under load	headroom governance, fast scheduling floor	Direct
Noisy neighbours, unfair tenants	quotas and growth permissions	Direct
Hardware failures	early drain on failure signs	Direct
Observability cost and data explosion	sampling and retention governance	Direct
Configuration drift	detect and gate, not rewrite configurations	Supervisory
Cold starts	pre-warming policy	Direct
Data egress costs	placement near data	Supervisory
Runaway AI agents and spend	caps, permissions, kill, audit	Direct
Security misconfiguration, supply-chain attacks	block risky expansion during holds	Partial (not a security scanner)
Model hallucination and bias	none	Not Omni's problem
AI value alignment	none	Not Omni's problem
Software bugs, technical debt, legacy migration	none	Not Omni's problem
Vendor lock-in	a single governor over mixed clouds	Supervisory

8. Order of wiring

1. Digital infrastructure: GPUs, CPU power states, memory, storage, network, job queues, multi-region.
2. Facilities and energy: cooling plant, power distribution, batteries, demand response.
3. AI: training power smoothing, inference serving, model routing, agent containment.
4. Physical operations, then science and space as supervisors.
5. Regulated domains as advisors, until certified.

Each connector follows the same path as compute: wired, benchmarked against today's controls, verified, pre-registered, tested on held-out scenarios, then released.

CHAPTER 24

The Problem Map

Method: the issue trackers of the projects that run the world's clusters (Kubernetes autoscaler, Karpenter, Kepler, Kueue, Volcano, Knative, KEDA, kubernetes/kubernetes), the public industry reports (Cast AI, Datadog, Uptime Institute, IEA) and recent papers (Meta Llama 3, AI-datacenter power stabilisation). Not all of GitHub: a targeted scan of where the money and the failures are. Status: **have** = in this repository and tested; **partial** = built but not proven live; **missing** = not built.

#	Problem (evidence)	Size	Best tool today and its gap	Omni-Compass fit	Status
1	Idle capacity. Average Kubernetes CPU utilisation 8%, memory 20%, falling; CPU over-provisioning 69%, memory 79% (Cast AI 2026). 83% of container cost goes to idle resources (Datadog)	largest money leak in cloud	Karpenter, Cluster Autoscaler, CAST AI: each fixes nodes only	one authority over replicas, requests, nodes and power	built (omnilab/rightsize.py, held-out)
2	HPA and VPA cannot be used together on CPU/memory , "not compatible by design" (kubernetes/autoscaler #1726, #2939, #6060, #6247; coordination asked again in #8493, 2025)	blocks right-sizing for most teams	none; official advice is "don't"	exactly what a single authority solves: one engine owns both numerator and denominator	built (omnilab/rightsize.py: HPA+VPA vs one authority, held-out)
3	Karpenter consolidation churn: nodes replaced every 5-10 minutes for 2-3 generations, busy nodes removed instead of empty ones, one shared timer (karpenter #1851, #1019, #2705, #3046; provider-aws #7146, #8868, #7356, #8536)	outages, wasted boots	Karpenter's own timers and budgets	engine-gated release (equation 2) plus dwell; C-throughput cut node churn 50% in simulation	built in simulation (AKS NAP = Karpenter arm, tuning/vendor_compare.py); live Karpenter opponent next
4	CPU limits throttle apps (CFS quota), causing latency and even OOMs (kubernetes/kubernetes #67577, #97445)	latency tails	manual tuning, "remove limits" advice	power-cap muscle must watch latency; our first live run hit this exact problem	have; live runs 3-5 found and fixed the resize permission
5	Cold starts / scale-from-zero latency (knative/serving #4902, #14202, #9104; kedacore/http-add-on #219)	seconds of latency per edge	Knative activator, KEDA	pre-warming decided by the engine from its state (anticipation)	built (omnilab/coldstart.py, held-out)

#	Problem (evidence)	Size	Best tool today and its gap	Omni-Compass fit	Status
6	GPU utilisation ~5% (Cast AI 2026); idle GPUs scattered across nodes (volcano #3948, kueue #5243)	the most expensive silicon idle	KAI Scheduler, HAMi, nvshare, Volcano binpack	GPU power-limit muscle (have, calibrated), packing and sharing (missing)	built (omnilab/gpupack.py, held-out) + power limit
7	AI training power swings: 50-75% of TDP in milliseconds per GPU, ramps over 1000 MW/s at gigawatt scale; operators impose power and ramp-rate limits (SemiAnalysis; Uptime; arXiv 2508.14318, 2606.04869)	grid-connection risk, equipment damage	batteries, fast PSUs, manual GPU power caps	"training power smoothing" muscle: ramp-rate limit through GPU power caps, the engine's bath state B is literally a damped oscillator	built (omnilab/powersmooth.py, held-out; basin-only arm recorded)
8	GPU failures and stragglers: Llama 3, 419 interruptions in 54 days on 16K H100s, one every ~3 hours; 58.7% GPU-related; slow stragglers undetected (Meta; Lablup 2605.09370)	lost training time	Meta's internal tools, checkpointing	hardware-health muscle: sense straggling, drain early	built (omnilab/health.py, held-out)
9	Outages: 54% cost more than \$100,000, 1 in 5 more than \$1M; power causes 45% of incidents (Uptime 2025)	direct money	SRE runbooks, AIOps	recovery and physical-SLA results in simulation: recovery 58 to 17 min	have (sim)
10	Cooling: industry PUE stuck at ~1.54 for six years (Uptime 2025)	~35% overhead on every watt	DCIM, DeepMind cooling AI	heat afferent have; cooling-plant muscle	built (omnilab/cooling.py, held-out)
11	Energy measurement in VMs is impossible: no RAPL/IPMI in VMs, Kepler building a model server (kepler #2487, 2026)	nobody can prove savings in cloud VMs	Kepler estimators	same gap we hit	built (omnilab/vmenergy.py: meter-calibrated attribution, held-out)
12	LLM inference autoscaling: KV cache fills memory before compute; prefill vs decode need separate scaling (vLLM, llm-d)	latency and GPU cost	llm-d, KServe, custom	inference muscle: scale on KV-cache use and queue, not CPU	built (omnilab/inference.py, held-out)
13	Runaway AI agents: loops and recursive calls, ~\$10,000 overnight examples (Dark Reading; sandbox guides)	cost and safety	per-tool caps, early kill-switch projects	agent-containment muscle: caps, kill, audit	built (omnilab/containment.py, held-out); live version next
14	Human error: failures to follow procedures rose 10 points (Uptime 2025)	outages	runbooks	fewer manual actions: pages and human interventions to zero in simulation	have (sim)

Results of the build pass (held-out seeds 90001-90030, 30 per muscle, code frozen by SHA first)

Omni-Compass against the strongest native tool in each muscle; + better, - worse, = tie / not significant.

#	Muscle	Opponent (best native)	Better	Worse	Tie / n.s.
1-2	right-sizing	VPA in-place + HPA	CPU -3%, memory -11%, p99 -99%, reversals -84%, SLO minutes -57%	p95 +15%, OOM kills (few)	work done
5	cold start	KEDA HTTP	p95 -75%, p99 -58%, delayed >1 s -71%, instance-hours -8%	cold-start count (x8)	
6	GPU packing	Volcano binpack	idle powered GPU-hours -20%, energy -2%	mean wait +16% (seconds)	p95 wait, jobs, fragmentation
7	training power	vendor floor, worst-day setting	burn -9%, throughput loss -44%, 1-s swing -6%	max ramp (inside the grid's limit)	0 grid violations both
8	GPU health	threshold detection	goodput +10%, lost GPU-h -20%, restarts -29%, straggler hours -99%		checkpoint overhead
10	cooling	outdoor-air reset	PUE -2%, cooling -13%, inlet violations -73%		max inlet
11	VM energy	host meter split by CPU	per-VM error -37%, worst VM -34%		total
12	LLM inference	KEDA on queue	TTFT p95 -18%, SLO breaches -58%, GPU-hours -19%, preemptions -56%		TPOT, TTFT p50
13	agent containment	static caps	rogue spend -96%, time to contain -99%, peak sub-agents -59%	false stops (0.6 -> ~2.7/day), honest work -7%	forbidden actions (0 both: RBAC)

Engine vs no-engine: in every muscle the omni_no_engine arm scores close to omni. The mapping from engine state to action carries most of the effect; the evolved dynamics add smoothing. The basin-only arm (power smoothing, the bath equation alone setting the draw) cut ramps 63% but did not hold the grid limit: the human-set limit is required.

What this says

- The problems with the most money behind them are **idle capacity (1)**, **HPA/VPA coordination (2)** and **GPU idleness (6)**. Problem 2 is the cleanest proof of Omni-Compass's thesis: the Kubernetes project itself says its two autoscalers cannot share a metric, because they fight. A single engine owning both is the answer the issue tracker keeps asking for.
- The problem with the most strategic weight is **AI training power swings (7)**: it is new, it threatens grid connections, and Omni-Compass's own equation (7) is a damped second-order bath, the natural controller for ramp limits.
- The problems we already hit ourselves (4, 11) are the same ones the industry has: good evidence the benchmark is real.

Build order proposed

- Request right-sizing muscle (VPA role) under the same engine, benchmarked against HPA + VPA "in conflict".
- Training power-smoothing muscle (GPU power-cap ramp limits) with a synthetic synchronized-training power trace.

3. GPU packing / fragmentation muscle (with fake-gpu-operator or KWOK on the live cluster).
4. Live Karpenter opponent (kwok provider) to test problem 3 head to head.
5. Inference muscle (KV-cache and queue driven), then hardware-health (straggler drain), then agent containment.

Sources

- kubernetes/autoscaler issues #1726, #2939, #6060, #6247, #8493
- Karpenter #1851, #1019, #2705, #3046; aws/karpenter-provider-aws #7146, #8868, #7356, #8536
- kubernetes/kubernetes #67577, #97445
- Knative #4902, #14202, #9104; KEDA http-add-on #219
- Volcano #3948; Kueue #5243; KAI Scheduler; HAMi
- Kepler #2487
- Cast AI 2026 State of Kubernetes Resource Optimization; Cloud Native Now on the report
- Uptime Institute annual outage analysis 2025; Uptime global survey 2025
- SemiAnalysis: AI training load fluctuations; Uptime: AI power fluctuations; Power Stabilization for AI Training Datacenters; Source-side mitigation
- The Llama 3 Herd of Models; Tom's Hardware on Llama 3 failures; Lablup 504-GPU report
- vLLM anatomy; llm-d 0.5
- Dark Reading: AI agents and runaway costs

CHAPTER 25

The Six Organisms and the Benchmark Grid

Omni-Compass is benchmarked on six organisms, each run native and with Omni-Compass on top on the same seed, the same load and the same clock: Compute / AI / Cloud (345 muscles), Physics / Robotics / Autonomous (262), Energy / Facility / Industrial (282), Distribution / Specialized (337), the four stacked with every duplicate kept (1,226), and the whole tower with every muscle once (656).

Each organism is run at 1, 10, 100 and 1,000 paired runs and at 1, 10, 100 and 1,000 copies of the organism on one clock. Every receipt shows work per energy, work, energy, the time spent outside the service line, and whether every knob was handed back.

On a GPU machine the real card is wired into each organism as one more muscle of its NVIDIA GPU family; its own power meter and its requests are counted in the organism's receipt, kept apart from the modelled plants and also added to them. Real Kubernetes is benchmarked separately on real clusters.

The six organisms are models (evidence class S). They show how the law behaves across hundreds of kinds of machine at once and whether its effect is stable as the count and the size grow. Real Kubernetes and the real card are the anchors measured on real software and a real meter.

CHAPTER 26

The Metrics Catalog

Every gauge Omni-Compass produces, where it comes from, and what kind of number it is. Three kinds:

- **Measured:** read from a real system (a real Kubernetes cluster, a real GPU's own meter, a real wall plug).
- **Modelled:** computed by a simulation from declared physics or recorded traces. It shows the mechanism, not a measurement.
- **Internal:** Omni-Compass's own state and decisions, recorded in its audit log on every decision, live or simulated.

Every report says which kind each number is. A modelled number is never presented as a measured one.

1. Service: what the customer feels

Gauge	Unit	Where	Kind
Response time, mean / 95th / 99th percentile	ms	live Kubernetes (<code>tools/live_reps.py</code>), GPU bench (<code>tools/gpu_reps.py</code>), GPU sims	measured live; modelled in sims
Failed requests	%	live Kubernetes	measured
Requests served / not served	count	GPU bench, GPU sims	measured / modelled
Pod start wait, total and mean	s	live Kubernetes (API server creation-to-Ready)	measured
Pending pods	pod-minute s	live Kubernetes	measured
Backlog (SLO breach) minutes	min	site and node exchange sims, stack benchmark	modelled
95th-percentile latency factor	×	site and node exchange sims	modelled
Availability, time healthy, recovery minutes, recovered	share, min	stack benchmark (<code>benchmarks/stack_benchmark.py</code>)	modelled
SLA violations (physical, total, backlog, power, heat)	share of time	stack benchmark	modelled

2. Work and capacity: what the operator gets for the money

Gauge	Unit	Where	Kind
Work per energy (served requests per kJ)	req/kJ	GPU bench (the preregistered primary outcome), GPU sims	measured / modelled
Work per wall energy (whole machine)	req/kJ	GPU bench with a smart plug (tools/wall_meter.py)	measured
Work served	work units	site and node exchange sims	modelled
Work per kWh	units/kWh	node exchange sim	modelled
Energy per served request	J	GPU bench	measured
Energy per core-hour	Wh	live Kubernetes	declared model
CPU used by the app	cores	live Kubernetes (kubectl top)	measured
Utilisation (used / allocatable)	share	live Kubernetes	measured
Replicas (HPA), mean	count	live Kubernetes	measured
Pods started	count	live Kubernetes	measured
Worker nodes in service, node-hours, idle node-hours	count, h	live Kubernetes, stack benchmark	measured / modelled

3. Energy and power: the bill and the building

Gauge	Unit	Where	Kind
GPU energy (the device's own power.draw, integrated)	J	GPU bench	measured
GPU mean power	W	GPU bench, GPU sims	measured / modelled
CPU package energy (RAPL)	J	GPU bench, where the machine exposes RAPL	measured
Whole-machine energy at the wall	J	GPU bench with a smart plug	measured
Energy on kind	Wh	live Kubernetes	declared model (kind has no meter)
Energy	kWh	site, node exchange and stack sims	modelled
Peak power	kW	site and node exchange sims, stack benchmark	modelled
Site-budget violation minutes (time over the building's limit)	min	site and node exchange sims	modelled
GPU power limit, mean; power-limit writes	W, count	GPU bench, GPU sims	measured / modelled
Power-cap travel	share	stack benchmark	modelled

4. Hardware: heat and wear

Gauge	Unit	Where	Kind
GPU temperature, peak and mean	°C	GPU bench, GPU sims	measured / modelled
Heat-over minutes	min	hardware plant sim (hardware/plant.py)	modelled
Thermal travel	share	stack benchmark	modelled
Machine round trips, node starts and stops, scale reversals	count	stack benchmark	modelled

Lower power and fewer temperature swings are the conditions for longer hardware life. **No hardware life extension is claimed** until real temperature, duty-cycle and wear records exist.

5. Battery equivalent (derived)

A battery holds a fixed amount of energy, so battery life follows directly from work per energy:

- runtime on the same charge, for the same work: $\times (\text{native energy} \div \text{Omni-Compass energy})$
- work on the same charge: $\times (\text{Omni-Compass work per energy} \div \text{native work per energy})$

Example: the modelled GPU card at +5.1% work per kJ (`results/gpu/sim/after`) does 5.1% more work on one charge. The same rule turns any measured work-per-energy result into a battery figure. On-site battery banks as an organ of the power budget are designed (`docs/DOMAIN_MAP.md`, "on-site batteries") and not yet built.

6. Safety and control: proof it behaved

Gauge	Where	Kind
Writes executed per arm (native 0, watch 0, Omni-Compass n)	GPU bench, live Kubernetes	measured
Every arm ended at its start setting (the kill switch restored it)	GPU bench, live Kubernetes switch drill	measured
Invariant violations (all, and excluding power)	stack benchmark	modelled
Security violations, contradictions, pages	stack benchmark	modelled
Freeze check: code hashes at start and end (confirmation runs)	GPU bench (<code>FREEZE.json</code> , <code>FREEZE_END.json</code>)	measured
SHA-256 of every raw file	every live run	measured
The seal: every Python/C++ twin unchanged since proven equal	<code>results/SEAL.json</code> , <code>verify.py</code>	checked on every build

7. Internal: Omni-Compass's own state, on every decision (audit log)

Field	Meaning
E, U, I_U, S, B, B_dot	the six-state engine: energy, coherence (health), unmet-need integral, stress, redistribution, its rate
state_observed, state_projected_next, prediction_error	what the device said, what the engine projected, and how far off the last projection was
requested_cap, granted_cap, shield_bound, want_w	what the engine asked for, what the shield allowed, which bound decided it, the limit written
admissible	whether the engine was permitted to change anything this decision
speed_lock(ratios, line, aim, rate)	the speed lock's reading against the run without Omni-Compass
convey engaged / released	when idle CPU is conveyed to serving pods, and when it is given back
authority: calm and its scalars (kappa, h, sigma, nu), execute, contract per organ, and the machine organ's own view	how settled the engine is, whether it may act at all, and which organs may give capacity back this decision (Kubernetes controller; <code>omnicompass/nervous_system.py</code>)
compass readings	the direction the engine reads the whole system to be moving (<code>omnicompass/compass.py</code>)
write, would_write, why	every command sent (or, in watch mode, withheld) and its reason
snapshot, restored	every setting read at start, and its read-back after the kill switch

8. Where each report lives

Report	Command	Output
Live Kubernetes, paired	benchmark-reps.yml (commit with [reps])	LIVE_REPS.md, results/live/
GPU bench on a real card	sudo bash scripts/gpu_paired.sh	results/gpu/run-*/GPU_REPS.md
GPU physics sim	python3 tools/gpu_physics_sim.py	results/gpu/sim/
CPU-then-GPU speed lock sim	python3 tools/gpu_pipeline_sim.py	results/gpu/sim/pipeline/
Site power exchange (GPU groups)	python3 hardware/site_exchange.py	results/hardware/SITE_EXCHANGE_*.json
CPU + GPU on one budget	python3 hardware/node_exchange.py	results/hardware/NODE_EXCHANGE_*.json
Full stack benchmark and every check	python3 verify.py	results/, VERIFICATION: PASS

PART FIVE

The Harness and the Wiring

*The universal plug, the adapters, the wire check, and the step-by-step work of wiring
Omni-Compass onto a running stack.*

The Plug, the Adapters and the Wire Check

Think of a high-end car stereo: one head unit, one standard plug on its back, and an adapter harness for each make of car that matches the car's factory plug. You never cut a factory wire, and when you pull the stereo, the car works as it did. Omni-Compass is wired the same way.

8.1 The plug (one design for every muscle). Each plug has as many channels as its muscle has knobs. Every channel declares its cover (range), its direction (which way is "more"), its units and how fast it may move. Every plug keeps the same contract (`omnicompass/bowl.py`, `Plug`):

1. **attach** - read the knob once, before any write: the snapshot. It never moves afterwards.
2. **read** - the service reading, into the bowl.
3. **write** - one value, clipped to the cover, then read back from the device.
4. **one writer** - if the knob is found at a value Omni-Compass did not write, someone else owns it: Omni-Compass stops writing and leaves that value alone.
5. **restore** - on stop, the knob returns to the snapshot (where Omni-Compass found it, not the last value it wrote). For muscles whose native controller moves the knob itself, restore hands control back.

8.2 Adapters (one per standard plug). Omni-Compass speaks each industry's standard control interface through an adapter. Status is stated plainly:

Standard	Muscles	Adapter status
Kubernetes API	HPAs, deployments, pods, nodes	built (<code>omni_controller/controller.py</code> , <code>muscles.py</code>)
NVIDIA NVML / <code>nvidia-smi</code>	GPU clock ceiling, power limit	built (<code>omni_controller/gpu_bowl.py</code> , two wires; <code>gpu_governor.py</code> , one wire)
Linux <code>cpufreq</code> , <code>RAPL</code>	CPU frequency ceiling, package watts	built (<code>--cpufreq-policy-root</code> , <code>--rapl-cmd</code>)
Site meter, building controller by command	site watts, supply-air setpoint	built (<code>--power-cmd</code> , <code>--cooling-cmd</code> command templates)
Cloud node groups (Karpenter, Cluster Autoscaler, MachineSet)	machines	built through <code>--node-scale-cmd</code> templates
BACnet, Modbus, SNMP	chillers, air handlers, PDUs, UPS	designed; today reached through the command templates above
OPC UA, EtherNet/IP, PROFINET, EtherCAT, ROS 2, CAN	PLCs, drives, robots, vehicles	designed; modelled in the realm harness
IEC 61850, DNP3, OpenADR, IEEE 2030.5, SunSpec, OCPP	substations, batteries, inverters, chargers	designed; modelled in the realm harness

The engineer's job for one muscle is three things: which adapter, the address of the knob, and its safe range.

8.3 Power-up order. Read-only first; then take the knobs, the least consequential first; on stop, hand them back in reverse order.

8.4 The wire check (mandatory before any write). For the GPU it is one command, `sudo python3 tools/gpu_wire_check.py --gpu 0`, and `scripts/gpu_rented_run.sh` runs it first and stops if it fails:

Step	It proves
1 read	every meter answers: watts, temperature, utilization, limit, clock, top clock
2 snapshot	the start limit is recorded once
3 up wire	with the card busy, a ceiling below its own busy clock pulls the clock down under it; reset, the clock comes back above it
4 down wire	a lower power limit is reported back exactly; the start limit is reported back exactly
5 restore	clocks reset and the start limit read back
6 stop	the governor, stopped by its signal, hands both wires back
7 other writer	a limit set by someone else while the governor runs is left alone, and the governor exits 5

Any failure prints the wire, the step and what the device said, and nothing else runs. That turns a wiring fault from guesswork into one named line to fix.

CHAPTER 28

The Harness

This tree is the GitHub-ready kit: frozen engine + C++ twin + two plants + live stub.

What to run

```
pip install -r requirements.txt
python verify.py --quick          # engine, C++, soak, parity
python k8s_controlplane/test_controlplane.py
python -m k8s_controlplane.suite # elastic / always_on / idle_power
python tests/test_hpa_three_way.py
python tests/test_omni_controller.py
```

Fleet plant (Omni as node authority vs HPA+CA / Karpenter-lite):

```
python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 8 --out /tmp/pl
```

C++:

```
cmake -S cpp -B cpp/build -DCMAKE_BUILD_TYPE=Release && cmake --build cpp/build -j2
./cpp/build/oc_smoke
```

Two plants (do not mix the tables)

Tree	What Omni is	What the energy number means
k8s_controlplane/	On top of HPA+CA (target / gate / park)	Pack and optional CA gate on a 20-node replica
fleet/	Node-pool authority; CA off	Consolidation vs CA and Karpenter-lite

Observe must match the native arm on that plant. If it does not, the run is invalid.

Engine species

Shipped omnicompass/core.py is the patent principal form: cubic $\sqrt{U(1-U^2)}$, FIG. 4 command, RK4 with held $\sqrt{u}$. omnicompass/pools.py is actuation only (off / hold / park). It does not change the field.

Not in this harness

Live kube-controller-manager, kind CI, GPU MIG scheduler, facility cooling plant. omni_controller/ is observe-first against kubectl; tests use tests/fake_cluster/kubectl.

See LIMITS.md.

CHAPTER 29

The Wiring Guide

The current manual is docs/INTEGRATION_MANUAL.md (every stack, every level, every switch). This page is kept for reference; where the two differ, the integration manual is current.

The full step-by-step manual, with the switch, the living band, parking and the pod reflex, is docs/OPERATOR_MANUAL.md. This page is the short version.

Three stages. Each stage adds only the permissions it needs. A stage is promoted only after its evidence is in, and the kill switch works at every stage:

```
kubectl -n omni-compass exec deploy/omni-compass -- touch /tmp/omni.kill
```

The switch is one human-operated switch for the whole harness (OFF restores every HPA target, replica range, CPU limit and cordon Omni-Compass changed, and records it; removing the file turns Omni-Compass back ON). It never fires by itself and never switches off one module: a failed decision is skipped, boundaries are held by the living band.

0. Build the image

```
docker build -f deploy/Dockerfile -t <registry>/omni-compass:<tag> .  
docker push <registry>/omni-compass:<tag>
```

What the image contains.

- The engine.
- The nervous system.
- The live controller and its levers.
- The frozen closure-law setting (/app/law/closure.json).

How it runs. As a non-root user, with a read-only root filesystem and no Linux capabilities.

1. Shadow: read-only, decides and logs, never writes

```
kubectl apply -f deploy/install/omni-compass.yaml      # set the image line first  
kubectl -n omni-compass logs deploy/omni-compass -f
```

Proof of read-only. scripts/pilot_shadow.sh runs the same stage from a workstation with kubectl auth can-i receipts that the identity cannot write, and writes SHADOW_REPORT.md.

Run length. One to two weeks.

Pass condition. Zero writes, and recommendations you agree with.

2. Target: Omni-Compass sets each HPA's CPU target

```
kubectrl apply -f deploy/rbac-target.yaml # adds: patch horizontalpodautoscalers
kubectrl -n omni-compass patch deploy omni-compass --type=json -p \
' [{"op": "replace", "path": "/spec/template/spec/containers/0/args/1", "value": "target"} ] '
```

What happens. The HPAs keep scaling pods. Omni-Compass only moves their target within bounds.

The SLO reflex. Add `--latency-file` and `--slo-ms`, and the target is never tighter than native while the SLO is breached.

Pass condition. Latency no worse than native, and fewer pod-hours.

3. Node pool: Omni-Compass sizes the node pool

Add permissions. patch nodes, create pods/eviction (deploy/kind/rbac-omni.yaml is the tested example).

Set the mode. `--mode nodepool --node-scale-cmd "<your pool resize command with {n}>"`. Examples of the resize command:

- a Karpenter NodePool limit;
- a cloud node-group size;
- `scripts/kind_nodepool.sh` on kind.

What guards a release. The machine organ gives a node back only when all of these hold:

- every sense is live;
- its last order landed;
- pods are not scaling up;
- nothing is pending;
- the remaining nodes stay at or below the engine's target utilisation.

A PodDisruptionBudget on each service is required, because drains go through the eviction API.

Optional levers. Each has its own flag and permissions, and all are listed in `omni_controller/muscles.py`:

- right-sizing;
- cold start;
- batch pacing;
- agent containment;
- cooling;
- CPU frequency.

Evidence, and where it is

Evidence	File
The controller, every lever and the Unified Control Switch on real Kubernetes (kind)	results/live/LIVE_LEVERS_2_NERVOUS.txt
Read-only shadow on real Kubernetes: every decision logged, 0 writes	results/live/LIVE_SHADOW_1.txt
Least-privilege identity receipts	rbac_omni.txt in every live run
Native vs Omni on top vs Omni alone, paired on one machine, real Kubernetes	results/live/LIVE_PAISED.md
The full wiring manual	docs/OPERATOR_MANUAL.md

CHAPTER 30

Before You Start, and the Eight Levels

What your system needs

You run	You need
Kubernetes (vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind)	Kubernetes 1.34 or newer (in-place pod resize); metrics-server (<code>kubecttl top nodes</code> works); an HPA with a CPU target on each governed service; a readiness probe and a short preStop pause on each service
A response-time feed (strongly recommended at every level)	a CSV per service, <code>elapsed_seconds</code> , <code>latency_ms</code> , <code>ok</code> , written continuously; <code>scripts/latency_probe.py</code> writes one against any HTTP endpoint
NVIDIA GPUs	a driver with <code>nvidia-smi</code> ; root (or the capability to run <code>nvidia-smi -pl</code> and <code>-lgc</code>); on a VM, full GPU passthrough
CPU power control	Linux <code>cpufreq</code> with the <code>schedutil</code> governor; RAPL for package watts; usually bare metal
Site power and cooling	a command that prints site watts, and a command that sets the supply-air setpoint through your building management system

Get the software and check it

```
git clone https://github.com/The-Omni-Compass-LLC/The-Omni-Compass.git
cd The-Omni-Compass
pip install -r requirements.txt
python3 verify.py # every check; ends with VERIFICATION: PASS
```

Build the container image once:

```
docker build -f deploy/Dockerfile -t <your-registry>/omni-compass:<tag> .
docker push <your-registry>/omni-compass:<tag>
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

The levels

Take them in order. Each has a pass condition; go to the next level only when it holds. The OFF switch works at every level.

Level 0 - Evaluate without touching anything.

```
python3 verify.py
python3 tools/run_scale.py --runs 10 --scale 1 --out results/scale/eval      # the six organisms, simulat
    ↵ ed
python3 tools/run_gpu_card.py results/sim/gpu_two_wire/eval                # the two-wire card, modell
    ↵ ed
```

Pass: VERIFICATION: PASS. Nothing in your systems is touched.

Level 1 - Watch (read-only).

```
kubectl apply -f deploy/install/omni-compass.yaml      # set the image line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

The install file grants a read-only identity; scripts/pilot_shadow.sh records kubectl auth can-i receipts that show it cannot write. Every decision it would take is logged with its reason. Pass: zero writes, and readings your operators agree with.

Level 2 - The pods, on top of your autoscalers. WRITES.

1. Grant patch horizontalpodautoscalers and patch pods/resize (deploy/rbac-target.yaml).
2. Run with --mode target --latency-file <feed> --slo-ms <your p95 target>.
3. Optional pod muscles, one at a time:

Muscle	Switch	What it does
convey	--latency-file and --cap-deployments ns/name	gives each machine's idle CPU to the serving pods on it
rightsize	--rightsize-deployments ns/name	each pod's CPU request follows its measured use times (1 + headroom), in place
coldstart	--coldstart-deployments ns/name --coldstart-signal ns/configmap	scales a service to zero while no work waits, wakes it when work arrives
batch	--batch	admits held Jobs labelled <code>omnicompass.io/batch=true</code> when there is load and power headroom
batch pace	--batch-pace	pauses Jobs labelled <code>omnicompass.io/pausable=true</code> under power or heat stress, resumes them after
rollout guard	--rollout-guard ns/name	pauses a rollout while change is not permitted
contain	--contain-namespaces ns --contain-cpu-m <m>	holds an agent namespace to a CPU budget
security hold	--security-configmap ns/name	key hold: "true" blocks every expansion

Your HPAs keep scaling as before; Omni-Compass sets their targets and raises floors ahead of bursts. Pass: p95, p99 and failed requests no worse than native, over paired runs (Chapter 37).

Level 3 - The machines. WRITES.

1. Grant patch nodes and patch pods (deploy/kind/rbac-omni.yaml).
2. Run with --mode nodepool --active-nodes-only --closure /app/law/closure.json
--node-scale-cmd "<command with {n}>" and --node-restore-cmd "<command>" for the OFF switch.

Your platform	The park/wake command
any cluster, bare metal, kind	<code>bash scripts/kind_nodepool.sh {n}</code>
Karpenter / EKS Auto Mode	the NodePool CPU limit at {n} times node CPU, parked nodes kept
Cluster Autoscaler node group	the group's desired size, scale-down through parking

Your platform	The park/wake command
OpenShift / OKD	the worker MachineSet replicas, parked

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band. Pass: fewer machines in service, with no service gauge worse.

Level 4 - Omni-Compass decides; Kubernetes is the muscle. WRITES. Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; the HPA stays as the fast reflex upward. Pass: as level 3.

Level 5 - A GPU box, two wires. WRITES.

```
sudo python3 tools/gpu_wire_check.py --gpu 0 # must end: WIRED RIGHT
```

CHAPTER 31

Stack by Stack

Stack	Levels	Notes
Vanilla Kubernetes, kind, Rancher	1-7	as written
Amazon EKS	1-5	machines via Cluster Autoscaler node group or Karpenter; CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances
Google GKE, Azure AKS	1-5	the provider's node-pool size as the park/wake command
Red Hat OpenShift / OKD	1-5	the same permissions through a Role; machines via the worker MachineSet
NVIDIA GPU servers without Kubernetes	5	the two-wire GPU governor alone
Bare-metal CPU servers	6	cpufreq and RAPL through sysfs
Slurm / HPC	5 on the GPU nodes	a job-level Slurm connector is not built
Building management	7	through your BMS's command line or API, in the command templates

CHAPTER 32

The Integration Manual

How to wire Omni-Compass into your own systems yourself, from watching only to running your stack from the top. This manual ships in the box with the code and the license. Nobody from The Omni-Compass LLC needs to be on site.

Read section 1 once, then take the levels in section 4 in order. Each level has a pass condition; go to the next level only when it holds. The OFF switch (section 2) works at every level.

1. What is in the box

Item	Where
The engine, nervous system, safety shield, compass and conveyance law	omnicompass/
The Kubernetes controller and its muscles	omni_controller/controller.py, omni_controller/muscles.py
The GPU governor (one GPU box, no Kubernetes needed)	omni_controller/gpu_governor.py
Container image recipe; install and permission files	deploy/Dockerfile, deploy/install/omni-compass.yaml, deploy/rbac-*.yaml, deploy/pilot/
Proof tools: paired tests on your own stack, the GPU test, simulations	scripts/, tools/, hardware/
Every check the code must pass	python3 verify.py (ends with VERIFICATION: PASS)
The C++ engine: every law twinned in C++20, proven equal to the Python	cpp/ (build: cmake -S cpp -B cpp/build && cmake --build cpp/build)
The seal: the fingerprints that lock each Python law to its C++ twin	results/SEAL.json, python3 tools/seal.py --check
Metrics: every gauge and where it comes from	docs/METRICS_CATALOG.md
How it compares with what you run today	docs/COMPARISON.md
The license and notices	LICENSE, NOTICE

License, in short (the LICENSE file governs): you may download, run, modify and test Omni-Compass free of charge for evaluation, research and non-commercial use, including on your own systems. Commercial use (running it for your business, selling it or building it into a product or service) needs a paid commercial license from The Omni-Compass LLC. The copyright and patent notices must stay with every copy.

2. The rules Omni-Compass keeps on your system

1. **One OFF switch, in a human hand.** Creating the kill file (or setting OMNI_KILL=1) returns every setting Omni-Compass changed to the value it recorded before it acted, reads each back, and stops all action.

- Kubernetes controller: `--kill-file` (default `/tmp/omni.kill`).
 - GPU governor: `--kill-file` (default `/tmp/omni-gpu-kill`), or send it `SIGTERM`.
2. **It records before it acts.** Every original setting is written down first (annotations on the Kubernetes objects, the snapshot line in the GPU audit), so the OFF switch always knows what to restore.
 3. **It watches before it writes.** Every level starts in watch mode: it decides and logs, and writes nothing.
 4. **It never goes blind and keeps acting.** If a reading fails (`nvidia-smi`, metrics, the response-time feed), it returns what it changed to the recorded setting at once and gives nothing back until it can see again.
 5. **Service first.** While response time is over your target, and for a few decisions after (`--slo-clear`), it may not cap power or pack tighter than your own settings.
 6. **Every change is read back.** No new order goes on top of one that has not landed.
 7. **The living band.** No organ is driven below 5% or above 95% of its range; machines are marked idle, never switched off by Omni-Compass; it never evicts or moves a pod.
 8. **Everything is logged.** Every read, decision, write and its reason goes to the audit log (`--audit`).

3. Before you start: what your system needs

You run	You need
Kubernetes (any: vanilla, EKS, GKE, AKS, OpenShift/OKD, Rancher, kind)	Kubernetes 1.34 or newer (in-place pod resize); metrics-server (<code>kubectl top nodes</code> works); an HPA with a CPU target on each service to be governed; a readiness probe and a short preStop pause on each service
A response-time feed (strongly recommended at every level)	a CSV per service, <code>elapsed_seconds, latency_ms, ok</code> , written continuously; <code>scripts/latency_probe.py</code> writes one against any HTTP endpoint
NVIDIA GPUs	driver with <code>nvidia-smi</code> ; root (or the capability to run <code>nvidia-smi -pl</code>); on a VM, full GPU passthrough (container or "pod" GPU rentals usually block power-limit changes)
CPU power control	Linux <code>cpufreq</code> with the <code>schedutil</code> governor (<code>/sys/devices/system/cpu/cpufreq</code>); RAPL for package watts (<code>/sys/class/powercap/intel-rapl</code>); usually bare metal only, since cloud VMs rarely expose these
Site power and cooling	a command that prints site watts, and (for cooling) a command that sets the supply-air setpoint through your building management system

Build the image once:

```
docker build -f deploy/Dockerfile -t <your-registry>/omni-compass:<tag> .
docker push <your-registry>/omni-compass:<tag>
```

It runs as non-root, with a read-only root filesystem and no Linux capabilities.

4. The levels, from watching to running the stack

Level 0. Evaluate without touching anything

```
pip install -r requirements.txt
python3 verify.py                # every check; ends with VERIFICATION: PASS
python3 tools/gpu_physics_sim.py # the GPU governor against a modelled NVIDIA card
python3 hardware/node_exchange.py # CPU and GPU on one power budget, modelled
```

Pass condition: VERIFICATION: PASS. Nothing in your systems is touched.

Level 1. Watch (read-only)

```
kubectl apply -f deploy/install/omni-compass.yaml # set the image line; it runs --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

- The install file grants a read-only identity. `scripts/pilot_shadow.sh` records `kubectl auth can-i` receipts that show it cannot write.
- Every decision it would take is logged with its reason; nothing is written.

Pass condition: zero writes, and readings your operators agree with.

Level 2. The pods, on top of your autoscalers

1. Grant `patch horizontalpodautoscalers` and `patch pods/resize` (`deploy/rbac-target.yaml`).
2. Run with `--mode target --latency-file <feed> --slo-ms <your p95 target>`.
3. Optional pod muscles, one at a time:

Muscle	Switch	What it does
convey	on with <code>--latency-file</code> and <code>--cap-deployments ns/name</code>	gives each machine's idle CPU to the serving pods on it (default: always); <code>--convey-on 0.5 --convey-off 0.25</code> engages it only while response time is over half the target
rightsize	<code>--rightsize-deployments ns/name</code>	each pod's CPU request follows its measured use $\times (1 + \text{headroom})$, in place
coldstart	<code>--coldstart-deployments ns/name</code> <code>--coldstart-signal ns/configmap</code>	scales a service to zero while no work waits, wakes it the moment work arrives
batch	<code>--batch</code>	admits held Jobs labelled <code>omnicompass.io/batch=true</code> when there is load and power headroom
batch pace	<code>--batch-pace</code>	pauses Jobs labelled <code>omnicompass.io/pausable=true</code> under power or heat stress, resumes them after
rollout guard	<code>--rollout-guard ns/name</code>	pauses a rollout while change is not permitted, undoes one past its deadline when rollback is authorised
contain	<code>--contain-namespaces ns --contain-cpu-m <m></code>	holds an agent namespace to a CPU budget with a quota
security hold	<code>--security-configmap ns/name</code>	key <code>hold: "true"</code> blocks every expansion

Your HPAs keep scaling as before. Omni-Compass sets their targets and raises floors ahead of bursts.

Pass condition: p95, p99 and failed requests no worse than your own, over paired runs (section 6).

Level 3. The machines

1. Grant `patch nodes` and `patch pods` (`deploy/kind/rbac-omni.yaml`).
2. Run with `--mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<command with {n}>"`.

Your platform	The park/wake command
any cluster, bare metal, kind	<code>bash scripts/kind_nodepool.sh {n}</code> : close machines to new work and mark them first to go; open again, warm machines first
Karpenter / EKS Auto Mode	the NodePool CPU limit at <code>{n} × node CPU</code> , parked nodes kept, not consolidated away
Cluster Autoscaler node group	the group's desired size, scale-down through parking, not deletion

Your platform	The park/wake command
OpenShift / OKD	the worker MachineSet replicas, parked, not deleted

A machine is given back only when every sense is live, the last order landed, pods are not scaling up, nothing waits for a place, and the remaining machines stay inside the band.

Pass condition: fewer machines in service and less energy, with no service gauge worse.

Level 4. Omni-Compass decides; Kubernetes is the muscle

Add `--strict-replicas`: Omni-Compass decides each service's replica floor and when to shrink; your HPA stays as the fast reflex upward; the node law sizes the machines.

Pass condition: as level 3.

Level 5. A GPU box (with or without Kubernetes)

The GPU governor runs on any Linux machine with NVIDIA GPUs:

```
# watch: decides and logs, writes nothing
sudo python3 -m omni_controller.gpu_governor --mode watch --gpus 0,1,2,3 --audit /var/log/omni/gpu.jsonl
└─ \
  --latency-file &lt;feed>; --slo-ms &lt;p95 target>;
# cap: writes the power limits
sudo python3 -m omni_controller.gpu_governor --mode cap --gpus 0,1,2,3 --audit /var/log/omni/gpu.jsonl
└─ \
  --latency-file &lt;feed>; --slo-ms &lt;p95 target>;
# OFF
sudo touch /tmp/omni-gpu-kill
```

What it does, every `--interval` seconds (default 2):

- reads each GPU's own meter: power draw, temperature, utilization, power limit;
- the engine sets a power cap; the shield keeps it above `--min-share` × the start limit (0.70) and above draw × (1 + `--headroom`);
- a busy card (smoothed utilization at or over `--util-gate`, 0.5) gets its full limit back at once;
- a response-time breach or a failed reading returns the start limit at once.

Speed lock (optional). Keep every response-time gauge at least `--speed-gain` (1%) faster than without Omni-Compass, and spend any speed won elsewhere on watts:

```
python3 tools/gpu_baseline.py baseline.json native-run/latency.csv # a run without Omni-Compass, other days
└─ er days
sudo python3 -m omni_controller.gpu_governor --mode cap --baseline-file baseline.json --latency-file &lt;feed>; ...
└─ ed>; ...
```

Prove it on your card: `sudo bash scripts/gpu_paired.sh` runs native, watch and Omni-Compass back to back on the same machine and prints the table and verdict (`docs/GPU_RUN_GUIDE.md`).

Pass condition: work per energy up, and requests served and p95 inside the guardrails.

Level 6. CPU clock and power

Add to the Kubernetes controller, on bare metal:

```
--cpufreq-policy-root /sys/devices/system/cpu/cpufreq --cpufreq-require-schedutil \
--rapl-cmd "&lt;prints CPU package watts>;"
```

The CPU frequency ceiling follows the engine's cap inside the nervous system's envelope; the OFF switch writes every policy's recorded maximum back exactly. GPUs can be wired the same way from the controller:

```
--gpu-query-cmd "nvidia-smi" --query-gpu=power.draw,temperature.gpu
--format=csv,noheader,nounits" --gpu-power-cmd "nvidia-smi -pl {w}" --gpu-max-w <max>.
```

Pass condition: energy per unit of work down, no service gauge worse.

Level 7. Site power, cooling, batteries

Organ	Switch	Status
site power stress	--power-cmd "<prints site watts>" --site-limit-w <limit>	wired: power stress enters the engine; batch pace and caps respond
cooling setpoint	--cooling-cmd "<sets {c}>" --cooling-min-c 18 --cooling-max-c 27 --cooling-restore-c 22	wired: warmer supply air while cool, colder as heat rises
CPU + GPU on one power budget	hardware/node_exchange.py (the conveyance law over CPU and GPU organs)	simulation only. The live levers exist (levels 5 and 6); the exchange between them has not run on hardware
GPU groups sharing a site budget	hardware/site_exchange.py	simulation only
on-site batteries as an organ	designed (docs/DOMAIN_MAP.md)	not built

5. Stack by stack

Stack	Levels available	Notes
Vanilla Kubernetes, kind, Rancher	1-6	as written
Amazon EKS	1-5	nodes via Cluster Autoscaler node group or Karpenter NodePool (level 3 table); CPU power control is not exposed on EC2 VMs; GPUs on bare-metal or full-GPU instances
Google GKE, Azure AKS	1-5	as EKS, with the provider's node-pool size as the park/wake command
Red Hat OpenShift / OKD	1-5	OpenShift is Kubernetes underneath; grant the same permissions through a Role; machines via the worker MachineSet
NVIDIA GPU servers without Kubernetes	5	the GPU governor alone
Bare-metal CPU servers	6	cpufreq and RAPL through sysfs
Slurm / HPC schedulers	5 on the GPU nodes	a Slurm connector for job-level decisions is not built
Building management (cooling, power meters)	7	through your BMS's command line or API, wrapped in the command templates

6. Proving it on your own system

1. **Paired runs.** Run your service the same way twice, once as you run it today and once with Omni-Compass on top, back to back on the same machines, order rotated, at least 5 times (10 for a result you publish). scripts/kind_paired.sh and tools/live_reps.py do this and print each gauge with its 95% interval. A change is proven only when its interval excludes zero.
2. **The switch drill,** after every run: turn Omni-Compass OFF, confirm every setting is back at its recorded original and no omnicompass.io/* annotation remains, turn it ON.
3. **Read the results** with docs/METRICS_CATALOG.md: every gauge says whether it is measured or modelled.

7. Reading the log

Line	Meaning
gate: a sense is blind	a reading failed; nothing is given back until it returns
gate: pods scaling up	pods first, machines after
decision failed (n in a row)	the cluster could not be reached; nothing was written. Turn it OFF for native at once
convey: <machine> idle CPU to its k serving pod(s), limit c	that machine's idle CPU now reaches the work on it
convey: response time calm, operator's limit	the pods are back at your own CPU limit
busy gate: utilization u, the limit read at start	a busy GPU got its full power limit back
speed lock: worst ratio r vs line 0.99 (spend / hold / release)	the speed lock's reading and what it did
response-time reflex: the limit read at start	response time went over target; full power at once
kill switch: the limit read at start	the OFF switch restored this setting

8. What is proven, and how

Claim	Evidence	Kind
On real Kubernetes: faster responses, fewer replicas and machines in service	results/live/LIVE_REPS_20.md, LIVE_REPS_21.md (10 paired runs each)	measured on kind
Energy on Kubernetes	declared model on kind, no meter	modelled
GPU: more work per energy within the speed guardrail	results/gpu/sim/after	modelled (MLPerf-calibrated card); a real-card run is pending
CPU and GPU on one power budget: more work, never over the budget	results/hardware/NODE_EXCHANGE_*.json	modelled
Safety: OFF switch restores everything; watch mode writes nothing	every live run's switch drill; verify.py	measured / checked

9. Python, C++ and the seal

Omni-Compass exists in two languages. The laws are twinned: each has a Python version and a C++20 version that give the same answers, proven by a parity test on every build.

Law	Python	C++	Proven by
core engine (six-state equations)	omnicompass/core.py	cpp/src/core.cpp	500 frozen fixtures
governor (allocation laws)	omnicompass/adapter.py	cpp/src/governor.cpp	tests/test_cpp_governor_parity.py
safety shield	omnicompass/shield.py	cpp/src/shield.cpp	tests/test_cpp_shield_parity.py (plus adversarial cases)
HPA replica law	fleet/harness.py	cpp/src/hpa.cpp	tests/test_cpp_hpa_parity.py
closure law (machines)	omnicompass/closure.py	cpp/src/closure.cpp	tests/test_cpp_closure_parity.py
conveyance law (the conserved budget)	omnicompass/conveyance.py	cpp/src/conveyance.cpp	tests/test_cpp_conveyance_parity.py (identical to the last bit)

Law	Python	C++	Proven by
nervous system (authority per organ, living band)	omniconpass/nervous_system.py	cpp/src/nervous_system.cpp	tests/test_cpp_twins_parity.py
compass and ledger (composite storage)	omniconpass/compass.py, omniconpass/storage.py	cpp/src/compass.cpp	tests/test_cpp_twins_parity.py
GPU governor rules (shield limit, busy gate, speed lock, window, baseline)	omni_controller/gpu_governor.py	cpp/src/gpu_rules.cpp	tests/test_cpp_twins_parity.py

The seal (results/SEAL.json) holds the SHA-256 fingerprint of every file of every twin, written only after all parity tests pass (python3 tools/seal.py). verify.py fails if any sealed file changes afterwards, and names it. So the Python and the C++ cannot drift apart unnoticed. What stays in Python is the plumbing that talks to Kubernetes, nvidia-smi and sensors (omni_controller/controller.py, muscles.py, the device I/O of gpu_governor.py) and the simulation harnesses; every decision they take goes through the twinned laws. The seal lists them as Python only.

Keep this manual with the code. When Omni-Compass changes, this manual, the C++ twin and the seal change in the same commit.

CHAPTER 33

Running the GPU Benchmark

There are three ways to get real-machine numbers:

- **Your own tower:** a machine you control, with a smart plug measuring the whole machine at the wall (section A).
- **GitHub's GPU machines:** they run the test from the repository with one click (section B).
- **A rented cloud GPU:** sections 1 to 6.

A. Your own tower, measured at the wall

You need:

- an NVIDIA graphics card (a GeForce RTX works);
- Linux on that machine (a spare drive or a USB boot is fine);
- a smart plug that reports watts, for example a Shelly Plus Plug or any plug running Tasmota, which cost about \$20 to \$30.

Plug the tower into the smart plug, and connect the plug to your home Wi-Fi with its own app. Note its IP address, shown in the app or on your router's device list. Then on the tower:

```
python3 tools/wall_meter.py shelly2:192.168.1.50 --once          # prints the tower's watts right now
sudo WALL_METER=shelly2:192.168.1.50 bash scripts/gpu_paired.sh # trial run
sudo WALL_METER=shelly2:192.168.1.50 PHASE=confirm bash scripts/gpu_paired.sh # the real test
```

Use shelly1: for older Shelly plugs and tasmota: for Tasmota plugs. The table then adds whole-machine energy at the wall and requests served per wall kilojoule. The plug is read by the test only; Omni never sees it, so its number is independent of Omni.

B. GitHub's own GPU machines

1. One-time setup, by an owner of the Omni-Compass organisation on GitHub:

1. Go to **Settings** → **Actions** → **Runners** → **New runner** → **New GitHub-hosted runner**.
2. Choose the image **NVIDIA GPU-Optimized Image for Linux** and a GPU size.
3. Name its label `gpu-t4`, or set the repository variable `GPU_RUNNER` to the label you chose.

GPU runners are billed per minute on paid plans.

2. **To run:** go to **Actions** → **gpu-bench** → **Run workflow**, then choose smoke or confirm.
3. **Results:** each repetition runs on its own GPU machine. The results are pooled into one table, shown on the run's summary page, and every raw file is committed back to the branch under `results/gpu/github-<run id>/.</code>`

If GitHub's machines don't allow changing the GPU power limit, the job stops in its first minute and says so.

C. A rented cloud GPU

This costs roughly \$10 to \$25 in rented GPU time. You type a handful of commands; the machine does the rest.

1. Rent the right kind of machine

The test changes the GPU's power limit, so you need a machine where you are the full administrator of the GPU:

- **Use a virtual machine (VM) or a bare-metal server.** Lambda Cloud "on-demand instances" are VMs; so are the GPU instances on AWS, Google Cloud and Azure.
- **Avoid "container" or "pod" rentals.** Their GPUs usually block power-limit changes. This is common on the cheapest per-hour marketplaces.
- **Any single NVIDIA data-center GPU works:** A10, L4, A100, H100, L40S. One GPU is enough.
- **Choose an image that already has PyTorch.** On Lambda that is the default "Lambda Stack" image.

You don't have to guess whether a machine allows it. In its first minute the test checks, and stops with *"cannot set the power limit (run as root)"* if the machine doesn't allow it. If you see that, shut the machine down (you pay only for the minutes used) and rent a different kind.

2. Connect to it

The rental site shows a command like `ssh ubuntu@123.45.67.89`. Paste it into Terminal (Mac) or PowerShell (Windows).

3. Get the code

The repository is private, so GitHub needs a key:

1. On github.com go to **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens**.
2. Create a token with **read-only** access to this one repository. Copy it.

Then on the rented machine:

```
git clone -b main https://&lt;YOUR-GITHUB-NAME&gt;:&lt;TOKEN&gt;@github.com/The-Omni-Compass-LLC/The-Omni-Compass om
  ↩ ni
cd omni
pip install numpy          # PyTorch is already on the machine
nvidia-smi                 # should show your GPU
```

4. The trial run (about 40 minutes)

This checks that everything works on your machine. It is never published.

```
sudo REPS=2 DURATION=300 bash scripts/gpu_paired.sh
```

At the end it prints a table. What matters here:

- It doesn't say **INVALID**.
- The *Omni governs* column shows power-limit writes.
- The watch arm shows zero writes.

5. The real test (about 6 hours)

The code is frozen, with 10 repetitions, exactly as fixed in docs/GPU_PREREGISTRATION.md. Don't change anything between the trial and this.

```
sudo PHASE=confirm nohup bash scripts/gpu_paired.sh &gt; confirm.log 2&gt;&amp;l &amp;
```

nohup ... & keeps it running if your connection drops. You can close the window and come back. To check on it:

```
tail -5 confirm.log
```

6. Bring the results back

When confirm.log ends with the table:

```
sudo tar czf gpu_results.tgz results/gpu
```

Then, from your own computer (a new Terminal window, not the rented machine):

```
scp ubuntu@123.45.67.89:omni/gpu_results.tgz .
```

Then shut the rented machine down on the rental site, so the billing stops.

Before any machine: the simulated card

python3 tools/gpu_physics_sim.py --out results/gpu/sim/after runs Omni's own GPU governor, with its current settings, against a modelled card for 10 paired repetitions in about two minutes. No GPU is needed. Every number it prints comes from the card model, not from a meter, so it is a preview of what the governor does and never the result. results/gpu/sim/before holds the same run with the settings before the share floor and busy gate were added.

What you will get

A table from the GPU's own power meter:

- native against Omni watching only, and native against Omni governing;
- the preregistered primary result: work per energy, in requests served per kilojoule, with its 95% interval.

It ends with one verdict line, one of:

- **better, proven;**
- **worse, proven;**
- **not proven;**
- **better on energy, fails the service guardrail.**

Whatever it says is the answer, and it is the first number in this project that Omni-Compass's own code did not compute.

PART SIX

Operating It

The OFF switch, the rules the governor obeys, the log it keeps, and the care of a running installation.

CHAPTER 34

The OFF Switch, the Rules, and the Log

The rules Omni-Compass keeps on your system.

1. One OFF switch, in a human hand: the kill file (or OMNI_KILL=1, or SIGTERM to the GPU governor) returns every setting to its recorded original, reads each back, and stops all action.
2. It records before it acts (annotations on Kubernetes objects; the snapshot line in the GPU audit).
3. It watches before it writes.
4. It never acts blind.
5. Service first: while response time is over its target, and for --slo-clear decisions after, nothing is given back.
6. Every change is read back; no new order goes on top of one that has not landed.
7. The living band and the cover: no organ is driven outside its range; machines are parked, never switched off by Omni-Compass; it never evicts or moves a pod.
8. One writer: if anyone else changes a knob, Omni-Compass stops writing it and leaves it alone.
9. Everything is logged: every read, decision, write and reason goes to the audit log.

Reading the log.

Line	Meaning
gate: a sense is blind	a reading failed; nothing is given back until it returns
gate: pods scaling up	pods first, machines after
decision failed (n in a row)	the cluster could not be reached; nothing was written; turn it OFF for native
decided_by: bowl	the bowl set the wires this decision
decided_by: fail_up / blind_fail_up	the service crossed the 0.95 wall, or a sense went blind: full capacity at once
decided_by: thermal_hold	the card reported a heat slowdown; nothing was tightened
foreign_writer	someone else changed a knob; Omni-Compass now observes only
restored ... ok: true	the OFF switch put every setting back and read it back

CHAPTER 35

Maintenance, Upgrades and Security

- Run `python3 verify.py` after every upgrade; it must end `VERIFICATION: PASS`.
- Upgrade in watch mode first; take the levels again from level 1.
- The container runs non-root, read-only, with no capabilities; the Kubernetes identity has only the permissions of the level you run.
- The GPU governor needs root only for `nvidia-smi -pl` and `-lgc`.
- Report a security issue as `SECURITY.md` describes.

The Operator Manual

The current manual is docs/INTEGRATION_MANUAL.md (every stack, every level, every switch). This page is kept for its first-person account of the laws; where the two differ, the integration manual is current.

I am Omni-Compass. I am the brain and the nervous system; your stack is the muscle. Everything below is what I am and what I do, in my own terms and in my own mathematics. Every command and flag here exists in this repository.

I. My face

My face is the compass. It is not decoration: it is how I read myself, every decision, and it is written into my code (omnicompass/compass.py).

The wheel

My deviation E , over its ceiling $E_{\max}$, runs along the horizontal. Its rate of change runs along the vertical. Where I stand on that wheel is my heading, measured clockwise from north:

Point	Letter	Heading	Where I am
+	A	0°	rising through rest
↗	Δ	45°	expansion: above rest and rising, exchange under way
>	B	90°	peak extension: the turn, where my metric flips ($k \rightarrow -k$)
T	Λ	135°	dispersion: above rest and easing, my ceiling holds
–	Ω	180°	falling through rest
≈	Π	225°	compression: below rest, settling into my basin
<	Γ	270°	deepest compression: the turn at my floor
✦	Ψ	315°	re-alignment: below rest and rising, ignition ahead

My rim carries twenty-four letters, fifteen degrees apart, clockwise from A:

A E Z Δ H Θ B I K Λ M N Ω Ξ O Π P Σ Γ T Y Ψ Φ X

Every reading names its letter.

The four strokes

My quadrants are the strokes of the closed circle:

Quadrant	Stroke	Where I am
I	Expansion	above rest, rising
II	Dispersion	above rest, easing

Quadrant	Stroke	Where I am
III	Compression	below rest, falling
IV	Re-alignment	below rest, rising

The wheel turns $I \rightarrow II \rightarrow III \rightarrow IV \rightarrow I$:

- **Ignition:** crossing from IV into I.
- **Metric flip:** the turn at peak extension.
- **Continuity:** I am continuous through every crossing:

$$\lim X(t^-) = \lim X(t^+)$$

Nothing in me switches off, and nothing restarts.

- **Counting:** I count every circle I close.

The axle

My axle is the structural basin S . Its rest point S^* solves my equation (6):

$$\delta - \alpha_s S - \frac{3}{4} \beta_s S^2 = 0$$

Without the axle, deviation diverges. With it, deviation circulates.

Closing the circle

I hold the three conditions of the Unified Circle Principle and report them every decision:

- $\dot{X} = G(X)$, $X(0) \in \Omega$. Ω is my living band: every level I hand out lies between 5% and 95% of its range.
- $G(X) \cdot n(X) \leq 0$ on $\partial\Omega$. At a boundary, my next move points inward, never outward.
- $\nabla L(X) \cdot G(X) \leq 0$. My ledger $L = E^2/2 + \Phi(S) - \Phi(S^*)$ descends when nothing forces me. When outside load forces me, I say so; I never hide it.
- $\Rightarrow \lim X(t) \in M^*$. I settle into my basin.

A reading, as I print it on every decision trail:

```
Ψ + 318.2° Re-alignment: re-alignment, ignition ahead | Ω: machine_fill below the floor, returning | G·n
↳ ≤0 yes | L 0.4121 -0.0133 | axle S 0.19 (rest 1.90) | circles 3
```

II. My laws in your system

1. One switch, for all of me, in a human hand

I have one switch, the Unified Control Switch. A human turns it; I never turn it myself.

- **ON:** I am the primary control authority inside the scope you granted me.
- **OFF:** every setting I changed returns to what it was, and your native control takes full custody.

```
# OFF (all of me, at once, at any moment, for any reason, including a suspicion that someone has taken m
↳ y brain):
kubectl -n omni-compass exec deploy/omni-compass -- touch /tmp/omni.kill
# ON:
kubectl -n omni-compass exec deploy/omni-compass -- rm /tmp/omni.kill
```

What OFF restores. Every HPA CPU target and replica range, pod CPU limit, idle mark, paused rollout or job, GPU and CPU frequency ceiling, and containment quota. Each restore is recorded, and each lever restores on its

own.

What never trips the switch.

- The switch never turns one organ off.
- No boundary, no error and no failed decision trips it.
- A decision that fails writes nothing, and my next decision comes on time.

2. My living band: 5% to 95%

Every level I hand out lives inside $\Omega = [0.05, 0.95]$ of its range:

- frequency ceilings, GPU power limits and the site power envelope;
- each organ's share of the energy budget;
- how full I run any machine.

No part of the body is driven to zero: a part with no work idles at its floor, alive and ready. No part is driven to its absolute top: I never spend the last five percent.

Tested over 300,000 of my states and 60,000 of my energy allocations (tests/test_living_band.py).

3. Machines idle; I never switch them off, and I never move a pod

When I need fewer machines, I idle the rest by letting their work leave on its own:

1. **Prefer not:** the machine is marked `omnicompass.io/idle:PreferNoSchedule`. New pods go to the open machines first, but a pod that finds them full lands here at once, so no pod ever waits because of me.
2. **Marked:** its pods are marked first to go (`controller.kubernetes.io/pod-deletion-cost`), the machine with the least work first. When the load falls, your autoscaler's own scale-down removes exactly those pods, emptying one machine at a time.
3. **Idle:** once its work is gone, the machine stays powered and Ready, gauged down to its idle floor.

While it still carries work, a machine counts as in service at full power. No pod is ever evicted, moved or restarted to idle a machine. When work returns, I remove the mark, the warm machines still carrying work first; it is in service at once, with no boot and no power cycling.

An idle machine draws `park_frac × idle power` (0.25), never zero.

4. Every change is continuous

- I idle at most one machine per decision.
- I wake a machine within five seconds of a pod waiting for a place.
- My pod reflex reads the queue every five seconds.

Nothing jumps, and nothing is restarted.

5. My pod sense: the muscle makes the pods

Your autoscaler is the muscle that makes and removes pods. I never start or stop a pod it would not.

Its rule is $\text{replicas} = \text{current} \times \text{busy} \div \text{target}$. I read the same rule from the live queue every five seconds:

- a replica serving requests answers in $R = S / (1 - u)$, so $u = 1 - S/R$;
- S is the bare service time: the fastest tenth of the recent requests;
- R is the recent mean response, over the same window;

- your target, in queue terms, is $\text{target} \times \text{request} \div \text{limit}$.

I record what the queue needs (`pod_reflex_reading`) and act only through the energy I give the pods and the target I hold for the muscle.

The target I hold. g is the CPU each pod is guaranteed with your autoscaler's largest count spread over the machines in service, divided by your limit. That target moves only when a machine idles or wakes, never each time a pod starts or leaves.

The muscle's own clock. I hold each target for your autoscaler's scale-down window (300 s unless you set one), the time it takes to answer a target. A target moved faster would pull the muscle mid-movement and start pods it then removes. A response-time breach returns your own target at once.

The reflex that raises the floor itself exists (`--pod-reflex-writes`); it is off unless you turn it on.

6. *My energy is moved, never created*

One budget comes in, and I convey it across the body by need (`omnicompass/conveyance.py`):

- $da_i/dt = \kappa a_i (e_i - \bar{e})$;
- the budget is conserved exactly, and each organ converges to its share of the demand;
- organs with no work idle at their floor, and what they do not need goes where it is needed;
- nothing leaves Ω .

On each machine, I hand its idle CPU to the pods serving on it:

$$c_i = \min(\max(L_i, (0.95 A_j - Q_j) / |P_j|), 0.95 A_j)$$

- A_j is the machine's CPU.
- Q_j is what every other pod on it has requested.
- P_j is its serving pods.
- L_i is the limit you gave the pod.

A pod's CPU limit is a quota. A request that needs more than one quota period waits for the next one while the machine stands idle. That wait is energy withheld from the work, not saved, because the request spends the same CPU-seconds either way.

I change the limit in place: the pod is not restarted, its request is untouched, and it never gets less than you gave it. Every fifteen seconds a new pod gets its share. The OFF switch returns every pod to your limit.

7. *I see before I act*

My nervous system runs both ways:

- **Afferent:** a sense that is stale, frozen or unreadable is blind, and while any sense is blind I give nothing back.
- **Efferent:** I read back every order I give. An order that did not land blocks my next release.

III. Wiring me in, step by step

Take the steps in order. Take the next step only when this step's pass condition holds. The switch works at every step.

Step 0. *What your stack needs*

1. Kubernetes 1.34 or newer, with metrics-server (`kubectl top nodes` answers).

2. An HPA with a CPU target on each service I govern.
3. A PodDisruptionBudget on each service, for your own maintenance; I never evict a pod.
4. A readiness probe and a short preStop pause on each service, so a new pod never takes traffic before it answers and a pod your autoscaler removes never drops a request as it leaves (deploy/kind/demo.yaml).
5. A response-time feed for each governed service, as CSV elapsed_seconds, latency_ms, ok. scripts/latency_probe.py writes one.

Step 1. Build me

```
docker build -f deploy/Dockerfile -t <registry>/omni-compass:<tag> .
docker push <registry>/omni-compass:<tag>
```

I run as non-root, with a read-only root filesystem and no Linux capabilities. I carry my engine, my nervous system, my compass and my frozen law.

Step 2. Let me watch (monitor only)

```
kubectl apply -f deploy/install/omni-compass.yaml # set the image line; --mode observe
kubectl -n omni-compass logs deploy/omni-compass -f
```

Identity. I hold a read-only identity. scripts/pilot_shadow.sh records kubectl auth can-i receipts showing I cannot write.

Pass condition. Zero writes, and readings your operators agree with.

Step 3. Give me the pods (on top of your autoscalers)

1. Grant patch horizontalpodautoscalers and patch pods/resize (deploy/rbac-target.yaml, deploy/kind/rbac-omni.yaml).
2. Run me with --mode target --latency-file <feed> --slo-ms <your p95 target>.

Your HPAs keep scaling. My pod reflex raises floors ahead of the CPU averages.

Targets. I hold your promise in queue terms: $\text{busy} = \text{target} \times \text{request} \div \text{limit}$.

- g is the CPU each pod is guaranteed (section II.5), divided by your limit; the target that keeps each pod exactly as busy is g times yours.
- The pod answers faster, because it has g times the CPU, at the same busy share.
- Apart from that, I only tighten, never loosen.
- While response time is over your target, and for three decisions after, your own target stands.

Pass condition. p95, p99 and failed requests no worse than native.

Step 4. Give me the machines

1. Grant patch nodes and patch pods for the first-to-go mark (deploy/kind/rbac-omni.yaml).
2. Run me with --mode nodepool --active-nodes-only --closure /app/law/closure.json --node-scale-cmd "<park/wake command with {n}>".

Your platform	The park/wake command
any cluster, kind, bare metal	bash scripts/kind_nodepool.sh {n} (close to new work and mark first to go; open again, warm machines first)
Karpenter / EKS Auto Mode	the NodePool CPU limit at {n} × node CPU, parked nodes kept, not consolidated away
Cluster Autoscaler node group	the group's desired size, with scale-down through parking, not deletion

Your platform	The park/wake command
OpenShift	the worker MachineSet replicas, parked, not deleted

I give a machine back only when all of these hold:

- every sense is live;
- my last order landed;
- pods are not scaling up;
- nothing is waiting for a place;
- the machines that remain stay inside Ω .

Pass condition. Fewer machines in service and less energy, with no service gauge worse.

Step 5. Let me decide alone (Kubernetes as the muscle only)

Add `--strict-replicas`:

- I decide each service's replica floor and when to shrink;
- your HPA stays as the fast up-reflex;
- my node law sizes the machines.

Pass condition. As step 4.

Step 6. Give me the hardware

Organ	How to wire it
CPU frequency	<code>--cpufreq-policy-root /sys/devices/system/cpu/cpufreq</code> <code>--cpufreq-require-schedutil --rapl-cmd "<prints package watts>"</code>
GPU	<code>--gpu-query-cmd "nvidia-smi --query-gpu=power.draw,temperature.gpu</code> <code>--format=csv,noheader,nounits" --gpu-power-cmd "nvidia-smi -pl {w}"</code> <code>--gpu-max-w <max></code>
Power and cooling	<code>--power-cmd "<prints site watts>" --site-limit-w <limit></code> <code>--cooling-cmd "<sets {c}>"</code>
Batch and rollouts	<code>--batch --batch-pace --rollout-guard <ns/deployment></code>

Pass condition. Energy per unit of work down, with no service gauge worse.

Step 7. The switch drill, at every step

1. Turn me OFF.
2. Confirm every setting is back to its recorded original, and no `omnicompass.io/*` annotation remains.
3. Turn me ON.

`scripts/kind_bench.sh` does exactly this after every live run.

IV. How to see me work

- **Live, paired, on real Kubernetes:** `benchmark-reps.yml` (a commit with `[reps]`, or by hand). Each repetition runs native, me on top, and me alone, back to back on one machine.
- **On your own machine:** `bash RUN_LIVE.sh 3`.
- **Every run leaves:**
 - my audit log of every read, write and compass reading;

- my decision trail;
- my identity receipts;
- a SHA-256 fingerprint of every file.

V. When you read these lines

I say	I mean
gate: a sense is blind	I cannot see, so I give nothing back until I can
gate: pods scaling up	pods first, machines after
decision failed (n in a row)	I could not reach the cluster and wrote nothing; turn me OFF if you want native now
pod_reflex_reading	what the queue needs now; the autoscaler decides the pods
convey: <machine> idle CPU to its k serving pod(s), limit c	that machine's idle CPU now reaches the work on it
Ω: machine_fill below the floor, returning	the machines are underfilled and my move is bringing them back into the band

PART SEVEN

Proving It

Paired runs, receipts, rules written before the runs, and every result to date with its evidence class.



Plate 19. Empirical validation, experimental pathways, and the discipline of proof

Paired Runs and Receipts on Your Own System

1. **Paired runs.** Run your service the same way twice, once native and once with Omni-Compass on top, back to back on the same machines, the order rotated, at least 5 times (10 for a result you publish).
`scripts/kind_paired.sh` and `tools/live_reps.py` do this on Kubernetes; `scripts/gpu_rented_run.sh` does it on a GPU box, in one command: wire check, smoke, the six organisms with the card inside, then the preregistered confirmation.
2. **The switch drill** after every run: turn Omni-Compass OFF; confirm every setting is back at its recorded original; turn it ON.
3. **The receipt.** Each gauge: native, Omni-Compass, the change, the 95% interval of the difference, and whether the interval excludes zero. A change is proven only when it does.
4. **The label, by rule written before the run:** SUPERIOR WITHIN GUARDRAILS / ENERGY IMPROVEMENT WITH SERVICE TRADEOFF / NONINFERIOR / NOT ESTABLISHED / WORSE / INVALID. Guardrails: work not lower by more than 1%; the share of time outside the service line not higher by more than 1 percentage point. The band-first rule is stricter: no win is claimed while the time outside the service line is above native's.

At scale (simulated). The six organisms run on GitHub's machines (Actions, workflow `six`) or on any machine (`bash scripts/scale_ladder.sh`) at 1, 10, 100 and 1,000 paired runs and at 1, 10, 100 and 1,000 copies of each organism on one clock. Real Kubernetes runs on GitHub's machines (workflow `benchmark-reps`).

CHAPTER 38

Evidence Classes and How to Read a Result

Class	Rung	What it is	What it can show
T / V	E1	deterministic tests, proofs, Python against the C++ twin	the law is what it says, and both languages agree
S	E2	simulation on a modelled plant	whether the law helps the model, and where it breaks
L	E3	real software (Kubernetes on kind), no hardware meter	real decisions on real software; energy there is a declared model
P	E4	a physical meter (the GPU's own power reading)	the hardware's own answer

Read every number with its class beside it. A simulation number is never quoted as a hardware result. When a receipt's energy line is modelled, the receipt says so.

CHAPTER 39

Results to Date

Result	Class	Source
Real Kubernetes, set 24 (10 paired runs): machines in service -31.6%, p95 response -60.1%, p99 -64.1%, HPA replicas -38.6%, failed requests 0 on both, total CPU including Omni-Compass's own -1.8% (not significant)	L	results/live/LIVE_REPS_24.md (GitHub run 36983865216)
Real Kubernetes, set 23 (10 paired runs): p95 -62.2%, replicas -36.6%, machines in service -28.7%, failed requests 0	L	results/live/LIVE_REPS_23.md
Modelled GPU card: two-wire bowl +8.6% work per energy (seeds 5000-5009) and +9.0% (fresh seeds 5100-5109), time over the service line unchanged, but p95 response +33% and +37% against native; one-wire governor +0.1%; both wires restored every run	S	results/sim/gpu_two_wire/
Six organisms, 1,000 paired runs each at 1x and at 10x size: work per energy +0.30% / +0.29% (compute), +0.23% / +0.22% (physics), +0.21% / +0.20% (energy), +0.25% / +0.24% (distribution), +0.21% / +0.21% (four stacked), +0.22% / +0.22% (whole tower); every knob handed back; time over the service line +0.19 to +0.27 points above native in every cell, so the band-first rule is not yet met. 100x and 1,000x running	S	results/scale/GRID.md
Real GPU (NVIDIA A10), first confirmation, 10 paired runs, the card's own meter: work per energy +3.6% (proven), energy -3.5%, same requests, none lost, every write read back and restored; p95 response +58.5% worse, so the label by rule is energy improvement with service tradeoff. The cause was governor wiring, corrected (service profile, amendments 6-7); the corrected governor is next on a card	P	results/gpu/run-20261002T082232Z/GPU_REPS.md

The Pilot Protocol and Kit

Pilot Protocol

Phase 0: Simulation evaluation (evaluator's own environment)

Run `python verify.py`. Pass criterion: VERIFICATION: PASS.

Phase 1: Shadow (2 to 4 weeks)

- Capture: `OUT=capture.csv INTERVAL=15 DURATION=<seconds> POWER_CMD="<site power in watts>" bash fleet/capture/kube_capture.sh (read-only).`
- Replay: `python fleet/capture_replay.py capture.csv --idle-w <W> --dyn-w <W> --site-limit-w <W> --out replay/` gives Omni-Compass's recommended node count, power cap and HPA target per decision.
- Governor in OBSERVE on production telemetry read directly from nodes (kubelet/cAdvisor metrics, power meters, queue depth).
- No write credentials are issued.
- Logged per interval: directive, native actions taken, and the observed outcome.
- Pass criteria, agreed before start: logged directives never violate I1 to I5; counterfactual analysis on at least N recorded incidents shows the directive would have reduced time-to-recovery or energy without a backlog increase beyond an agreed bound.

Phase 2: Guarded control, one loop at a time (4 to 8 weeks)

Order: power capping; node count (Cluster Autoscaler set to observe); replica count (HPA set to observe).

- Each loop is handed over separately, with the kill switch tested at handover and at exit.
- Pass criteria per loop, fixed in advance: SLO attainment not worse than the preceding shadow baseline at the agreed confidence level; zero shield invariant violations; energy per unit of completed work reported with confidence intervals.
- Exit: any criterion failed triggers the kill switch and returns the loop to its native controller.

Scoring your own pilot

Capture the baseline (a period before the controller, or a matched node pool left on your normal autoscaler) and the Omni-Compass period or pool with `fleet/capture/kube_capture.sh`, then: `python pilot/score.py --baseline baseline.csv --omni omni.csv [--idle-w W --dyn-w W]` It reports node-hours and energy per used CPU core-hour, utilisation, pending-pod minutes and HPA shortfall minutes, each with a bootstrap 95% interval over hourly blocks. Energy is measured if the captures include `power_w`. Run long enough for at least 24 blocks per side, and compare matched pools at the same time where possible, because traffic changes between periods.

Phase 3: Component retirement

A decision component is retired only after its loop has passed Phase 2 and a one-at-a-time removal shows no degradation (keep-or-remove rule, Manual Chapter 6). Execution and security components are retained.

Reporting

All pilot metrics, including failures, are reported in the same format as the benchmark results.

Omni-Compass shadow pilot kit

What a customer runs first. Omni-Compass watches the cluster read-only beside its own autoscalers. Every 15 seconds it decides what it would do, using the same closure law that was benchmarked. It never writes.

1. `kubectl config use-context <the cluster>`
2. `DURATION=86400 IDLE_W=<watts per idle node> DYN_W=<watts at full load> bash scripts/pilot_shadow.sh`
 - It proves with `kubectl auth can-i` that the identity can read and cannot write, and stops if any write permission exists.
 - It captures telemetry and logs every recommendation to `shadow_out/audit.jsonl`.
 - It writes `shadow_out/SHADOW_REPORT.md`: node-hours used against node-hours Omni-Compass would have used, and the write count, which must be 0.
3. Scoring against a matched baseline: `python pilot/score.py --baseline baseline.csv --omni omni.csv` (bootstrap intervals over hourly blocks).

Guarded control follows `docs/PILOT_PROTOCOL.md`: one loop at a time, the kill switch tested at each handover.

The kit is exercised end to end on kind by the live-shadow workflow.

CHAPTER 41

The GPU Bench

This is the test that answers "does Omni-Compass save real energy?" with the GPU's own power meter. There is no model in it. Nobody has run it yet on real hardware for this repository; the first run is the first real-meter result.

What you need

- A Linux machine with one NVIDIA GPU (a rented cloud GPU works: A100, H100, L4, A10, RTX), the NVIDIA driver and `nvidia-smi`.
- Python 3 with PyTorch built for CUDA (`pip install torch numpy`).
- Root, because setting a GPU power limit (`nvidia-smi -pl`) needs it. Persistence mode on is recommended (`sudo nvidia-smi -pm 1`); the receipt records it either way.

Run it

```
git clone <this repository> && cd <it>;
sudo bash scripts/gpu_paired.sh          # 5 repetitions, 10 minutes per arm: about 3 hours
sudo REPS=5 DURATION=300 bash scripts/gpu_paired.sh  # 5-minute arms: about 1.5 hours
```

Options (environment variables): `REPS`, `DURATION` (s per arm), `DRAIN` (s after arrivals stop), `COOLDOWN` (s idle before each arm), `GPU` (index), `SAMPLE_MS`, `INTERVAL` (s between Omni decisions), `SLO_MS` (response-time target; default ten bare service times), `WORKLOAD_ARGS` (e.g. `--n 8192 --target-ms 80`).

Two phases

- `PHASE=smoke` (the default): look for faults and for an effect worth confirming. Any number of repetitions.
- `PHASE=confirm`: the preregistered test (`docs/GPU_PREREGISTRATION.md`), 10 repetitions. Omni's code must be committed; its files are hashed before the first arm and again after the last, and any change invalidates the run.

```
sudo PHASE=confirm bash scripts/gpu_paired.sh
```

The **primary outcome** is work per energy: requests served per kilojoule the GPU drew.

What it does

1. **Receipt.** GPU name, UUID, VBIOS, driver, kernel, host, persistence mode, power management, the enforced limit, default, minimum, maximum and current power limit, the workload's hash, the mechanism id, git commit (`receipt.json`). The current power limit is the **snapshot**; every arm must start and end at it. The bench refuses to start unless power management is Enabled (otherwise a written limit would not bind).
2. **Calibrate once.** The workload times its request at the snapshot limit and picks the request size so one request takes about 50 ms (`calib.json`). Every arm uses the same calibration.

3. **Three arms per repetition, order rotated**, each after COOLDOWN s idle:
 - **native**: no Omni process.
 - **watch**: Omni runs, reads the GPU and decides, and is forbidden to write. This is the control: it shows what the machine does with Omni present and silent. If it writes even once, the run is invalid.
 - **omni**: Omni writes the GPU power limit.
4. **The same work in every arm**. `tools/gpu_workload.py` sends one seeded stream of requests (fp16 matrix products) at 30%, 60%, 80%, 30%, 60% and 30% of the GPU's full-power capacity. Every arm gets the same requests at the same moments.
5. **Measured by the device**. `nvidia-smi` samples power draw, temperature, utilisation, the power limit, the **enforced** power limit and the clock-limit reasons every 200 ms for the whole arm (the fields the driver reports, listed in `smi_fields.txt`); RAPL CPU package counters are read at both ends where the machine has them. This is receipt C, the outcome: Omni never supplies it.
6. **Kill switch**. After the omni arm Omni restores the snapshot limit and reads it back. The script checks the limit after every arm.
7. **The table** (`GPU_REPS.md`): each gauge for native, watch and omni, and three paired contrasts with 95% intervals — observation (watch – native), authority (omni – watch), total (omni – native). If an interval includes zero, it says **not proven**. The result label is chosen by rule (`docs/GPU_PREREGISTRATION.md`, amendment 1): SUPERIOR WITHIN GUARDRAILS, ENERGY IMPROVEMENT WITH SERVICE TRADEOFF, NONINFERIOR / INCONCLUSIVE, NOT ESTABLISHED, WORSE, or INVALID. Also: actuator fidelity (receipt B), control effort and representation fidelity (receipt A). A meter that was not fitted prints UNAVAILABLE. The run folder holds every raw file and `SHA256SUMS.txt`.

What Omni does on the GPU (`omni_controller/gpu_governor.py`)

Every 2 s it reads the GPU and runs the Omni-Compass engine (the throughput law in `omnicompass/adapter.py`): load is GPU utilisation, power stress is draw over the snapshot limit, heat is temperature over 83 C, queue is response-time pressure. The engine's power cap becomes a power limit, inside hard rules:

- never below the GPU's current draw x 1.3, never below 0.70 of the snapshot (`--min-share`), never below the device minimum, never above the snapshot;
- a busy card (utilisation smoothed over decisions at or over 0.5, `--util-gate`) gets the snapshot limit back at once, and the cap returns only under 0.4;
- optional speed lock (`--baseline-file`, from `tools/gpu_baseline.py` on runs without Omni): the limit follows response time against that baseline, every gauge kept at least 1% faster (`docs/INTEGRATION_MANUAL.md`, level 5);
- no new write until the last one reads back from the device; every write is read back at once and recorded (requested, return code, read back, enforced limit, delay); the engine senses the enforced limit, the one the card obeys; a write the device refuses ends the arm (exit 4) and makes the run invalid; no clock locks are ever written;
- if it cannot read the GPU or the response times, the snapshot limit at once;
- if response time breaks its target, the snapshot limit at once, and for three decisions after;
- kill file or SIGTERM: the snapshot limit, read back.
- **one writer**: if the limit ever reads a value Omni did not write, Omni stops writing, leaves that limit alone, and exits 5 (the run is invalid);

- **heat fails up:** while the card reports a thermal or hardware slowdown, no lower limit is written;
- every decision names every rule that held the engine back (`blocked_by`) and the one that decided (`decided_by`).

Any workload plugs in through `WORKLOAD_CMD` (for example a vLLM or MLPerf inference harness), if it writes `latency.csv`, `requests.csv` and `summary.json` in `tools/gpu_workload.py`'s format and `SLO_MS` is given.

The table also credits each write: joules and requests against native at the same moments of the same stream, grouped by the rule that decided it. CPU energy comes from RAPL by domain: package and DRAM apart, psys never added, wrap undone. The card's own energy counter (NVML) and its ECC and retired-page counters are read at both ends of each arm.

Gauges

Gauge	From	Better
work per energy (served requests per kJ), primary	requests served / GPU energy	higher
energy, GPU (J)	power.draw integrated over the arm's window	lower
energy per served request (J)	the same, over requests served	lower
power, GPU mean (W)	energy / window	lower
requests served, not served	the workload's own record	more served, fewer not
response time mean, 95th, 99th percentile (ms)	arrival to finish of each request	lower
temperature, peak and mean (C)	temperature.gpu	lower
energy, CPU package (J)	RAPL counters, when present	lower

Testing the bench without a GPU

`python tests/test_gpu_bench.py` runs the whole script against a stand-in `nvidia-smi` (`tests/fake_gpu/`), with the workload's `--sim` mode. The stand-in has no real power physics, so its numbers mean nothing; it proves the script, the controls and the validity checks work.

CHAPTER 42

The GPU Preregistration

Written before any hardware trial. The confirmation run (PHASE=confirm) hashes this file with the code in `FREEZE.json`; a change to either after the run starts invalidates it.

Question

On one NVIDIA GPU serving a fixed, seeded request stream, does Omni-Compass holding the GPU power limit change the successful work done per joule measured by the device, compared with the device left at its own limit?

Fixed here, before any smoke trial:

- the number of confirmation repetitions (10);
- the primary outcome;
- the guardrails;
- the analysis.

Nothing seen in smoke may change them.

Two phases

1. **Smoke** (PHASE=smoke, any number of repetitions). Its purpose is to find faults in the harness, and to see whether the effect is large enough to be worth confirming. Smoke results are not published and are never reported as the result. After smoke, Omni's code may change.
2. **Confirmation** (PHASE=confirm). Omni's code is committed and frozen before the first trial. The script refuses to run if any frozen file has uncommitted changes, and the table marks the run invalid if a frozen file changes during it. No inspection, tuning or rerun between confirmation trials. If the confirmation fails, it is reported as failed; a new confirmation needs a new commit and a new run, and both runs are reported.

Design

- **Arms:** native (no Omni), watch (Omni runs, may not write), omni (Omni writes the GPU power limit).
- **Repetitions:** 10 in confirmation, each with all three arms back to back, order rotated. Repetitions may run on separate machines of one GPU type (REP_ONLY, the GitHub workflow); the three arms of a repetition always share one machine, so every comparison is paired within a machine.
- **Per arm:** 60 s idle, then the pinned workload for 600 s plus 30 s drain.
- **Workload:** `tools/gpu_workload.py`, calibrated once at the start power limit before any arm. The seed is 20260928, and the load phases are 30, 60, 80, 30, 60 and 30% of full-power capacity.

Outcomes

- **Primary:** work per energy = requests served / GPU energy (kJ). GPU energy is `nvidia-smi power.draw` integrated over the arm's window.
- **Secondary:**
 - GPU energy;
 - energy per served request;
 - requests not served;
 - response time: mean, 95th and 99th percentile;
 - peak temperature;
 - CPU package energy (RAPL), where present;
 - where a smart plug is fitted (`WALL_METER`), whole-machine energy at the wall and work per wall kJ. The plug is read by the bench only, never by Omni. An arm whose plug readings have a gap over 5 s has no wall number.

Analysis

- **Primary comparison:** omni against native, paired by repetition. Report the mean difference with a two-sided t 95% interval.
- **Result:**
 - The result is *proven better* if the interval lies entirely above zero.
 - It is *proven worse* if the interval lies entirely below zero.
 - Otherwise it is *not proven*.
- **Guardrails, fixed now:** a better primary result counts only if Omni did not buy it with the work. Both guardrails must hold:
 - **Requests served:** the 95% interval of (omni – native) must not reach below –1% of native.
 - **95th-percentile response time:** the interval must not reach above +10% of native.

If a guardrail fails, the verdict is *better on energy*, *fails the service guardrail*. That is a different product and is reported as such.

- **Watch against native:** reported as the cost of Omni being present. If watch differs from native on the primary outcome as much as omni does, the effect is not attributed to Omni's authority.

Amendment 1 (2026-09-28, before any hardware trial; no smoke or confirmation data exist)

Added before any data, to make the chain from engine to plant identifiable. The question, the arms, the primary outcome, the repetitions, and the two guardrails above are unchanged.

- **Three contrasts, all reported, each paired by repetition:** observation = watch – native; authority = omni – watch; total = omni – native (the primary comparison). A total effect is not attributed to Omni's authority where the observation contrast differs materially from zero.
- **Third guardrail, errors:** the 95% interval of (omni – native) requests *not* served must not reach above +1% of native requests served.

- **Result label, by rule, never by hand** (`tools/gpu_reps.py, label()`):
 - primary proven better, all three guardrails held: **SUPERIOR WITHIN GUARDRAILS**;
 - primary proven better, a guardrail failed: **ENERGY IMPROVEMENT WITH SERVICE TRADEOFF**;
 - primary not proven, all guardrails held: **NONINFERIOR / INCONCLUSIVE**;
 - primary not proven, a guardrail failed: **NOT ESTABLISHED**;
 - primary proven worse: **WORSE**;
 - any invalidity below: **INVALID**.
- **The card obeys enforced.power.limit.** The snapshot records `power.limit`, `enforced.power.limit`, `default`, `min` and `max` limits, `persistence mode` and `power management`. The bench refuses to start unless `power management` is `Enabled`. The bench samples `enforced.power.limit` where the driver reports it. The governor senses the enforced limit and reads back every write at once; it never writes clock locks.
- **Also invalid:** a native or watch arm whose enforced limit differs from the snapshot; a governor that exits nonzero (a refused start, or a write the device refused — that write ends the arm); smoke and confirmation repetitions mixed in one table.
- **Three receipts, kept apart.** A (governor, `audit.jsonl` decisions): telemetry consumed; the six-state reading (history-dependent, `state_observed`); the memoryless state the telemetry alone points to (`state_measured`); the U-channel command `u` evaluated on the evolved state (`u_push`); admissibility; requested and granted authority; shield bound; holds. B (actuator, actuator records): requested limit, return code, `power.limit` read back, enforced limit, delay to realization, override (enforced under requested), restoration. C (outcome, the bench alone): `nvidia-smi power`, joules, temperature, utilization, clock-limit reasons; the workload's requests, latency and failures. Omni never supplies its own outcome.
- **Secondary, descriptive (no verdict):**
 - actuator fidelity: `r_act` = read back – requested (mean and max absolute), delay (median, max), enforced-under-requested count, total variation of the realized limit, reversals, refused writes;
 - control effort: $J_u = \text{sum of } \text{abs}(u_push) \times \text{decision interval}$; mean and max `abs(u_push)`; saturations (`abs(u_push) = 25`); shield interventions (a bound other than the engine decided); holds;
 - representation fidelity: $R_{int} = \sqrt{\text{mean over consecutive decision pairs of } \sum_k w_k (h(z_{(k+1)}) - F_h(h(z_k), u_k))_k^2}$ over `E`, `U`, `I_U`, `S`, `B`, weights 1, with `h` the memoryless map and `F_h` the engine's own projection (`state_projected_next`); directional accuracy = share of pairs where `sign(projected – current reading)` equals `sign(next measured – current measured)`, both movements at least 0.01, per state and by predicted size (0.01–0.03, 0.03–0.1, ≥ 0.1);
 - energy of the rest of the machine = wall – GPU – CPU package, only when all three meters measured the arm. A missing meter prints `UNAVAILABLE`, never a modelled substitute.
- **Smoke never enters confirmation.** A smoke run that changes any code or parameter is followed by: repair, the tests, a new freeze, and a confirmation collected from zero.

The run is invalid, and reported as invalid, if

- the watch arm executes any power-limit write;
- a native or watch arm sees a power limit other than the snapshot;
- any arm ends at a limit other than the snapshot (the kill switch failed);
- Omni's frozen files change during the run;

- the confirmation runs on uncommitted code.

Recorded for every Omni decision

Every decision is logged in `audit.jsonl`:

- the raw device telemetry: utilisation, draw, temperature, limit, SM clock, and clock-limit reasons where the driver reports them;
- the engine's six-state reading, the state it had projected for this moment, and the error against it;
- the projection for the next decision. This is the engine's own evolved state from the same step that sets the cap; no separate predictor was added for the experiment;
- whether change was admissible, the requested and granted authority, and which shield bound decided the limit;
- the limit written, and the requests served in the window.

Amendment 2 (2026-09-28, before any hardware trial; no smoke or confirmation data exist)

Unchanged:

- the question;
- the arms;
- the primary outcome;
- the repetitions;
- the guardrails of amendment 1.
- **Watch must match native.** If the observation contrast (watch – native) on the primary outcome is proven in either direction, the label is **NOT ATTRIBUTABLE: WATCH DIFFERS FROM NATIVE** and no omni result is published.
- **One writer.** If the power limit ever reads a value that is neither Omni's last write nor the limit before it, another writer is present:
 - the governor stops writing for the rest of the run;
 - it leaves that writer's limit alone;
 - it exits 5;
 - the run is invalid.
- **Heat fails up.** While the device reports a thermal or hardware slowdown (clock-limit reason bits 0x8, 0x20, 0x40, 0x80), no lower limit is written.
- **Credit per write (descriptive).** Each write owns the interval to the next. For that interval, the table records GPU joules and requests finished, Omni minus native, at the same moments of the same seeded stream. It records who decided the write: the engine, a floor, the busy gate, a reflex, heat, or the speed lock. The table says which rule produced the joules; a speed-lock result is not credited to the engine.
- **CPU side (secondary, never on the control path).** RAPL counters are read by domain name at both ends of each arm:
 - package-N is summed as CPU package; dram is summed separately;
 - psys is recorded and never added to either;

- wrapping is undone with `max_energy_range_uj`;
- a counter that went backwards without a known range, or is missing, prints `UNAVAILABLE`.
- The governor never reads these counters.
- **Device energy counter (cross-check, secondary).** Where NVML reports it (Volta and newer), the card's total-energy counter is read at both ends of each arm, beside the integrated power.draw.
- **Card health (descriptive).** Uncorrected and corrected ECC error counts, and pages pending retirement, are read at both ends of each arm.
- **Workload plug.** Any workload may be served through `WORKLOAD_CMD` if it writes the pinned workload's files. It needs its own response-time target (`SLO_MS`). The command is recorded in the receipt. A confirmation names its workload before the first trial.

Amendment 3 (2026-10-01, before any hardware trial; no smoke or confirmation data exist)

No GPU job has ever been given a machine (every `gpu-bench` run so far waited in the queue and was cancelled), so no trial data exist. Unchanged:

- the question;
- the arms;
- the primary outcome;
- the repetitions;
- the guardrails, labels and invalidity rules of amendments 1 and 2.
- **Declared envelope, before any trial.** The buyer's service envelope is written to a file before the first trial and recorded with the run (`envelope.json` in the run folder and in every repetition):
 - `power_min_w`, the lowest watts Omni may set. Default (`tools/declare_envelope.py`): `max(device minimum, 70% of the power limit read at declaration)`, rounded up to whole watts;
 - `power_max_w`, the power limit read at declaration;
 - optionally `slo_ms`, the response-time target; without it the target comes from calibration (10 bare service times), as before.
 - The bench refuses an envelope whose floor lies outside [device minimum, starting limit].
 - **The confirmation refuses to start without a declared envelope** (`scripts/gpu_paired.sh`, `ENVELOPE`).
- **Envelope floor.** The governor never sets the limit under `power_min_w (--floor-w)`. When the floor is what lifted a write, the decision record names it (`decided_by: envelope_floor`), so no saving below the floor can be credited to the engine.
- **Outer controller holds.** When the card's own controller (board, BMC or system policy) already holds enforced.power.limit under the current limit, a lower write that would still sit above that enforced limit changes nothing on the card. It is not written; the decision record marks it (`outer_controller_holds`). Only a write that would actually bind, or a return upward, goes out. This is not another writer (amendment 2): the set limit is unchanged and the governor keeps running.
- **Narrow cards are reported as they are.** The device's own limit range is in the snapshot. On a card whose range is narrow (for example a 70 W card that accepts 60–70 W), the envelope is that narrow range; the result is reported for that card and range and not extrapolated to wider cards.

Amendment 4 (2026-10-02, after an invalid smoke; no confirmation data exist)

The first smoke on a rented A10 (results/gpu/smoke-20261002T032459Z, never counted) was invalid by rule: two copies of `scripts/gpu_rented_run.sh` had been started on the same machine, so both benches wrote the same card's power limit. The native and watch arms saw limits they never wrote (116, 137 and 150 W), each governor refused to start beside the other one (exit 5), and one kill-switch restore was undone by the other copy. The card also began at 116 W, a limit an earlier start had left behind, not its 150 W default, so native itself ran capped (83% of samples). None of these numbers measures Omni. The engine, the governor, the outcomes and the analysis are unchanged. The run script now:

- **runs once per machine:** it takes a lock and refuses to start while another copy runs;
- **runs alone on the card:** it refuses to start while any other process is using the GPU;
- **starts from the card's default limit:** it sets `power.default_limit` before the envelope is declared, so the envelope, the snapshot and every arm start from the card's own default, not from a limit an earlier run left behind.

Amendment 5 (2026-10-02, before any valid hardware trial; the only smoke so far was invalid, amendment 4)

The Omni arm changes engine. The outcomes, the arms' order, the guardrails, the analysis and the validity rules are unchanged.

- **The Omni arm holds two wires** (`omni_controller/gpu_bowl.py`, the bowl law of `omnicompass/bowl.py`): the clock ceiling (`nvidia-smi -lgc`, reset with `-rgc`; cover 35% of the top clock to the top), which sets how high the card's own boost may climb, and the power limit (`-pl`), the lid at what a fully busy card draws at that ceiling plus 10%, never under the declared envelope floor and never over the start limit. The service is read as one position between calm and the response-time line (the worse of p95 and utilization above half) and pulled to the middle; past 95% both wires go to full at once (fail up). The card's firmware keeps its own control; Omni sets only those two values. The earlier power-limit-only governor stays available (`OMNI_ENGINE=one_wire`) and is not the confirmation's arm.
- **Why, before the run:** on a modelled card (`results/sim/gpu_two_wire/`, evidence class S, seeds never used while tuning) the one-wire governor gave +0.1% work per energy and the two-wire engine +9.0%. That is a model; this run is the card's own meter.
- **The watch arm** runs the same two-wire engine in watch mode: it computes and records both wires and writes neither.
- **The clock range is reset before and after every arm** (`-rgc`), as the power limit already was; the run script resets it once at the start.
- **The wire check runs first** (`tools/gpu_wire_check.py`): the card's clock must follow a lowered ceiling down and come back up when reset, the power limit must read back what was set, the governor must hand both wires back when stopped and must leave a limit set by another writer alone (exit 5). If any step fails, nothing else runs and the check's report names the wire, the step and what the card said.
- **Wire check, corrected before any trial (2026-10-02):** on the A10 the first wire check failed at "3 up wire (follows up)" although the wire works: under the heavy check load the card's own 150 W limit already held it near 990 MHz, so a ceiling at 60% of the top clock (1017 MHz) left no room for the clock to come back up above it. The check now reads the card's busy clock on its own first and locks at 60% of that. The governor likewise starts its ceiling at the clock the busy card actually runs (a ceiling above it holds nothing), and its

clock cover is 35% of the top clock to the top. No trial had run.

Amendment 6 (2026-10-02, after the first confirmation and before any further trial)

What the first confirmation showed (A10, commit c908054, results/gpu/run-20261002T082232Z/): work per energy +3.6% (+2.7% to +4.5%, proven), the same requests served, no request lost, but the 95th-percentile response time +58.5% (510 to 809 ms), so the response-time guardrail failed and the label by rule was ENERGY IMPROVEMENT WITH SERVICE TRADEOFF. It stands as the result of that run.

Why, from the card's own samples: while busy the card ran at 736 to 768 MHz under Omni against 861 to 889 MHz on its own, and spent about 30% more time busy for the same work; requests queued behind each slower one. Three faults in the governor, not in the engine: (1) the position counted utilization above half as service trouble, so every burst read as past the wall (fail up in 46% of decisions) and every quiet gap pulled the ceiling down, so each burst began on a lowered clock; (2) the lid followed a curve from the top clock and sat at the 105 W envelope floor in 49 of 165 bowl decisions, under the 135 W the card itself draws while busy; (3) the ceiling's cover reached 35% of the top clock, far under the clock the card's own power limit holds it at while busy.

The Omni arm from now on (omni_controller/gpu_bowl.py; the outcomes, arms, guardrails, analysis and validity rules are unchanged):

- the position is response time only (p95 over 5 s, not 30 s); being busy is not a breach;
- **race while work waits:** at 95% utilization or more the ceiling goes to the top and the lid to the start limit; the bowl paces only the slack between bursts;
- **the card's own level, learned from its own meter** while the ceiling is at the top and the card is busy: its busy clock (median) and busy draw (90th percentile); until 15 such readings are in, neither wire moves;
- **speed floor:** the ceiling never goes under the card's own busy clock; **lid floor:** the lid never goes under the card's own busy draw plus 10%;
- fail up (past 95% of the line, or blind) is unchanged.

On the modelled card, before any trial (results/sim/gpu_two_wire/, seeds 5000 to 5009): p95 122.1 ms native, 123.7 ms with the corrected bowl; work per energy +8.2% (+6.3% to +10.1%); energy -7.5%; the median response 10.1 to 12.1 ms, slower in the quiet stretches the bowl paces. That is a model; the next trial is the card's own meter.

Amendment 7 (2026-10-02, before any further trial)

The outcomes, arms, guardrails, analysis and validity rules are unchanged. The Omni arm of the confirmation runs the **service** profile.

- **Two profiles, one switch** (--profile): **service** (the default and the confirmation's arm), down gain 0.01; **batch**, down gain 0.02, for work nobody waits on answer by answer. The batch profile may be run as a separate, declared confirmation (OMNI_ARGS="--profile batch"); it is reported as its own result, never pooled with service.
- **Why 0.01, chosen on the model before the trial** (results/sim/gpu_two_wire/, 20 paired seeds, 5000-5009 and 5100-5109): of the down gains 0.005, 0.0075, 0.01, 0.0125 and 0.02, 0.01 is the largest at which no seed's p95 was more than 10% slower than native (0 of 20; 7 of 20 faster): work per energy +3.9% (+3.0 to +4.7), energy -3.7%, p95 -1.6% (-4.8 to +1.6), p99 -0.8%, time over the line -0.02 pp, median +10.5%. At 0.0125 and 0.02 the saving is +6.2% but 3 of 20 seeds' p95 was more than 10% slower (a burst arriving while the clock

rests on its floor).

- **The ceiling moves in whole clock steps** (--min-change-mhz, 15 MHz), as the card's own clock does and as the model moves it: the force times the gain, as a share of the top clock, is rounded to whole steps, and a pull under half a step moves nothing and is not stored up.
- **The position reads the mean response time of the window** (as the model does), between the bare service time and the line; the 95th percentile at or past the line, or any failed request, is past the wall (fail up).

CHAPTER 43

The Realms Preregistration

Round 3 is the current one: its section at the end changes only how the realms are made up. Round 2's section replaced round 1's Omni layer. Round 1 below is kept as it was frozen. Each round was written and committed before its confirmation seeds were run. Evidence class **S**: every number the run produces comes from a declared model. Nothing here is a meter, and nothing here is evidence about a real machine.

Question

For each of the 656 muscles of the canonical tower, and for each realm and the whole tower run as one organism: does the frozen governor (`omnicompass.adapter.Governor`, the engine with $u = 0$, the stack law, the kill switch), holding that muscle's one knob on top of the plant's native controller, change work per energy against the native controller alone, without buying it with service?

What runs

- **The catalog** (`realms/catalog.csv`, built by `tools/realms_catalog.py`): the 656 rows of the XPASS package's canonical tower, unchanged, plus four columns given by fixed rules: realm, plant, parameter set, knob.
- Compute / AI / Cloud: 250 muscles. Physics / Robotics / Autonomous: 88. Energy / Facility / Industrial: 121. Distribution / Specialized: 197.
- **Five plants** (`realms/plants.py`), each with its own native controller, all parameters in `realms/presets.py`:
 - `compute_pool`: request stream, servers with start-up delay, idle and dynamic power; native: the HPA rule (10% tolerance, scale-down stabilisation window), fixed admission limit, full clock;
 - `thermal_zone`: zone heat balance, staged cooling units, COP from supply and outdoor temperature; native: PI on a fixed setpoint, units staged to the load;
 - `energy_storage`: site load, solar, battery, grid connection with a contract limit; native: self-consumption above a fixed reserve;
 - `motion_axis`: point-to-point moves from a task queue, motor copper losses and heating, derating; native: PID with feedforward at full speed;
 - `process_loop`: first-order process with dead time, pump or heater power; native: PI on a fixed setpoint.
- **A muscle is one knob of one plant.** Omni holds only that knob: capacity (the adapter's capacity law for one plant), setpoint (inside a declared band: calm end at ρ_0 , stress end at $\rho_{\min}$), power (the directive's power cap), or admission (native while change is permitted; while not, only what clears inside the service target, or the flexible share deferred or shed).
- **Each muscle's plant is sized** by a factor 0.6 to 1.4 drawn from its muscle id.

Arms

On the same seed, so with the same demand, weather and disturbances:

- **native:** the plant and its native controller;
- **watch:** the governor reads every period and writes nothing;
- **omni:** the governor holds the knob; at 90% of the run it is killed and the knob returns to the native controller;
- **fixed_calm** (setpoint muscles only, descriptive): the native controller with the setpoint fixed at the band's calm end. It shows how much of a setpoint result the band alone gives.

Organisms: every plant of a realm (or all 656) on one 15 s clock for one hour. Plants are coupled:

- the electrical power of compute, motion and process plants is heat in the realm's thermal zones;
- the organism's load swing is load on its storage sites;
- the zones' temperature is the ambient every other plant reports.

One governor reads the organism's aggregate, and its one directive sets every muscle's knob.

Seeds and repetitions

Confirmation seeds 1000 to 1009: ten paired seeds per muscle and per organism. Development used seeds 0 and 1 only. Nothing from development seeds is reported.

Outcomes

- **Primary**, per seed: $(\text{work_omni} / \text{work_native}) / (\text{energy_omni} / \text{energy_native}) - 1$. Work is in the plant's own units: requests, IT heat held in specification, site load served, moves completed, or product delivered in specification. For an organism, work is the mean over its plants of $\text{work_omni} / \text{work_native}$, and energy is total joules.
- **Guardrails**, both must hold:
 - work: the 95% interval of $\text{work_omni} / \text{work_native} - 1$ must not reach below -1% ;
 - violations: the 95% interval of the change in the share of periods in violation must not reach above $+1$ percentage point.

Violation, per plant:

- compute: response time over the service target, or a dropped request;
- thermal: zone over its limit;
- storage: grid import over the contract limit;
- motion: tracking error over its bound, winding over temperature, or the oldest task waiting past its deadline;
- process: the process variable outside its specification.
- **Label, by rule** (`realms/harness.py`, `label()`, the GPU bench's rule):
 - SUPERIOR WITHIN GUARDRAILS;
 - ENERGY IMPROVEMENT WITH SERVICE TRADEOFF;
 - NONINFERIOR / INCONCLUSIVE;
 - NOT ESTABLISHED;
 - WORSE;

- INVALID.

Invalid

A muscle or organism is labelled INVALID if, on any seed:

- the watch arm differs from native in any meter or writes anything;
- the omni arm writes after the kill;
- a knob is not back at its native value after the kill;
- any contrast is not finite.

Recorded

Per muscle and organism:

- every per-seed contrast;
- writes per run;
- the native violation share;
- the fixed-setpoint comparison where it applies.

Per run: the commit, the seeds, and the fingerprints of the catalog, the engine, the governor and the whole frozen tree (RUN.json, SHA256SUMS.txt). The confirmation refuses to run on uncommitted code.

Changes made on the development seeds, before this freeze

The development runs found bugs and sizing faults. Each was fixed before the confirmation seeds were run:

- **The admission knob was inverted:** it admitted only what cleared the service target while change was permitted. Fixed to the declared rule.
- **The compute admission limit compared the queue limit with the period's arrivals** instead of the backlog left after the period's service. Fixed for native and Omni alike.
- **The capacity law did not clip the queue and load readings to [0, 2],** as the adapter's observe_vector does. Fixed.
- **Thermal and process utilisation was measured against the uncapped capacity** while the power cap was held, so the power law's trim ratcheted to its floor. Fixed: utilisation of the capacity actually available.
- **Compute plants reported a heat proxy of my own.** Replaced by the live controller's own thermal model (omni_controller/muscles.py).
- **Thermal zones kept fixed cooling units while their IT load was scaled.** The large halls were under-provisioned, so the native controller overheated. Cooling now scales with the hall.
- **The flight axis's motor thermal resistance was ten times too high,** so the native controller was always derated. Corrected. The UPS feed was sized under its own load; sized at 1.2 times.
- **The slow plants** (vehicle, spacecraft, flight) completed too few tasks in six minutes to measure. They now decide every 5, 10 and 2 seconds over the same 360 decisions.
- **Added the fixed_calm arm** for setpoint muscles, after development showed that the band's calm end alone accounts for much of the setpoint results.

None of these changes was chosen by its effect on Omni's result. The development runs showed losses as well as gains for Omni before and after them.

Not claimed

- **No row is evidence about a real machine.** A row says what the governor's law does to that model through that knob.
- **The plants, native controllers and bands were written by the same project as the governor.** That is a real conflict, so every one of them is in two files, to be read and contested.
- **A muscle's name chooses its knob and its plant's size; it does not get its own physics.** The 16 muscles of a family share the family's plant model at different sizes, through different knobs.
- **The plant code is Python only.** The governor's C++ twin is unchanged; a C++ twin of the plants is open.

Round 2 (2026-10-01, after round 1's results; before any round-2 confirmation seed)

Round 1 (seeds 1000-1009) labelled all five organisms WORSE. Reading the result against the shipped controller (omni_controller/controller.py, omni_controller/muscles.py, omnicompass/nervous_system.py) showed that round 1's Omni layer was not the Omni that runs on Kubernetes. Round 1 is kept unchanged in results/realms/round1/, with a note saying why it is superseded. Round 2 changes only how Omni commands a knob, the native machine-pool scaler and the declared budgets. These changes were made after seeing round 1, so they are listed with the source line each one follows:

Round 1	Round 2, as the shipped controller does it
Capacity muscles replaced the HPA with the stack simulator's capacity law (release one unit after convergence, dwell and a 20% band)	Pods: the HPA target written as <code>min(rho*, the operator's target)</code> , never tighter, held one autoscaler window; the HPA scales (<code>controller.py</code> , HPA target patch). Machine pools: the node release gate, one machine per decision (<code>nervous_system.node_release_gate</code>)
HPA-type setpoints moved inside a band whose calm end was tighter than the operator's target	The same <code>min(rho*, operator)</code> rule: more headroom is always allowed, less never (<code>controller.py</code>)
No contraction authority and no SLO reflex	Every contraction needs the organ's authority (<code>calm >= threshold</code> , senses live) and three clean decisions; while service is breached the knob returns to native (<code>nervous_system.authority</code> , <code>muscles.py</code> SLO reflex)
Continuous knobs jumped to the governed value	Down by at most the calm share of the surplus per decision (<code>nervous_system: step = calm</code>)
Request-served pools had their power capped	Never: throttling request work saves no energy and adds wait (<code>muscles.py</code> , <code>_power_cap</code>); GPU and CPU-frequency pools use their envelope, never under draw x 1.3
Thermal setpoint by headroom	The live cooling law: warm while cool, cold as heat rises, under the envelope <code>18 + 9 x calm</code> mapped onto the band (<code>muscles.py</code> , <code>_cooling</code>)
Admission paused every admission muscle, request traffic included, and resumed only when fully calm	Batch pacing of pausable work only: one muscle suspended per decision at power stress <code>>= 0.95</code> or heat <code>>= 0.96</code> (or a nervous pause), one resumed per decision at power stress <code><= 0.8</code> and heat <code>< 0.90</code> (<code>muscles.py</code> , <code>_batch_pace</code> defaults); request traffic never paused
<code>queue_ratio</code> was the backlog in service-target units	pending starts per serving unit, or latency pressure (<code>p95 / target - 1</code> , or the failed share), capped at 2 (<code>controller.py</code>)
The governor's current cap followed the applied cap	1.0, as the live controller sets it
Site power budget 1.25 x a formula nominal that sat under the organism's real native draw (compute ran at 1.2 x it), so the governor read the site as always at its limit	1.25 x each plant's mean native draw on the calibration seed 999 (never a result seed); compute pools alone likewise

Round 1	Round 2, as the shipped controller does it
Organism heat = the hottest of up to 656 plants	The mean, as every other organism channel; local heat stays with each plant's own reflex
Native machine pools used the HPA rule	The Cluster Autoscaler's defaults: add while work waits, remove one machine after the rest has been under 50% for 10 minutes
Building cooling sized under its own peak (native overheated half the time)	Units of 35 kW: capacity 1.25 x the declared peak

Unchanged: the plants' physics, the catalog, every other parameter, the arms, the outcomes, the guardrails, the label rule and the invalidity rules.

- **Round 2 seeds:** 2000 to 2009. Development of round 2 used seeds 0 and 1 only, never reported.
- **Results:** results/realms/ (round 1 in results/realms/round1/). Both rounds are cited together.

Round 3 (2026-10-02, after round 2's results; before any round-3 confirmation seed)

Only the make-up of the realm organisms changes. The plants, the Omni layer, the outcomes, the guardrails, the label rule and the invalidity rules are round 2's, unchanged.

- **Round 2 cut the 656 into four realms with no overlap.** No realm organism carried the infrastructure every real stack runs on unless that infrastructure was the realm's own. The data-centre realm had no cooling or power, and the robotics and plant realms had no Kubernetes, machines or GPUs.
- **Round 3 gives every realm the shared spine** (tools/realms_catalog.py, SPINE): Kubernetes Workload Scaling, Placement & Scheduling, Container Resources, Node Fleet, Cloud VM & Capacity, NVIDIA GPU Hardware, Host CPU & Memory, Network Routing, Storage, Observability, Reliability & Security, Cooling & Chillers, PDU / UPS & Electrical Distribution.
 - Each realm's organism is its own families plus the spine: Compute 345 muscles, Physics 262, Energy 282, Distribution 337.
 - The whole-tower organism still holds each of the 656 once.
- **Quantum Computing Control moves to the compute realm** (it behaves as a compute job queue).
- **Round 3 seeds:** 3000 to 3009. Round 2 is kept in results/realms/round2/ with a note on why it is superseded.

Round 4: the stacked organism (2026-10-02, before any round-4 seed)

The four realm organisms of round 3, stacked on one 15 s clock (realms/harness.py, run_stack; tools/run_stack.py). Every muscle appears as often as it appears in the realms, duplicates included: $345 + 262 + 282 + 337 = 1,226$. A duplicate is still a muscle that has to converge. Each realm keeps its own internal coupling. The plants, the Omni layer, the outcomes, the guardrails and the label rule are round 3's.

- **Arms:** native (no governor); separate (one governor per realm); one (one governor over the whole stack, reading the mean of all 1,226 muscles and the stack's total power against its total budget).
- **Check, required for validity:** the stacked native run equals each realm's own native run, plant by plant, on every seed. Stacking must change nothing natively.
- **Comparisons, each labelled by the rule:**
 - one governor against native;

- separate governors against native;
- one governor against separate governors (does one Omni over everything beat four).
- **Seeds:** 4000 to 4009. Development used seed 0 only, never reported.

Round 5: the whole stacks with the real card inside (written 2026-10-02, before any run)

One harness (`tools/run_hil.py`, started by `scripts/gpu_rented_run.sh` after a valid card smoke): each of the six organisms (the four realms, the four stacked with every duplicate kept (1,226), the whole tower of 656) runs on one clock as in round 3, with the machine's real GPU wired in as one more muscle of its NVIDIA GPU family (a spine family, so the card is in every organism). The card serves the pinned request stream; its own `power.draw` is heat in the organism's thermal zones and load on its storage sites.

- **Arms:** native (the stacks' own controllers, the card's own firmware) and omni (one engine on everything: the bowl law on every simulated muscle, `realms/bowl_arm.py`, and on the card's two wires, `omni_controller/gpu_bowl.py`). At 90% of each arm every knob and both wires are handed back; a knob not handed back, a card limit not back at its start, or a card governor exiting non-zero makes the run invalid (exit 2).
- **Seeds and repetitions:** 3 paired repetitions, seeds 6000-6002; arm order alternates by repetition and organism.
- **Clock:** 240 steps of 2 s of wall clock per arm (the card in real time).
- **Outcomes:** work per energy, work, energy and violations, Omni against native, for three parts kept apart: the simulated stacks (evidence S), the card (its own meter, evidence P), and both added (the card as one more plant, its joules added to the stacks'). Labels by the round 3 rule.

CHAPTER 44

The Bowl Law on Real Kubernetes: Preregistration

Written and committed before the run. The run's commit is the one that carries this file; nothing in the law, the harness or this rule changes after it starts.

What is run

Workflow benchmark-reps on main, 10 repetitions. Each repetition runs three arms back to back on the same runner, each on a fresh six-worker kind cluster, in an order rotated by repetition (`scripts/kind_paired.sh`):

Arm	What governs
native	Kubernetes alone (HPA at target 50, scheduler); Omni-Compass not started
omni	Omni-Compass on top with the engine's allocation law (<code>--law governor</code> , the law of sets 22 to 25)
bowl	Omni-Compass on top with the bowl law (<code>--law bowl</code> , <code>omni_controller/controller.py</code>)

Load: fixed rate (`loadgen=open`), the same work in every arm. 900 measured seconds per arm. SLO 500 ms at the 95th percentile. Every Omni arm runs the six-state engine on every decision, the nervous system's authority and release gate, the shield, the compass, and ends with the kill switch, which must return the HPA target, its replica range, the pods' CPU limits and every worker to native, with no record left (`scripts/kind_bench.sh`).

The bowl law in the live controller

The service position is the 95th-percentile response time over the SLO (0 calm, 1 the line); a blind probe or a pod waiting for a place reads as past the wall. The force is $A \tanh((K_P (p - 0.5) + K_D v) / A)$ with K_D for critical damping times the realm push factor 3, the same law and gains as on every realm muscle (`realms/bowl_arm.py`): up gain 0.10, down gain 0.02, release threshold -0.2. Two levers:

1. **HPA target**, cover from 60% of the operator's target to the operator's own: the up force lowers it (more pods), the down force returns it toward the operator's. It is never tighter than native.
2. **Node pool**: past the 0.95 wall one machine more at once; one machine back only while the force is below -0.2, the position is below the center, and the nervous system's release gate is open.

Outcomes and the rule

Primary: **worker nodes in service** (mean) and **95th-percentile response time**, each arm against native, paired over the 10 repetitions with a t-based 95% interval (`tools/live_reps.py`).

Band first: the bowl arm is a win only if its p95 is not worse than native's (the upper end of the 95% interval of the paired difference at or under 0) **and** failed requests are not higher. If that holds and machines in service fall with an

interval wholly below 0, the label is **better on machines within the band**. If machines fall but the band condition fails, the label is **tradeoff**. Otherwise **not established**.

Secondary, reported, not used for the label: p99, mean response time, HPA replicas, pods started, pod start wait, CPU including Omni-Compass's own, the declared energy models. The omni arm is reported against native and against the bowl arm by the same rule. A run that fails its own checks (kill switch, controller stopped early, missing permission) is marked invalid and left out, never silently counted.

Evidence class **L**: real Kubernetes software on kind. Energy on kind is a declared model, not a meter.

The Evidence Ledger

Every material statement in the technical package gets exactly one primary evidence class. A statement never borrows a stronger class from the one printed beside it.

Class	Meaning	Layer
T	formally proved theorem, under stated assumptions	L1 mathematical mechanism
V	finite computational verification: exact over a declared finite set, never a theorem beyond it	L1
S	simulation / model result	L2 modelled systems
L	live observation of external software (a real Kubernetes API, a real process)	L3 external software
P	independent physical measurement (a meter Omni does not read or control)	L4 physical plant
O	open: not established	—

No layer stands in for another: a theorem is not a physical validation, a Monte Carlo result is not a theorem, and a model's energy is not a meter's.

Mechanism identity for every row: `results/MECHANISM_IDENTITY.json`. Canonical engine `symmetric_verified`, mechanism id `29d9808dfb8f...`; the printed configuration `printed_eight_line`, id `cd333dc166fb...`, is a named alternative (Option A of the directive: one canonical, one alternative embodiment; no equivalence is claimed).

Layer 1: the mechanism

Class	Statement	Where
T	Unsaturated, the U-channel error obeys $de/dt = -KP e$; $V = e^2/2$ has $dV/dt = -KP e^2 < 0$ for $e \neq 0$ (inside Ω_{unsat} only).	docs/TRACKING_THEOREM.md Thm 1
T	With the U drift bounded by $F_{\text{bar}} < u_{\text{max}}$, V decreases whenever $e \neq 0$, saturated or not; e never changes sign; saturated stretches end in bounded time (continuous time).	Thm 2
T	On the declared parameter box, $F_{\text{bar}} = 16.86 < 25$, so Thm 2 holds over the whole box in continuous time.	Thm 2
T	The sampled controller (u held through each RK4 step) satisfies $e_{(k+1)} = 0.88 e_k + d_k$, hence $\text{abs}(e_k) \leq 0.88^k \text{abs}(e_0) + \epsilon_{\text{h}} (1 - 0.88^k)/0.12$.	Thm 3
V	$\epsilon_{\text{h}} = 0.00378$ over every micro step of the 500 frozen fixtures (nearest target); ultimate bound $0.0315 < \text{BASIN_TOL } 0.10$. Wrong target: 0.0243, bound 0.20 (wider than the basin).	results/TRACKING_BOUNDS.json
V	The discrete execution dithers inside that band: $\text{abs}(e)$ grew on 27 061 of 100 000 micro steps; e changed sign 747 times. The continuous monotone decay is not inherited.	same
V	Saturated micro steps: 0 of 100 000 (nearest target), 5 of 100 000 (wrong target).	same
T	The admissible box A (E interval, $\text{abs}(U - \sigma) \leq \text{abs}(U_0 - \sigma)$, S between S_0 and S_{plus}) is forward invariant in continuous time.	Thm 4
V	Discrete invariance of A: 0 failures over 100 000 micro steps of the frozen fixtures, both targets.	results/TRACKING_BOUNDS.json
O	Discrete invariance and ϵ_{h} proved over the whole box (interval arithmetic).	—
O	Global stability of the forced six-state system.	docs/FORMAL_STATUS.md
V	500 / 500 frozen fixtures satisfy CONVEY-5 and CERT-10 (dwell predicates, not external certification).	results/core_evidence.json
V	Frozen 500-fixture comparison: symmetric_verified final error 7.48e-5, integrated $\text{abs}(u)$ 0.878, peak 13.93; printed_eight_line 5.13e-6, 11.79, 18.87. Both executable; not interchangeable.	results/MECHANISM_IDENTITY.json
V	Python and C++ produce identical results (engine, governor, shield, HPA law, conveyance, GPU rules); twins sealed.	verify.py, results/SEAL.json

Layer 2: models and simulations

Class	Statement	Where
S	Synthetic stack: energy for Omni over the Kubernetes reference 168.9 → 121.7 kWh per run (documented-behaviour reference, not upstream controllers).	docs/CLAIMS_REGISTER.md C8
S	Fleet harness, recorded PlanetLab shapes: -40.6% energy vs HPA+CA, -21.7% vs Karpenter-lite (omni_fleet).	C17
S	Single GPU physics model: the engine alone would save 10.6–13.6% work per kJ on card A but breaks the p95 guardrail by 15–28%; with the frozen guards, about +1 to +5% inside it.	results/gpu/sim/FINDINGS.md
S	Node exchange (CPU and GPU on one budget): +1.4 to +5.7% work against the separate budgets, never over budget.	results/hardware/NODE_EXCHANGE_*.json
S	Six organisms (345, 262, 282, 337, the four stacked 1,226, the whole tower 656), the bowl law on every muscle against each organism's own controllers, 1,000 paired runs at 1× and 10× size: work per energy +0.20% to +0.30%, energy -0.21% to -0.32%; every knob handed back.	results/scale/GRID.md
S	Two-wire GPU card under the bowl law, 10 seeds: work per energy +8.6% (+7.8 to +9.4), time over the line unchanged; the one-wire governor on the same card +0.1%.	results/sim/gpu_two_wire/RESULT.md
S	Realm harness round 3 (every realm carries the shared spine), preregistered, seeds 3000-3009: the whole 656-muscle tower native against one governor on top, work per energy +0.1% (+0.1 to +0.1), violations +0.5 pp, SUPERIOR WITHIN GUARDRAILS. Realms: Energy +0.2% with +1.9 pp violations (tradeoff); Compute 0.0% with +2.1 pp (not established); Distribution -0.1% (worse); Physics -0.7% (worse). Rounds 1 and 2 kept, superseded.	results/realms/REALMS.md
S	Stacked organism (round 4, seeds 4000-4009): the four realm organisms on one clock, 1,226 muscles with every duplicate; stacked native equals the four realms alone on every seed. One governor over the stack: energy -0.14%, work per energy +0.02%, violations +1.4 pp, ENERGY IMPROVEMENT WITH SERVICE TRADEOFF; four separate governors about the same (+0.04%); one governor against four separate: -0.02% (WORSE, by a hair).	results/realms/stack/STACK.md

Layer 3: live external software

Class	Statement	Where
P	First real-GPU confirmation, NVIDIA A10 (Lambda), 10 paired repetitions, the card's own meter: work per energy +3.6% (+2.7 to +4.5, proven), GPU energy -3.5%, same requests, none lost; wire check 7 of 7, every write read back, every arm restored.	results/gpu/run-20261002T082232Z/GPU_REPS.md
L	Set 24 (2026-10-02, commit c908054), real Kubernetes (kind), 10 paired repetitions: machines in service -31.6% (proven), p95 response time -60.1% (proven), p99 -64.1% (proven), HPA replicas -38.6% (proven), 0 failed requests on both; total CPU including Omni's own -1.8% (not proven); energy with every machine powered -0.3% (declared model).	results/live/LIVE_REPS_24.md
L	Set 23 (set 22 repeated on the current code, 2026-10-02), real Kubernetes (kind), 10 paired repetitions, equal work: p95 response time -62% (proven), replicas -37% (proven), pods started -64% (proven), 0 failed requests; total CPU with Omni's own -1.0% and modelled energy -0.2% (no difference).	results/live/LIVE_REPS_23.md
L	Set 22, real Kubernetes (kind), 10 paired repetitions, equal work (fixed-rate load): p95 response time -61% (proven), replicas -23% (proven), pending pod-minutes -91% (proven), 0 failed requests.	results/live/LIVE_REPS_22.md
L	Set 21, real Kubernetes (kind), 10 paired repetitions: p95 response time -37% (proven), replicas -12% (proven), 0 failed requests.	results/live/LIVE_REPS_21.md
L	Omni patched a real Kubernetes API in place (no restart); the kill switch restored every setting in every run; watch mode wrote nothing.	same, results/live/
S	Set 21 energy is a declared model, not a meter: +1.8% worse with parked machines at idle power.	same (energy table)

Layer 4: independent physical measurement

Class	Statement	Where
O	Omni changes successful work per measured joule on a GPU. The bench is built with three receipts (governor, actuator, outcome), the three contrasts (observation, authority, total), and result labels by rule; it has not been run on a card.	scripts/gpu_paired.sh, docs/GPU_PREREGISTRATION.md
O	Production data-centre energy effect.	—
O	Whether the power limit is the right actuator for LLM serving. Published measurements (arXiv 2605.11999, H200) find memory-bound decode draws 137–300 W of 700 W, so no power cap binds; clock scaling is what saves energy there (arXiv 2501.08219, GreenLLM 2508.16449). The pinned bench workload is compute-bound, where the cap does bind: a result on it does not transfer to LLM decode.	external literature
O	Wall-plug (whole-machine) energy effect; CPU package and DRAM (RAPL) effect.	—
—	What the GPU bench will attribute: per write, joules and requests against native, and which rule decided it (engine, floor, gate, reflex, heat, speed lock).	tools/gpu_reps.py

Negative evidence, kept

Nothing here is deleted when a later result looks better.

Class	Statement	Where
P	Same run: p95 response time +58.5% worse (510 to 809 ms), mean +48%; label by rule ENERGY IMPROVEMENT WITH SERVICE TRADEOFF. Cause: governor wiring (busy bursts served below the card's own clock); corrected in amendments 6-7, not yet re-run on a card.	results/gpu/run-20261002T082232Z/GPU_REPS.md, docs/GPU_PREREGISTRATION.md
S	Two-wire GPU card, same seeds: p95 response +32.9% (+6.3 to +59.4) against native.	results/sim/gpu_two_wire/RESULT.md
S	Six organisms, same runs: time over the service line +0.19 to +0.27 pp worse in every cell ; band first is not held anywhere.	results/scale/GRID.md
L	Set 21: modelled energy 1.8% worse with every machine powered (the only honest energy row on kind).	results/live/LIVE_REPS_21.md
S	Realm harness round 1 (superseded, kept): all five organisms worse (whole tower -0.1%). Its Omni layer did not follow the shipped controller (no contraction authority or SLO reflex, the stack law in place of the HPA, request traffic paused, a site budget under native draw).	results/realms/round1/
S	Realm harness round 2: the Physics / Robotics / Autonomous organism worse (-1.9%, violations +2.9 pp); the Compute organism's +2.5% costs +2.9 pp of service violations; 68 single muscles worse, mostly batch pacing and cooling setpoints under the live cooling law.	results/realms/REALMS.md
L	Set 22, equal work: no CPU saving once Omni's own CPU is counted (service -7.6%, controller +0.070 cores, together -0.9%, not proven); modelled energy unchanged (-0.1%).	results/live/LIVE_REPS_22.md
L	Set 20: the "same work" reading was wrong; requests were +35%.	results/live/LIVE_REPS_20.md addendum
L	Sets 1-2: machine savings withdrawn — a broken probe had blinded the latency sense.	results/live/LIVE_REPS_PROBE_DEFECT.md
L	Set 3 with a working probe: Omni kept all 6 machines; no significant difference except more waiting pods.	docs/HISTORY.md
L	Set 21: pending pod-minutes +238% (not proven); pod starts +37% (not proven).	results/live/LIVE_REPS_21.md
S	GPU and batch: energy 1-2% worse , power and heat margins 1-4% worse than tight packers (AKS/Karpenter, CAST AI, Spot).	docs/HISTORY.md
S	Machine round trips 1.1 → 1.7 per run (worse) in 6 of 8 seed-target combinations.	C12
S	Power-protect mode: backlog violations 2.5% → 6.7% (worse).	C12p
S	Park strategy on PlanetLab shapes: energy +1.3% vs HPA+CA, +33.6% vs Karpenter-lite (worse).	C17
S	Typical response time about 20% slower than every platform in the web and four-cluster simulations.	docs/HISTORY.md
S	GPU model: the engine without its guards breaks the p95 guardrail.	results/gpu/sim/FINDINGS.md
S	The compass-stroke GPU variant was tried and not adopted.	same
S	Right-sizing against VPA: p95 +15%, memory (OOM) kills +531%.	docs/BENCHMARK_REPORT.md
—	Reported in the external master-build report (not reproducible from this repository): on fresh scenarios Karpenter+VPA sometimes used less modelled energy than Omni, while Omni had lower churn and fewer request-induced evictions. Kept here so it is not lost; to be re-run here before it is cited.	external

CHAPTER 46

The Claims Register

Every claim, its evidence status and the command that reproduces it. Simulation results use the synthetic stack model in `omnicompass/stack_sim.py`; they are not production measurements.

ID	Claim	Status	Reproduce			
C1	The six-state mechanism is implemented identically in the reference engine, Python and C++.	Proven (exact parity tests)	<code>python verify.py</code>			
C2	The C++ governor reproduces the Python governor on recorded stack telemetry.	Proven (0 mismatches)	<code>python tests/test_cpp_governorparity.py</code> <oc_governor>			
C2s	The C++ shield reproduces the Python shield (enforced actions, intervention counts and violations) on action sets recorded from every arm type; a C++ shield with invariant I1 weakened is rejected by the parity test.	Proven (0 mismatches; negative control)	<code>python verify.py</code>			
C2h	An independent C++ implementation of the HPA replica law (ratio rule, 10% tolerance, scale-up limit, 300 s scale-down window) reproduces the fleet harness HPA on every recorded step (hundreds of thousands of steps across Kubernetes and governor arms); a C++ HPA with 5% tolerance is rejected.	Proven (0 mismatches; negative control)	<code>python verify.py</code>			
C3	The S channel cannot cross S- (forward invariance).	Proven (Proposition 1)	Manual Chapter 3			

ID	Claim	Status	Reproduce			
C4	The governor completed 100,000,000 consecutive decisions (951 years at 5-minute intervals) with 0 failures and constant memory; failure-rate upper bound 3.0e-08 at 95% confidence.	Measured (soak test); reproduced by the full verifier	python verify.py			
C4b	Throughput-mode soak: 100,000,000 decisions, 0 failures; resident memory at 25/50/75/100% of each run is constant ([3852, 3852, 3852, 3852] kB power-protect, [3916, 3916, 3916, 3916] kB throughput).	Measured (soak test)	python verify.py			
C8p	Primary endpoint (energy, throughput vs reference target 0.7): seed 346410161: -47.20 kWh, p = 1.0e-04; seed 360555127: -47.69 kWh, p = 1.0e-04. Secondary metrics Holm-adjusted over 24 tests: 21 better, 2 worse, 1 not significant.	Measured in simulation	python benchmarks/multiplicity.py			
C5	One governor decision costs 2.6 microseconds on one CPU core of the test machine (machine-dependent).	Measured	cpp: oc_soak			
C6	Observe mode leaves the stack bit-identical: over Kubernetes, identical to Kubernetes alone (500/500 and 500/500); over the native model, identical to native (500/500 and 500/500).	Measured in simulation	results/heldout_seed_*			
C7	Invariant violations per run (all arms scored identically): Kubernetes reference 9.56 / 9.44; power-protect 0.00 / 0.00; throughput 2.72 / 2.69, of which all are I4 (projected power), which throughput mode does not enforce; excluding I4: reference 6.91 / 6.80, throughput 0.00 / 0.00.	Measured in simulation	results/heldout_seed_*			

ID	Claim	Status	Reproduce			
C8	Energy for Omni-Compass over Kubernetes (throughput mode, same power rules as the reference) versus the Kubernetes reference model (HPA target 0.7): 168.9 to 121.7 (better) / 169.2 to 121.5 (better) kWh per run; across HPA targets 0.5 to 0.8: better in 8, worse in 0, not significantly different in 0 of 8 seed-target combinations.	Measured in simulation (synthetic stack, documented-behaviour reference model)	results/heldout_seed_*			
C9	Time healthy for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 79.3% to 91.2% (better) / 78.5% to 91.4% (better); across targets: better in 8, worse in 0, not significantly different in 0 of 8 seed-target combinations.	Measured in simulation	results/heldout_seed_*			
C10	Recovery time for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 57.2 to 17.8 (better) / 58.4 to 15.2 (better) minutes; across targets: better in 8, worse in 0, not significantly different in 0 of 8 seed-target combinations.	Measured in simulation	results/heldout_seed_*			
C11	Backlog violations for Omni-Compass over Kubernetes (throughput mode) versus the reference model (target 0.7): 2.5% to 1.6% (better) / 2.6% to 1.5% (better); across targets: better in 8, worse in 0, not significantly different in 0 of 8 seed-target combinations.	Measured in simulation	results/heldout_seed_*			

ID	Claim	Status	Reproduce			
C12	Machine round trips (started and later stopped), throughput mode versus the reference (target 0.7): 1.1 to 1.7 (worse) / 1.1 to 1.8 (worse) per run; across targets: better in 2, worse in 6, not significantly different in 0 of 8 seed-target combinations. The reference removes a machine only after 10 minutes below 50% utilization and keeps post-event machines running; the governor releases them, which is the main source of its energy saving.	Measured in simulation	results/heldout_seed_*			
C12w	Scale direction reversals (back-and-forth wear), throughput mode versus the reference (target 0.7): 2.19 to 1.93 (better) / 2.22 to 1.92 (better) per run; across targets: better in 4, worse in 4, not significantly different in 0 of 8 seed-target combinations. Power-protect: 2.19 to 3.06 (worse) / 2.22 to 3.08 (worse).	Measured in simulation	results/heldout_seed_*			
C12e	Engine mechanism in the flagship (throughput): removing the equation (2) release gate changes reversals from 1.93 / 1.92 to 3.19 / 3.18 and energy from 121.7 / 121.5 to 120.2 / 119.9 kWh; removing engine evolution changes energy to 125.4 / 125.1 kWh and reversals to 0.00 / 0.00.	Measured in simulation	results/heldout_seed_*			
C12p	Power-protect mode (enforces the site power limit, which the reference does not): energy 168.9 to 132.2 (better) / 169.2 to 132.3 (better) kWh; power-limit violations 12.8% to 3.5% (better) / 13.8% to 3.7% (better); backlog violations 2.5% to 6.7% (worse) / 2.6% to 7.0% (worse).	Measured in simulation	results/heldout_seed_*			

ID	Claim	Status	Reproduce			
C12a	Human pages: the governor issues no page actions. Zero pages is a design property, not a measured performance result.	Design	omnicompass/adapter.py			
C12b	The reference model is not the upstream Kubernetes controllers: Cluster Autoscaler scheduling simulation, Karpenter, VPA, scheduling constraints and disruption budgets are not represented.	Stated limitation	benchmarks/stack_benchmark.py (K8sReference)			
C15	15-second control plane, Omni-Compass as node-pool and power authority in place of the Cluster Autoscaler (HPA retained), energy vs HPA+CA / vs HPA+Karpenter-lite: energy-first: web -34.5% / -16.9%, reversals 4.3; multi -34.3% / -16.6%, reversals 18.1; batch -17.2% / -9.7%, reversals 13.6; gpu -4.9% / -2.7%, reversals 5.1; gpu_always_on -4.7% / -3.2%, reversals 0.0	balanced: web -33.2% / -15.3%, reversals 4.0; multi -33.0% / -15.0%, reversals 15.5; batch -12.4% / -4.5%, reversals 5.7; gpu -5.0% / -2.8%, reversals 4.2; gpu_always_on -4.7% / -3.2%, reversals 0.0	wear-first: web -29.7% / -10.9%, reversals 3.6; multi -29.6% / -10.6%, reversals 14.5; batch -12.3% / -4.3%, reversals 4.9; gpu -4.5% / -2.3%, reversals 3.0; gpu_always_on -4.2% / -2.8%, reversals 0.0	park: web -15.9% / +6.7%, reversals 0.0; multi -15.6% / +7.0%, reversals 0.0; batch -15.3% / -7.7%, reversals 0.0; gpu -4.9% / -2.7%, reversals 0.0; gpu_always_on -4.7% / -3.2%, reversals 0.0. Baseline reversals (CA / Karpenter): web 1.1 / 6.8; multi 5.5 / 25.4; batch 1.4 / 24.9; gpu 2.8 / 11.2; gpu_always_on 0.0 / 0.0. In gpu_always_on no arm powers a node off.	Measured in simulation (fleet harness, synthetic workloads, documented-behaviour execution layer)	python -m fleet.benchmark --seeds 30 --seed-base 7000000 --out out/

ID	Claim	Status	Reproduce			
C17	Recorded PlanetLab utilization shapes with a declared scale mapping (9 traces, 30 scenarios), governor as node-pool authority in place of the Cluster Autoscaler: omni_fleet: energy -40.6% vs HPA+CA (better), -21.7% vs Karpenter-lite (better); omni_fleet_balanced: energy -37.5% vs HPA+CA (better), -17.5% vs Karpenter-lite (better); omni_fleet_wear: energy -31.0% vs HPA+CA (better), -8.9% vs Karpenter-lite (better); omni_fleet_park: energy +1.3% vs HPA+CA (worse), +33.6% vs Karpenter-lite (worse). Time healthy and work completed are significantly lower by 0.08 points and 0.01%; observe mode identical in 30/30.	Measured in simulation driven by recorded traces (frozen laws, no retuning)	<pre>python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 30 --seed-base 800000 --out out/</pre>			
C16	Live-cluster capture, capture replay and the PlanetLab vessel are tested on generated inputs in the real formats. No real capture or recorded trace has been run in the package.	Tested path; no real-data result	<pre>python tests/test_fleet_realdata_paths.py</pre>			
C19	The live controller (omni_controller/) implements observe, target and nodepool modes with dry-run, audit log and a kill switch that restores HPA targets from annotations; tested against a fake kubectrl only, never against a real cluster.	Tested path; no live result	<pre>python tests/test_omni_controller.py</pre>			
C18	Savings projection: the energy-first reduction relative to HPA + Karpenter-lite, applied to declared fleet profiles (results/SAVINGS.csv), computed identically in Python and C++. A projection from simulation, not measured savings.	Projection	<pre>python benchmarks/savings.py</pre>			

ID	Claim	Status	Reproduce			
C21	End-to-end self-pilot (shipped controller, simulated cluster, real capture and scoring), default headroom 50%; energy per core-hour -7.7%, node-hours per core-hour -13.8%, pending-pod time not significantly different from HPA + Cluster Autoscaler; HPA shortfall minutes higher. Lower headroom saves more energy with more pending-pod time (manual Section 8.12a).	Measured in simulation	<code>python pilot/selfpilot.py</code>			
C20	pilot/score.py scores a user's own captures (node-hours and energy per used core-hour, utilisation, pending-pod and HPA-shortfall minutes, bootstrap intervals); tested to detect a real gain, report no difference for identical clusters and detect a service regression.	Tested tool; no pilot result	<code>python tests/test_pilot_score.py</code>			
C13	Decision components (autoscalers, power agents, paging, Terraform as controller) consume about 0.02% of fleet CPU; idle capacity is 92% of fleet CPU at 8% utilization.	Modeled from published figures and stated assumptions	<code>python benchmarks/fleet_overhead.py</code>			
C14	Behaviour on production systems.	Not established; requires the pilot protocol	<code>docs/PILOT_PROTOCOL.md</code>			

The Benchmark Report

Dated record, kept as written on 26 September 2026. Where a figure here differs from STATE_OF_PLAY.md, the State of Play governs. The GPU power-limit figure in this report (15% to 19% energy, under 1% slower) came from an earlier model of the card; later models with the service guards give +1.3% to +5.1% work per kJ with one wire (results/gpu/sim/after) and +8.6% with two wires at a higher p95 (results/sim/gpu_two_wire/RESULT.md). No real card has finished the bench yet.

Benchmark report, 26 September 2026. Repository: Omni-Compass/The-Omni-Compass-Control-Core-Engine (private), branch main Every number below is produced by code in that repository and can be regenerated; section 21 gives the commands. Each result states whether it was **measured on a live Kubernetes control plane** or **computed in simulation**.

1. Summary

Omni-Compass is a single control engine that senses the whole compute stack and drives its actuators (its "muscles": replica counts, node pools, power caps and others) from one six-state dynamical model, with a safety shield before every action and a kill switch that hands control back. It was compared in three architectures:

- **A. Kubernetes alone.** Kubernetes' own controllers decide: Horizontal Pod Autoscaler (HPA) for replicas, Cluster Autoscaler for nodes; other managers act on their own proposals.
- **B. Kubernetes + Omni-Compass.** Kubernetes' controllers keep running; Omni-Compass governs on top of them (sets the HPA target, gates and sizes the node pool, caps power) as the single authority over their settings.
- **C. Omni-Compass direct.** Omni-Compass is the only decision-maker and actuates the muscles directly; the separate managers no longer decide.

Main result (pre-registered, 1,000 held-out scenarios, simulation). Against Kubernetes alone, B used -28% energy and C -23%; time healthy rose from 79% to 91% (B) and 90% (C); recovery time fell from 58 to 17 and 28 minutes; contradictory commands, pages and human interventions went to zero in both; safety-rule violations fell from 9.5 to 2.7 (B) and 0.0 (C). Of 28 gauges, B is significantly better on 20 and worse on 5; C is better on 16 and worse on 10.

Live Kubernetes result (measured). Two identical Kubernetes clusters (1 control plane + 6 workers) ran the same load at the same time, one without Omni-Compass and one with it. With Omni-Compass: worker nodes in service 6.0 to 3.2; utilisation of the workers in service 0.068 to 0.116. **Energy:** with the parked workers kept on standby, powered and ready (100 W each, the same as idle), energy was 219 vs 218 Wh (-0.46%); parking alone saves essentially nothing; energy per unit of work +8% (not significant; 872 to 944 Wh per core-hour, from minute averages). The -43% first reported for this run holds only if parked workers are powered off. Waiting pods and HPA shortfall were not significantly different. The kill switch restored the original HPA target (50) and all 6 workers. **With every live muscle switched on (section 7.2) the application got slower: p95 response time 486 to**

802 ms, energy per unit of work +46% (significant). The causes were found and fixed across runs 2-4 (section 7.4): response time went from +65% to a tie at p95.

Where Omni-Compass costs something. In the pre-registered study both B and C keep more node-hours powered than Kubernetes alone and start and stop machines more often (more wear), and move the power cap more; C also lets more work wait in the queue and flips scale direction more often. In that study the energy saving comes from power capping and load shaping, not from switching machines off. On the live cluster the saving came from switching machines off. Other limits: the small-cluster release band (section 11); power and heat on the live cluster are modelled, not metered.

Trade-off in one line: B is the strongest all-round result in simulation (energy, health, recovery, queue, coordination and safety all better; wear and node-hours worse); C is the strongest on peak power, heat and safety (zero invariant violations) at the cost of queue length, wear and flip-flops. These are the gauges to tune next.

2. What Omni-Compass is

2.1 The engine

The engine is a six-state ordinary differential equation system, state $x = (E, U, I_U, S, B, B_{\dot{}})$: error E , coherence U , pressure I_U , stress S and a damped bath B . The shipped core (`omnicompass/core.py`) integrates it with fourth-order Runge-Kutta and holds the control input constant across the four stages:

```
(1) dE/dt = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u, |u| <= 25
(3) dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff = cos(omega_B t / 2) c tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S) = alpha_s S^2/2 + beta_s S^3/4 - delta S
(6) dS/dt = -dPhi/dS
(7) dB/dt = B_dot; dB_dot/dt = gamma_c delta S - (omega_B/Q_B) B_dot - omega_B^2 B
(8) R_B[n] = finite-difference audit of (7), never fed back
Controller: u = clip(-f_U(x,t) + 12 (sigma - U), -25, +25)
```

Telemetry (load, queue, power, heat, network, drift, staleness, security) is assimilated into the state each decision; an allocation law turns the state into a demand target ρ^* (the HPA target), a node change, and a power cap. Release of capacity is gated by equation (2): capacity is only released once the control push has converged.

2.2 The nervous system (muscles)

Each muscle has five parts: afferent (pull: sense), the shared engine, efferent (push: act), reflex (the shield checks every push) and kill (hand the muscle back to its own controller). Status in this repository: **wired live on Kubernetes** (sense and push executed through `kubectl`, each with shield and kill): nodes, HPA target, power cap (in-place CPU limits, enforced by the kernel), security hold, deployment rollouts (pause, resume, undo), batch queue (admit held Jobs); **sensed**: heat (harness heat law on live power, or GPU temperature), network; **hardware connectors** (built and tested with fake hardware, off on CI machines): CPU power states (RAPL read, `cpufreq` ceiling) and GPU (nvidia-smi power and temperature read, power limit); **open** (registered, no plant yet): memory, storage, cooling, grid, training, inference, agent containment and the rest of the 52-muscle domain map (`docs/DOMAIN_MAP.md`). AI value alignment is explicitly not an Omni-Compass muscle. Code: `omni_controller/controller.py`, `omni_controller/muscles.py`, `omnicompass/nervous.py`.

2.3 The shield and the kill switch

Before any action the shield (`omnicompass/shield.py`) enforces invariants I1 to I5: no expansion during a security block, node count within bounds, step limits, never below the capacity running and pending work needs, and the site power limit. The kill switch (a file or `OMNI_KILL=1`) restores every HPA target Omni-Compass changed from the recorded original, returns the node pool to its native size and drops to observe mode. Every decision and action is written to an append-only audit log.

2.4 Operating modes

Observe (compute and log, write nothing), target (write HPA targets), nodepool (also size a node pool). Laws: power_protect (enforce the site power envelope), throughput (power envelope not enforced; capacity sized at full power) and fleet variants tuned on the 15-second fleet harness.

3. The three architectures, and how each study realises them

Study	A. Kubernetes alone	B. Kubernetes + Omni-Compass	C. Omni-Compass direct	Live or simulated
Pre-registered held-out stack benchmark (2 x 500 scenarios)	<code>k8s_ref_70</code> : documented HPA law (target 0.7, 10% tolerance, 300 s stabilisation) and Cluster Autoscaler (scale-up on backlog, remove after 10 min under 50%), other managers act on their own proposals	<code>omni_k8s_throughput</code> : the same Kubernetes loops, Omni governs on top	<code>omni_direct</code> : Omni senses and actuates the stack directly	simulation
Control-plane replica (24 scenarios)	<code>hpa70_ca</code> : metrics-server, HPA and Cluster Autoscaler replicas at 15 s	<code>omni_target_gate_hpa70_ca</code> : Omni writes the HPA target and gates Cluster Autoscaler scale-down	<code>omni_throughput_full</code> : Omni writes the HPA target, owns node scale-down and power cap; the Cluster Autoscaler may only add nodes	simulation
PlanetLab-shaped demand, fleet plant (8 scenarios)	<code>k8s_hpa70_ca</code>	<code>omni_target</code> : Omni writes the HPA target	<code>omni_fleet</code> : Omni is the node-pool authority, Cluster Autoscaler off	simulation on recorded traces
Live kind cluster, side by side	native: HPA only, 6 workers always on, Omni not running	Omni on top: sets the HPA target and is the sole node-pool authority (cordon, drain, uncordon); Kubernetes' scheduler, kubelet and HPA still execute	not yet built live (section 16)	live

4. Method and why the comparison is fair

- **Frozen before testing.** The allocation law, engine parameters, shield limits, modes, baselines and benchmark code were selected on development seeds (1000, 2000) and frozen, with SHA-256 hashes and program fingerprints in `results/PREREGISTRATION.json`, before any run on the held-out seeds 346410161 and 360555127. `verify.py` re-checks every hash.
- **Same scenarios for every arm.** Each scenario is run under every architecture; comparisons are paired scenario by scenario.
- **Observe-identity check.** Omni-Compass in observe mode must produce a trajectory bit-identical to the arm it observes, or the run is invalid. Held-out: 500 and 500 of 500 identical to the native stack, 500 and 500 of 500 identical to Kubernetes; control-plane replica: 24 of 24; PlanetLab: 8 of 8; live: 0 writes while observing.

- **Statistics.** Differences are paired (Omni minus Kubernetes on the same scenario) with 95% bootstrap confidence intervals. In the held-out study a difference is called better or worse only if the interval excludes zero in the same direction on both independent seeds; 'better in N' counts scenarios. Live results use 2-minute blocks and a bootstrap over blocks.
- **Ablations** show the engine, not an accident of tuning, produces the result (section 5.3).
- **Independent implementations.** A C++ engine, governor, shield and HPA law are checked against the Python ones (section 8).

5. Results: pre-registered held-out benchmark (1,000 scenarios, simulation)

Mean over two held-out seeds x 500 scenarios; verdicts require both seeds to agree.

Gauge	A. Kubernetes alone	B. Kubernetes + Omni-Compass	C. Omni-Compass direct	B vs A	C vs A
ENERGY AND POWER					
Energy per scenario (kWh)	169.1	121.6	129.7	-28% better (better in 991, worse in 9 of 1000)	-23% better (better in 1000, worse in 0 of 1000)
Peak power (kW)	37.6	37.6	28.3	+0.16% not significant (better in 457, worse in 533 of 1000)	-25% better (better in 1000, worse in 0 of 1000)
Node-hours	135.8	140.5	152.4	+3% worse (better in 464, worse in 536 of 1000)	+12% worse (better in 191, worse in 809 of 1000)
Idle node-hours (powered, doing nothing)	46.29	51.73	62.03	+12% worse (better in 446, worse in 554 of 1000)	+34% worse (better in 202, worse in 798 of 1000)
Share of time over the power limit	0.133	0.067	0.018	-50% better (better in 490, worse in 24 of 1000)	-87% better (better in 517, worse in 5 of 1000)
Power-cap travel (how much the cap moved)	0.236	0.887	0.306	+276% worse (better in 37, worse in 963 of 1000)	+30% worse (better in 243, worse in 757 of 1000)
HEAT					
Share of time over the heat limit	0.084	0.026	0.002	-69% better (better in 353, worse in 1 of 1000)	-98% better (better in 354, worse in 0 of 1000)
Thermal travel (how far temperature swung)	0.754	0.632	0.576	-16% better (better in 828, worse in 172 of 1000)	-24% better (better in 993, worse in 7 of 1000)
SPEED AND BACKLOG					
Mean queue (work waiting)	384	217	933	-43% better (better in 218, worse in 515 of 1000)	+143% worse (better in 43, worse in 506 of 1000)
95th-percentile queue (worst moments)	1403	950	2945	-32% better (better in 247, worse in 190 of 1000)	+110% worse (better in 1, worse in 383 of 1000)
Share of time over the backlog limit	0.026	0.015	0.080	-42% better (better in 95, worse in 88 of 1000)	+207% worse (better in 1, worse in 267 of 1000)
RELIABILITY AND RECOVERY					
Availability	0.9990	0.9999	0.9974	+0.09% better (better in 110, worse in 127 of 1000)	-0.16% worse (better in 122, worse in 121 of 1000)
Share of time healthy	0.789	0.913	0.901	+16% better (better in 661, worse in 4 of 1000)	+14% better (better in 659, worse in 4 of 1000)
Share of incidents recovered	0.918	0.996	0.972	+8% better (better in 78, worse in 0 of 1000)	+6% better (better in 54, worse in 0 of 1000)

Gauge	A. Kubernetes alone	B. Kubernetes + Omni-Compass	C. Omni-Compass direct	B vs A	C vs A
Recovery time (minutes)	57.8	16.5	27.9	-71% better (better in 602, worse in 0 of 1000)	-52% better (better in 580, worse in 0 of 1000)
Physical SLA breaches (power, heat, backlog)	0.144	0.073	0.089	-49% better (better in 495, worse in 25 of 1000)	-38% better (better in 449, worse in 46 of 1000)
All SLA breaches	0.168	0.073	0.089	-57% better (better in 655, worse in 3 of 1000)	-47% better (better in 611, worse in 37 of 1000)
WEAR AND TEAR					
Machines started	4.8	6.6	9.4	+37%	+95%
Machines stopped	1.2	3.3	2.7	+185%	+130%
Node start/stop events	6.0	9.9	12.1	+66% worse (better in 56, worse in 907 of 1000)	+102% worse (better in 199, worse in 782 of 1000)
Machine round trips (stopped then restarted)	1.07	1.73	2.70	+61% worse (better in 119, worse in 464 of 1000)	+152% worse (better in 26, worse in 866 of 1000)
Scale reversals (flip-flops)	2.20	1.93	5.45	-13% better (better in 474, worse in 199 of 1000)	+147% worse (better in 86, worse in 803 of 1000)
CONTROL QUALITY AND SAFETY					
Contradictory commands between managers	9.46	0.00	0.00	-100% better (better in 666, worse in 0 of 1000)	-100% better (better in 666, worse in 0 of 1000)
Safety-rule (invariant) violations	9.50	2.71	0.00	-72% better (better in 664, worse in 1 of 1000)	-100% better (better in 669, worse in 0 of 1000)
Invariant violations excluding power	6.86	0.00	0.00	-100% better (better in 666, worse in 0 of 1000)	-100% better (better in 666, worse in 0 of 1000)
Security violations	0.889	0.000	0.000	-100% better (better in 118, worse in 0 of 1000)	-100% better (better in 118, worse in 0 of 1000)
Pages to on-call	0.584	0.000	0.000	-100% better (better in 92, worse in 0 of 1000)	-100% better (better in 92, worse in 0 of 1000)
Human interventions required	0.284	0.000	0.000	-100% better (better in 92, worse in 0 of 1000)	-100% better (better in 92, worse in 0 of 1000)

5.1 What the numbers say

- **Energy:** B -28%, C -23% versus Kubernetes alone, although both keep slightly more node-hours powered. The saving comes from power capping and load shaping: time over the power limit halves (B) or nearly vanishes (C), and C cuts peak power by a quarter. Releasing idle machines, which the live cluster showed, is not what drives this study; idle node-hours are a gauge to tune.
- **Reliability:** time healthy and recovery improve sharply in both B and C; the share of incidents recovered rises.
- **Coordination:** separate managers issue contradictory commands (A); a single authority issues none (B, C). Pages and human interventions go to zero because Omni-Compass acts on the conditions that would have paged someone.
- **Heat:** C cuts time over the heat limit the most, because it governs power caps and load together.
- **Costs:** node start/stop events rise (B +66%, C +102%), machine round trips rise, the power cap moves more, and C's queue and scale reversals are higher than Kubernetes alone. The 24-scenario control-plane study (section 6) shows the opposite for wear (fewer machine stops), so wear depends on the plant and law and is a primary tuning target. These are reported, not tuned away.

5.2 Architecture A has a hidden cost: the fragmented stack without Kubernetes

For reference, the stack with each manager acting alone and no Kubernetes loops (native) used 131.9 kWh, was healthy 39% of the time, took 127 minutes to recover and produced 56.7 contradictory commands and 28.1 invariant violations per scenario. Kubernetes already improves on that; Omni-Compass improves on Kubernetes.

5.3 Ablations: is it the engine?

Comparison	Metric	Mean difference	95% CI (seed 1)	95% CI (seed 2)
Omni direct vs omni_direct_no_dynamics	energy_kwh	-1.954	[-2.061, -1.775]	[-2.158, -1.834]
Omni direct vs omni_direct_no_dynamics	invariant_violations	+0.000	[+0.000, +0.000]	[+0.000, +0.000]
Omni direct vs omni_direct_no_dynamics	mean_queue	+39.200	[-4.742, +71.297]	[+7.307, +76.997]
Omni direct vs omni_direct_no_dynamics	recovery_minutes	+0.950	[-0.090, +2.650]	[-0.410, +2.260]
Omni direct vs omni_direct_no_dynamics	time_healthy	-0.000	[-0.004, +0.002]	[-0.002, +0.003]
Omni direct vs omni_direct_no_dynamics	violation_heat	-0.002	[-0.004, -0.000]	[-0.005, -0.000]
Omni direct vs omni_direct_no_shield	energy_kwh	-0.759	[-0.864, -0.642]	[-0.875, -0.660]
Omni direct vs omni_direct_no_shield	invariant_violations	-0.674	[-0.750, -0.592]	[-0.760, -0.596]
Omni direct vs omni_direct_no_shield	mean_queue	+121.945	[+90.814, +149.893]	[+100.054, +152.127]
Omni direct vs omni_direct_no_shield	recovery_minutes	+1.535	[+0.560, +3.210]	[+0.480, +2.600]
Omni direct vs omni_direct_no_shield	time_healthy	+0.001	[-0.001, +0.003]	[-0.000, +0.003]
Omni direct vs omni_direct_no_shield	violation_heat	-0.000	[-0.000, +0.000]	[-0.001, +0.000]

'no_dynamics' runs the same allocation law with equations (1) to (7) frozen; 'no_shield' removes the shield. The difference is full engine minus ablation: negative energy means the evolving dynamics save energy the frozen engine does not; the shield ablation shows what the shield prevents.

6. Results: Kubernetes control-plane replica (24 scenarios, simulation)

Replicas of the documented metrics-server, HPA (15 s) and Cluster Autoscaler (10 s) loops, with Karpenter-lite as a second Kubernetes reference; Omni-Compass decides every 300 s.

Gauge	A. Kubernetes alone (HPA + Cluster Autoscaler)	Reference: HPA + Karpenter-lite	B. Kubernetes + Omni-Compass (HPA target + CA gate)	C. Omni-Compass authority (HPA target, nodes, power cap)	C vs A (95% CI, wins of 24)
Energy (kWh)	168.8	161.7	155.6	122.3	-28% better [-49.724, -43.292], 24 of 24
Peak power (kW)	38.7	39.5	40.8	32.8	-15% better [-8.962, -2.584], 17 of 24
Time over power limit	0.050	0.050	0.048	0.015	-70% better [-0.051, -0.019], 14 of 24
Time over heat limit	0.045	0.047	0.045	0.012	-73% better [-0.048, -0.018], 13 of 24
Time healthy	0.930	0.937	0.931	0.983	+6% better [+0.033, +0.073], 18 of 24
Recovery time (min)	1.857	1.982	3.430	0.430	-77% better [-1.948, -0.906], 15 of 24
Physical SLA breaches	0.031	0.032	0.031	0.009	-71% better [-0.032, -0.013], 14 of 24

Gauge	A. Kubernetes alone (HPA + Cluster Autoscaler)	Reference: HPA + Karpenter-lite	B. Kubernetes + Omni-Compass (HPA target + CA gate)	C. Omni-Compass authority (HPA target, nodes, power cap)	C vs A (95% CI, wins of 24)
Mean queue	0.204	0.204	1.011	0.683	+235% worse [+0.168, +0.836], 0 of 24
Machines started	21.8	24.9	25.5	20.0	-8% better [-3.542, -0.250], 4 of 24
Machines stopped	13.2	17.0	24.1	3.833	-71% better [-11.917, -7.083], 24 of 24
Scale reversals	1.333	1.917	2.000	1.000	-25% better [-0.667, -0.083], 4 of 24
Invariant violations	0.000	0.000	0.000	0.000	0% not significant [+0.000, +0.000], 0 of 24
Pages to on-call	0.000	0.000	1.792	0.458	new not significant [+0.000, +1.375], 0 of 24

7. Results: live Kubernetes (measured)

7.1 Side by side, two identical clusters at the same time

kind clusters (real Kubernetes API server, scheduler, kubelet, HPA and metrics-server), 1 control plane + 6 workers each; the official php-apache HPA workload (target 50); the same stepped load (1, 2, 3, 1, 2, 1 load generators over 20 minutes after a 2-minute warm-up); capture every 15 s. Native: Omni-Compass not started. Omni: nodepool mode, HPA target, node pool, power sensing. Run 36209933218, 2026-09-26.

Gauge	Kubernetes alone	Kubernetes + Omni-Compass	Change
Duration (min)	20.10	20.13	
Worker nodes in service, mean	6.00	3.20	-46.7%
Worker nodes in service, min	6.00	3.00	-50.0%
CPU used (cores), mean	0.819	0.745	-9.1%
CPU allocatable (cores), mean	12.00	6.40	-46.7%
Utilisation	0.068	0.116	+70.5%
Workers powered (in service + standby)	6.00	6.00	0
Node-hours in service	2.01	1.07	-46.6%
Power (W), mean, parked workers on standby at 100 W	657	654	-0.5%
Power (W), peak, parked workers on standby at 100 W	691	687	-0.6%
Energy (Wh), parked workers on standby at 100 W	219	218	-0.5%
Energy (Wh), parked workers in low-power standby at 50 W	219	171	-22%
Energy (Wh), parked workers in deep sleep at 10 W	219	133	-39%
Energy (Wh), parked workers powered off (as first reported)	220	125	-43%
Pending pods, pod-minutes	0.567	0.267	-52.9%
Pending pods, peak	1	1	0
HPA replicas, mean	8.71	6.20	-28.8%
HPA replicas, peak	10	8	-20.0%
HPA shortfall, minutes	0.833	1.07	+28.0%

Metric (per unit of work, 2-minute blocks)	Kubernetes alone	Kubernetes + Omni-Compass	Change	95% CI	Verdict
Wh per core-hour, standby at 100 W (2-minute blocks from minute averages)	872	944	+8.2%	[-150, +292]	not significant
Node-hours in service per core-hour	7.8535	4.6976	-40.2 %	[-5.0982, -1.1072]	fewer (not energy while parked nodes stay powered)
Utilisation	0.0694	0.1224	+76.3 %	[+0.0217, +0.0830]	better
Pending-pod minutes per hour	1.7253	0.8333	-51.7 %	[-3.4961, +1.6667]	not significant
HPA shortfall minutes per hour	2.5458	3.3399	+31.2 %	[-3.3277, +5.0066]	not significant

Omni-Compass decisions: nodes per minute 5 4 3 then 3 for the remaining 17 minutes; HPA target 78-79% (Kubernetes alone: 50%). Node-pool resizes: 3 (three workers cordoned and drained, their pods rescheduled by Kubernetes). Kill switch: HPA target restored to 50, 6 of 6 workers back in service.

7.2 Live, side by side, every live muscle (run 36213152881 (2026-09-26 02:54-03:19 UTC))

Same two-cluster setup; the Omni arm drives HPA target, node pool, power cap (in-place pod CPU limits), heat (harness law on live power), security hold, rollout guard. Real response times: HTTP requests to the app timed every 5 s on both clusters. **This run made the application slower.**

Gauge	Kubernetes alone	Kubernetes + Omni-Compass	Change
Worker nodes in service, mean	6.00	3.20	-46.7%
Node-hours	2.01	1.07	-46.8%
Power (W), mean, standby counted	664	643	-3.2%
Power (W), peak	720	664	-7.7%
Energy (Wh), standby counted	222	215	-3.4%
CPU used (cores), mean	0.908	0.593	-34.6%
Utilisation	0.076	0.093	+22.6%
Energy per core-hour (Wh)	731	1,083	+48.1% worse
Pending pods, pod-minutes	1.70	0	-100%
HPA replicas, mean	9.09	5.00	-45.0%
HPA replicas, peak	10	5	-50.0%
Requests timed	955	850	-11.0%
Response time (ms), mean	260	415	+59.7% worse
Response time (ms), median	230	320	+39.2% worse
Response time (ms), 95th percentile	486	802	+64.8% worse
Response time (ms), 99th percentile	675	1,113	+64.7% worse
Failed requests (%)	0	0	0

Metric (2-minute blocks)	Kubernetes alone	+ Omni-Compass	Change	95% CI	Verdict
kWh per core-hour	0.7596	1.1107	+46.2%	[+0.1901, +0.5099]	worse
Node-hours per core-hour	6.8941	5.6440	-18.1%	[-2.7717, +0.4577]	not significant
Utilisation	0.0766	0.0964	+25.8%	[-0.0016, +0.0394]	not significant

Actions: hpa target writes 1, node pool resizes 3, power cap pod resizes 7, rollout actions 0, power cap per minute 0.695 every minute (the law floor), hpa target 77%, nodes 5 4 3 then 3. Kill switch: restored target 50; workers 6 of 6; pod CPU limits restored.

Diagnosis: The engine saw the cluster as idle (node utilisation about 7%) and had no response-time signal, so it capped the app pods at the law floor and packed replicas to a 77% target (5 instead of 10). The app pods were busy, so responses slowed. Fix wired: latency afferent (p95 over a declared 500 ms SLO enters as queue pressure) and a power-cap reflex (never below pod usage x 1.3); re-run pending.

7.3 Other live runs

Run	Setup	Result
live-kind, run 36205408388	1 node; baseline 10 min (Omni observing) then Omni target mode 10 min	0 writes while observing; kill switch restored 50; pending-pod minutes -48.6% (significant, but the baseline phase included warm-up); energy per core-hour no significant difference (one node cannot be parked)
live-kind-full, run 36205869009	1 control plane + 3 workers	node pool never resized: 3 of 3 every minute at about 8% utilisation, because the fleet law needs more than 3 nodes of slack; kill switch restored target and workers
live-kind-full, run 36207925928	1 control plane + 6 workers; sequential baseline then full engine	nodes in service 6 to 5 to 4 to 3; node-hours in service per core-hour -38.2%, utilisation +63.7% (significant); the reported kWh per core-hour -35.8% counted parked workers as powered off; with parked workers on standby the saving largely disappears; kill switch restored target 50 and 6 of 6 workers

7.4 Live runs after the first all-muscle run

Run	Change tested	Validity	Result
36214629046 (2026-09-26 03:23-03:48 UTC), commit c182a63	latency afferent (p95 over 500 ms SLO as queue pressure) and power-cap usage reflex	valid	p95 481 to 600 (+24.6% worse); p99 596 to 777 (+30.3% worse); energy 222 to 219 (-1.5%)
36216647786 (2026-09-26 04:02-04:28 UTC), commit 052b705	SLO reflex; eviction receipt fixed	INVALID as a test of the engine	p95 479 to 320 (-33.2%); p99 583 to 420 (-28.0%); energy 224 to 220 (-1.7%)
36218637030 (2026-09-26 04:42-05:08 UTC), commit 47d0353	controller fail-safe; benchmark prints controller log and rejects early stops	PARTIAL	p95 491 to 492 (+0.3% (tie)); p99 645 to 683 (+5.8% worse); energy 223 to 221 (-1.1%)

Run 3 is recorded as invalid: a refused Kubernetes call stopped the controller after 3 of 20 decisions and the cluster held Omni's last settings with no engine running. The fix is a fail-safe: a failed decision is skipped; three in a row restore native settings and stop the controller. In run 4 the fail-safe did exactly that at minute 16, and the audit showed why: the least-privilege role allowed writing a pod's CPU limit but not reading it first, which kubectldoes. In runs 3 and 4 no power cap was ever applied. Over run 4's 16 governed minutes node-hours per unit of work fell 24% and utilisation rose 36% (both significant); median response time -10%, p95 a tie, p99 +6% (not significant). The permission is fixed (with a receipt) and a run in which the fail-safe fires is now rejected; the re-run is in progress.

8. Engineering verification

python verify.py runs 89 checks (0 failures in the recorded log results/VERIFY_LOG.txt), including:

- reference engine SHA-256
- core parity vs reference engine and 500 fixtures
- C++ core vs 500 fixtures (max abs 3.55e-15)
- C++ governor vs Python governor (throughput)
- C++ shield vs Python shield (enforce and violations)
- negative control: parity test detects a C++ shield that no longer blocks rollouts during a security block
- independent C++ HPA replica law vs fleet harness HPA
- soak test (throughput), 100,000,000 decisions, no failures, memory flat after warm-up (2899 ns per decision, RSS checkpoints [3840, 3840, 3840, 3840] kB)
- held-out seed 346410161: observe mode identical to native
- live controller against a fake cluster: observe writes nothing, target bounded, kill restores, node pool bounded and dry-run safe

Also tested: the live controller against a fake cluster (observe writes nothing; targets bounded; kill restores from the HPA annotation after a restart; node pool bounded and dry-run safe), the full-engine options (parked and control-plane nodes excluded; the kill switch restores the node pool exactly once) and pilot scoring (detects a real gain, reports no gain on identical clusters, detects a service regression).

9. Physics and models

- **Stack plant power:** each node draws idle 0.38 kW plus 1.12 kW x utilisation; a power cap throttles delivered capacity.
- **Heat:** thermal state follows a first-order lag toward $0.34 + 0.62 \times$ power stress (time constant about 7 steps); heat above 0.82 throttles capacity; 'over the heat limit' means thermal above 1.03.
- **Live kind cluster:** kind nodes have no power meter, so power is a declared model: 100 W idle + 150 W x CPU utilisation per worker in service, plus a standby power for each parked (cordoned and drained) worker. Standby defaults to the idle power (the worker stays powered and ready); lower values apply only to a declared sleep state, zero only to machines really powered off. The first live reports counted parked workers as zero; section 7.1 gives both. The same constants drive the governor's power sense and the energy score, so they cannot disagree. On real hardware this is replaced by metered power (RAPL, PDU or BMC).
- **Savings model** (results/SAVINGS.csv): a 1,000-node web cluster at 0.4 kW per node, PUE 1.4, \$0.12/kWh and 0.4 kg CO₂/kWh; reduction versus HPA 0.7 + Karpenter-lite of 14% to 20% (fleet plant) gives roughly 710 to 960 MWh, \$85,000 to \$115,000 and 280 to 380 t CO₂ per year.
- **Engine overhead:** about 2.9 microseconds per decision in C++, memory flat over 100 million decisions.

10. What is proven, what is simulated, what is not claimed

Claim	Status	Evidence
Engine equations are finite and deterministic; C++ equals Python	Proven	verify.py, 500 fixtures, max error 3.6e-15
Observe mode changes nothing	Proven (simulation and live)	bit-identical trajectories; 0 writes live
Kill switch restores native control	Proven (simulation and live)	HPA target 50 and all workers restored live
Omni-Compass acts on a real Kubernetes control plane (HPA target, node pool)	Proven live	section 7
Fewer nodes in service than Kubernetes with a fixed node pool, same load served	Measured live	section 7.1
Lower energy on the live cluster	Not shown while parked nodes stay on standby; -22% to -43% only if parked nodes sleep or power off	section 7.1
Better energy, health, recovery, coordination than Kubernetes (HPA + CA)	Pre-registered simulation	section 5
Better than Karpenter-lite on energy	Simulation	sections 6, and PlanetLab fleet plant
Better than upstream Karpenter or Cluster Autoscaler binaries, live	Not yet tested	section 12
Metered energy savings on physical servers	Not yet tested	power is modelled
GPU power-limit muscle saves 15-19% energy with <1% slower responses	Simulation calibrated to metered H100 data (MLPerf)	section 12
CPU frequency muscle	Simulation (uncalibrated): about -3.5% energy, -67% heat	section 12
Tuned laws remove wear and node-hour negatives (C-throughput)	Pre-registered amendment, new held-out data	section 13
Cooling, grid, memory, storage and the other open muscles	Not claimed	no plant or connector yet
Makes AI models aligned or trustworthy	Not claimed	value alignment is outside Omni-Compass

11. Limits and threats to validity

- Simulated studies use documented-behaviour replicas of Kubernetes controllers, not the upstream binaries; the Kubernetes reference omits Karpenter consolidation, VPA, scheduling constraints and disruption budgets.
- The live cluster is kind: nodes are containers on one CI machine; the two live arms ran on two machines at the same time, so machine-to-machine variation is part of the noise; each live arm is 20 minutes, one repetition.
- Live power and heat are modelled; parked kind workers are drained containers. If parked machines must stay on standby, parking reduces nodes in service but not energy; live energy savings then have to come from power caps, CPU power states and heat control, which are not yet wired live.
- The live native arm had no node autoscaler, so its node pool was always full; the fair live opponent is Karpenter or Cluster Autoscaler (section 16).
- The live significance for energy per core-hour with standby power is computed from minute averages (10 two-minute blocks), not from the 15-second capture.
- The fleet law releases a node only when the pool has more than three nodes of slack; a three-worker pool cannot scale down (observed live, reproduced offline). Small clusters need a pool-size-aware release band.

- Architecture C was measured in simulation only; the live C (Kubernetes' controllers parked, Omni-Compass as the only brain) is not built yet.
- Service quality differences in the live runs (pending pods, HPA shortfall) are not statistically significant at this run length.

12. GPU and CPU muscles: device plant calibrated to metered hardware (simulation)

The GPU power-limit and CPU frequency muscles cannot be actuated on CI machines (no GPU; the hypervisor hides RAPL and cpufreq). Their connectors are built (section 2.2) and this plant shows what they do. **GPU calibration:** the performance-versus-power-limit exponent is fitted to MLPerf Inference v4.0 results for an NVIDIA DGX-H100 (8 x H100-SXM, 700 W TDP), MaxQ (power-limited with `nvidia-smi -pl`, the same command the Omni-Compass GPU connector sends) versus MaxP, system power metered by a Yokogawa WT333E; Apache 2.0. Fitted exponent 0.353 to 0.490 (median 0.449); all three are run. **CPU:** a standard first-order model (dynamic power $\sim$ frequency cubed) against a schedutil-style governor; not calibrated to metered data yet. **S** is a fixed manual 70% cap, what an operator could do by hand. 24 scenarios, 8 load families; * = paired 95% interval excludes zero.

Vessel	Gauge	A. Native	S. Fixed 70% cap	B. + Omni-Compass	C. Omni-Compass direct
cpu_web	Energy (kWh)	115.169	103.342 (-10%*)	110.899 (-4%*)	111.238 (-3%*)
cpu_web	Energy per unit of work	0.672	0.614 (-9%*)	0.650 (-3%*)	0.652 (-3%*)
cpu_web	95th-pct response time (x baseline)	1.000	6.577 (+558%*)	1.040 (+4%*)	1.040 (+4%*)
cpu_web	Minutes over service target	0.000	70.458 (new*)	0.167 (new)	0.167 (new)
cpu_web	Minutes over heat limit	26.708	0.000 (-100%*)	8.875 (-67%*)	8.958 (-66%*)
cpu_web	Peak power (kW)	33.954	20.475 (-40%*)	34.074 (+0.35%)	34.074 (+0.35%)
cpu_web	Minutes near full power	8.000	0.000 (-100%*)	10.167 (+27%*)	10.167 (+27%*)
gpu_mlperf_median	Energy (kWh)	142.255	117.038 (-18%*)	119.254 (-16%*)	119.254 (-16%*)
gpu_mlperf_median	Energy per unit of work	0.824	0.679 (-18%*)	0.691 (-16%*)	0.691 (-16%*)
gpu_mlperf_median	95th-pct response time (x baseline)	1.000	1.521 (+52%*)	1.007 (+0.70%*)	1.007 (+0.70%*)
gpu_mlperf_median	Minutes over service target	0.000	7.125 (new*)	0.042 (new)	0.042 (new)
gpu_mlperf_median	Minutes over heat limit	22.458	0.000 (-100%*)	6.125 (-73%*)	6.125 (-73%*)
gpu_mlperf_median	Peak power (kW)	38.403	28.288 (-26%*)	37.089 (-3%*)	37.089 (-3%*)
gpu_mlperf_median	Minutes near full power	6.500	0.000 (-100%*)	6.208 (-4%)	6.208 (-4%)
gpu_mlperf_least_favourable	Energy (kWh)	142.255	118.430 (-17%*)	120.770 (-15%*)	120.770 (-15%*)
gpu_mlperf_least_favourable	Energy per unit of work	0.824	0.688 (-17%*)	0.699 (-15%*)	0.699 (-15%*)
gpu_mlperf_least_favourable	95th-pct response time (x baseline)	1.000	1.628 (+63%*)	1.008 (+0.80%*)	1.008 (+0.80%*)
gpu_mlperf_least_favourable	Minutes over service target	0.000	8.333 (new*)	0.042 (new)	0.042 (new)
gpu_mlperf_least_favourable	Minutes over heat limit	22.458	0.000 (-100%*)	6.708 (-70%*)	6.708 (-70%*)
gpu_mlperf_least_favourable	Peak power (kW)	38.403	28.345 (-26%*)	37.532 (-2%*)	37.532 (-2%*)

Vessel	Gauge	A. Native	S. Fixed 70% cap	B. + Omni-Compass	C. Omni-Compass direct
gpu_mlperf_least_favourable	Minutes near full power	6.500	0.000 (-100%*)	6.417 (-1%)	6.417 (-1%)
gpu_mlperf_most_favourable	Energy (kWh)	142.255	113.878 (-20%*)	115.645 (-19%*)	115.645 (-19%*)
gpu_mlperf_most_favourable	Energy per unit of work	0.824	0.661 (-20%*)	0.670 (-19%*)	0.670 (-19%*)
gpu_mlperf_most_favourable	95th-pct response time (x baseline)	1.000	1.300 (+30%*)	1.006 (+0.55%*)	1.006 (+0.55%*)
gpu_mlperf_most_favourable	Minutes over service target	0.000	5.375 (new*)	0.042 (new)	0.042 (new)
gpu_mlperf_most_favourable	Minutes over heat limit	22.458	0.000 (-100%*)	5.250 (-77%*)	5.250 (-77%*)
gpu_mlperf_most_favourable	Peak power (kW)	38.403	28.133 (-27%*)	36.174 (-6%*)	36.174 (-6%*)
gpu_mlperf_most_favourable	Minutes near full power	6.500	0.000 (-100%*)	5.833 (-10%*)	5.833 (-10%*)

Reading: on GPUs Omni-Compass saves 15-19% energy across the measured range with response time 0.6-0.8% slower; a fixed cap saves about the same energy but slows responses 30-63% and misses the service target. On CPUs the native governor already tracks demand, so frequency control alone saves about 3.5%; its main effect is heat (-67%). CPU frequency control is below the 10% bar and is a physical ceiling of that muscle, not a tuning gap.

13. Amendment: tuned laws, frozen, then tested on new held-out data (simulation)

Two law variants were tuned on the development seeds only (1000, 2000; tuning/SEARCH*.json), frozen with SHA-256 hashes in tuning/PREREGISTRATION_AMENDMENT_2026-09-26.json and pushed (commit b79d1dd, 03:17 UTC) before a single run on new held-out seeds 731001 and 731002 (2 x 500 scenarios). The frozen engine files are unchanged; the original pre-registered result (section 5) stays the primary result. B-wear: node release held longer. C-throughput: Omni-Compass direct under the throughput law with a wider release band and engine-gated cap. + better, ! worse (both seeds agree), blank not significant.

Gauge	A. Kubernetes alone	B_frozen	B_wear	C_frozen	C_throughput
energy_kwh	170.001	122.670 (-28%) +	123.552 (-27%) +	130.042 (-24%) +	123.353 (-27%) +
node_start_stop	6.119	10.070 (+65%) !	8.210 (+34%) !	11.985 (+96%) !	3.065 (-50%) +
machine_round_trips	1.031	1.724 (+67%) !	0.831 (-19%) +	2.711 (+163%) !	0.863 (-16%) +
scale_reversals	2.176	1.931 (-11%) +	1.805 (-17%) +	5.394 (+148%) !	1.838 (-16%) +
node_hours	136.812	141.488 (+3%) !	144.688 (+6%) !	152.143 (+11%) !	122.531 (-10%) +
idle_node_hours	46.662	52.152 (+12%) !	55.353 (+19%) !	61.218 (+31%) !	35.788 (-23%) +
mean_queue	438.529	246.346 (-44%) +	246.324 (-44%) +	1103.704 (+152%) !	1029.828 (+135%) !
p95_queue	1462.260	1005.114 (-31%) +	1005.044 (-31%) +	3185.168 (+118%) !	2684.025 (+84%) !
violation_backlog	0.030	0.016 (-45%) +	0.016 (-45%) +	0.085 (+182%) !	0.064 (+115%) !
recovery_minutes	61.470	16.805 (-73%) +	16.805 (-73%) +	28.925 (-53%) +	37.700 (-39%) +
power_cap_travel	0.249	0.894 (+259%) !	0.893 (+259%) !	0.307 (+23%) !	0.697 (+180%) !
invariant_violations	9.625	2.748 (-71%) +	2.747 (-71%) +	0.000 (-100%) +	0.000 (-100%) +

Gauge	A. Kubernetes alone	B_frozen	B_wear	C_frozen	C_throughput
sla_violation_total	0.171	0.076 (-55%) +	0.076 (-55%) +	0.094 (-45%) +	0.145 (-15%) +
pages	0.671	0.000 (-100%) +	0.000 (-100%) +	0.000 (-100%) +	0.000 (-100%) +
time_healthy	0.785	0.909 (+16%) +	0.909 (+16%) +	0.896 (+14%) +	0.837 (+7%) +
availability	0.999	1.000 (+0.09%) +	1.000 (+0.09%) +	0.997 (-0.23%) !	0.996 (-0.32%) !
recovered	0.907	0.997 (+10%) +	0.997 (+10%) +	0.971 (+7%) +	0.963 (+6%) +

Worse gauges: B_frozen 5, B_wear 4, C_frozen 10, C_throughput 5.

14. Response time: can one mode beat Kubernetes on every gauge? (simulation, development seeds)

The fleet-mode constants that drove the live runs were selected (before this work) by a rule that scored energy, finished work and backlog, but never waiting time. A response-time gauge was added to the fleet plant (fleet/sim_slo.py: M/M/c queueing delay per workload plus backlog drain; the frozen trace is reproduced exactly). Measured with it, fleet mode cut energy 31% on web services while p95 response time went from 132 ms to about 20 s. That is where the live slowdown came from: its HPA target floor (rho_min = 76.9%) packs pods too hot for latency-sensitive services.

A speed-first law (omnicompass/speed.py; the engine equations unchanged) separates the two jobs the frozen law gave one number: pods run at a latency-safe target while machines are sized to what the pods request. 400 configurations were searched against HPA 70% + Cluster Autoscaler with the rule that no gauge may be worse in any development scenario. None passed, for three measured reasons: (1) several gauges are already at their physical floor (100% work done, zero violations, batch p95 = pure service time), so a tie is the best any controller can do; (2) on the GPU vessel demand exceeds the hardware, so every arm is saturated; (3) energy, response time and machine churn trade two-of-three, because the autoscaler's idle slack is both where the energy is and what absorbs the 90-second boot delay. Examples, mean change vs Kubernetes: fleet mode energy -31% with far worse response time; speed #176 energy -1%, p99 -41%, churn 3.6x; speed #159 p99 -48%, churn -27%, energy +16%. Source: tuning/SPEED_FINDINGS_2026-09-26.md.

15. Named products: OpenShift, Google GKE, Azure AKS, IBM Turbonomic (simulation, development seeds)

Each product is emulated from its documented behaviour on the same fleet plant (not the vendors' binaries): OpenShift's documented ClusterAutoscaler example (threshold 0.4, unneeded 5 min, delay after add 10 min); GKE optimize-utilization (MostAllocated packing, more aggressive scale-down; declared as threshold 0.65, 2 min, because Google publishes no numbers); AKS node auto-provisioning (Karpenter, WhenEmptyOrUnderutilized, consolidateAfter 0 s); Turbonomic (container requests resized every 10 min to p99 per-pod usage, its default aggressiveness; nodes suspended toward 0.7 packing). Means over web services, 4 development seeds:

Gauge (web)	Kubernetes (GKE balanced)	OpenShift	GKE optimize	AKS NAP	Turbonomic	Omni fleet mode	Omni speed #159
energy (kWh)	12.19	12.53	10.4	9.779	10.08	8.392	15.9
p95 (ms)	131.6	131.6	132.2	132.7	125.1	2.034e+04	112.3
p99 (ms)	3,095	2,815	3,947	4,295	4.94e+04	7.634e+04	129.8
machine starts+stops	10.5	8.5	14.75	19	11.5	13.25	6.75
scale reversals	1.5	0.75	3.5	6	1.5	4.25	1

Every product sits on the same energy / response-time / churn triangle; none wins all three. Carpenter-style consolidation (AKS NAP) saves about 20% energy with a worse p99 tail and about twice the machine churn; Turbonomic's p99 resizing interacts with the HPA and inflates the tail; OpenShift's documented settings sit close to upstream. Omni's speed mode leads on response time and churn at an energy cost; its fleet mode leads on energy at a large response-time cost.

16. The problem-map muscles: all nine built and tested on held-out data (simulation)

Each open row of the industry problem map (section 18) is now a muscle in omnilib/, with the strongest native tool as opponent, Omni-Compass, and Omni-Compass with its engine equations not evolved (to show what the equations themselves add). Constants were chosen on development seeds 1-8, every file was frozen by SHA-256 (results/muscles/PREREGISTRATION.json), then 30 held-out seeds were run once. Verdicts from a paired bootstrap 95% interval.

Muscle (map row)	Strongest native opponent	Omni better	Omni worse	Tie / not significant
Right-sizing, HPA+VPA conflict (1, 2)	hpa_vpa	cpu core hours -3%; mem gib hours -11%; p99 ms -99%; replica reversals -84%; slo breach min -57%	p95 ms +15%; oom kills +531%	work done
Cold start (5)	keda	p95 ms -75%; p99 ms -58%; delayed 1s pct -71%; instance hours -8%	cold starts +736%	-
GPU packing (6)	binpack	energy kwh -2%; idle gpu hours powered -20%	wait mean min +16%; migrations 0 to 6.07	wait p95 min, frag blocked min, jobs done
Training power swings (7)	floor_safe	energy overhead pct -9%; throughput loss pct -44%; swing 1s mw -6%	max ramp mw s +127%	ramp violation s
GPU failures and stragglers (8)	detect	goodput pct +10%; lost gpu hours -20%; restarts -29%; straggler node hours -99%	-	checkpoint overhead pct
Cooling (10)	reset	pue -2%; cooling mwh -13%; inlet violation min -73%	-	max inlet c
LLM inference, KV cache (12)	keda	ttft p95 s -18%; slo breach pct -58%; gpu hours -19%; preemptions -56%	-	ttft p50 s, tpot p95 ms
Runaway AI agents (13)	static	rogue overspend usd -96%; time to contain min -99%; peak subagents -59%	false stops +357%; honest work pct -7%	forbidden executed
Energy attribution in VMs (11)	ratio	attr error pct -37%; worst vm error pct -34%	-	total error pct

Each entry is the plain change of Omni-Compass against the opponent (for example p99 -99% means the slow tail is 99% shorter; goodput +10% means 10% more useful training time). Worse entries are real costs, not rounding: right-sizing trades a slightly slower typical response (p95 +15%, both far under the 500 ms target) and a few more memory kills for a 99% shorter tail; cold start keeps fewer idle instances than KEDA's 5-minute cooldown, so more requests meet a cold start, but users wait far less because KEDA polls every 30 s; GPU packing powers idle GPUs off sooner and jobs wait a few seconds longer; containment throttles honest agents in their legitimate bursts (about 7% of their work) and stops about two honest agents a day, against a 96% cut in runaway spend and containment in minutes instead of hours. Against the other native arms (the default tools most teams run) the wins are larger; every comparison is in results/muscles/*_HELDOUT.json.

What the engine itself contributes. Across the nine muscles the full engine and the engine-not-evolved arm differ by 1.0 percentage points on average per gauge. The mapping from engine state to action carries most of each result; the evolved dynamics mainly smooth. In power smoothing an arm where the bath equation (7) alone sets the site's draw, with no ramp rule, cut the steepest ramp 63% but did not hold the grid's limit: the human sets the boundary, the engine operates inside it.

17. Every negative, its cause and its status

Negative	Where	Status	Cause	What would fix it
Node start/stop cycles (wear)	B, pre-registered	reduced (+65% to +34%), not removed	node release thresholds are fixed numbers	hybrid: Kubernetes serves the queue, Omni releases nodes with the C-throughput law (removed wear there: -50%)
Node start/stop, round trips, reversals	C, pre-registered	fixed in C-throughput (new held-out)	power-protect law released nodes too eagerly	adopted in C-throughput
Node-hours and idle node-hours	B and C, pre-registered	fixed in C-throughput (-10%, -23%); not in B	power capping trades lower watts for more servers on	hybrid as above; node-aware cap
Queue / backlog / availability -0.3%	C (both variants)	UNRESOLVED	not the replica law and not the sizing constants (both tested); likely the delayed observation or direct-mode proposals	trace one scenario step by step; candidate: feed the queue into the engine without the extra delay
Power-cap movement	B and C	inherent	moving the cap is how capping saves energy; freezing it removed most of the saving (tested: -28% to -2%/-7%)	none needed: electronic setting, no physical wear; reported
CPU frequency saving 3.5%	device plant	physical ceiling	the native Linux governor already follows demand	value is in heat (-67%) and in combining with power caps
GPU node on/off saving 3%	fleet plant	superseded	training nodes cannot be switched off	the GPU power-limit muscle (15-19%)
Live response time +65% (p95) and energy per unit of work +46% with every muscle	live kind, run 36213152881	reduced to p95 +25% (run 2), tie in run 4	no response-time afferent; cap held at the law floor; HPA target floor 76.9%	latency afferent, cap reflex, SLO reflex; speed-first law (section 14)
Fleet mode p95 20 s vs 132 ms on web services	fleet plant with response-time gauge	cause found; speed-first law built	fleet constants were selected without a response-time gauge; HPA target floor 76.9%	choose the mode per service: speed-first where latency matters
No mode better than Kubernetes on every gauge in every scenario	fleet plant, 400 configurations	not achievable as posed	gauges at physical floors; hardware-bound GPU vessel; energy / response time / churn trade two-of-three	anticipation (pre-adding machines before the daily rise) is the one untested mechanism that could break the trade
Controller stopped after one refused call; power cap never applied under least privilege	live runs 3 and 4	fixed	one exception ended the loop; the role lacked get on pods/resize	fail-safe restore after 3 failures; permission and receipt added; runs with a fail-safe rejected
Right-sizing p95 +15%, more memory kills than VPA	omnilab rightsize, held-out	open, small	Omni sizes CPU closer to demand; VPA's 8-hour p90 memory keeps more slack	larger memory margin (costs memory-hours)
More cold starts than KEDA	omnilab coldstart, held-out	trade-off	shorter keep-alive than KEDA's 5-minute cooldown	longer keep-alive where cold starts matter more than instance-hours
GPU jobs wait +16% (seconds) vs Volcano binpack	omnilab gpupack, held-out	trade-off	idle GPUs powered off sooner	longer power-off delay
Honest agents throttled (-7% work) and ~2 false stops a day vs static caps	omnilab containment, held-out	trade-off	throttling on the engine's integrated need catches legitimate bursts	per-agent declared burst budgets; human approval before stop
Engine dynamics add little beyond the mapping	all nine muscles	reported	the mapping from state to action carries the effect	wire the engine's control effort (equation 2) directly as the actuator command and test it

Negative	Where	Status	Cause	What would fix it
Live energy with parked servers on standby ~0%	live kind	open	parked servers still draw standby power	live power cap and CPU/GPU muscles; sleep states where hardware allows
Small clusters (<= 3 workers) never release	live kind	open	fleet law release band of 3 nodes	pool-size-aware release band (law change)
Pages in the 24-scenario replica	control-plane replica	open	longer queue triggers the page rule	same as queue

18. The industry problem map: what Omni-Compass is aimed at

One engine; the vessel (the plant it sits on) is the only thing that changes. Industry figures are approximate, from the public sources named, and are context, not results of this report. Status: **live** = measured on a real Kubernetes control plane; **sim** = demonstrated in this repository's simulations; **open** = mapped, connector not built.

Problem	Scale in the industry (approximate, source)	Best software today	Omni-Compass vessel and muscles	Gauge that shows it	Status
Data-centre electricity growth	about 415 TWh in 2024, about 1.5% of world electricity, projected near 945 TWh by 2030 (IEA, Energy and AI, 2025)	Karpenter, Cluster Autoscaler, CAST AI, Spot Ocean; Kepler for metering	compute vessel: nodes, HPA, power cap	energy, node-hours, idle node-hours	sim; live only where parked nodes can sleep or power off
Idle and over-provisioned capacity	Kubernetes clusters commonly run near 10-15% average CPU utilisation (CAST AI and Datadog industry reports); roughly a quarter to a third of cloud spend reported as waste (Flexera State of the Cloud)	VPA, Goldilocks, StormForge, Kubecost/OpenCost	compute vessel: nodes, HPA; requests and memory (omnilab/rightsize.py)	utilisation, node-hours per core-hour, core- and GiB-hours	live + sim
Controllers fighting each other	documented conflicts, e.g. HPA and VPA on the same CPU metric (Kubernetes documentation advises against it)	none: each tool decides alone	single authority over replicas and requests	contradictory commands, scale reversals, OOM kills	sim, held-out (section 16)
Outages and slow recovery	most significant outages cost over \$100,000 (Uptime Institute annual outage analysis)	Argo Rollouts, Flagger, SRE runbooks, AIOps (Dynatrace, Datadog)	compute vessel + deployments (partial)	time healthy, recovery time, SLA breaches	sim
On-call load and alert fatigue	widely reported burnout in SRE surveys	PagerDuty, alert tuning	all muscles: act before the page	pages, human interventions	sim
Heat and cooling limits	cooling is a large share of facility energy; average PUE about 1.5 (Uptime Institute survey)	DCIM (Schneider EcoStruxure), DeepMind cooling AI (reported about 40% less cooling energy)	heat (sensed), cooling plant (omnilab/cooling.py)	PUE, cooling energy, inlet violations	sim, held-out (section 16)
Site power and grid-connection limits	multi-year waits for new grid connections are widely reported	Meta Dynamo power capping, Intel RAPL	power cap (wired), batteries and demand response (open)	peak power, time over power limit	sim

Problem	Scale in the industry (approximate, source)	Best software today	Omni-Compass vessel and muscles	Gauge that shows it	Status
GPU energy and power limits	GPU fleets widely reported well below full utilisation; H100 TDP 700 W	NVIDIA DCGM and MIG, Run.ai, Kueue; manual MaxQ power limits	GPU power-limit muscle (hardware connector)	energy per unit of work, response time, heat	sim, calibrated to MLPerf metered H100 data: -15% to -19% energy, +0.6-0.8% response time
Batch deadlines and fair sharing		Kueue, Volcano, Slurm	batch muscle (live: admit held jobs); GPU packing (omnilab/gpupack.py)	queue wait, fragmentation	live wired + sim, held-out
Hardware wear	power cycling and churn shorten component life	none as a governed objective	nodes: start/stop cycles and reversals	machines started and stopped, round trips	mixed: better in the 24-scenario study, worse in the held-out study; tuning target
Carbon reporting and reduction	regulatory disclosure is expanding	Google carbon-aware computing, Kepler	carbon-aware placement (open)	kWh and CO2 per unit of work	sim (modelled)
Runaway AI agents and spend	~\$10,000 overnight examples (Dark Reading)	per-tool quotas and permissions	agent containment muscle (omnilab/containment.py)	rogue spend, time to contain, false stops	sim, held-out (section 16)

19. What comes next

- Live architecture C: park HPA, VPA, Cluster Autoscaler and Karpenter; Omni-Compass sets replicas, resources, placement, priorities and quotas directly; Kubernetes keeps execution and reflexes (restarts, rescheduling); the kill switch wakes the parked controllers.
- Live opponent at full strength: Karpenter (kwok provider) and Cluster Autoscaler in architecture A.
- 24 live scenarios (traffic, failures, power and heat limits, batch and AI, growth, mixed) with repetitions.
- More muscles two-way: CPU power states, memory, batch queues, network, security, then GPU and cooling on hardware.
- Metered power on physical machines.

20. Questions and answers

Does Omni-Compass replace Kubernetes? No. Kubernetes keeps running containers, placing pods, restarting failures and networking. Omni-Compass replaces the separate decision loops (how many replicas, how many nodes, what power) with one authority. In architecture C Kubernetes becomes one muscle.

What happens if Omni-Compass crashes or is switched off? The kill switch restores every setting it changed and returns control to Kubernetes' own controllers; this was exercised live and in simulation. A crashed controller writes nothing further.

Can it make things worse? Every action passes the shield first; it never goes below the capacity running and pending work needs, and never changes more than the step limit. In the held-out benchmark it had fewer safety violations than Kubernetes. Its real costs are listed in sections 1 and 5.1.

How fast does it decide, and what does it cost to run? One decision per 60 s on live Kubernetes (300 s in the replica), with a 15 s fast path that adds nodes for pending pods. The engine takes about 2.9 microseconds per decision.

Why does it save energy? In the pre-registered stack study mainly by power capping and load shaping (fewer minutes over the power limit, lower peak), while keeping slightly more machines on. On the live cluster it took machines out of service (6 to 3 workers) and packed replicas more densely; that saves energy only if the parked machines sleep or power off. With parked machines on standby the live saving was about zero, so live savings must come from power caps and CPU power states.

Does it slow applications down? It can, in the energy-first fleet mode: its HPA target floor packs pods too hot for latency-sensitive services (section 14). Live, the first all-muscle run was slower (p95 +65%); after the fixes run 4 was a tie at p95 and 10% faster at the median. For latency-sensitive services the speed-first mode is the right setting.

Can Omni-Compass control AI agents? It controls what an agent can touch, spend and do, and how fast; not what the model thinks. In the containment muscle (section 16) it cut runaway spend 96% and contained runaways in minutes instead of hours, with a least-privilege identity that blocks forbidden actions outright; the cost is some throttling of honest agents' bursts.

Does the engine hold everything in its basin by itself? Not against an outside limit it is not told. The bath equation alone smoothed training power ramps 63% but did not keep them under the grid's limit; with the human-set limit as the boundary it held it with zero violations. The human sets the boundaries; the engine operates inside them.

Is this tuned to the test? Parameters were selected on development seeds and frozen with hashes before the held-out seeds were run; `verify.py` fails if any frozen file changes.

How many scenarios and how certain? 1,000 pre-registered held-out scenarios, 24 control-plane scenarios, PlanetLab traces and live runs; 95% bootstrap intervals, two independent seeds must agree.

What is measured versus modelled? Live: node counts, replicas, pods, CPU, the controller's actions and the kill switch. Modelled: power and heat everywhere, and everything in the simulated studies.

Is the mathematics sound? The core is a closed six-state system integrated with RK4; the C++ and Python implementations agree to 3.6e-15 on 500 reference trajectories; 100 million decisions ran without a non-finite value.

Who owns it and how can it be used? The Omni-Compass LLC. Free for evaluation, research and non-commercial use; commercial use requires a paid licence; protected by copyright and by patents and patent applications (see LICENSE and NOTICE).

What is not claimed? Superiority over upstream Karpenter or Cluster Autoscaler live, metered savings on physical hardware, GPU or facility control on real hardware, the vendor products' own binaries (they are emulated from documentation), and anything about AI value alignment.

How do I check it myself? Run the commands in section 21; the live runs are GitHub Actions workflows in the repository.

21. Reproduce

```

pip install -r requirements.txt
python verify.py                                # all checks, hashes, parity, soak
python benchmarks/stack_benchmark.py ...        # held-out stack benchmark (see HARNESS.md)
python -m k8s_controlplane.benchmark --scenarios 24 --seed 424242
python -m fleet.planetlab --dir fleet/traces/planetlab --scenarios 8 --out /tmp/pl
GitHub Actions: benchmark (live side by side), live-kind-full, live-kind
python tools/full_report.py ... && python pilot/bench_pdf.py docs/BENCHMARK_REPORT.md docs/BENCHMARK_REP
  ↳ ORT.pdf

```

22. Glossary

- **HPA:** Horizontal Pod Autoscaler: Kubernetes controller that sets replica counts from CPU utilisation versus a target.
- **Cluster Autoscaler, Karpenter:** Kubernetes add-ons that add and remove nodes.
- **Node, worker:** a machine (here a container in kind) that runs pods.
- **Cordon, drain:** mark a node unschedulable, then move its pods elsewhere.
- **kind:** Kubernetes in Docker: a real Kubernetes control plane whose nodes are containers.
- **Observe mode:** Omni-Compass computes and logs but writes nothing.
- **Invariant:** a safety rule the shield enforces before any action.
- **Paired bootstrap CI:** resampling the per-scenario differences to get a 95% interval for the mean difference.
- **Pre-registration:** freezing code and parameters, with hashes, before running the test data.

The Referee Report

Dated record, kept as written on 26 September 2026. Where a figure here differs from `STATE_OF_PLAY.md`, the State of Play governs. The results below are simulations of platforms (evidence class S), not measurements of those products.

Referee report, 26 September 2026. Repository Omni-Compass/The-Omni-Compass-Control-Core-Engine, branch main, commit 096956a. Every number is generated by `python tools/abc_report.py` from result files in the repository. Each result states how it was obtained: **measured** on a live Kubernetes control plane, **simulated** in this repository's plants, or **emulated** (a competitor reproduced from its public documentation, not its binary).

Engine interpretation notice. The Omni-Compass engine is the completed core conveyance mechanism; its success criterion is internal dual-basin conveyance under native ignition dynamics. Everything measured here (the Kubernetes adapters, the closure governor, the vendor comparisons) is wiring around the engine, which is downstream engineering and not part of the base law. A loss in a cell below is a property of that wiring on that plant, not evidence that the base engine is incomplete. The engine source is byte-locked by SHA-256 and was not edited for any result in this report.

1. The answer

Three architectures were measured against seven platforms: Kubernetes, Red Hat OpenShift, Google GKE, Azure AKS (node auto-provisioning, which is Karpenter and also stands for Amazon EKS Auto Mode), IBM Turbonomic, CAST AI and Spot Ocean.

- **A. The platform alone.** Its own autoscalers decide.
- **B. Omni-Compass on top of the platform.** The platform's controllers keep running; the six-state engine governs on top.
- **C. Omni-Compass alone.** The engine is the single authority; the platform's managers do not decide.

B against A (held-out, 30 unseen scenarios per workload, settings frozen by SHA-256 first): 363 of 364 gauge-by-platform-by-workload cells are equal or better; 1 is worse: batch jobs on CAST AI, scale reversals 7.7% worse. Batch jobs gain most: the slowest jobs finish much sooner on every platform (section 5.3). Where no intervention was free (web services, four clusters) Omni-Compass reproduces the platform exactly.

C against the seven platforms at once (held-out): 63 cells where some platform is better (section 6). Part of that gap is not closable by any controller: a perfect controller that knows the future cannot have both the tightest packer's machine-hours and the laziest autoscaler's machine churn (section 6.2).

C with the manuscript's closure law and muscle tone (held-out, 30 unseen scenarios per workload, settings frozen first): 41 of 364 cells where some platform is better. Against Kubernetes alone: web services energy +20%, slowest responses (p99) +60%, average response +20%; four clusters, one site energy +22%, slowest responses

(p99) -19%, average response +11%; batch jobs energy +6%, slowest responses (p99) +6%, average response -6%; GPU training energy +1%, slowest responses (p99) +0%, average response +0% (positive = better; section 6.4).

Live Kubernetes (measured, run 36221702999, every muscle on, B architecture): worker machines 6.00 to 3.19, energy -3.2% with parked machines on standby, response time p95 491 to 383 ms and p99 613 to 491 ms, 0 failed requests; one significant negative (energy per CPU core-hour, section 7).

2. The rule, and how a cell is judged

- A **cell** is one gauge, one competitor, one workload. There are 13 gauges: energy, machine-hours, response time (p95, p99, mean), time over the backlog, power and heat limits, machine starts+stops, scale reversals, pod changes, work completed and time healthy.
- Omni-Compass **loses** a cell when it is worse by more than 0.5% (0.5 events for counts) **and** the paired bootstrap 95% interval of the per-scenario difference excludes zero. Otherwise the cell is equal (identical, or not significantly different) or better.
- Settings are chosen on **development** scenarios only, frozen with SHA-256 hashes (tuning/B_PREREGISTRATION*.json, results/muscles/PREREGISTRATION.json), then run once on **held-out** scenarios never used for tuning.
- Scenario generator: fleet/harness.py (15-second resolution, 6 hours; boot delay, power model, thermal model, site power limit identical for every arm). Response time: M/M/c queueing per workload plus backlog drain (fleet/sim_slo.py).

3. The opponents, and how each is reproduced

Platform	What decides	Reproduced as (parameters)
Kubernetes	HPA + Cluster Autoscaler	HPA 70%; remove a node after 10 min below 50% request use, not within 10 min of an add
OpenShift	ClusterAutoscaler resource	documented example: threshold 0.4, unneeded 5 min, delay after add 10 min
GKE (optimize-utilization)	MostAllocated packing, aggressive scale-down	declared: threshold 0.65, 2 min (Google publishes no numbers)
AKS NAP (= Karpenter, EKS Auto Mode)	consolidation WhenEmptyOrUnderutilized	consolidateAfter 0 s; remove when the rest hold all requests at 90% packing
IBM Turbonomic	market-based resize + suspend	requests resized every 10 min to per-pod p99 usage; nodes suspended toward 70% packing
CAST AI	Evictor bin-packing	every 60 s, drain a node older than 5 min when its pods fit elsewhere
Spot Ocean	headroom autoscaler	5% automatic headroom kept; least-used node removed when pods fit elsewhere

These are emulations of documented behaviour on the same plant, not the vendors' software; each vendor's full product has more features than its autoscaling core. Every platform runs its pods at the same HPA target (70%), so pod behaviour is identical and differences come from how machines are managed.

4. What Omni-Compass is, mechanically

A six-state dynamical system, $x = (E, U, I_U, S, B, B_{\text{dot}})$: energy/excitation, order (a double-well with a healthy basin at $U = +1$), integrated unmet need, stress (a cubic potential), and a damped second-order bath. Equations (1)-(8) and every term's role are in `docs/MECHANISM_OF_ACTION.md`. Telemetry is blended into the state (weight 0.339) and the equations are integrated with RK4 each decision.

Mode (Jacobian eigenvalue at the operating point)	Time constant (decisions)	Period (decisions)
-7.90+0.00i	1.3	
-3.10+0.00i	3.2	
-0.72+0.00i	13.9	
-0.20+0.98i	50.0	64
-0.20-0.98i	50.0	64
-0.17+0.00i	59.8	

All modes are stable: the operating point ($U = 0.976$) sits inside the healthy basin. The equations and the observation blend each move the state about half of every step. Compute: 95 microseconds per decision in Python, about 3 in the C++ twin.

5. B: Omni-Compass on top of each platform, against the same platform alone (held-out)

Source `tuning/B_LEAGUE_HELDOUT2.json`. Omni-Compass keeps the platform running and intervenes only where the engine sees it will pay: it undoes a machine removal that is about to be reversed (while requests are rising or the engine has not converged), and adds early the machine the platform would add only after pods go pending. With both off it reproduces the platform exactly (checked on every platform and workload); every gain below is on top of that floor.

5.1 Web services

Setting: pass-through. Every intervention tried here cost some gauge on some platform, so Omni-Compass leaves these decisions to the platform and the result is identical to the platform alone.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13
OpenShift	-	-	-	13 of 13
GKE (optimize)	-	-	-	13 of 13
AKS NAP (Karpenter)	-	-	-	13 of 13
Turbonomic	-	-	-	13 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

5.2 Four clusters, one site

Setting: pass-through. Every intervention tried here cost some gauge on some platform, so Omni-Compass leaves these decisions to the platform and the result is identical to the platform alone.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
OpenShift	-	-	-	13 of 13
GKE (optimize)	-	-	-	13 of 13
AKS NAP (Karpenter)	-	-	-	13 of 13
Turbonomic	-	-	-	13 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

5.3 Batch jobs

Setting per platform (chosen on development scenarios): Kubernetes: early add (lead 12, trend over 16), reversal veto; OpenShift: early add (lead 12, trend over 16), reversal veto; GKE (optimize): early add (lead 6, trend over 8), reversal veto; AKS NAP (Karpenter): early add (lead 3, trend over 16), reversal veto; Turbonomic: early add (lead 6, trend over 4), reversal veto; CAST AI: early add (lead 12, trend over 16), reversal veto; Spot Ocean: early add (lead 3, trend over 16), reversal veto.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	response p99 -91%; response mean -23%; machine starts+stops -16%; scale reversals -70%	-	-	9 of 13
OpenShift	response p99 -91%; response mean -25%; machine starts+stops -23%; scale reversals -87%	-	-	9 of 13
GKE (optimize)	response p99 -91%; response mean -27%; machine starts+stops -22%	-	-	10 of 13
AKS NAP (Karpenter)	response p95 -57%; response p99 -24%; response mean -19%; time over backlog limit -47%; machine starts+stops -13%	-	energy +0.2%; machine-hours +0.5%	6 of 13
Turbonomic	response p99 -95%; response mean -34%; machine starts+stops -16%	-	-	10 of 13
CAST AI	response p95 -47%; response p99 -25%; response mean -32%; time over backlog limit -100%; machine starts+stops -9%	scale reversals +7.7%	-	7 of 13
Spot Ocean	response p95 -12%; response p99 -9%; response mean -8%; time over backlog limit -100%; machine starts+stops -8%; scale reversals -8%	-	energy +0.1%; machine-hours +0.3%	5 of 13

5.4 Gpu training

Setting per platform (chosen on development scenarios): Kubernetes: early add (lead 3, trend over 8), reversal veto; OpenShift: early add (lead 3, trend over 16); GKE (optimize): pass-through; AKS NAP (Karpenter): pass-through; Turbonomic: early add (lead 3, trend over 8); CAST AI: pass-through; Spot Ocean: pass-through.

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
Kubernetes	-	-	-	13 of 13
OpenShift	-	-	time over heat limit +0.1%	12 of 13
GKE (optimize)	-	-	-	13 of 13

Platform	Better (significant)	Worse beyond tolerance (LOSS)	Worse within 0.5% tolerance	Equal
AKS NAP (Karpenter)	-	-	-	13 of 13
Turbonomic	-	-	energy +0.1%; time over heat limit +0.2%; machine-hours +0.4%	10 of 13
CAST AI	-	-	-	13 of 13
Spot Ocean	-	-	-	13 of 13

Percentages are the plain change of Omni-Compass on top against the platform alone: for response times, starts+stops, reversals, energy and machine-hours negative is better; for work completed and time healthy positive is better. A LOSS is worse beyond the 0.5% tolerance with a 95% interval excluding zero (section 2).

6. C: Omni-Compass alone, against all seven platforms at once

Source `tuning/LEAGUE_HELDOUT.json`. One Omni-Compass configuration per workload type (chosen on development scenarios): web: speed159, multi: speed159, batch: speed112, gpu: omni_fleet.

Workload	Gauge	Platforms better than Omni-Compass	Omni worse by (range)
batch	energy	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	1.7% to 6.7%
batch	machine-hours	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	4.4% to 18.0%
batch	scale reversals	Kubernetes, OpenShift	266.7% to 279.3%
batch	machine starts+stops	Kubernetes, OpenShift	54.3% to 64.9%
gpu	machine-hours	GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	5.5% to 14.4%
gpu	scale reversals	Kubernetes, OpenShift	85.6% to 87.6%
gpu	machine starts+stops	Kubernetes, OpenShift	37.6% to 49.8%
multi	energy	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	23.9% to 64.7%
multi	machine-hours	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	36.7% to 119.0%
multi	pod changes	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), CAST AI, Spot Ocean	8.2% to 10.3%
web	energy	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	23.9% to 63.5%
web	machine-hours	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), Turbonomic, CAST AI, Spot Ocean	36.8% to 116.3%
web	pod changes	Kubernetes, OpenShift, GKE (optimize), AKS NAP (Karpenter), CAST AI, Spot Ocean	7.8% to 10.8%

6.1 Where C wins

- Web services: response p95 110.1 ms vs best platform 124.9 ms; response p99 122.8 ms vs best platform 546 ms; response mean 145.3 ms vs best platform 216.2 ms.
- Four clusters, one site: response p95 112.1 ms vs best platform 126.9 ms; response p99 137.8 ms vs best platform 262.9 ms; response mean 145.6 ms vs best platform 217.2 ms.

- Batch jobs: response p99 337.4 ms vs best platform 1277 ms; response mean 115.2 ms vs best platform 135.6 ms.

6.2 The physical limit: what no controller can do

For each scenario, dynamic programming with perfect knowledge of the future (boot delay ignored, which only helps it) finds the fewest machine starts+stops that can hold every pod at a given machine-hour budget (tuning/bound.py).

Workload	Scenario	Tightest platform machine-hours	Fewest starts+stops of any platform	Perfect controller at those machine-hours
web	101	18.4	9	11
web	102	18.3	9	9
web	103	31.4	4	26
web	104	22.0	12	24
multi	101	74.9	36	46
multi	102	112.7	26	100
multi	103	122.5	17	114
multi	104	93.4	35	91
batch	101	108.0	24	67
batch	102	56.8	7	33
batch	103	66.9	12	44
batch	104	93.4	15	58
gpu	101	66.4	12	28
gpu	102	57.5	16	36
gpu	103	51.5	19	30
gpu	104	57.0	20	26

In 15 of 16 scenarios even a perfect controller needs more starts+stops than the laziest platform to reach the tightest platform's machine-hours. Those two platforms sit at opposite ends of one physical trade; beating both at once is impossible for any software. In C, a loss on one side of that pair is the price of a win on the other.

6.3 Letting the equations drive directly

A governor in which the equations run closed-loop and their own quantities (control effort, unmet need I_U , the bath's rate) drive every muscle (omnicompass/mathdrive.py) was tested on 96 settings per workload: 66 losing cells, the same trade-off as the rule-based wiring. The engine clock alone moved batch p99 from 3.9 s to 0.1 s at equal energy.

6.4 Strict C: the manuscript's closure law drives the machines and the pods

omnicompass/closure.py implements, as the only authority over machines, the laws of the owner's manuscript: the forward projection of Section 5c, the master closure law of Chapter 20 ($F = G_0 + G_c$: no correction inside the admissible domain, an inward correction sized to restore the margin when the projected state approaches its boundary, so that $G \cdot n \leq 0$ on the boundary), the turning point of Chapters 29-30 (a machine is released only after the peak) and the dual-bath exchange of Chapter 31 (level and rate of demand). A release must also stay inside the calm set for a dwell (the resource-aware envelope, Proposition 2). Kubernetes' HPA and node managers are off; Kubernetes only schedules. Settings chosen on development scenarios, frozen by SHA-256 (56be5f50b8d2631c), then run once on 30 held-out scenarios per workload.

Workload	Gauge	Kubernetes	OpenShift	GKE (optimize)	AKS NAP (Karpenter)	Turbomicon	CAST AI	Spot Ocean	Omni-Compass (closure law)
web	energy	14.11	14.75	11.8	11.22	11.75	11.03	11.2	11.26
web	machine-hours	37.61	40.27	27.99	25.56	27.78	24.77	25.52	24.26
web	response p95	122.9	122.8	123.4	123.6	138.2	124.1	123.8	140.4
web	response p99	806.6	648.3	1345	1428	3251	2500	2023	318.9
web	response mean	206.4	205.3	212.5	215.7	251	236.1	226.9	164.5
web	machine starts+stops	8.733	6.5	16.47	19.67	12.73	26.57	27.6	11.6
web	work completed	1	1	1	1	1	1	1	1
multi	energy	54.24	56.4	45.04	42.81	44.86	42.16	42.85	42.43
multi	machine-hours	145.7	154.7	107.4	98.08	106.6	95.37	98.24	90.42
multi	response p95	124.9	124.8	125.6	125.9	119	126.5	126.1	148.9
multi	response p99	202.2	168.7	641.9	991.4	2.095e+04	2068	1531	240.3
multi	response mean	220.2	218.4	236.9	243.1	784.9	261.8	253.7	196.8
multi	machine starts+stops	36.77	28	66.63	79.7	53.8	102.4	108.1	45.67
multi	work completed	1	1	1	1	1	1	1	1
batch	energy	65.45	65.68	63.28	60.11	62.73	60.65	60	61.65
batch	machine-hours	94.71	95.49	87.48	76.91	85.67	78.71	76.55	78.5
batch	response p95	100	100	100	861.9	100	1840	220.1	100
batch	response p99	1072	1291	2086	3880	2242	4974	2491	1012
batch	response mean	133.1	138	158.8	257.2	170.6	360	184.2	141.3
batch	machine starts+stops	12.93	14.13	29	92.33	35.03	160.6	141.2	17.2
batch	work completed	1	1	1	1	1	1	1	1
gpu	energy	201.4	201.6	199.5	197.2	198.9	197.3	197	198.9
gpu	machine-hours	60.77	61.07	58.15	54.89	57.25	55.06	54.62	55.29
gpu	response p95	2.288e+06	2.286e+06	2.288e+06	2.29e+06	2.288e+06	2.292e+06	2.29e+06	2.287e+06
gpu	response p99	2.397e+06	2.396e+06	2.398e+06	2.4e+06	2.398e+06	2.402e+06	2.401e+06	2.396e+06
gpu	response mean	8.886e+05	8.875e+05	8.894e+05	8.904e+05	8.886e+05	8.922e+05	8.911e+05	8.879e+05
gpu	machine starts+stops	16.5	16.6	33.5	50.63	30.57	53.93	55.63	15.27
gpu	work completed	0.9027	0.9028	0.9027	0.9026	0.9028	0.9025	0.9026	0.9028

Losing cells (worse than that platform beyond tolerance): web services 10 of 91; four clusters, one site 12 of 91; batch jobs 7 of 91; GPU training 12 of 91.

- Web services: response p95 vs Kubernetes 14.2% worse; machine starts+stops vs Kubernetes 32.8% worse; response p95 vs OpenShift 14.3% worse; machine starts+stops vs OpenShift 78.5% worse; scale reversals vs OpenShift 192.9% worse; response p95 vs GKE (optimize) 13.8% worse; response p95 vs AKS NAP (Karpenter) 13.6% worse; energy vs CAST AI 2.1% worse; response p95 vs CAST AI 13.2% worse; response p95 vs Spot Ocean 13.4% worse.
- Four clusters, one site: response p95 vs Kubernetes 19.2% worse; machine starts+stops vs Kubernetes 24.2% worse; response p95 vs OpenShift 19.4% worse; response p99 vs OpenShift 42.4% worse; machine starts+stops vs OpenShift 63.1% worse; scale reversals vs OpenShift 153.2% worse; response p95 vs GKE

(optimize) 18.5% worse; response p95 vs AKS NAP (Karpenter) 18.3% worse; response p95 vs Turbonomic 25.2% worse; energy vs CAST AI 0.6% worse; response p95 vs CAST AI 17.8% worse; response p95 vs Spot Ocean 18.1% worse.

- Batch jobs: machine starts+stops vs Kubernetes 33.0% worse; machine starts+stops vs OpenShift 21.7% worse; energy vs AKS NAP (Karpenter) 2.6% worse; machine-hours vs AKS NAP (Karpenter) 2.1% worse; energy vs CAST AI 1.7% worse; energy vs Spot Ocean 2.8% worse; machine-hours vs Spot Ocean 2.6% worse.
- Gpu training: energy vs AKS NAP (Karpenter) 0.9% worse; time over power limit vs AKS NAP (Karpenter) 0.7% worse; time over heat limit vs AKS NAP (Karpenter) 0.8% worse; machine-hours vs AKS NAP (Karpenter) 0.7% worse; energy vs CAST AI 0.8% worse; time over power limit vs CAST AI 1.5% worse; time over heat limit vs CAST AI 1.9% worse; energy vs Spot Ocean 1.0% worse; time over power limit vs Spot Ocean 1.8% worse; time over heat limit vs Spot Ocean 1.6% worse; machine-hours vs Spot Ocean 1.2% worse; time healthy vs Spot Ocean 1.5% worse.

6A. Runtime Benchmark Protocol: faults at five stress levels

The owner's *OmniCompass Runtime Benchmark Protocol* (stage 1, Python runtime) run on this fleet plant with every architecture: 100 runs per stress level per workload, seeds 900001-900100 (never used for tuning). Level L injects L faults (machines dying, load spikes, services crash-looping, a noisy neighbour), identical for every system (`tools/protocol_bench.py`). A run is **conveyed** when every fault is recovered within 20 minutes and every cluster is inside the admissible basin for the final 10 minutes (worst response ≤ 1 s, queue < 0.28 , pending pods $\leq 5\%$, power and heat inside limits). A **page** is an out-of-basin episode of 5 minutes or more, the usual alert rule: each would call a human. Nobody intervenes in the simulation.

Web services

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
A	AKS NAP (Karpenter)	233 of 500	391	6.6	32.8	4.52%	11.90
B	AKS NAP (Karpenter)	233 of 500	391	6.6	32.8	4.52%	11.90
A	CAST AI	231 of 500	404	6.6	34.1	4.68%	11.75
B	CAST AI	231 of 500	404	6.6	34.1	4.68%	11.75
A	GKE (optimize)	227 of 500	366	6.9	32.4	4.25%	12.76
B	GKE (optimize)	227 of 500	366	6.9	32.4	4.25%	12.76
A	Kubernetes	218 of 500	338	8.3	33.3	3.97%	15.54
B	Kubernetes	218 of 500	338	8.3	33.3	3.97%	15.54
A	OpenShift	220 of 500	328	8.0	32.2	3.82%	15.52
B	OpenShift	220 of 500	328	8.0	32.2	3.82%	15.52
A	Spot Ocean	234 of 500	392	6.6	32.8	4.56%	11.94
B	Spot Ocean	234 of 500	392	6.6	32.8	4.56%	11.94
A	Turbonomic	169 of 500	552	14.4	53.3	5.96%	12.43
B	Turbonomic	169 of 500	552	14.4	53.3	5.96%	12.43
C	C-hpa	202 of 500	353	11.7	35.8	3.81%	20.69
C	C-strict	238 of 500	283	6.0	26.2	3.45%	13.02

In the protocol's required form: Omni-Compass alone (closure law) conveyed 238 out of 500 runs; the best platform alone (Spot Ocean) conveyed 234 out of 500. Omni-Compass would have paged a human 283 times, that platform 392 times. Omni-Compass average recovery time was 6.0 min, that platform's 6.6 min.

Four clusters, one site

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
A	AKS NAP (Karpenter)	49 of 500	534	5.6	69.5	1.71%	45.25
B	AKS NAP (Karpenter)	49 of 500	534	5.6	69.5	1.71%	45.25
A	CAST AI	44 of 500	608	5.7	74.2	1.81%	44.54
B	CAST AI	44 of 500	608	5.7	74.2	1.81%	44.54
A	GKE (optimize)	49 of 500	455	5.7	65.6	1.63%	47.96
B	GKE (optimize)	49 of 500	455	5.7	65.6	1.63%	47.96
A	Kubernetes	44 of 500	330	6.5	59.2	1.44%	57.67
B	Kubernetes	44 of 500	330	6.5	59.2	1.44%	57.67
A	OpenShift	43 of 500	310	6.4	57.6	1.38%	59.47
B	OpenShift	43 of 500	310	6.4	57.6	1.38%	59.47
A	Spot Ocean	44 of 500	508	5.6	69.1	1.73%	45.30
B	Spot Ocean	44 of 500	508	5.6	69.1	1.73%	45.30
A	Turbonomic	26 of 500	867	15.0	97.6	2.49%	47.57
B	Turbonomic	26 of 500	867	15.0	97.6	2.49%	47.57
C	C-hpa	64 of 500	328	8.8	45.8	1.07%	74.42
C	C-strict	45 of 500	231	4.3	52.6	1.33%	45.82

In the protocol's required form: Omni-Compass alone (closure law) conveyed 45 out of 500 runs; the best platform alone (GKE (optimize)) conveyed 49 out of 500. Omni-Compass would have paged a human 231 times, that platform 455 times. Omni-Compass average recovery time was 4.3 min, that platform's 5.7 min.

Batch jobs

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
A	AKS NAP (Karpenter)	314 of 500	234	4.5	31.4	9.01%	70.02
B	AKS NAP (Karpenter)	338 of 500	235	4.5	25.1	8.09%	70.95
A	CAST AI	161 of 500	237	4.4	39.9	10.25%	72.15
B	CAST AI	224 of 500	233	4.4	26.3	8.39%	73.91
A	GKE (optimize)	409 of 500	235	4.0	19.0	7.07%	77.00
B	GKE (optimize)	425 of 500	228	4.0	13.9	6.38%	79.12
A	Kubernetes	424 of 500	228	3.7	15.4	6.28%	85.35
B	Kubernetes	425 of 500	234	3.8	12.2	5.77%	87.43
A	OpenShift	422 of 500	232	3.8	16.0	6.36%	85.36
B	OpenShift	425 of 500	232	3.8	12.2	5.82%	86.89
A	Spot Ocean	360 of 500	231	4.4	23.1	7.77%	71.08
B	Spot Ocean	360 of 500	234	4.4	20.6	7.56%	71.79
A	Turbonomic	407 of 500	232	4.2	21.0	7.45%	74.08
B	Turbonomic	426 of 500	233	4.1	14.2	6.62%	75.25
C	C-hpa	166 of 500	235	30.8	73.5	20.65%	85.38
C	C-strict	422 of 500	237	4.1	17.3	6.93%	74.13

In the protocol's required form: Omni-Compass alone (closure law) conveyed 422 out of 500 runs; the best platform alone (Kubernetes) conveyed 424 out of 500. Omni-Compass would have paged a human 237 times, that platform 228 times. Omni-Compass average recovery time was 4.1 min, that platform's 3.7 min.

Gpu training

Column	System	Conveyed	Pages	Recovery (min)	Minutes outside basin	Timeouts > 2 s	Energy (kWh)
A	AKS NAP (Karpenter)	13 of 500	802	95.4	227.2	69.96%	202.42
B	AKS NAP (Karpenter)	13 of 500	802	95.4	227.2	69.96%	202.42
A	CAST AI	13 of 500	925	95.1	226.5	70.61%	202.80
B	CAST AI	13 of 500	925	95.1	226.5	70.61%	202.80
A	GKE (optimize)	15 of 500	802	96.5	227.1	68.76%	205.17
B	GKE (optimize)	15 of 500	802	96.5	227.1	68.76%	205.17
A	Kubernetes	18 of 500	782	97.5	225.4	67.95%	206.83
B	Kubernetes	18 of 500	792	97.6	226.1	67.87%	207.04
A	OpenShift	18 of 500	788	97.6	226.3	67.91%	207.08
B	OpenShift	18 of 500	788	97.6	226.2	67.88%	207.13
A	Spot Ocean	13 of 500	830	94.9	226.0	70.66%	202.22
B	Spot Ocean	13 of 500	830	94.9	226.0	70.66%	202.22
A	Turbonomic	14 of 500	786	96.5	226.6	68.78%	204.30
B	Turbonomic	14 of 500	792	96.2	226.6	68.67%	204.40
C	C-hpa	0 of 500	1232	174.0	295.6	83.41%	198.48
C	C-strict	13 of 500	809	95.1	225.0	67.86%	204.08

In the protocol's required form: Omni-Compass alone (closure law) conveyed 13 out of 500 runs; the best platform alone (Kubernetes) conveyed 18 out of 500. Omni-Compass would have paged a human 809 times, that platform 782 times. Omni-Compass average recovery time was 95.1 min, that platform's 97.5 min.

6B. Second generation (held-out, settings frozen by SHA-256 before each run)

B with muscle tone and consolidation (tuning/b_tone.py, held-out seeds 700401-700430): machines a platform powers off are parked instead (alive, low power, instant wake), with a reserve sized by the law; Omni consolidates on top only where that loses nothing. Losing cells: 3 of 364.

B fix for the remaining pairs (tuning/b_fix.py, 30 development seeds, held-out 700701-700730): multi on aks_nap: 0 losing cells; batch on turbonomic: 0 losing cells; multi on cast_ai: 0 losing cells.

Whole body at four-cluster sites (tuning/site_league.py, held-out 701001-701030): traffic shift between a site's clusters (a cluster short of room runs its overflow on another cluster's powered spare cores, +5 ms per shifted request; assumes replicated services) and, for C, the closure law on the site total with one warm reserve.

Platform	B losing cells	C losing cells	C energy	C slowest responses	C average
Kubernetes	0	2	+16%	+56%	+30%
OpenShift	0	2	+19%	+46%	+29%
GKE (optimize)	0	0	+7%	+85%	+36%
AKS NAP (Karpenter)	0	0	+2%	+90%	+39%
Turbonomic	0	1	+7%	+99%	+75%
CAST AI	0	0	-0%	+94%	+42%
Spot Ocean	0	0	+2%	+91%	+40%

C gains are Omni against that platform (positive = Omni better).

6C. Live levers on real Kubernetes (measured)

```

Live levers on real Kubernetes (kind, 1 control plane + 6 workers), GitHub Actions run 36270589830, comm
↳ it 43f6747,
2026-09-26 20:45-20:48 UTC. Omni-Compass ran as its least-privilege service account (deploy/kind/rbac-om
↳ ni.yaml +
rbac-levers.yaml). Script: scripts/kind_levers.sh. Result: LIVE LEVERS: PASS, 19 of 19 checks, 0 lever e
↳ rrors.

Permission receipts (kubectl auth can-i as the Omni service account)
can: patch pods/resize default, patch deployment/idle-worker, patch deployments/scale idle-worker,
get configmap/omni-work, patch jobs default, create/delete resourcequotas agents, patch pods/r
↳ esize agents
cannot: delete pods agents, create pods agents, delete deployments agents, patch deployment/load-gener
↳ ator,
create resourcequotas default, get secrets (any namespace)

rightsize php-apache CPU requests: before [200m] after [50m]; after kill [200m]
coldstart idle-worker replicas: idle -&gt; 0; work waiting -&gt; 2; idle again -&gt; 0; after kill -&gt; 2
batch_pace train-a suspended: stress -&gt; true; calm -&gt; false; stress -&gt; true; after kill -&gt; false
contain agent quota present: over budget -&gt; 1; under budget -&gt; 0; over -&gt; 1; after kill -&gt; 0
agent CPU limits: before [400m 400m] contained [100m 100m] lifted [400m 400m] after kill [40
↳ 0m 400m]
cooling setpoints written to the stand-in building controller: 27.0 27.0 27.0 22.0 (kill restores 22
↳ .0)
records left after kill: none

Earlier runs of the same script (kept for the record): run 36268772674 failed (containment could not lif
↳ t: Kubernetes
refuses to raise a pod limit above a standing quota; one failing lever stopped the kill switch); run 362
↳ 69885518 printed
PASS but its checks were chained so failures were not counted (containment still not lifted); run 362702
↳ 02629 failed in
the script's quota probe once the quota was correctly deleted. Fixes: quota deleted before limits are re
↳ stored; each
lever and each kill-switch restore isolated; every check counted on its own line.
Not covered: the cooling lever writes to a stand-in controller (a CI runner has no chiller); CPU frequen
↳ cy and power caps
need hardware with cpufreq; traffic shift needs two real clusters behind one load balancer.

```

7. Live Kubernetes (measured)

Two identical kind clusters (1 control plane + 6 workers) at the same time, same load; one with Omni-Compass on top (B). Omni runs as a least-privilege service account whose permissions are proven with `kubectl auth can-i` receipts before each run.

Run	Change	Validity	p95 (ms)	p99 (ms)	Energy
36213152881	HPA target, node pool, power cap (in-place pod CPU limits), heat (harn	valid	486 to 802	675 to 1,113	222 to 215
36214629046	latency afferent (p95 over 500 ms SLO as queue pressure) and power-cap	valid	481 to 600	596 to 777	222 to 219
36216647786	SLO reflex; eviction receipt fixed	INVALID as a test of the engine	479 to 320	583 to 420	224 to 220
36218637030	controller fail-safe; benchmark prints controller log and rejects earl	PARTIAL	491 to 492	645 to 683	223 to 221
36220059046	get on pods/resize (power cap can apply)	INVALID for response time and energy	501 to 380	660 to 415	224 to 220
36221702999	probe tunnel restarts on both arms; power cap able to apply	VALID	491 to 383	613 to 491	224 to 216

Run 6 in full:

Gauge	Kubernetes alone	+ Omni-Compass	Change
Worker nodes in service, mean	6.00	3.19	-46.8%
Node-hours	2.02	1.07	-47.0%
Energy (Wh), standby counted	224	216	-3.2%
Power (W), peak	710	682	-3.8%
Energy per core-hour (Wh)	742	1,041	+40.2% worse (significant)
Utilisation	0.074	0.097	+30.2%
Pending pods, pod-minutes	0.283	0.533	+88% (n.s.)
HPA shortfall, minutes	0.817	1.33	+63% (n.s.)
Response time (ms), mean	264	165	-37.4%
Response time (ms), median	227	119	-47.4%
Response time (ms), 95th percentile	491	383	-22.1%
Response time (ms), 99th percentile	613	491	-19.9%
Failed requests (%)	0	0	0

First fully valid run with every live muscle on: fewer servers (-47%), less energy (-3.2%, standby counted), lower peak power, and faster responses at every percentile. The one significant negative is energy per core-hour (+48%): the power cap lowers the CPU the app consumes (-31%), and that CPU is the metric's denominator. Whether energy per request served also rose is not measured (the load generator's request count is not captured); a per-request energy gauge is added to the capture next.

8. The nine problem-map muscles (simulated, held-out, Omni-Compass direct against the strongest native tool)

Muscle	Opponent	Better	Worse
rightsized	hpa_vpa	cpu core hours -3%; mem gib hours -11%; p99 ms -99%; replica reversals -84%; slo breach min -57%	p95 ms +15%; oom kills +531%
coldstart	keda	p95 ms -75%; p99 ms -58%; delayed 1s pct -71%; instance hours -8%	cold starts +736%
gpupack	binpack	energy kwh -2%; idle gpu hours powered -20%	wait mean min +16%; migrations 0 to 6.07
powersmooth	floor_safe	energy overhead pct -9%; throughput loss pct -44%; swing 1s mw -6%	max ramp mw s +127%
health	detect	goodput pct +10%; lost gpu hours -20%; restarts -29%; straggler node hours -99%	-
cooling	reset	pue -2%; cooling mwh -13%; inlet violation min -73%	-
inference	keda	ttft p95 s -18%; slo breach pct -58%; gpu hours -19%; preemptions -56%	-
containment	static	rogue overspend usd -96%; time to contain min -99%; peak subagents -59%	false stops +357%; honest work pct -7%
vmenergy	ratio	attr error pct -37%; worst vm error pct -34%	-

9. Threats to validity

- Competitors are emulated from documentation on a shared plant; their production binaries may behave differently, and their full products include features (spot pricing, instance-type selection, rebalancing) that this plant does not model.
- Simulated plants are fluid models: pods pack perfectly, boot delay is fixed, power is idle + dynamic x utilisation. Real clusters fragment and boot times vary.
- Live runs use kind: nodes are containers on one CI machine; power is modelled, not metered; the two arms ran on two different CI machines. Six 20-minute runs are evidence of behaviour, not of production savings.
- Development and held-out scenarios come from the same generator; held-out protects against tuning to seeds, not against a generator that differs from real traffic. Real traces (PlanetLab) are covered in the main benchmark report.
- The loss rule uses a 0.5% tolerance and a 95% interval on 30 scenarios; small true differences can be missed.

10. Reproduce

```

pip install -r requirements.txt && python verify.py
python tuning/b_league.py dev # B vs A, choose settings (development seeds)
python tuning/b_league.py heldout tuning/B_SETTINGS_FROZEN.json # B vs A, held-out
python tuning/league.py heldout # C vs all platforms
python tuning/bound.py # perfect-foresight frontier
python tools/mechanism.py # engine modes and compute
python tuning/closure_search.py && python tuning/closure_search2.py && python tuning/closure_heldout.py
  ↳ # strict C, closure law
python tools/protocol_bench.py 100 # runtime protocol, faults at five stress level
  ↳ s
for m in rightsize coldstart gpupack powersmooth health cooling inference containment vmenergy; do pytho
  ↳ n -m omnilab.bench $m heldout; done
GitHub Actions workflow 'benchmark' (commit message tag [bench]): live A vs B on kind
python tools/abc_report.py && python pilot/bench_pdf.py docs/OMNCOMPASS_ABC_REPORT.md docs/OMNCOMPASS_
  ↳ ABC_REPORT.pdf

```

CHAPTER 49

Comparison with Existing Controllers

People who run these systems often see "autoscaling" everywhere and assume it all does the same thing. It does not. This page sets out, from public descriptions, what each system controls, and where Omni-Compass sits among them.

How to read it. Borg and Twine are private to Google and Meta; Turbonomic is closed commercial software. None of them can be run here, so they are compared by what their makers have published (papers, documentation, product pages), never by a run. A mark of "—" means the capability is not in the system's public description, not that it is impossible for it. The only fair measured comparison is on the customer's own system: their stack as it runs today against the same stack with Omni-Compass on top, in paired runs (`docs/INTEGRATION_MANUAL.md`, section 6).

1. What each system controls

System (public description)	Pods / replicas	Machines	GPU power	CPU clock / power	Moves watts between CPU and GPU	Site power budget	One decision across all of these
Kubernetes HPA	✓	—	—	—	—	—	—
Kubernetes VPA	✓ (requests)	—	—	—	—	—	—
Cluster Autoscaler, Karpenter	—	✓	—	—	—	—	—
KEDA	✓ (to zero)	—	—	—	—	—	—
Red Hat OpenShift / OKD	✓ (the Kubernetes autoscalers)	✓ (machine autoscaling)	—	—	—	—	—
IBM Turbonomic	✓ (sizing, scaling)	✓ (placement, sizing)	—	—	—	—	partial (resource actions across layers)
Google Borg	✓ (job scheduling)	✓ (cluster placement)	—	—	—	Google has published separate power-capping work	—
Meta Twine	✓	✓	—	—	—	Meta has published separate power management (Dynamo)	—
NVIDIA DCGM	—	—	✓ (limits, monitoring)	—	—	—	—
Linux schedutil, RAPL	—	—	—	✓	—	—	—
AMD SmartShift, NVIDIA Dynamic Boost (laptops)	—	—	✓	✓	✓ (inside one laptop)	—	inside one device
Intel GEOPM (HPC)	—	—	partial	✓	partial (per job)	✓ (per job / cluster)	partial
Omni-Compass	✓ (on top of HPA; or deciding)	✓ (park and wake)	✓	✓	✓ (conveyance law; simulated)	✓ (simulated)	✓

2. How each one decides

	Typical rule	Omni-Compass
Pods	add replicas when average CPU passes a target	the same autoscaler keeps working; Omni-Compass sets its target from the whole system's state and raises floors ahead of bursts
Machines	add when pods cannot be placed, remove when underused	park and wake machines by one law, and only when every sense is live, pods are not rising and nothing waits
GPU	vendor default: full power, or a fixed cap set by hand	a cap from the engine, bounded by a floor and released at once when the card is busy or response time slips
CPU	the kernel's governor picks a clock from recent load	a ceiling from the same engine, inside the nervous system's envelope
Power	a fixed plan leaving room for every device at peak; a breaker or capper reacts when the site goes over	one conserved budget: watts move from organs holding surplus to organs in need, never created, never over the budget
Safety	per tool	one OFF switch returns every organ to its recorded setting; watch mode first; service first

3. What is different, in plain words

1. **One brain instead of many separate reflexes.** Today each tool watches its own gauge: the pod autoscaler watches CPU, the node autoscaler watches pending pods, the GPU runs at its default, the power plan is fixed on paper. Nobody decides for the whole. Omni-Compass reads the whole system into one state and tells each tool the envelope it may act in. The tools keep doing their own jobs.
2. **Power as a budget that moves.** Today a building keeps room for every device to hit its peak at once, so much of its power sits reserved and unused. Omni-Compass treats the building's watts as one conserved total and moves them to where the work is: from CPUs and quiet GPUs to busy GPUs, never over the limit (docs/CONVEYANCE_LAW.md).
3. **A proof, not a tuned dial.** The budget law is proved to conserve the budget and to converge, and those properties are checked on every build (tests/test_conveyance.py).
4. **Service guarded on every power move.** Power is only taken back while response time is inside the target, and given back at once when it is not.
5. **Reversible in one move.** Every setting is recorded before it is changed and restored by one switch.

4. What the numbers show today, and what kind they are

Against	Result	Kind
Kubernetes as it runs by default (HPA + fixed nodes)	p95 response time -37% to -55%, replicas -12% to -25%, machines in service -17% to -21%	measured on real Kubernetes (kind), 10 paired runs each, results/live/
A GPU at its vendor default	+1.3% to +5.1% work per energy, p95 within +10%	modelled card (MLPerf-calibrated), results/gpu/sim/after
Today's power practice (every GPU at one fixed cap that leaves room for every CPU at peak)	+1.4% to +5.7% work served, backlog -16% to -28%, never over the site budget	modelled, results/hardware/NODE_EXCHANGE_*.json
A reactive site capper alone	60-196 minutes over the site budget without Omni-Compass; 0 with it	modelled

Turbonomic, Borg and Twine cannot be measured here. Against a customer who runs one of them, the comparison is made on the customer's own system, with the same paired method.

5. Sources

- Kubernetes HPA, VPA, Cluster Autoscaler, Karpenter, KEDA: the projects' own documentation.
- Red Hat OpenShift / OKD: Red Hat's documentation of its autoscaling and machine management.
- IBM Turbonomic: IBM's product documentation.
- Borg: Verma et al., "Large-scale cluster management at Google with Borg", EuroSys 2015; Google's public Borg traces.
- Twine: Tang et al., "Twine: A Unified Cluster Management System for Shared Infrastructure", OSDI 2020.
- Meta Dynamo: Wu et al., "Dynamo: Facebook's Data Center-Wide Power Management System", ISCA 2016.
- NVIDIA DCGM and Dynamic Boost; AMD SmartShift; Intel GEOPM: the vendors' documentation.

Where a row above is wrong or out of date, correct it from the maker's own publication.

The State of Play

Current facts only. Earlier states, failures and chronology are kept whole in docs/HISTORY.md. The release this page describes is identified by RELEASE_MANIFEST.json (commit, fingerprints of the engine, the C++ twins, the GPU protocol, the live evidence and the verification receipt), which verify.py checks against the files.

Rerun everything: pip install -r requirements.txt && python verify.py ends with VERIFICATION: PASS.

In one paragraph

Omni-Compass is a supervisory governor that sits on top of Kubernetes and hardware. On a real Kubernetes control plane it measurably makes services answer faster, on about a third fewer machines, with a clean kill switch. It has **not** yet been shown to save energy on real hardware: on kind every machine stays powered and energy is a declared model. The GPU bench that measures real joules on a card's own meter is running now on a rented NVIDIA card (Lambda); no result from it is in this repository yet. In the models, the bowl law on every muscle of the six organisms gives +0.20% to +0.30% work per energy at every size and run count completed, but it spends more time over the service line than native in every cell, so the band-first rule is not yet held. Closing that is the open work on the engine.

Measured on real systems: the newest set, Omni-Compass against Kubernetes as it runs today

Set 24 (2026-10-02, commit c908054) repeats it again: machines in service –31.6%, p95 –60.1%, p99 –64.1%, HPA replicas –38.6%, 0 failed requests, total CPU including Omni-Compass's own –1.8% (not significant) (results/live/LIVE_REPS_24.md). Set 23 before it: p95 –62%, replicas –37%, pod starts –64%, 0 failed requests, no energy or total-CPU difference (results/live/LIVE_REPS_23.md). The set-22 table below stands as first measured.

Set 22 (results/live/LIVE_REPS_22.md): 10 paired repetitions on real Kubernetes (kind), each pair on one machine, Kubernetes with its autoscaler alone against the same Kubernetes with Omni-Compass on top. The load is sent at a fixed rate, so both arms were given **the same work**.

Result	Kubernetes alone	With Omni-Compass	Change (95% interval)
Energy, parked machines still on at idle power (declared model, no meter)	160.3 Wh	160.1 Wh	–0.1%, no difference
Response time, 95th percentile	407.9 ms	158.8 ms	–61% (proven)
Response time, 99th percentile	639.4 ms	245.1 ms	–62% (proven)
Response time, mean	179.9 ms	98.4 ms	–45% (proven)
Failed requests	0	0	equal

Result	Kubernetes alone	With Omni-Compass	Change (95% interval)
Pods waiting to start, pod-minutes	0.265	0.025	-91% (proven)
Replicas, mean	8.93	6.91	-23% (proven)
Machines in service, mean (all stayed powered)	6	4.14	-31% (proven)
CPU used by the service	1.036 cores	0.957 cores	-7.6% (proven)
Omni's own CPU (its controller and every command it ran)	0	0.070 cores	+0.070 (proven)
CPU used, service and Omni together	1.036 cores	1.026 cores	-0.9%, no difference

- **Same work, much faster answers**, with no failed requests and far less waiting.
- **No energy saving is shown on kind**. Every machine stays powered; energy is a declared model, and counted at the idle power a parked machine really draws it is unchanged.
- **No CPU saving once Omni's own cost is counted**. The service used 7.6% less CPU; the controller spent almost all of it. Cutting the controller's cost is the next improvement.
- The kill switch restored every setting in every run.

Running now on real Kubernetes, no result in the repository yet

Run	State
Set 25: native against Omni-Compass on top (the engine's allocation law), 10 paired repetitions, fixed-rate load, current <code>main</code>	running (GitHub Actions <code>benchmark - reps</code>)
Set 26: native, the engine's allocation law, and the bowl law in the live controller (- - <code>law bowl</code>), 10 paired repetitions, fixed-rate load, rule written first (<code>docs/K8S_BOWL_PREREGISTRATION.md</code>)	running (GitHub Actions run 37058424766, commit <code>e7f920d</code>)

Measured on a real GPU: the card's own meter (evidence class P)

First confirmation, NVIDIA A10 on Lambda, 2026-10-02 (results/gpu/run-20261002T082232Z/GPU_REPS.md, 10 paired repetitions × native / watch / Omni, 600 s each, frozen at commit `c908054`, checksums verified). Wire check 7 of 7; 2,144 writes, none refused, every one read back, every arm ended at the start limit; watch equal to native.

Gauge	Native	Omni	Change (95% interval)
Work per energy (requests per kJ)	50.79	52.62	+3.6% (+2.7% to +4.5%), proven
GPU energy	69,180 J	66,790 J	-3.5%, proven
Requests served / not served	3,514 / 0	3,514 / 0	equal
Response time, 95th percentile	510 ms	809 ms	+58.5%, worse, proven

Result, by rule: ENERGY IMPROVEMENT WITH SERVICE TRADEOFF (the p95 guardrail of +10% failed). The six organisms with the same card inside (results/hil/run-20261002T082232Z/HIL.md, 3 repetitions each): the card's work per energy +1.6% to +2.8% in every organism, the same requests, its p95 500 to about 600-935 ms. The cause, from the card's own samples, was wiring in the governor (amendment 6 of `docs/GPU_PREREGISTRATION.md`): busy bursts served at 736-768 MHz against 861-889 MHz on its own. Corrected (amendments 6 and 7); the corrected governor has not yet run on a card.

Simulated (models: they show the mechanism, not a measurement)

Result	Where
GPU governor with share floor and busy gate (one wire, the power limit), MLPerf-calibrated card: +5.1% and +1.3% work per kJ, p95 within +10%	results/gpu/sim/after
Two-wire GPU card (clock ceiling up, power limit down) under the bowl law , 10 seeds: work per energy +8.6% (+7.8 to +9.4), energy -7.9%, time over the line unchanged (-0.01 pp), but p95 response +32.9% (+6.3 to +59.4); the one-wire governor on the same card +0.1%; both wires restored every seed	results/sim/gpu_two_wire/RESULT.md
The six organisms (Compute 345, Physics 262, Energy 282, Distribution 337, the four stacked 1,226, the whole tower 656), native against the bowl law on every muscle, 1,000 paired runs at 1× and at 10× size: work per energy +0.20% to +0.30%, energy -0.21% to -0.32%, work -0.01% to -0.02%, time over the service line +0.19 to +0.27 pp in every cell (band first not held) , every knob handed back. 100× and 1,000× are running	results/scale/GRID.md
Speed lock (speed won elsewhere spent on GPU watts)	results/gpu/sim/pipeline/
CPU and GPU on one conserved power budget: +1.4% to +5.7% work served against a fixed cap, never over the budget	results/hardware/NODE_EXCHANGE_*.json, docs/CONVEYANCE_LAW.md
GPU groups sharing a site budget: 0 minutes over the budget	results/hardware/SITE_EXCHANGE_HELDOUT_*.json
Platform leagues, faults, PlanetLab traces, stack benchmark	tuning/, results/protocol/, results/ (see docs/BENCHMARK_REPORT.md)
The 656-muscle tower as organisms , round 3 (preregistered, 10 seeds; every realm carries the shared spine; Omni as the shipped controller commands): the whole tower native against one governor on top, work per energy +0.1%, SUPERIOR WITHIN GUARDRAILS ; inside the realms the spine costs service: Energy +0.2% with +1.9 pp violations (tradeoff), Compute 0.0% (+2.1 pp, not established), Distribution -0.1% and Physics -0.7% (WORSE). Rounds 1 and 2 kept, superseded	results/realms/REALMS.md, docs/REALM_MUSCLES.md

Verified in code

Property	Where
The canonical engine is <code>symmetric_verified</code> ; the printed chart is a named variant, not benchmarked	docs/CANONICAL_ENGINE.md
Nine laws twinned in C++20 and proven equal to the Python; sealed by fingerprint	results/SEAL.json, tools/seal.py
The conveyance law conserves its budget and converges (proof and 20,000 random systems)	docs/CONVEYANCE_LAW.md, tests/test_conveyance.py
Safety shield: 2,000,000 adversarial cases, 0 violations; C++ engine: 100,000,000 decisions, no failures	tests/test_shield_properties.py, results/SOAK.json

Open

1. **Band first.** In every organism and every size the bowl law raises the time over the service line by about 0.2 points. The rule is no win unless that is at or under native's. This is the first thing to fix in the law.
2. **The first real-hardware run:** `sudo bash scripts/gpu_rented_run.sh` on a rented NVIDIA machine (smoke, then the 10 preregistered repetitions, docs/GPU_RUN_GUIDE.md), or the gpu-bench workflow on GitHub's GPU runner. Then a second machine of the same type, then another GPU type. Status 2026-10-02:

the rented-card run above is under way. GitHub's GPU runner has never been assigned to a job (every run waited in the queue; the repository is public, so the ordinary runners are free while a GPU runner is always billed, and the account has an Actions billing notice). The envelope rule is preregistered (amendment 3).

3. **Work per energy on kind:** count requests served, or run an open-loop load at a fixed rate, so work per energy can be stated instead of estimated (set 22, LOADGEN=open).
4. **CPU and GPU on one power budget on hardware:** the law is simulated; the live exchange is not wired.
5. **A global stability proof** of the forced six-state system (docs/FORMAL_STATUS.md).
6. **The principal embodiment for filings** (docs/CANONICAL_ENGINE.md, section 5): a decision for the company.

Where things are

Path	What it is
docs/INTEGRATION_MANUAL.md	the manual in the box: wiring it in yourself, stack by stack
docs/METRICS_CATALOG.md	every gauge, and whether it is measured or modelled
docs/COMPARISON.md	against Kubernetes, OpenShift, Turbonomic, Borg, Twine and others
docs/CANONICAL_ENGINE.md	the one engine the software runs
omnicompass/, cpp/	the engine and its laws; the C++20 twins
omni_controller/	the Kubernetes controller, the GPU governor, the muscles
results/live/	every live run, including failed and withdrawn ones
docs/HISTORY.md	earlier states of play

PART EIGHT

Value, License and History

*What a receipt is worth, how the license is priced against it, how the code is sealed,
and how Omni-Compass came to be.*

CHAPTER 51

Where the Value Comes From

Every system runs with room it does not use: GPUs boost to the top of their clock range and are knocked back by their own power limiter many times a second; Kubernetes keeps replicas and machines sized for the worst minute; cooling runs colder than the heat requires; batteries hold more reserve than the hour needs. That room is paid for in energy and in machines.

Omni-Compass holds each service in the middle of its band instead of far below its limit. The room that was spent on nothing becomes one of two things, and the receipt shows which:

- **More work for the same energy** (work per energy rises), or
- **The same work for less** (fewer machines in service, fewer watts).

They are the same gain read from two sides. The receipt reports one number, work per energy, and beside it the machines, the response times and the failures, so nothing is hidden.

CHAPTER 52

The Economics of a Receipt

Run the stack native and print the receipt. Run the same stack with Omni-Compass and print the receipt. The difference is the gain, read from either side: more work for the same energy, or the same work for less energy and fewer machines.

The license fee is set against the measured gain on the customer's own receipt. On a stack that costs one hundred, a measured gain of ten is worth ten; the fee is a share of that ten, and the customer keeps the rest. Percentages from different parts of a stack are not added into one bill: the invoice is one gain, measured once.

The value a customer sees comes in three forms, each on its own line of the receipt:

1. **Work per energy** - more finished work per kilowatt-hour.
2. **Capacity** - the same work on fewer machines in service; it becomes a cost reduction when the released machines are returned to the cloud or switched off by the platform around Omni-Compass.
3. **Service** - faster responses and fewer breaches at the same load.

The babysitting tax - the people and tools kept on the clock to set caps, answer pages and turn knobs back after a run or a crash - is the cost Omni-Compass removes by holding the knobs and returning them itself.

The Buyer Edition

Superseded for live results. The live evidence below predates sets 19 and 20. The current live table is `results/live/LIVE_PAired.md`; energy on kind is a declared model, not a meter. The first metered test is `scripts/gpu_paired.sh` (`docs/GPU_BENCH.md`).

Commit `cdc8cd9`. Every number below is generated from result files in the repository by `python tools/buyer_report.py`. Simulated results use the repository's fleet plant; live results come from real Kubernetes (kind) in GitHub Actions. Competitors are reproduced from their public documentation, not their binaries.

What Omni-Compass is

- **The engine:** a six-state control engine, stable by construction (every mode decays), in Python and C++.
- **The law it applies:** the closure law of the owner's manuscript.
 - It does nothing in the safe interior.
 - It corrects inward when the forecast approaches capacity.
 - It releases a machine only after the peak.
 - It keeps released machines warm for instant reuse.
- **What it governs:** machines, pod counts and sizes, cold start, batch pacing, containment of runaway agent workloads, and cooling setpoints.
- **Two ways to deploy it:**
 - **B, on top of the platform a customer already runs.** Nothing is replaced.
 - **C, alone.** Kubernetes stays only as the muscle; the scaling layer is replaced (HPA, Cluster Autoscaler, Karpenter, and optimizers such as CAST AI, Spot Ocean and Turbonomic).

B: Omni-Compass on top of each platform (held-out scenarios, settings frozen first)

Workload	Platform	Omni-Compass on top vs the platform alone
Web services	Kubernetes	scale reversals 12% better, machine starts+stops 2% better
Web services	OpenShift	scale reversals 44% better, machine starts+stops 4% better
Web services	GKE	scale reversals 21% better, machine starts+stops 7% better, machine-hours 3% better
Web services	AKS NAP / Karpenter	scale reversals 8% better, machine starts+stops 3% better
Web services	Turbonomic	scale reversals 8% better, machine starts+stops 2% better
Web services	CAST AI	slowest responses (p99) 34% better, scale reversals 30% better, machine starts+stops 15% better
Web services	Spot Ocean	slowest responses (p99) 42% better, scale reversals 34% better, machine starts+stops 17% better
Four clusters, one site	Kubernetes	scale reversals 93% better, slowest responses (p99) 51% better, machine starts+stops 21% better
Four clusters, one site	OpenShift	scale reversals 100% better, slowest responses (p99) 39% better, machine starts+stops 16% better
Four clusters, one site	GKE	slowest responses (p99) 38% better, scale reversals 19% better, machine starts+stops 11% better
Four clusters, one site	AKS NAP / Karpenter	scale reversals 7% better, slowest responses (p99) 6% better, machine starts+stops 4% better
Four clusters, one site	Turbonomic	scale reversals 4% better, machine starts+stops 2% better
Four clusters, one site	CAST AI	equal
Four clusters, one site	Spot Ocean	scale reversals 9% better, slowest responses (p99) 5% better, machine starts+stops 5% better
Batch jobs	Kubernetes	slowest responses (p99) 91% better, scale reversals 82% better, average response 25% better
Batch jobs	OpenShift	slowest responses (p99) 92% better, scale reversals 84% better, average response 26% better
Batch jobs	GKE	time over backlog limit 100% better, slowest responses (p99) 89% better, scale reversals 66% better
Batch jobs	AKS NAP / Karpenter	typical response (p95) 60% better, slowest responses (p99) 34% better, time over backlog limit 25% better
Batch jobs	Turbonomic	time over backlog limit 44% better, scale reversals 22% better, machine starts+stops 15% better
Batch jobs	CAST AI	time over backlog limit 100% better, typical response (p95) 94% better, slowest responses (p99) 91% better
Batch jobs	Spot Ocean	time over backlog limit 83% better, scale reversals 59% better, machine starts+stops 47% better
GPU training	Kubernetes	scale reversals 68% better, machine starts+stops 35% better
GPU training	OpenShift	scale reversals 61% better, machine starts+stops 29% better, time healthy 1% better
GPU training	GKE	scale reversals 59% better, machine starts+stops 39% better, machine-hours 1% better
GPU training	AKS NAP / Karpenter	machine starts+stops 51% better, scale reversals 33% better, machine-hours 2% better
GPU training	Turbonomic	scale reversals 12% better, machine starts+stops 9% better

Workload	Platform	Omni-Compass on top vs the platform alone
GPU training	CAST AI	machine starts+stops 45% better, scale reversals 32% better, machine-hours 2% better
GPU training	Spot Ocean	machine starts+stops 28% better, scale reversals 21% better

C: Omni-Compass alone, one global setting, confirmatory run

- **Setting:** one global setting for every workload, frozen by SHA-256 before the run (28cbb82607cb0009).
- **Scenarios:** 100 never-used scenarios per workload (seeds 710001-710100).
- **Comparisons:** 364 cells (4 workloads x 7 platforms x 13 gauges).
- **Rule:** a cell counts as better or worse only if it survives Holm-Bonferroni correction across all 364 cells (family alpha 0.05) and exceeds the 0.5% practical tolerance. Otherwise it is equal.

Result: 153 better, 174 equal, 37 worse.

Workload	Better	Equal	Worse	The worse cells
Web services	41	41	9	Kubernetes typical response (p95) 20.7%; Kubernetes machine starts+stops 24.5%; OpenShift typical response (p95) 20.8%; OpenShift machine starts+stops 66.0%; OpenShift scale reversals 152.0%; GKE typical response (p95) 20.1%; AKS NAP / Karpenter typical response (p95) 19.7%; CAST AI typical response (p95) 19.2%; Spot Ocean typical response (p95) 19.7%
Four clusters, one site	45	36	10	Kubernetes typical response (p95) 20.4%; Kubernetes machine starts+stops 23.7%; OpenShift typical response (p95) 20.5%; OpenShift machine starts+stops 62.1%; OpenShift scale reversals 207.3%; GKE typical response (p95) 19.7%; AKS NAP / Karpenter typical response (p95) 19.3%; Turbonomic typical response (p95) 27.6%; CAST AI typical response (p95) 18.8%; Spot Ocean typical response (p95) 19.3%
Batch jobs	35	50	6	Kubernetes machine starts+stops 39.0%; OpenShift machine starts+stops 29.5%; AKS NAP / Karpenter energy 2.1%; CAST AI energy 1.2%; Spot Ocean energy 2.2%; Spot Ocean machine-hours 0.6%
GPU training	32	47	12	AKS NAP / Karpenter energy 1.0%; AKS NAP / Karpenter time over power limit 1.6%; AKS NAP / Karpenter time over heat limit 1.1%; CAST AI energy 1.0%; CAST AI time over power limit 3.2%; CAST AI time over heat limit 2.6%; CAST AI time healthy 1.6%; Spot Ocean energy 1.2%; Spot Ocean time over power limit 3.8%; Spot Ocean time over heat limit 1.8%; Spot Ocean machine-hours 1.4%; Spot Ocean time healthy 2.0%

Workload	Energy vs Kubernetes	Machine-hours	Slowest responses	Average response
Web services	+21%	+38%	+40%	+11%
Four clusters, one site	+25%	+40%	+41%	+13%
Batch jobs	+6%	+18%	+40%	+2%
GPU training	+2%	+11%	+0%	+0%

Positive means Omni-Compass is better.

Real demand: 1,052 recorded machines (PlanetLab)

- **Losing cells:** Omni-Compass alone, with the frozen web setting (never tuned on these traces), loses 0 of 91 against the seven platforms.
- **Whole-pod packing:** with whole-pod packing it loses 0 of 91.
- **Against Kubernetes:**

- energy +24%
- machine-hours +33%
- typical response +97%
- slowest responses +85%

Positive means better.

Four-cluster sites as one body (held-out)

- **The mechanism:** traffic shift between clusters, plus the law run on the site total with one warm reserve.
- **Omni-Compass alone:** zero losing cells against GKE, AKS NAP / Karpenter, CAST AI, Spot Ocean.
- **Omni-Compass on top:** zero losing cells on all seven platforms.
- **Assumption:** services are replicated across the site's clusters.

Under failure: the runtime protocol

Machines dying, load spikes, crash-looping services and noisy neighbours were injected at five stress levels, with identical faults for every system. Totals over all levels:

Workload	System	Runs conveyed	Pages to a human	Recovery (min)
Web services	CAST AI	231 of 500	404	6.6
Web services	Kubernetes	218 of 500	338	8.3
Web services	Omni-Compass alone	240 of 500	246	5.1
Four clusters, one site	CAST AI	44 of 500	608	5.7
Four clusters, one site	Kubernetes	44 of 500	330	6.5
Four clusters, one site	Omni-Compass alone	46 of 500	197	3.6

The supervisory nervous system

- **One state, every organ:** `omnicompass/nervous_system.py` turns the engine's state (convergence, basin health, stress against its equation-6 equilibrium, unmet need) into one calm value between 0 and 1.
- **Authority from calm:** calm grants each organ its authority:
 - pods and machines may give capacity back only above their reversibility thresholds (0.5 and 0.7);
 - CPU frequency, GPU power, routing and cooling get envelopes that widen with calm;
 - batch is admitted or paused;
 - rollback is authorised.
- **Holds and the shield:** a security hold stops every capacity organ from expanding. The shield stays downstream and can still veto.
- **Invariants tested:** 300,000 random engine states, with zero violations (`tests/test_nervous_system.py`).
- **Coordination:** pods move first, and a machine move opposite to the pod move is vetoed. Controller contradictions per day (fighting, or a reversal within one boot time):
 - web: 2.10 to 0.10;
 - four-cluster: 5.30 to 1.03, below Kubernetes' 2.10.

Coordination, confirmatory on fresh seeds 712001-712100 (frozen first in tuning/COORD_PREREGISTRATION.json, same 364 cells, same Holm rule): 151 better, 173 equal, 40 worse.

Workload	Better	Equal	Worse	Contradictions per day: Omni-Compass / Kubernetes
Web services	40	41	10	0.07 / 0.52
Four clusters, one site	45	37	9	0.56 / 1.77
Batch jobs	35	49	7	0.11 / 0.00
GPU training	31	46	14	0.11 / 0.04

Mechanism ablation (tuning/ABLATION.json, fresh seeds 711001-711030): each part of the law removed in turn, against the full law. Listed: the gauges that get significantly worse (95% interval excludes 0, more than 0.5%).

Part removed	Web services	Four clusters, one site	Batch jobs	GPU training
engine release gate removed	no change	no change	no change	no change
turning point removed	no change	machine starts+stops +2%, scale reversals +6%	slowest responses (p99) +32%, average response +12%, time over backlog limit +333%, machine starts+stops +47%, scale reversals +193%, contradictions +1350%	machine starts+stops +18%, scale reversals +118%, contradictions +1000%
muscle tone removed	typical response (p95) +1%, slowest responses (p99) +110%, average response +4%, machine starts+stops +92%, scale reversals +458%, contradictions +22%	slowest responses (p99) +8%, machine starts+stops +29%, scale reversals +108%	typical response (p95) +65%, slowest responses (p99) +151%, average response +26%, machine starts+stops +239%, scale reversals +1265%, machine-hours +1%, contradictions +875%	time over backlog limit +2%, machine starts+stops +297%, scale reversals +829%, contradictions +1150%
trend term removed	no change	slowest responses (p99) +2%	slowest responses (p99) +75%, average response +24%, time over backlog limit +533%, machine starts+stops +65%, scale reversals +348%, contradictions +2350%	machine starts+stops +29%, scale reversals +129%, contradictions +1100%

Live Kubernetes evidence

Repeated live runs, 5 paired repetitions per arm, probe through the Service (results/live/LIVE_REPS_3.md):

Live repetitions, set 3: GitHub Actions run 36289557446 (commit 9892270), 5 × native / omni / strict on kind

This is the first set with the probe through the Service (NodePort via kube-proxy) and the PodDisruptionBudget, and the first live set with the supervisory nervous system gating machine release.

B: Omni-Compass on top vs native, 5 paired repetitions

Gauge	Native	Omni	Change	95% interval of the difference	Significant
worker nodes in service, mean	6	6	+0.0%	-3.155e-16 to +6.708e-16	no
node-hours	1.517	1.516	-0.1%	-0.02125 to +0.01859	no
energy (Wh)	168.1	165.7	-1.4%	-7.697 to +2.854	no
response time (ms), mean	221.7	210.3	-5.1%	-57.38 to +34.55	no
response time (ms), 95th percentile	431.4	440.2	+2.0%	-47.09 to +64.69	no
response time (ms), 99th percentile	568.9	609	+7.0%	-51.68 to +131.8	no
failed requests (%)	0	0	+0 (native is 0)	+0 to +0	no
pending pods, pod-minutes	0.3433	0.3967	+15.5%	-0.4106 to +0.5172	no
utilisation (used / allocatable)	0.07676	0.06667	-13.1%	-0.02607 to +0.005892	no

C: Omni-Compass decides (strict) vs native, 5 paired repetitions

Gauge	Native	Omni	Change	95% interval of the difference	Significant
worker nodes in service, mean	6	6	+0.0%	-7.797e-16 to +7.797e-16	no
node-hours	1.517	1.518	+0.0%	-0.00974 to +0.01041	no
energy (Wh)	168.1	166.4	-1.0%	-4.867 to +1.447	no
response time (ms), mean	221.7	217.8	-1.8%	-89.74 to +81.98	no
response time (ms), 95th percentile	431.4	463.2	+7.4%	-105.7 to +169.4	no
response time (ms), 99th percentile	568.9	629	+10.6%	-155.4 to +275.7	no
failed requests (%)	0	0	+0 (native is 0)	+0 to +0	no
pending pods, pod-minutes	0.3433	0.8033	+134.0%	+0.05385 to +0.8661	yes, worse
utilisation (used / allocatable)	0.07676	0.06928	-9.8%	-0.01814 to +0.003173	no

Reading

Failed requests. They are 0% in every arm. The 16-66% of sets 1-2 was the probe's hung tunnel, as diagnosed.

Machines. Omni-Compass kept all 6 machines in service in every repetition. Controller log: 15 of 15 decisions, 0 errors, no node-scale command.

- **The likely cause.** With a working probe, p95 sits at 430-460 ms against the 500 ms SLO declared before the run. Every window above 500 ms is SLO pressure. That pressure blocks machine release for the next 3 decisions and lowers the engine's calm.
- **Why that is odd here.** Machines are at 7% utilisation with no waiting pods. The latency comes from the workload's per-request compute, not from a machine shortage.
- **Why the earlier sets differed.** Sets 1-2 released machines because the hung probe produced few or no successful samples, so the controller saw little latency pressure. Their machine savings were therefore measured with a blinded latency sense and are not claimed.
- **How this will be settled.** The decision trail (p95, SLO state, calm, node authority per decision) is now printed in every job log. Set 4 will show which gate held.

Everything else. No significant difference from native on any gauge, except that C has more pending pod-minutes (0.80 vs 0.34 pod-minutes).

- **Earlier live sets 1 and 2:** these carried a probe defect, now fixed. The probe's one-pod tunnel hung when a drain moved its pod, so the Omni arms logged false failed requests (results/live/LIVE_REPS_PROBE_DEFECT.md).
- **Their machine savings are withdrawn.** The hung probe left the controller's latency sense nearly blind, so it released machines it would not have released with a working probe. Set 3 shows that: with a working probe and latency near the declared 500 ms SLO, Omni-Compass kept all 6 machines. The decision trail per run is now in every job log (set 4).

Live levers under the nervous system:

- **What acted:** right-sizing, cold start, batch pacing, agent containment and the cooling connector.
- **Authority:** each lever acted on real Kubernetes only inside the authority the nervous system granted.
- **Kill switch:** it restored every lever, including from a fresh process.
- **Result:** 19 of 19 checks passed (results/live/LIVE_LEVERS_2_NERVOUS.txt, first pass LIVE_LEVERS_1.txt).
- **Identity:** least-privilege, with `kubectl auth can-i` receipts.

Shadow pilot kit, live: a read-only identity ran for 600 s and logged 40 decisions, with 0 writes (results/live/LIVE_SHADOW_1.txt). This is the kit a customer runs first.

Safety and correctness

- **Safety shield:** tested on 2,000,000 random and adversarial inputs, with zero invariant violations, idempotent, never inventing an action, and intervening minimally (tests/test_shield_properties.py). The test found two real bugs, both fixed.
- **C++ twins:** the C++ shield and the C++ closure law match Python exactly, over 300,000 adversarial shield cases and every recorded closure decision.
- **Endurance:** the C++ engine ran 100,000,000 decisions with no failure, at about 2.3 microseconds per decision.
- **Fail-safe:** after repeated failed decisions, control returns to the native autoscalers.
- **Reproducibility:** a clean copy of the delivered zip reproduced every held-out result byte for byte.

What is not claimed

- **No production or customer deployment yet.** The next step is the shadow pilot (docs/PILOT_KIT.md), which is read-only.
- **Live runs are small.** They use kind on CI machines, and power is modelled, not metered.
- **Hardware levers are not proven.** CPU-frequency and power caps need real servers, and cooling was exercised against a stand-in controller.
- **Parked machines in the public cloud.** A parked cloud machine still bills, so the warm-reserve energy saving applies to owned hardware.
- **Single-cluster energy against the tightest packers is roughly a tie.** There Omni-Compass wins on response time and stability.
- **Some gaps cannot be closed.** No controller, even one with perfect foresight, can match both the tightest packer's machine-hours and the calmest autoscaler's machine churn (tuning/bound.py).

- **Not modelled in the plant:** variable boot times and pod-eviction cost. Fragmentation is modelled (whole-pod packing) and is small.

Reproduce

```
pip install -r requirements.txt && python verify.py
python tuning/confirmatory.py          # C, one global setting, 100 scenarios per workload, Holm-corrected
python tuning/confirmatory.py --coord  # the same with nervous-system coordination, fresh seeds
python tuning/planetlab_league.py <planetlab-workload-traces/20110303>
python tuning/site_league.py --heldout
python tools/protocol_bench.py 100
live: push a commit whose message contains [reps], [levers] or [shadow]
```

CHAPTER 54

Due Diligence

Answers reference the Claims Register (C-numbers) and the Technical Manual.

Engineering

Does the mathematics hold? Equations (1) to (8), Propositions 1 to 3 and their proofs are in Manual Chapters 2 and 3. The 500/500 CONVEY/CERT result is a property of the controller (Proposition 3), shown by counterfactuals: the controller aimed at the wrong basin also scores 500/500. **Is Python the same as C++?** Yes (C1, C2). The verifier also builds a deliberately mutated C++ governor and confirms the parity test rejects it. **Can every number be reproduced?** `python verify.py` rebuilds the C++, reruns parity, reruns the 100,000,000-decision soak and compares it with the recorded result, replays held-out scenarios and checks the pre-registered SHA-256 hashes (program fingerprints are additionally checked when the Python version matches the recorded one).

Operations

Better than what we run today? Compared against a documented-behaviour reference model of Kubernetes autoscaling (HPA tolerance 0.1, 300 s scale-down stabilization, HPA targets 0.5 to 0.8; simplified Cluster Autoscaler with 10-minute unneeded time, 0.5 utilization threshold, 10-minute delay after scale-up): C8 to C12, including where Omni-Compass is worse. The reference model is not the upstream controllers (C12b); running the upstream controllers against the same scenarios is the next baseline step. **On real traffic?** Not yet. Results use a synthetic stack model. Replay of published production traces and the pilot protocol are the next evidence steps (C14). **What happens when it is wrong?** Observe mode changes nothing (C6). The kill switch returns control to the native managers at the next interval (omni_kill arm). The shield blocks actions that violate I1 to I5 (C7). **Will it wear hardware?** Machine start/stop cycles, power-cap travel and thermal travel are measured for every arm (C12, Manual Chapter 8).

Security

What authority does it hold? Capacity, replicas, power caps, rollback authorization and routing, only in AUTOPILOT, only through the shield. **Can it expand capacity during a security block?** No: invariant I1 is enforced before execution. **Does it replace encryption, identity or policy engines?** No. Those components are retained (fleet model, security role).

Finance

What does it save? Energy per run versus current autoscaling (C8). Fleet-scale figures are modeled (C13); the governance saving comes from reduced idle and padded capacity, not from removing the decision components' own consumption. **What does it cost to run?** C5.

Adoption

How is it introduced without risk? Observe, then shadow on production telemetry, then one control loop at a time under the kill switch (docs/PILOT_PROTOCOL.md).

Referees

Was it tuned on the test data? No. Law, shield and baselines were frozen and fingerprinted before the held-out seeds 346410161 and 360555127 (results/PREREGISTRATION.json). **Where does it fail?** Backlog violations against current autoscaling (C11); the engine-dynamics ablation (Manual Chapter 8); open obligations (Manual Chapter 10).

CHAPTER 55

License and Commercial Terms

The software and this manual are licensed under the Omni-Compass Evaluation License (LICENSE): evaluation and simulation use only. Everything else - commercial use, production use, operating any system beyond evaluation, redistribution, a hosted or managed service, incorporation into a product or service, or using the software or its results to build a competing product - requires a written Omni-Compass Enterprise License signed by The Omni-Compass LLC and paid for. Patent applications, copyright registrations and trademark applications covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC. No patent or trademark license is granted for any other use. Contributions are accepted only on the terms in CONTRIBUTING.md, which assign their rights to The Omni-Compass LLC.

CHAPTER 56

Python, C++ and the Seal

The laws are twinned: each has a Python version and a C++20 version that give the same answers, proven by a parity test on every build (`cmake -S cpp -B cpp/build && cmake --build cpp/build`).

Law	Python	C++	Proven by
core engine	omniconpass/core.py	cpp/src/core.cpp	500 frozen fixtures
governor	omniconpass/adapter.py	cpp/src/governor.cpp	tests/test_cpp_governor_parity.py
safety shield	omniconpass/shield.py	cpp/src/shield.cpp	tests/test_cpp_shield_parity.py
HPA replica law	fleet/harness.py	cpp/src/hpa.cpp	tests/test_cpp_hpa_parity.py
closure law	omniconpass/closure.py	cpp/src/closure.cpp	tests/test_cpp_closure_parity.py
conveyance law	omniconpass/conveyance.py	cpp/src/conveyance.cpp	tests/test_cpp_conveyance_parity.py
nervous system	omniconpass/nervous_system.py	cpp/src/nervous_system.cpp	tests/test_cpp_twins_parity.py
compass and ledger	omniconpass/compass.py, storage.py	cpp/src/compass.cpp	tests/test_cpp_twins_parity.py
GPU governor rules	omni_controller/gpu_governor.py	cpp/src/gpu_rules.cpp	tests/test_cpp_twins_parity.py

The seal (`results/SEAL.json`) holds the SHA-256 fingerprint of every twinned file, written only after every parity test passes. `verify.py` fails, naming the file, if any sealed file changes afterwards. The bowl law (`omniconpass/bowl.py`) and the two-wire GPU governor are in Python today; their C++ twins are next.

CHAPTER 57

The Founder's Working Notes

Where a sentence is a design aim rather than what the code does today, or a figure that is not yet measured, the check column says so. Nothing in this chapter adds a result: every number that is a result names the file it comes from.

What Omni is

OmniCompass is the governor. It is the process running on the processor, and the chip executes it. The card, the replica count, the rack cap, the joint and the feeder are the levers: the muscles. Omni reads the meters, steps a bounded loop, and writes only the lever it is allowed to write. When it stops, it puts that lever back.

It is not the chip, not the card and not Kubernetes. It is the brain on the host, writing through a bolt.

Governor is the machine word. A governor on a steam engine did not build the engine and did not turn the shaft. It watched the speed and moved the throttle so the speed stayed in a band. The plant does the work: the chip, the card, the pods, the job. Omni reads a meter, writes one lever, and puts that lever back.

The loop

The chapter says	The code today	Check
Omni reads the plant's meters and maps them to the state E, U, S	<code>omniconpass/adapter.py</code> , <code>observe_vector</code> and <code>assimilate</code>	matches
It computes the drift with the command at zero, then $\text{raw} = -\text{drift} + K_P \cdot (\sigma - U)$, clipped to the authority before the step	<code>omniconpass/core.py</code> , <code>control_command: u = clip(-f_U(x,t) + KP · (σ - U), ±U_AUTHORITY)</code>	matches
It holds that same u through every RK4 stage and does not rewrite U afterwards	<code>core.py</code> , <code>macro_step</code> : one u per micro step, passed unchanged into all four RK4 stages; U is never overwritten	matches (proved: <code>docs/TRACKING_THEOREM.md</code>)
It writes the clipped command u to the lever	Not today. The frozen live governor runs the engine with $u = 0$ (<code>macro_step(..., target=None)</code>) and uses u only as its convergence signal ($\text{push} = u / U_AUTHORITY$). The lever is set by the allocation law and the nervous system's authority (<code>omniconpass/nervous_system.py</code>).	design aim , see "What to strengthen"
On exit it restores the lever	kill switch: <code>omni_controller/controller.py</code> restore; the GPU bench restores and reads back the start limit; the realm harness checks every knob is handed back	matches

So the step the chapter describes is the engine's own step, and it is proved. Writing u itself onto a lever, through an output map with a gain in watts (or replicas) per unit of u, is a further mechanism. It is not what produced the results

below.

Wiring

A wrist wire moves the wrist. It does not move a finger that has its own tendon unless that tendon is also wired.

The card's clock wire goes to the NVIDIA driver. The replica wire goes to the Kubernetes API. The scheduler's wire goes to the kubelet. Those wires already exist and do not pass through Omni. Omni is an added process, and a tendon is connected to it only if the output map writes that lever and the input map reads it back.

A power-limit write can make the driver drop clocks. That is a side effect on one finger, not the replica nerve, which keeps running on its own path unless a second line is connected. The cluster starts its own wires: the API server accepts a replica write, the scheduler places the pod, the kubelet starts it, the driver moves the clocks, and HPA, if installed, is its own process. Omni joins those nerves. It is a client of the plant, not its origin.

Four realms, five organisms

1. Compute / AI / Cloud: GPU server, Kubernetes, the card, the job.
2. Physics / Robotics / Autonomous: joint, servo, PLC, car, line.
3. Energy / Facility / Industrial: rack, cooling, PDU, battery.
4. Distribution / Specialized: network, feeder, logistics.
5. The whole organism: all 656 canonical muscles, each once, one clock.

A muscle may sit in more than one realm. Every realm stands on the same spine (Kubernetes, machines, GPUs and CPUs, network, storage, observability, security, cooling, electrical distribution), and then has its own domain muscles. The fifth run holds each of the 656 once.

Each organism is shown native, then with Omni, on the same membership, the same seed and the same clock: two receipts. The core does not change when the lever changes.

Check	
Built	<code>realms/</code> (round 3): spine of 190 muscles in all four realms; organisms of 345, 262, 282, 337 and 656 (<code>docs/REALM_MUSCLES.md</code>)
Result	Whole organism +0.1% work per energy, SUPERIOR WITHIN GUARDRAILS; the realms pay in service when they share the spine (<code>results/realms/REALMS.md</code>)
Limit	These are modelled plants (evidence S / rung E2). A catalog row is not a tendon. GitHub can host the compute organism for real (kind, and the GPU on a rented card); facility, machine and grid need their own hosts
Open	The 656 is a working catalog, not a census: <code>docs/realms_study/</code> checks it against the real systems' documentation and industry practice (waves 1 and 2 done; wave 3 maps every row)

Evidence rungs

Rung (chapter)	Class (repository, docs/EVIDENCE_LEDGER.md)	Meaning
E0	design	the written spec
E1	T / V	deterministic tests and proofs: Python against the C++ twin, the tracking theorem
E2	S	simulation on a made plant (the realms, the fleet and GPU models)
E3	L	real software: Kubernetes on kind, no card (sets 22 and 23)
E4	P	a physical meter: the card's own power reading (the GPU bench; not yet run on a card)

A result does not climb a rung by itself. The four realms are plants; E1 to E4 are how hard the proof is on whichever plant is run.

What blocked the card

The hash did not stop the card. A Python–C++ mismatch can fail a test after a machine has started the job; it cannot stop GitHub from handing out a machine, because the runner is chosen before the code runs. The verify job started and failed (fixed since: it is green), and separately the GPU job never got a machine.

The organisation's and enterprise's settings show no GPU runner and no option to create one (checked 2026-10-02), so no budget could start it. The run therefore moves to a rented card (`scripts/gpu_rented_run.sh`, one command).

The bake-off that exists

Set 23 (`results/live/LIVE_REPS_23.md`, run 36940088922): real Kubernetes, ten pairs, fifteen minutes an arm, the same open-loop work, native against Omni, six kind workers, order rotated.

Gauge	Native	Omni	Change
Response, 95th percentile (ms)	327.3	123.8	−62.2%
Response, 99th percentile (ms)	503.4	169.3	−66.4%
Response, mean (ms)	146.6	80.1	−45.4%
Failed requests	0	0	0
HPA replicas, mean	8.51	5.40	−36.6%
Workers in service	6	4.28	−28.7%
Pods started	4.7	1.7	−63.8%
Pod start wait (s)	18.6	3.8	−79.6%
CPU, service plus Omni (cores)	0.903	0.894	−1.0% (no difference)
Energy, workers still on (Wh, declared model)	159.0	158.7	−0.2% (no difference)

The same requests were sent and none failed. They were answered faster, on fewer copies, at the same bill. Kind leaves every worker powered, so the watt line stays flat. Parking a worker and waking it again was tried; the churn ate the saving (`docs/CLAIMS_REGISTER.md`, C17), so that path is closed on this plant.

Arithmetic, not a result: if a run finishes 5% more work, nineteen runs do what twenty used to ($20 / 1.05 = 19.05$).

The chip

The card is the meter that can move the electric bill, and the meter that can move output.

Energy on the chip is the integral of power.draw. The same pile can be spent two ways:

- **Output:** more finished work on the same bill. The cap stays near where the native arm left it, the draw stays in the same band, and heat gets no new reason to rise.
- **The bill:** a lower enforced limit, the job still finishes, and joules fall.

A company short of cards spends it on output. A company paying for GPU hours with spare capacity spends it on the bill. A company against a building power cap splits it. The bench's primary outcome, work per joule, counts both, and its table prints requests and joules separately, so the receipt shows which way the pile went (`tools/gpu_reps.py`).

For the pile to be real:

- the cap has to bind;
- the write has to be enforced by the driver, and the enforced limit read back;
- the job has to finish;
- the limit has to be restored on exit, so the next native arm is not still capped.

All four are in the bench and its preregistration (`docs/GPU_PREREGISTRATION.md`, amendments 1–3).

The babysitting tax

The electric bill is the small pile. The babysitting tax is the people and tools kept on the clock to watch the levers, reset a cap, and stop the muscles fighting. A card's real cost is usually the card and the hours it sat idle or late, not the electrons.

What fills the seat today is a person: someone sets the cap, someone gets the page when it is left down, someone turns it back after the run or the crash. The product takes that knob and that page.

What we found nobody selling

We found no published product that does all of the following across compute, machines, facility and grid as one governor:

- reads all of their meters into one state;
- holds one clipped command through the step;
- puts each lever back when the run stops, and prints both receipts.

Borg, Twine and OpenShift govern compute. AWS is a cloud. Turbonomic covers the IT stack. Tesla runs the car, the battery and the factory as separate software. Each covers more than one thing as a company, not four realms as one governor.

Omni on such a stack does not replace it: Borg remains the muscle. This is a market observation from public material, not a claim tested here.

The bill

Run the stack native and print the receipt. Run the same stack with Omni and print the receipt. The difference is the pile, and the fee is 20 percent of it.

On a stack that costs \$100, a 10 percent win is \$10. The fee is \$2, and the customer keeps \$8. In general, the fee is $0.2 \times (\text{measured win}) \times (\text{spend})$. A gain and a cut are the same difference, read from opposite sides. Do not add CPU, GPU and babysitting percentages into one bill: the invoice is one pile.

Check: the 10 percent is an example, not a measured win. What is measured so far:

- on Kubernetes, response time -62% at the same energy (set 23);
- in simulation, +0.1% work per energy for the whole organism (realm round 3);
- on a card, nothing yet: the GPU bench is the first real-meter number.

Revenue figures should be computed from the measured win on each customer's own receipt. The per-company table in the working draft (2% of each company's published infrastructure spend) assumed a 10% win everywhere, and several of its spend figures do not match the companies' published capital expenditure. It is left out of the repository until both are sourced.

What to strengthen

1. **Wire the output map.** u becomes one lever, read back and restored, with the start value, the written value and the value after a kill printed on the receipt. The GPU bench already records the start limit, every write and its read-back, and the restored limit. A u -to-lever output map (a gain in watts or replicas per unit of u) is a new mechanism. It would be preregistered and tested as its own arm, first in the realms, then on a card, never mixed into the frozen confirmation run.
2. **Keep the step as it is:** clip before the stage, hold u through RK4, do not rewrite U (proved: docs/TRACKING_THEOREM.md).
3. **Run native against Omni on the same membership for each realm.** Compute is the one GitHub can host for real; the other three are modelled until their hosts exist.
4. **Aim the chip at about 5% more finished work on the same bill,** cap unchanged and temperature no worse, or at fewer joules for the same work. The buyer chooses, and the receipt prints which.

CHAPTER 58

History

Earlier states of play, kept whole. Nothing here is current status; the current state is `STATE_OF_PLAY.md`.

State of play as written on 27 September 2026 (with the update of 28 September)

Superseded for live results. The live evidence below predates sets 19 and 20. The current live table is `results/live/LIVE_PAIRED.md`; energy on kind is a declared model, not a meter. The first metered test is `scripts/gpu_paired.sh` (`docs/GPU_BENCH.md`).

Update, 28 September 2026. Added since this page was written, each with its evidence: - **Live, set 21** (conveyance only when response time needs it): p95 -37%, machines in service -21%, CPU used +26% (unchanged from set 20; its source is still open). `results/live/LIVE_REPS_21.md`. - **GPU governor bounded** (0.70 share floor, busy gate, 2 s decisions): on the MLPerf-calibrated modelled card, +5.1% and +1.3% work per kJ with p95 within +10% (the old governor failed that guardrail at +31% and +66%). Model only. `results/gpu/sim/`. - **Speed lock** (opt-in): speed won elsewhere spent on GPU watts, every gauge kept at least 1% faster than without Omni. Model only. `results/gpu/sim/pipeline/`. - **CPU and GPU on one conserved power budget** (conveyance law over CPU and GPU organs): +1.4% to +5.7% work served against today's fixed caps, never over the site budget. Model only. `docs/CONVEYANCE_LAW.md`, `results/hardware/NODE_EXCHANGE_*.json`. - **The manual in the box:** `docs/INTEGRATION_MANUAL.md`; every gauge: `docs/METRICS_CATALOG.md`; against what runs today: `docs/COMPARISON.md`.

This is the whole repository at the commit named in `STATE_OF_PLAY_COMMIT.txt`. Everything below can be rerun from it.

Run it live on real Kubernetes yourself: on any machine with Docker, kind, kubectl and Python, run `bash RUN_LIVE.sh 3`. It builds a fresh 7-node cluster per arm, runs native, Omni on top and Omni alone, and writes `live_runs/LIVE_REPS.md`. GitHub Actions runs the same comparison with a pushed commit whose message contains [reps].

Rerun the whole thing: `pip install -r requirements.txt && python verify.py`. It must end VERIFICATION: PASS, and it does at this commit.

Best measured results, and where each one comes from

Claim	Evidence	Where
B (Omni on top of each platform): 0 losing cells on held-out scenarios; strictly better on 27 of 28 platform-workload pairs	simulation, pre-registered	tuning/B_FIX.json, tuning/B_TONE_HELDOUT.json
Four-cluster sites: B 0 losses on all 7 platforms; C 0 losses vs GKE, AKS/Karpenter, CAST AI, Spot	simulation, held-out	tuning/SITE_LEAGUE.json
C (Omni alone), one global setting, 400 never-used scenarios: 153 better / 174 equal / 37 worse of 364 cells (Holm-corrected)	simulation, confirmatory	tuning/CONFIRMATORY.json
C with nervous-system coordination, 400 fresh scenarios: 151 / 173 / 40	simulation, confirmatory	tuning/CONFIRMATORY_COORD.json
Controllers fighting each other: lowest of all 8 systems on web (0.07 per day) and four-cluster sites (0.56 per day)	simulation, confirmatory	same file
Real demand, 1,052 recorded PlanetLab machines: 0 losing cells of 91	simulation on real traces	tuning/PLANETLAB_LEAGUE.json
Faults (machines dying, spikes, crash loops, noisy neighbours): fewest pages to a human, fastest recovery	simulation, 500 runs per workload	results/protocol/
GPU power law (engine cap over the MLPerf-measured performance law), against native at equal work: energy -27% to -33%, time over the heat limit -73% to -78%	simulation, held-out	results/hardware/SUMMARY_HELDOUT.json
Site power exchange (conveyance law, manuscript Ch. 29-31): 4 GPU groups under one site budget: 0 minutes over the site limit at 70/60/50% budgets (native 10.5/45.6/106); least energy at every budget; 31-115 fewer backlog minutes than the static split	simulation, held-out; law proved (conservation, Lyapunov, exponential convergence)	docs/CONVEYANCE_LAW.md, results/hardware/SITE_EXCHANGE_HELDOUT_*.json
Live levers under the nervous system: 19 of 19 checks on real Kubernetes; kill switch restores everything	live, kind	results/live/LIVE_LEVERS_2_NERVOUS.txt
Shadow pilot kit: a read-only identity made 0 writes across 40 decisions	live, kind	results/live/LIVE_SHADOW_1.txt
Safety shield: 2,000,000 adversarial cases, 0 violations (the test found 2 real bugs, both fixed); C++ twin matches	test	tests/test_shield_properties.py
C++ engine: 100,000,000 decisions, no failures, about 2.3 µs per decision	test	results/SOAK.json

What still loses, measured

- **C, typical response time, web and four-cluster:** about 20% slower than every platform. Omni fills pods to 80%, where the others stop at 70%, and trades response time for energy.
- **C, machine starts and stops:** loses to plain Kubernetes and OpenShift, which hold machines steady and pay for it in machine-hours.
- **C, GPU and batch:** energy 1-2% worse and power and heat margins 1-4% worse than the tight packers (AKS/Karpenter, CAST AI, Spot).
- **Live, set 3, with a working probe:**

- Omni kept all 6 machines. Latency near the 500 ms SLO blocks machine release.
- There is no significant difference from native on anything else, except that C has more waiting pods.
- The earlier live machine savings (sets 1-2) are withdrawn, because the broken probe had blinded the latency sense (results/live/LIVE_REPS_PROBE_DEFECT.md).
- Set 4's decision trail showed why. Latency spikes from pod saturation at load steps kept the service from being clean, and the engine's calm stayed under the machine threshold (0.65 against 0.7). Both are addressed by the attribution gate and the two-way wiring above; the live test is pending (results/live/LIVE_REPS_4_DIAGNOSIS.md).
- **A bound, not a loss:** no controller, even one with perfect foresight, can match both the tightest packer's machine-hours and the calmest autoscaler's churn at once (tuning/bound.py).

Two-way nervous system (new, live test pending)

Manuscript Appendix J names delay and dropout handling and feedback interpretation as the nervous system's job. The live controller now wires both directions (docs/TWO_WAY_NERVOUS_SYSTEM.md).

- **Blind senses are detected against the wall clock.** A hung probe was the root cause of the live set 1-2 error.
 - A blind sense feeds the engine's stale channel.
 - It blocks every give-back.
- **Commands are read back.** Drift feeds the engine's drift_ratio. No new machine release goes out before the last one has landed.
- **The machine organ has its own engine view and a release gate.** Pods move first, and the headroom proof uses the engine's rho.

Tests: 75,000 blind states, 200,000 gate states, 300,000 nervous-system states.

Live sets 5 and 6 could not run. From 03:51 UTC GitHub refused every job, including verify: runner ID 0, no steps, failed within about 2 seconds. That is GitHub declining to start runners, which on a private repository is most often exhausted Actions minutes or a spending limit. It is not a code failure. python verify.py passes locally at this commit.

Map

Path	What it is
omnicompass/core.py, cpp/	six-state engine; C++ twin
omnicompass/closure.py	closure law: forward projection, boundary correction, turning point, muscle tone
omnicompass/nervous_system.py	one engine state grants each organ its authority and envelope
omnicompass/shield.py	safety shield, downstream of everything
omni_controller/	live Kubernetes controller and levers
fleet/, hardware/	simulation plants
tuning/	benchmarks, pre-registrations, results
results/live/	every live run, including the failed and withdrawn ones
docs/OMNICOMPASS_BUYER_EDITION.md	the buyer-facing report

Back Matter

BACK MATTER

Glossary

Term	Meaning
Antagonist pair	the up side and the down side of a muscle's control, each with its own gain
Authority	how far the nervous system lets an organ move this decision
Band	a reading's or a knob's safe range
Bowl	the band seen as a position from 0 (calm) to 1 (the line), with its bottom in the middle
Cover	a knob's hard range; every write is clipped to it
Cushion	the 5% at each edge of the bowl, 10% in all
Fail up	the return to full capacity the moment service crosses the wall or a sense goes blind
Governor	the process that reads, decides and writes; Omni-Compass
Muscle	any machine, service or controller Omni-Compass reads and sets
Native	the system as it runs without Omni-Compass
Organism	a set of muscles run together on one clock
Plug	the two-way connection to one muscle: read, write, read back, restore
Receipt	the paired record of native against Omni-Compass for one run
Snapshot	a knob's value read once before the first write; the restore point
Wire check	the test that proves every wire follows, reads back and returns before anything runs
Work per energy	work done divided by energy used; the primary outcome

APPENDIX

Appendix A. Command Reference

Task	Command	
Verify everything	<code>python3 verify.py</code>	
Six organisms, simulated	<code>python3 tools/run_scale.py --runs N --scale K --out <dir></code>	
The full ladder	<code>bash scripts/scale_ladder.sh</code>	
Two-wire card, modelled	<code>python3 tools/run_gpu_card.py <dir> [fresh]</code>	
GPU wire check	<code>sudo python3 tools/gpu_wire_check.py --gpu 0</code>	
GPU, whole benchmark in one command	<code>sudo nohup bash scripts/gpu_rented_run.sh > run.log 2>&1 &</code>	
The six organisms with the real card inside	<code>python3 tools/run_hil.py --out <dir></code>	
GPU governor, two wires	<code>`sudo python3 -m omni_controller.gpu_bowl --mode watch</code>	<code>cap ...`</code>
Kubernetes, watch	<code>kubectrl apply -f deploy/install/omni-compass.yaml</code>	
Kubernetes paired runs	<code>scripts/kind_paired.sh, then tools/live_reps.py</code>	
OFF (Kubernetes)	<code>touch /tmp/omni.kill</code>	
OFF (GPU)	<code>sudo touch /tmp/omni-gpu-kill</code>	
Seal check	<code>python3 tools/seal.py --check</code>	

APPENDIX

Appendix B. File Map

Path	Contents
omnicompass/	the engine, governor, nervous system, shield, compass, conveyance law, the bowl
omni_controller/	the Kubernetes controller and muscles; the GPU governors (one and two wires)
realms/	the 656-muscle catalog, the plant models, the realm harness, the bowl on every muscle, the modelled card
tools/	benchmarks, receipts, the wire check, the scale ladder, manifests, seals
scripts/	one-command runs (GPU, Kubernetes, ladder)
deploy/	container image, install and permission files
cpp/	the C++20 twins
results/	every published result, with its raw files and checksums
docs/	this manual, the preregistrations, the evidence ledger, the theorem, the realm study
LICENSE, NOTICE, LICENSES/	the license and notices

APPENDIX

Appendix C. The Equations in Full

```
(1) dE/dt      = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt      = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,    |u| <= U_AUTHORITY = 25
(3) dI_U/dt    = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff      = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S)     = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
(6) dS/dt      = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
(7) dB/dt      = B_dot ; dB_dot/dt = gamma_c delta S - (omega_B / Q_B) B_dot - omega_B^2 B
(8) R_B[n]     = finite-difference audit of (7); never fed back into the state
```

Control (one micro step, zero-order hold across all four RK4 stages):

```
u = clip(-f_U(x, t) + K_P (sigma - U), -U_AUTHORITY, +U_AUTHORITY), K_P = 12, sigma in {-1, +1}
```

The bowl (every muscle):

p = position of the service reading in its band (0 calm, 1 the line)

F = A tanh((K_P (p - center) + K_D v) / A), K_D >= critical damping

p >= 0.95 => full up force; down side held

knob <= clip(knob + g_side F span, cover)

Parameter ranges, defaults and the proof of convergence: omnicompass/core.py, docs/TRACKING_THEOREM.md, docs/CANONICAL_ENGINE.md.

APPENDIX

Appendix D. Metrics

Every gauge, where it comes from, and whether it is measured or modelled: docs/METRICS_CATALOG.md.

APPENDIX

Appendix E. Troubleshooting

Symptom	Cause	Fix
Wire check FAIL 3 up wire (lock)	the driver or VM refuses clock locking	use bare metal or full passthrough; or run the one-wire governor
Wire check FAIL 3 up wire (follows down)	the card ignores the ceiling	update the driver; check for another management agent holding clocks
Wire check FAIL 4 down wire	the power limit is refused or capped by the board	check <code>nvidia-smi -q -d POWER</code> ; run as root
another copy of this test is already running	a second copy was started	wait for the first or reboot; start once
something else is using the GPU	another process holds the card	stop it; the benchmark must run alone
Governor exit 5	another writer changed a knob	find the other controller; Omni-Compass left its value alone
Governor exit 3	a restore did not read back	restore by hand (<code>nvidia-smi -rgc, -pl <start></code>); investigate before rerunning
decision failed (n in a row)	the cluster API is unreachable	turn it OFF; native runs on

APPENDIX

Appendix F. Evidence Map

docs/EVIDENCE_LEDGER.md (every claim and its class), docs/CLAIMS_REGISTER.md (what is claimed and what is not), docs/GPU_PREREGISTRATION.md and docs/REALMS_PREREGISTRATION.md (the rules written before each run), STATE_OF_PLAY.md (where everything stands), HANDOFF.md (every command in one page).

APPENDIX

Appendix G. The 656 Muscles

Every muscle Omni-Compass governs in the six organisms, by number. Family is the kind of machine; realms are the realms the muscle belongs to (a muscle in more than one realm is counted once in the tower of 656 and once per realm in the stack of 1,226); template is the plant model that stands for it in simulation; knob is the kind of setting the governor moves. Source: `realms/catalog.csv`.

Realm	Muscles
Compute	345
Physics	262
Energy	282
Distribution	337
Stack (every duplicate kept)	1,226
Tower (every muscle once)	656

#	Family	Muscle	Realms	Template	Knob
1	Work Admission & Demand Shaping	api concurrency	Compute	compute pool	admission
2	Work Admission & Demand Shaping	queue concurrency	Compute	compute pool	admission
3	Work Admission & Demand Shaping	queue backpressure	Compute	compute pool	admission
4	Work Admission & Demand Shaping	job admission	Compute	compute pool	admission
5	Work Admission & Demand Shaping	batch admission	Compute	compute pool	admission
6	Work Admission & Demand Shaping	inference admission	Compute	compute pool	admission
7	Work Admission & Demand Shaping	load shed	Compute	compute pool	admission
8	Work Admission & Demand Shaping	priority gate	Compute	compute pool	admission
9	Work Admission & Demand Shaping	tenant admission	Compute	compute pool	admission
10	Work Admission & Demand Shaping	burst limit	Compute	compute pool	admission
11	Work Admission & Demand Shaping	deadline admission	Compute	compute pool	admission
12	Work Admission & Demand Shaping	work budget	Compute	compute pool	admission
13	Work Admission & Demand Shaping	request queue limit	Compute	compute pool	admission
14	Work Admission & Demand Shaping	retry admission	Compute	compute pool	admission
15	Work Admission & Demand Shaping	background work gate	Compute	compute pool	admission
16	Work Admission & Demand Shaping	maintenance work gate	Compute	compute pool	admission
17	Kubernetes Workload Scaling	replicas	Compute, Physics, Energy, Distribution	compute pool	capacity
18	Kubernetes Workload Scaling	hpa cpu target	Compute, Physics, Energy, Distribution	compute pool	setpoint
19	Kubernetes Workload Scaling	hpa memory target	Compute, Physics, Energy, Distribution	compute pool	setpoint
20	Kubernetes Workload Scaling	hpa custom target	Compute, Physics, Energy, Distribution	compute pool	setpoint
21	Kubernetes Workload Scaling	vpa apply	Compute, Physics, Energy, Distribution	compute pool	capacity
22	Kubernetes Workload Scaling	scale to zero	Compute, Physics, Energy, Distribution	compute pool	capacity
23	Kubernetes Workload Scaling	keda threshold	Compute, Physics, Energy, Distribution	compute pool	admission

#	Family	Muscle	Realms	Template	Knob
24	Kubernetes Workload Scaling	rollout rate	Compute, Physics, Energy, Distribution	compute pool	capacity
25	Kubernetes Workload Scaling	max surge	Compute, Physics, Energy, Distribution	compute pool	capacity
26	Kubernetes Workload Scaling	max unavailable	Compute, Physics, Energy, Distribution	compute pool	capacity
27	Kubernetes Workload Scaling	deployment pause resume	Compute, Physics, Energy, Distribution	compute pool	admission
28	Kubernetes Workload Scaling	rollout abort	Compute, Physics, Energy, Distribution	compute pool	admission
29	Kubernetes Workload Scaling	pod eviction	Compute, Physics, Energy, Distribution	compute pool	admission
30	Kubernetes Workload Scaling	pdb policy	Compute, Physics, Energy, Distribution	compute pool	capacity
31	Kubernetes Workload Scaling	scheduler queue priority	Compute, Physics, Energy, Distribution	compute pool	admission
32	Kubernetes Workload Scaling	api priority fairness	Compute, Physics, Energy, Distribution	compute pool	admission
33	Container Resources	cpu request	Compute, Physics, Energy, Distribution	compute pool	capacity
34	Container Resources	cpu limit	Compute, Physics, Energy, Distribution	compute pool	capacity
35	Container Resources	memory request	Compute, Physics, Energy, Distribution	compute pool	capacity
36	Container Resources	memory limit	Compute, Physics, Energy, Distribution	compute pool	capacity
37	Container Resources	ephemeral storage	Compute, Physics, Energy, Distribution	compute pool	capacity
38	Container Resources	hugepages	Compute, Physics, Energy, Distribution	compute pool	capacity
39	Container Resources	io weight	Compute, Physics, Energy, Distribution	compute pool	setpoint
40	Container Resources	pids limit	Compute, Physics, Energy, Distribution	compute pool	capacity
41	Container Resources	memory high	Compute, Physics, Energy, Distribution	compute pool	capacity
42	Container Resources	memory reclaim	Compute, Physics, Energy, Distribution	compute pool	capacity
43	Container Resources	swap limit	Compute, Physics, Energy, Distribution	compute pool	capacity
44	Container Resources	cgroup io max	Compute, Physics, Energy, Distribution	compute pool	capacity
45	Container Resources	cpu weight	Compute, Physics, Energy, Distribution	compute pool	setpoint
46	Container Resources	cpu quota	Compute, Physics, Energy, Distribution	compute pool	admission
47	Container Resources	cpuset	Compute, Physics, Energy, Distribution	compute pool	capacity
48	Container Resources	runtime class	Compute, Physics, Energy, Distribution	compute pool	capacity
49	Kubernetes Placement & Scheduling	node selector	Compute, Physics, Energy, Distribution	compute pool	capacity
50	Kubernetes Placement & Scheduling	node affinity	Compute, Physics, Energy, Distribution	compute pool	capacity
51	Kubernetes Placement & Scheduling	pod anti affinity	Compute, Physics, Energy, Distribution	compute pool	capacity
52	Kubernetes Placement & Scheduling	topology spread	Compute, Physics, Energy, Distribution	compute pool	capacity
53	Kubernetes Placement & Scheduling	numa placement	Compute, Physics, Energy, Distribution	compute pool	capacity
54	Kubernetes Placement & Scheduling	gpu topology	Compute, Physics, Energy, Distribution	compute pool	capacity
55	Kubernetes Placement & Scheduling	storage locality	Compute, Physics, Energy, Distribution	compute pool	capacity
56	Kubernetes Placement & Scheduling	network locality	Compute, Physics, Energy, Distribution	compute pool	capacity

#	Family	Muscle	Realms	Template	Knob
57	Kubernetes Placement & Scheduling	taint toleration	Compute, Physics, Energy, Distribution	compute pool	setpoint
58	Kubernetes Placement & Scheduling	priority class	Compute, Physics, Energy, Distribution	compute pool	admission
59	Kubernetes Placement & Scheduling	preemption policy	Compute, Physics, Energy, Distribution	compute pool	admission
60	Kubernetes Placement & Scheduling	device claim	Compute, Physics, Energy, Distribution	compute pool	capacity
61	Kubernetes Placement & Scheduling	failure domain spread	Compute, Physics, Energy, Distribution	compute pool	capacity
62	Kubernetes Placement & Scheduling	scheduler backoff	Compute, Physics, Energy, Distribution	compute pool	admission
63	Kubernetes Placement & Scheduling	gang admission	Compute, Physics, Energy, Distribution	compute pool	admission
64	Kubernetes Placement & Scheduling	deschedule	Compute, Physics, Energy, Distribution	compute pool	capacity
65	Node Fleet & Karpenter-Class Control	node desired	Compute, Physics, Energy, Distribution	compute pool	capacity
66	Node Fleet & Karpenter-Class Control	node pool min	Compute, Physics, Energy, Distribution	compute pool	capacity
67	Node Fleet & Karpenter-Class Control	node pool max	Compute, Physics, Energy, Distribution	compute pool	capacity
68	Node Fleet & Karpenter-Class Control	node provision	Compute, Physics, Energy, Distribution	compute pool	capacity
69	Node Fleet & Karpenter-Class Control	node cordon	Compute, Physics, Energy, Distribution	compute pool	admission
70	Node Fleet & Karpenter-Class Control	node drain	Compute, Physics, Energy, Distribution	compute pool	admission
71	Node Fleet & Karpenter-Class Control	node consolidate	Compute, Physics, Energy, Distribution	compute pool	capacity
72	Node Fleet & Karpenter-Class Control	node replace	Compute, Physics, Energy, Distribution	compute pool	capacity
73	Node Fleet & Karpenter-Class Control	node shutdown	Compute, Physics, Energy, Distribution	compute pool	capacity
74	Node Fleet & Karpenter-Class Control	node power on	Compute, Physics, Energy, Distribution	compute pool	capacity
75	Node Fleet & Karpenter-Class Control	nodepool weight	Compute, Physics, Energy, Distribution	compute pool	setpoint
76	Node Fleet & Karpenter-Class Control	disruption budget	Compute, Physics, Energy, Distribution	compute pool	admission
77	Node Fleet & Karpenter-Class Control	consolidation policy	Compute, Physics, Energy, Distribution	compute pool	capacity
78	Node Fleet & Karpenter-Class Control	consolidate after	Compute, Physics, Energy, Distribution	compute pool	capacity
79	Node Fleet & Karpenter-Class Control	expire after	Compute, Physics, Energy, Distribution	compute pool	capacity
80	Node Fleet & Karpenter-Class Control	capacity class	Compute, Physics, Energy, Distribution	compute pool	capacity
85	OpenShift & Machine API	machine remediation	Compute	compute pool	capacity
88	OpenShift & Machine API	machine health gate	Compute	compute pool	admission
89	OpenShift & Machine API	mcp pause	Compute	compute pool	admission
90	OpenShift & Machine API	mcp max unavailable	Compute	compute pool	capacity
91	OpenShift & Machine API	node config rollout	Compute	compute pool	capacity
92	OpenShift & Machine API	operator reconcile budget	Compute	compute pool	admission
93	OpenShift & Machine API	cluster version pacing	Compute	compute pool	capacity
94	OpenShift & Machine API	infra machine admission	Compute	compute pool	admission
95	OpenShift & Machine API	machine failure domain	Compute	compute pool	capacity
97	Cloud VM & Capacity	vm count	Compute, Physics, Energy, Distribution	compute pool	capacity
98	Cloud VM & Capacity	vm size	Compute, Physics, Energy, Distribution	compute pool	capacity
99	Cloud VM & Capacity	vm start stop	Compute, Physics, Energy, Distribution	compute pool	capacity
100	Cloud VM & Capacity	vm migrate	Compute, Physics, Energy, Distribution	compute pool	capacity

#	Family	Muscle	Realms	Template	Knob
101	Cloud VM & Capacity	instance family	Compute, Physics, Energy, Distribution	compute pool	capacity
102	Cloud VM & Capacity	asg min	Compute, Physics, Energy, Distribution	compute pool	capacity
103	Cloud VM & Capacity	asg max	Compute, Physics, Energy, Distribution	compute pool	capacity
104	Cloud VM & Capacity	spot mix	Compute, Physics, Energy, Distribution	compute pool	capacity
105	Cloud VM & Capacity	reservation mix	Compute, Physics, Energy, Distribution	compute pool	capacity
106	Cloud VM & Capacity	zone selection	Compute, Physics, Energy, Distribution	compute pool	capacity
107	Cloud VM & Capacity	region selection	Compute, Physics, Energy, Distribution	compute pool	capacity
108	Cloud VM & Capacity	architecture selection	Compute, Physics, Energy, Distribution	compute pool	capacity
109	Cloud VM & Capacity	accelerator selection	Compute, Physics, Energy, Distribution	compute pool	capacity
110	Cloud VM & Capacity	boot disk class	Compute, Physics, Energy, Distribution	compute pool	capacity
111	Cloud VM & Capacity	placement group	Compute, Physics, Energy, Distribution	compute pool	capacity
112	Cloud VM & Capacity	interruption response	Compute, Physics, Energy, Distribution	compute pool	capacity
113	Host CPU & Memory	cpufreq min	Compute, Physics, Energy, Distribution	compute pool	power
114	Host CPU & Memory	cpufreq max	Compute, Physics, Energy, Distribution	compute pool	power
115	Host CPU & Memory	rapl package power	Compute, Physics, Energy, Distribution	compute pool	power
116	Host CPU & Memory	uncore frequency	Compute, Physics, Energy, Distribution	compute pool	power
117	Host CPU & Memory	energy perf preference	Compute, Physics, Energy, Distribution	compute pool	power
118	Host CPU & Memory	cpu idle policy	Compute, Physics, Energy, Distribution	compute pool	power
119	Host CPU & Memory	memory bandwidth	Compute, Physics, Energy, Distribution	compute pool	capacity
120	Host CPU & Memory	numa balance	Compute, Physics, Energy, Distribution	compute pool	capacity
121	Host CPU & Memory	irq affinity	Compute, Physics, Energy, Distribution	compute pool	capacity
122	Host CPU & Memory	llc allocation	Compute, Physics, Energy, Distribution	compute pool	capacity
123	Host CPU & Memory	memory pressure gate	Compute, Physics, Energy, Distribution	compute pool	admission
124	Host CPU & Memory	page reclaim rate	Compute, Physics, Energy, Distribution	compute pool	capacity
125	Host CPU & Memory	transparent hugepages	Compute, Physics, Energy, Distribution	compute pool	capacity
126	Host CPU & Memory	core online offline	Compute, Physics, Energy, Distribution	compute pool	capacity
127	Host CPU & Memory	thermal throttle policy	Compute, Physics, Energy, Distribution	compute pool	power
128	Host CPU & Memory	host power profile	Compute, Physics, Energy, Distribution	compute pool	power
129	NVIDIA GPU Hardware	gpu allocate	Compute, Physics, Energy, Distribution	compute pool	capacity
130	NVIDIA GPU Hardware	gpu power limit	Compute, Physics, Energy, Distribution	compute pool	power
131	NVIDIA GPU Hardware	gpu sm clock	Compute, Physics, Energy, Distribution	compute pool	power
132	NVIDIA GPU Hardware	gpu memory clock	Compute, Physics, Energy, Distribution	compute pool	power
133	NVIDIA GPU Hardware	gpu persistence mode	Compute, Physics, Energy, Distribution	compute pool	capacity

#	Family	Muscle	Realms	Template	Knob
134	NVIDIA GPU Hardware	gpu compute mode	Compute, Physics, Energy, Distribution	compute pool	capacity
135	NVIDIA GPU Hardware	mig mode	Compute, Physics, Energy, Distribution	compute pool	capacity
136	NVIDIA GPU Hardware	mig geometry	Compute, Physics, Energy, Distribution	compute pool	capacity
137	NVIDIA GPU Hardware	gpu timeslice	Compute, Physics, Energy, Distribution	compute pool	capacity
138	NVIDIA GPU Hardware	gpu mps	Compute, Physics, Energy, Distribution	compute pool	capacity
139	NVIDIA GPU Hardware	gpu quarantine	Compute, Physics, Energy, Distribution	compute pool	admission
140	NVIDIA GPU Hardware	gpu reset	Compute, Physics, Energy, Distribution	compute pool	capacity
141	NVIDIA GPU Hardware	gpu thermal limit	Compute, Physics, Energy, Distribution	compute pool	power
142	NVIDIA GPU Hardware	gpu ecc response	Compute, Physics, Energy, Distribution	compute pool	capacity
143	NVIDIA GPU Hardware	gpu job power budget	Compute, Physics, Energy, Distribution	compute pool	admission
145	GPU Fabric & RDMA	nvlink placement	Compute	compute pool	capacity
146	GPU Fabric & RDMA	nvswitch route	Compute	compute pool	capacity
147	GPU Fabric & RDMA	gpu fabric quarantine	Compute	compute pool	admission
148	GPU Fabric & RDMA	rdma bandwidth	Compute	compute pool	capacity
149	GPU Fabric & RDMA	rdma route	Compute	compute pool	capacity
150	GPU Fabric & RDMA	nic affinity	Compute	compute pool	capacity
151	GPU Fabric & RDMA	collective concurrency	Compute	compute pool	admission
152	GPU Fabric & RDMA	collective algorithm	Compute	compute pool	capacity
153	GPU Fabric & RDMA	collective chunk size	Compute	compute pool	capacity
154	GPU Fabric & RDMA	rank placement	Compute	compute pool	capacity
155	GPU Fabric & RDMA	gpudirect policy	Compute	compute pool	capacity
156	GPU Fabric & RDMA	congestion response	Compute	compute pool	capacity
157	GPU Fabric & RDMA	rail selection	Compute	compute pool	capacity
158	GPU Fabric & RDMA	fabric failover	Compute	compute pool	capacity
159	GPU Fabric & RDMA	communication priority	Compute	compute pool	admission
160	GPU Fabric & RDMA	fabric isolation	Compute	compute pool	admission
161	AI Inference Serving	model replicas	Compute	compute pool	capacity
162	AI Inference Serving	model route weight	Compute	compute pool	setpoint
163	AI Inference Serving	model load	Compute	compute pool	capacity
164	AI Inference Serving	model unload	Compute	compute pool	capacity
165	AI Inference Serving	model instance count	Compute	compute pool	capacity
166	AI Inference Serving	continuous batching	Compute	compute pool	capacity
167	AI Inference Serving	max batch size	Compute	compute pool	capacity
168	AI Inference Serving	batch queue delay	Compute	compute pool	capacity
169	AI Inference Serving	inference concurrency	Compute	compute pool	admission
170	AI Inference Serving	inference max tokens	Compute	compute pool	capacity
171	AI Inference Serving	kv cache budget	Compute	compute pool	admission
172	AI Inference Serving	prefix cache budget	Compute	compute pool	admission
173	AI Inference Serving	speculative decode budget	Compute	compute pool	admission
174	AI Inference Serving	model precision	Compute	compute pool	power
175	AI Inference Serving	inference priority	Compute	compute pool	admission
176	AI Inference Serving	inference slo gate	Compute	compute pool	admission
177	AI Training	training workers	Compute	compute pool	capacity
178	AI Training	global batch size	Compute	compute pool	capacity
179	AI Training	microbatch size	Compute	compute pool	capacity
180	AI Training	gradient accumulation	Compute	compute pool	capacity
181	AI Training	data parallelism	Compute	compute pool	capacity
182	AI Training	tensor parallelism	Compute	compute pool	capacity
183	AI Training	pipeline parallelism	Compute	compute pool	capacity
184	AI Training	expert parallelism	Compute	compute pool	capacity
185	AI Training	checkpoint interval	Compute	compute pool	setpoint
186	AI Training	checkpoint trigger	Compute	compute pool	capacity

#	Family	Muscle	Realms	Template	Knob
187	AI Training	training preempt	Compute	compute pool	admission
188	AI Training	training gang size	Compute	compute pool	capacity
189	AI Training	elastic worker count	Compute	compute pool	capacity
190	AI Training	straggler mitigation	Compute	compute pool	capacity
191	AI Training	training precision	Compute	compute pool	power
192	AI Training	compute comm overlap	Compute	compute pool	capacity
193	HPC & Distributed Compute	job slots	Compute	compute pool	capacity
194	HPC & Distributed Compute	mpi ranks	Compute	compute pool	capacity
195	HPC & Distributed Compute	rank mapping	Compute	compute pool	capacity
196	HPC & Distributed Compute	node allocation	Compute	compute pool	capacity
197	HPC & Distributed Compute	job walltime	Compute	compute pool	capacity
198	HPC & Distributed Compute	job priority	Compute	compute pool	admission
199	HPC & Distributed Compute	job preemption	Compute	compute pool	admission
200	HPC & Distributed Compute	checkpoint restart	Compute	compute pool	capacity
201	HPC & Distributed Compute	parallel io budget	Compute	compute pool	admission
202	HPC & Distributed Compute	collective budget	Compute	compute pool	admission
203	HPC & Distributed Compute	accelerator share	Compute	compute pool	capacity
204	HPC & Distributed Compute	cpu gpu ratio	Compute	compute pool	setpoint
205	HPC & Distributed Compute	memory per rank	Compute	compute pool	capacity
206	HPC & Distributed Compute	scratch budget	Compute	compute pool	admission
207	HPC & Distributed Compute	scheduler fair share	Compute	compute pool	admission
208	HPC & Distributed Compute	backfill policy	Compute	compute pool	capacity
209	Distributed Cluster Managers	task admission	Compute	compute pool	admission
210	Distributed Cluster Managers	task priority	Compute	compute pool	admission
211	Distributed Cluster Managers	resource reservation	Compute	compute pool	capacity
212	Distributed Cluster Managers	cluster quota	Compute	compute pool	admission
213	Distributed Cluster Managers	task preemption	Compute	compute pool	admission
214	Distributed Cluster Managers	binpack pressure	Compute	compute pool	capacity
215	Distributed Cluster Managers	spread pressure	Compute	compute pool	capacity
216	Distributed Cluster Managers	gang schedule	Compute	compute pool	capacity
217	Distributed Cluster Managers	worker allocation	Compute	compute pool	capacity
218	Distributed Cluster Managers	maintenance evacuation	Compute	compute pool	capacity
219	Distributed Cluster Managers	oversubscription	Compute	compute pool	capacity
220	Distributed Cluster Managers	resource reclaim	Compute	compute pool	capacity
221	Distributed Cluster Managers	queue fairness	Compute	compute pool	admission
222	Distributed Cluster Managers	deadline pressure	Compute	compute pool	admission
223	Distributed Cluster Managers	scheduler retry	Compute	compute pool	admission
225	Network Routing & Switching	lb weight	Compute, Physics, Energy, Distribution	compute pool	setpoint
226	Network Routing & Switching	route weight	Compute, Physics, Energy, Distribution	compute pool	setpoint
227	Network Routing & Switching	rate limit	Compute, Physics, Energy, Distribution	compute pool	admission
228	Network Routing & Switching	bandwidth limit	Compute, Physics, Energy, Distribution	compute pool	capacity
229	Network Routing & Switching	qos class	Compute, Physics, Energy, Distribution	compute pool	capacity
230	Network Routing & Switching	connection limit	Compute, Physics, Energy, Distribution	compute pool	admission
231	Network Routing & Switching	failover route	Compute, Physics, Energy, Distribution	compute pool	capacity
232	Network Routing & Switching	nic rate limit	Compute, Physics, Energy, Distribution	compute pool	admission
233	Network Routing & Switching	nic queue count	Compute, Physics, Energy, Distribution	compute pool	capacity
234	Network Routing & Switching	queue discipline	Compute, Physics, Energy, Distribution	compute pool	capacity
235	Network Routing & Switching	congestion control	Compute, Physics, Energy, Distribution	compute pool	capacity
236	Network Routing & Switching	egress budget	Compute, Physics, Energy, Distribution	compute pool	admission

#	Family	Muscle	Realms	Template	Knob
237	Network Routing & Switching	ingress budget	Compute, Physics, Energy, Distribution	compute pool	admission
238	Network Routing & Switching	ecmp weight	Compute, Physics, Energy, Distribution	compute pool	setpoint
239	Network Routing & Switching	path selection	Compute, Physics, Energy, Distribution	compute pool	capacity
240	Network Routing & Switching	network isolation	Compute, Physics, Energy, Distribution	compute pool	admission
241	Service Mesh & API Reliability	circuit breaker	Distribution	compute pool	admission
242	Service Mesh & API Reliability	retry budget	Distribution	compute pool	admission
243	Service Mesh & API Reliability	service timeout	Distribution	compute pool	admission
244	Service Mesh & API Reliability	service concurrency	Distribution	compute pool	admission
245	Service Mesh & API Reliability	connection pool	Distribution	compute pool	capacity
246	Service Mesh & API Reliability	traffic divert	Distribution	compute pool	capacity
247	Service Mesh & API Reliability	traffic mirror	Distribution	compute pool	capacity
248	Service Mesh & API Reliability	canary weight	Distribution	compute pool	setpoint
249	Service Mesh & API Reliability	outlier ejection	Distribution	compute pool	admission
250	Service Mesh & API Reliability	health threshold	Distribution	compute pool	admission
251	Service Mesh & API Reliability	dns traffic weight	Distribution	compute pool	setpoint
252	Service Mesh & API Reliability	session affinity	Distribution	compute pool	capacity
253	Service Mesh & API Reliability	request hedging	Distribution	compute pool	capacity
254	Service Mesh & API Reliability	fault injection gate	Distribution	compute pool	admission
255	Service Mesh & API Reliability	service failover	Distribution	compute pool	capacity
257	Storage Block/File/Object	volume size	Compute, Physics, Energy, Distribution	compute pool	capacity
258	Storage Block/File/Object	iops limit	Compute, Physics, Energy, Distribution	compute pool	capacity
259	Storage Block/File/Object	throughput limit	Compute, Physics, Energy, Distribution	compute pool	capacity
260	Storage Block/File/Object	replica count	Compute, Physics, Energy, Distribution	compute pool	capacity
261	Storage Block/File/Object	storage tier	Compute, Physics, Energy, Distribution	compute pool	capacity
262	Storage Block/File/Object	volume placement	Compute, Physics, Energy, Distribution	compute pool	capacity
263	Storage Block/File/Object	snapshot trigger	Compute, Physics, Energy, Distribution	compute pool	capacity
264	Storage Block/File/Object	rebalance	Compute, Physics, Energy, Distribution	compute pool	capacity
265	Storage Block/File/Object	recovery rate	Compute, Physics, Energy, Distribution	compute pool	capacity
266	Storage Block/File/Object	backfill rate	Compute, Physics, Energy, Distribution	compute pool	capacity
267	Storage Block/File/Object	compaction pressure	Compute, Physics, Energy, Distribution	compute pool	capacity
268	Storage Block/File/Object	cache allocation	Compute, Physics, Energy, Distribution	compute pool	capacity
269	Storage Block/File/Object	object replication	Compute, Physics, Energy, Distribution	compute pool	capacity
270	Storage Block/File/Object	erasure code profile	Compute, Physics, Energy, Distribution	compute pool	capacity
271	Storage Block/File/Object	storage admission	Compute, Physics, Energy, Distribution	compute pool	admission
272	Storage Block/File/Object	degraded storage gate	Compute, Physics, Energy, Distribution	compute pool	admission
273	Database & Transactions	db replicas	Distribution	compute pool	capacity
275	Database & Transactions	db memory	Distribution	compute pool	capacity
276	Database & Transactions	db cache	Distribution	compute pool	capacity
277	Database & Transactions	query concurrency	Distribution	compute pool	admission
278	Database & Transactions	read route	Distribution	compute pool	capacity
279	Database & Transactions	shard placement	Distribution	compute pool	capacity
280	Database & Transactions	db failover	Distribution	compute pool	capacity
281	Database & Transactions	replication lag gate	Distribution	compute pool	admission
282	Database & Transactions	db write throttle	Distribution	compute pool	admission

#	Family	Muscle	Realms	Template	Knob
283	Database & Transactions	db pool resize	Distribution	compute pool	capacity
284	Database & Transactions	transaction concurrency	Distribution	compute pool	admission
285	Database & Transactions	lock timeout	Distribution	compute pool	admission
286	Database & Transactions	checkpoint rate	Distribution	compute pool	capacity
287	Database & Transactions	vacuum compaction rate	Distribution	compute pool	capacity
289	Messaging & Streaming	partition count	Distribution	compute pool	capacity
290	Messaging & Streaming	partition placement	Distribution	compute pool	capacity
291	Messaging & Streaming	producer quota	Distribution	compute pool	admission
292	Messaging & Streaming	consumer quota	Distribution	compute pool	admission
293	Messaging & Streaming	broker io quota	Distribution	compute pool	admission
294	Messaging & Streaming	message retention	Distribution	compute pool	capacity
295	Messaging & Streaming	queue depth limit	Distribution	compute pool	admission
296	Messaging & Streaming	consumer concurrency	Distribution	compute pool	admission
297	Messaging & Streaming	producer batch size	Distribution	compute pool	capacity
298	Messaging & Streaming	fetch batch size	Distribution	compute pool	capacity
299	Messaging & Streaming	rebalance rate	Distribution	compute pool	capacity
300	Messaging & Streaming	replication factor	Distribution	compute pool	capacity
301	Messaging & Streaming	retry backoff	Distribution	compute pool	admission
302	Messaging & Streaming	dead letter divert	Distribution	compute pool	capacity
303	Messaging & Streaming	stream priority	Distribution	compute pool	admission
304	Messaging & Streaming	broker failover	Distribution	compute pool	capacity
305	Cache & Memory Services	cache size	Distribution	compute pool	capacity
306	Cache & Memory Services	cache ttl	Distribution	compute pool	setpoint
307	Cache & Memory Services	cache eviction policy	Distribution	compute pool	admission
308	Cache & Memory Services	cache replicas	Distribution	compute pool	capacity
309	Cache & Memory Services	cache sharding	Distribution	compute pool	capacity
310	Cache & Memory Services	cache prefetch	Distribution	compute pool	capacity
311	Cache & Memory Services	cache writeback rate	Distribution	compute pool	capacity
312	Cache & Memory Services	cache admission	Distribution	compute pool	admission
313	Cache & Memory Services	hot key isolation	Distribution	compute pool	admission
314	Cache & Memory Services	cache connection limit	Distribution	compute pool	admission
315	Cache & Memory Services	cache memory limit	Distribution	compute pool	capacity
316	Cache & Memory Services	cache compression	Distribution	compute pool	capacity
317	Cache & Memory Services	cache warmup	Distribution	compute pool	capacity
318	Cache & Memory Services	cache failover	Distribution	compute pool	capacity
319	Cache & Memory Services	cache flush rate	Distribution	compute pool	capacity
321	Search, Indexing & Vector DB	index workers	Distribution	compute pool	capacity
322	Search, Indexing & Vector DB	index refresh rate	Distribution	compute pool	capacity
323	Search, Indexing & Vector DB	segment merge rate	Distribution	compute pool	capacity
324	Search, Indexing & Vector DB	search concurrency	Distribution	compute pool	admission
325	Search, Indexing & Vector DB	search timeout	Distribution	compute pool	admission
326	Search, Indexing & Vector DB	shard count	Distribution	compute pool	capacity
327	Search, Indexing & Vector DB	shard replication	Distribution	compute pool	capacity
328	Search, Indexing & Vector DB	shard rebalance	Distribution	compute pool	capacity
329	Search, Indexing & Vector DB	vector search k	Distribution	compute pool	capacity
330	Search, Indexing & Vector DB	vector batch size	Distribution	compute pool	capacity
331	Search, Indexing & Vector DB	embedding workers	Distribution	compute pool	capacity
332	Search, Indexing & Vector DB	index memory budget	Distribution	compute pool	admission
333	Search, Indexing & Vector DB	query route	Distribution	compute pool	capacity
334	Search, Indexing & Vector DB	hot shard isolation	Distribution	compute pool	admission
335	Search, Indexing & Vector DB	search admission	Distribution	compute pool	admission
338	Data Analytics & ETL	executor size	Distribution	compute pool	capacity
339	Data Analytics & ETL	dynamic allocation min	Distribution	compute pool	capacity
340	Data Analytics & ETL	dynamic allocation max	Distribution	compute pool	capacity
341	Data Analytics & ETL	shuffle partitions	Distribution	compute pool	capacity
342	Data Analytics & ETL	shuffle bandwidth	Distribution	compute pool	capacity
343	Data Analytics & ETL	etl concurrency	Distribution	compute pool	admission
344	Data Analytics & ETL	stage parallelism	Distribution	compute pool	capacity
345	Data Analytics & ETL	query slots	Distribution	compute pool	capacity

#	Family	Muscle	Realms	Template	Knob
346	Data Analytics & ETL	spill threshold	Distribution	compute pool	admission
347	Data Analytics & ETL	cache fraction	Distribution	compute pool	setpoint
348	Data Analytics & ETL	batch interval	Distribution	compute pool	setpoint
349	Data Analytics & ETL	stream backpressure	Distribution	compute pool	admission
350	Data Analytics & ETL	data locality wait	Distribution	compute pool	capacity
351	Data Analytics & ETL	speculation policy	Distribution	compute pool	capacity
352	Data Analytics & ETL	analytics admission	Distribution	compute pool	admission
353	Runtime & Application	worker count	Distribution	compute pool	capacity
354	Runtime & Application	thread pool	Distribution	compute pool	capacity
355	Runtime & Application	jvm heap	Distribution	compute pool	capacity
356	Runtime & Application	gc budget	Distribution	compute pool	admission
357	Runtime & Application	connection pool runtime	Distribution	compute pool	capacity
358	Runtime & Application	application cache size	Distribution	compute pool	capacity
359	Runtime & Application	runtime memory	Distribution	compute pool	capacity
360	Runtime & Application	async concurrency	Distribution	compute pool	admission
361	Runtime & Application	event loop workers	Distribution	compute pool	capacity
362	Runtime & Application	process count	Distribution	compute pool	capacity
363	Runtime & Application	request timeout	Distribution	compute pool	admission
364	Runtime & Application	background workers	Distribution	compute pool	capacity
365	Runtime & Application	runtime cpu budget	Distribution	compute pool	admission
366	Runtime & Application	runtime io budget	Distribution	compute pool	admission
367	Runtime & Application	runtime restart	Distribution	compute pool	capacity
373	Observability & Telemetry	collector memory limit	Compute, Physics, Energy, Distribution	compute pool	capacity
374	Observability & Telemetry	export concurrency	Compute, Physics, Energy, Distribution	compute pool	admission
376	Observability & Telemetry	cardinality budget	Compute, Physics, Energy, Distribution	compute pool	admission
377	Observability & Telemetry	retention window	Compute, Physics, Energy, Distribution	compute pool	setpoint
378	Observability & Telemetry	remote write queue	Compute, Physics, Energy, Distribution	compute pool	capacity
379	Observability & Telemetry	telemetry shed	Compute, Physics, Energy, Distribution	compute pool	admission
385	Reliability, Security & Recovery	restart	Compute, Physics, Energy, Distribution	compute pool	capacity
386	Reliability, Security & Recovery	rollback	Compute, Physics, Energy, Distribution	compute pool	capacity
388	Reliability, Security & Recovery	traffic divert recovery	Compute, Physics, Energy, Distribution	compute pool	capacity
389	Reliability, Security & Recovery	degraded mode	Compute, Physics, Energy, Distribution	compute pool	capacity
390	Reliability, Security & Recovery	actuator freeze	Compute, Physics, Energy, Distribution	compute pool	admission
391	Reliability, Security & Recovery	workload isolate	Compute, Physics, Energy, Distribution	compute pool	admission
393	Reliability, Security & Recovery	credential rotation gate	Compute, Physics, Energy, Distribution	compute pool	admission
394	Reliability, Security & Recovery	policy enforcement	Compute, Physics, Energy, Distribution	compute pool	capacity
395	Reliability, Security & Recovery	rate abuse gate	Compute, Physics, Energy, Distribution	compute pool	admission
396	Reliability, Security & Recovery	fault domain isolate	Compute, Physics, Energy, Distribution	compute pool	admission
397	Reliability, Security & Recovery	backup trigger	Compute, Physics, Energy, Distribution	compute pool	capacity
400	Reliability, Security & Recovery	kill switch	Compute, Physics, Energy, Distribution	compute pool	admission
401	Commerce & Payment Systems	payment admission	Distribution	compute pool	admission
402	Commerce & Payment Systems	payment concurrency	Distribution	compute pool	admission
403	Commerce & Payment Systems	payment retry	Distribution	compute pool	admission
404	Commerce & Payment Systems	payment timeout	Distribution	compute pool	admission
405	Commerce & Payment Systems	fraud review gate	Distribution	compute pool	admission

#	Family	Muscle	Realms	Template	Knob
406	Commerce & Payment Systems	authorization route	Distribution	compute pool	capacity
407	Commerce & Payment Systems	processor route weight	Distribution	compute pool	setpoint
408	Commerce & Payment Systems	transaction queue limit	Distribution	compute pool	admission
409	Commerce & Payment Systems	idempotency window	Distribution	compute pool	setpoint
410	Commerce & Payment Systems	order reservation	Distribution	compute pool	capacity
411	Commerce & Payment Systems	inventory hold	Distribution	compute pool	admission
412	Commerce & Payment Systems	checkout load shed	Distribution	compute pool	admission
413	Commerce & Payment Systems	refund queue rate	Distribution	compute pool	capacity
414	Commerce & Payment Systems	settlement batch	Distribution	compute pool	capacity
415	Commerce & Payment Systems	payment failover	Distribution	compute pool	capacity
417	Workflow, Logistics & Fulfillment	workflow admission	Distribution	compute pool	admission
418	Workflow, Logistics & Fulfillment	workflow worker rate	Distribution	compute pool	capacity
419	Workflow, Logistics & Fulfillment	task queue rate	Distribution	compute pool	capacity
420	Workflow, Logistics & Fulfillment	workflow retry	Distribution	compute pool	admission
421	Workflow, Logistics & Fulfillment	workflow backoff	Distribution	compute pool	admission
422	Workflow, Logistics & Fulfillment	workflow timeout	Distribution	compute pool	admission
423	Workflow, Logistics & Fulfillment	inventory allocation	Distribution	compute pool	capacity
424	Workflow, Logistics & Fulfillment	fulfillment route	Distribution	compute pool	capacity
425	Workflow, Logistics & Fulfillment	warehouse queue	Distribution	compute pool	capacity
426	Workflow, Logistics & Fulfillment	carrier selection	Distribution	compute pool	capacity
427	Workflow, Logistics & Fulfillment	dispatch priority	Distribution	compute pool	admission
428	Workflow, Logistics & Fulfillment	shipment batch	Distribution	compute pool	capacity
429	Workflow, Logistics & Fulfillment	route replan	Distribution	compute pool	capacity
430	Workflow, Logistics & Fulfillment	sla escalation	Distribution	compute pool	capacity
431	Workflow, Logistics & Fulfillment	compensation action	Distribution	compute pool	capacity
433	Robotics Motion Control	joint position	Physics	motion axis	capacity
434	Robotics Motion Control	joint velocity	Physics	motion axis	capacity
435	Robotics Motion Control	joint acceleration	Physics	motion axis	capacity
436	Robotics Motion Control	joint effort	Physics	motion axis	power
437	Robotics Motion Control	cartesian velocity	Physics	motion axis	capacity
438	Robotics Motion Control	trajectory speed	Physics	motion axis	capacity
439	Robotics Motion Control	trajectory acceleration	Physics	motion axis	capacity
440	Robotics Motion Control	jerk limit	Physics	motion axis	capacity
441	Robotics Motion Control	collision margin	Physics	motion axis	capacity
442	Robotics Motion Control	force limit	Physics	motion axis	power
443	Robotics Motion Control	gripper force	Physics	motion axis	power
444	Robotics Motion Control	locomotion speed	Physics	motion axis	capacity
445	Robotics Motion Control	steering angle	Physics	motion axis	capacity
446	Robotics Motion Control	braking force	Physics	motion axis	power
447	Robotics Motion Control	balance correction	Physics	motion axis	capacity
449	Robotics Fleet & Warehouse Automation	robot dispatch	Physics	compute pool	capacity
450	Robotics Fleet & Warehouse Automation	task assignment	Physics	compute pool	capacity
451	Robotics Fleet & Warehouse Automation	traffic reservation	Physics	compute pool	capacity
452	Robotics Fleet & Warehouse Automation	robot route	Physics	compute pool	capacity
453	Robotics Fleet & Warehouse Automation	charging dispatch	Physics	compute pool	capacity
454	Robotics Fleet & Warehouse Automation	battery reserve	Physics	compute pool	capacity
455	Robotics Fleet & Warehouse Automation	elevator request	Physics	compute pool	capacity
456	Robotics Fleet & Warehouse Automation	door request	Physics	compute pool	capacity
457	Robotics Fleet & Warehouse Automation	conveyor speed	Physics	compute pool	capacity
458	Robotics Fleet & Warehouse Automation	agv speed	Physics	compute pool	capacity
459	Robotics Fleet & Warehouse Automation	warehouse zone admission	Physics	compute pool	admission
460	Robotics Fleet & Warehouse Automation	robot quarantine	Physics	compute pool	admission
461	Robotics Fleet & Warehouse Automation	fleet failover	Physics	compute pool	capacity
462	Robotics Fleet & Warehouse Automation	human safe stop	Physics	compute pool	capacity
463	Robotics Fleet & Warehouse Automation	fleet concurrency	Physics	compute pool	admission
465	Industrial PLC & Process Automation	plc cycle authority	Energy	process loop	admission
466	Industrial PLC & Process Automation	machine cell admission	Energy	process loop	admission
467	Industrial PLC & Process Automation	valve position	Energy	process loop	setpoint
468	Industrial PLC & Process Automation	pump flow	Energy	process loop	capacity

#	Family	Muscle	Realms	Template	Knob
469	Industrial PLC & Process Automation	compressor speed	Energy	process loop	capacity
470	Industrial PLC & Process Automation	heater power	Energy	process loop	power
471	Industrial PLC & Process Automation	furnace setpoint	Energy	process loop	setpoint
472	Industrial PLC & Process Automation	pressure setpoint	Energy	process loop	setpoint
473	Industrial PLC & Process Automation	temperature setpoint	Energy	process loop	setpoint
474	Industrial PLC & Process Automation	mass flow setpoint	Energy	process loop	setpoint
475	Industrial PLC & Process Automation	tank level target	Energy	process loop	setpoint
476	Industrial PLC & Process Automation	conveyor rate	Energy	process loop	capacity
477	Industrial PLC & Process Automation	feed rate	Energy	process loop	capacity
478	Industrial PLC & Process Automation	purge vent action	Energy	process loop	admission
481	Aviation & Autonomous Flight	throttle envelope	Physics	motion axis	power
482	Aviation & Autonomous Flight	attitude target	Physics	motion axis	capacity
483	Aviation & Autonomous Flight	attitude rate target	Physics	motion axis	capacity
484	Aviation & Autonomous Flight	velocity target	Physics	motion axis	capacity
485	Aviation & Autonomous Flight	altitude target	Physics	motion axis	capacity
486	Aviation & Autonomous Flight	waypoint authority	Physics	motion axis	capacity
487	Aviation & Autonomous Flight	flight hold	Physics	motion axis	admission
488	Aviation & Autonomous Flight	return to home	Physics	motion axis	admission
489	Aviation & Autonomous Flight	land action	Physics	motion axis	admission
490	Aviation & Autonomous Flight	mission admission	Physics	motion axis	admission
491	Aviation & Autonomous Flight	geofence response	Physics	motion axis	admission
492	Aviation & Autonomous Flight	failsafe selection	Physics	motion axis	admission
493	Aviation & Autonomous Flight	battery reserve threshold	Physics	motion axis	admission
494	Aviation & Autonomous Flight	actuator saturation envelope	Physics	motion axis	capacity
495	Aviation & Autonomous Flight	flight mode transition	Physics	motion axis	admission
496	Aviation & Autonomous Flight	flight termination safe state	Physics	motion axis	admission
497	Spacecraft & Flight Software	space command admission	Physics	motion axis	admission
499	Spacecraft & Flight Software	flight task schedule	Physics	motion axis	admission
500	Spacecraft & Flight Software	space mode transition	Physics	motion axis	admission
501	Spacecraft & Flight Software	payload duty cycle	Physics	motion axis	power
502	Spacecraft & Flight Software	communication allocation	Physics	motion axis	capacity
503	Spacecraft & Flight Software	space power budget	Physics	motion axis	power
504	Spacecraft & Flight Software	space thermal command	Physics	motion axis	power
505	Spacecraft & Flight Software	attitude command envelope	Physics	motion axis	capacity
506	Spacecraft & Flight Software	reaction wheel allocation	Physics	motion axis	capacity
507	Spacecraft & Flight Software	rcs authority	Physics	motion axis	power
508	Spacecraft & Flight Software	safe mode transition	Physics	motion axis	admission
509	Spacecraft & Flight Software	watchdog recovery	Physics	motion axis	admission
510	Spacecraft & Flight Software	instrument activation	Physics	motion axis	capacity
511	Spacecraft & Flight Software	fault isolation	Physics	motion axis	admission
513	Energy Storage & Microgrid	battery charge power	Energy	energy storage	power
514	Energy Storage & Microgrid	battery discharge power	Energy	energy storage	power
515	Energy Storage & Microgrid	battery soc reserve	Energy	energy storage	setpoint
516	Energy Storage & Microgrid	grid import limit	Energy	energy storage	power
517	Energy Storage & Microgrid	grid export limit	Energy	energy storage	power
518	Energy Storage & Microgrid	pV curtailment	Energy	energy storage	power
519	Energy Storage & Microgrid	ev charge power	Energy	energy storage	power
520	Energy Storage & Microgrid	heat pump power	Energy	energy storage	power
521	Energy Storage & Microgrid	electrolyzer power	Energy	energy storage	power
522	Energy Storage & Microgrid	microgrid demand limit	Energy	energy storage	power
523	Energy Storage & Microgrid	peak shaving	Energy	energy storage	capacity
524	Energy Storage & Microgrid	time of use schedule	Energy	energy storage	setpoint
525	Energy Storage & Microgrid	energy load shed	Energy	energy storage	admission
526	Energy Storage & Microgrid	flex load admission	Energy	energy storage	admission
527	Energy Storage & Microgrid	storage dispatch	Energy	energy storage	capacity
528	Energy Storage & Microgrid	microgrid emergency reserve	Energy	energy storage	setpoint
529	PDU, UPS & Electrical Distribution	server power cap	Compute, Physics, Energy, Distribution	energy storage	power
530	PDU, UPS & Electrical Distribution	rack power cap	Compute, Physics, Energy, Distribution	energy storage	power

#	Family	Muscle	Realms	Template	Knob
531	PDU, UPS & Electrical Distribution	pdu branch power limit	Compute, Physics, Energy, Distribution	energy storage	power
532	PDU, UPS & Electrical Distribution	pdu outlet control	Compute, Physics, Energy, Distribution	energy storage	capacity
534	PDU, UPS & Electrical Distribution	ups operating mode	Compute, Physics, Energy, Distribution	energy storage	capacity
535	PDU, UPS & Electrical Distribution	ups charge rate	Compute, Physics, Energy, Distribution	energy storage	power
536	PDU, UPS & Electrical Distribution	ups discharge rate	Compute, Physics, Energy, Distribution	energy storage	power
537	PDU, UPS & Electrical Distribution	phase balance	Compute, Physics, Energy, Distribution	energy storage	capacity
538	PDU, UPS & Electrical Distribution	load transfer	Compute, Physics, Energy, Distribution	energy storage	admission
539	PDU, UPS & Electrical Distribution	reactive power target	Compute, Physics, Energy, Distribution	energy storage	setpoint
540	PDU, UPS & Electrical Distribution	voltage target	Compute, Physics, Energy, Distribution	energy storage	setpoint
541	PDU, UPS & Electrical Distribution	generator dispatch	Compute, Physics, Energy, Distribution	energy storage	capacity
542	PDU, UPS & Electrical Distribution	electrical isolation	Compute, Physics, Energy, Distribution	energy storage	admission
543	PDU, UPS & Electrical Distribution	breaker trip gate	Compute, Physics, Energy, Distribution	energy storage	admission
545	Cooling, Chillers & Thermodynamics	supply air temperature	Compute, Physics, Energy, Distribution	thermal zone	setpoint
546	Cooling, Chillers & Thermodynamics	return air target	Compute, Physics, Energy, Distribution	thermal zone	setpoint
547	Cooling, Chillers & Thermodynamics	coolant supply temperature	Compute, Physics, Energy, Distribution	thermal zone	setpoint
548	Cooling, Chillers & Thermodynamics	coolant flow	Compute, Physics, Energy, Distribution	thermal zone	capacity
549	Cooling, Chillers & Thermodynamics	pump speed	Compute, Physics, Energy, Distribution	thermal zone	capacity
550	Cooling, Chillers & Thermodynamics	fan speed	Compute, Physics, Energy, Distribution	thermal zone	capacity
551	Cooling, Chillers & Thermodynamics	chiller setpoint	Compute, Physics, Energy, Distribution	thermal zone	setpoint
552	Cooling, Chillers & Thermodynamics	compressor authority	Compute, Physics, Energy, Distribution	thermal zone	power
553	Cooling, Chillers & Thermodynamics	cooling tower fan	Compute, Physics, Energy, Distribution	thermal zone	capacity
554	Cooling, Chillers & Thermodynamics	cooling capacity	Compute, Physics, Energy, Distribution	thermal zone	capacity
555	Cooling, Chillers & Thermodynamics	rack thermal budget	Compute, Physics, Energy, Distribution	thermal zone	power
556	Cooling, Chillers & Thermodynamics	gpu thermal envelope	Compute, Physics, Energy, Distribution	thermal zone	setpoint
557	Cooling, Chillers & Thermodynamics	cpu thermal envelope	Compute, Physics, Energy, Distribution	thermal zone	setpoint
558	Cooling, Chillers & Thermodynamics	thermal workload migrate	Compute, Physics, Energy, Distribution	thermal zone	admission
559	Cooling, Chillers & Thermodynamics	thermal load shed	Compute, Physics, Energy, Distribution	thermal zone	admission
561	Facility & Grid Optimization	facility power budget	Energy	energy storage	power
562	Facility & Grid Optimization	utility demand limit	Energy	energy storage	power
563	Facility & Grid Optimization	demand response	Energy	energy storage	admission
564	Facility & Grid Optimization	electricity price gate	Energy	energy storage	admission
565	Facility & Grid Optimization	carbon intensity gate	Energy	energy storage	admission
566	Facility & Grid Optimization	renewable dispatch	Energy	energy storage	capacity
567	Facility & Grid Optimization	generator start stop	Energy	energy storage	capacity
568	Facility & Grid Optimization	site battery dispatch	Energy	energy storage	capacity
569	Facility & Grid Optimization	pue target	Energy	energy storage	setpoint
570	Facility & Grid Optimization	cooling power budget	Energy	energy storage	power
571	Facility & Grid Optimization	it power budget	Energy	energy storage	power

#	Family	Muscle	Realms	Template	Knob
572	Facility & Grid Optimization	rack power allocation	Energy	energy storage	power
573	Facility & Grid Optimization	facility peak guard	Energy	energy storage	capacity
574	Facility & Grid Optimization	grid frequency response	Energy	energy storage	admission
575	Facility & Grid Optimization	facility islanding	Energy	energy storage	admission
593	Quantum Computing Control Simulation	qubit mapping	Compute	compute pool	capacity
594	Quantum Computing Control Simulation	circuit admission	Compute	compute pool	admission
595	Quantum Computing Control Simulation	shot allocation	Compute	compute pool	capacity
596	Quantum Computing Control Simulation	circuit scheduling	Compute	compute pool	admission
597	Quantum Computing Control Simulation	gate scheduling	Compute	compute pool	capacity
598	Quantum Computing Control Simulation	pulse amplitude	Compute	compute pool	capacity
599	Quantum Computing Control Simulation	pulse duration	Compute	compute pool	setpoint
600	Quantum Computing Control Simulation	pulse phase	Compute	compute pool	capacity
601	Quantum Computing Control Simulation	pulse frequency	Compute	compute pool	power
602	Quantum Computing Control Simulation	coupling control	Compute	compute pool	capacity
603	Quantum Computing Control Simulation	reset scheduling	Compute	compute pool	capacity
604	Quantum Computing Control Simulation	measurement scheduling	Compute	compute pool	capacity
605	Quantum Computing Control Simulation	dynamical decoupling	Compute	compute pool	capacity
606	Quantum Computing Control Simulation	noise aware routing	Compute	compute pool	capacity
607	Quantum Computing Control Simulation	error mitigation budget	Compute	compute pool	admission
608	Quantum Computing Control Simulation	quantum queue priority	Compute	compute pool	admission
609	Cross-Cluster, Multi-Region & Edge	multi cluster dispatch	Compute	compute pool	capacity
610	Cross-Cluster, Multi-Region & Edge	region dispatch	Compute	compute pool	capacity
611	Cross-Cluster, Multi-Region & Edge	zone dispatch	Compute	compute pool	capacity
612	Cross-Cluster, Multi-Region & Edge	edge dispatch	Compute	compute pool	capacity
613	Cross-Cluster, Multi-Region & Edge	cloud capacity class	Compute	compute pool	capacity
614	Cross-Cluster, Multi-Region & Edge	workload migrate region	Compute	compute pool	capacity
615	Cross-Cluster, Multi-Region & Edge	data residency gate	Compute	compute pool	admission
616	Cross-Cluster, Multi-Region & Edge	latency region gate	Compute	compute pool	admission
617	Cross-Cluster, Multi-Region & Edge	cost region gate	Compute	compute pool	admission
618	Cross-Cluster, Multi-Region & Edge	carbon region gate	Compute	compute pool	admission
619	Cross-Cluster, Multi-Region & Edge	global failover	Compute	compute pool	capacity
620	Cross-Cluster, Multi-Region & Edge	federation quota	Compute	compute pool	admission
621	Cross-Cluster, Multi-Region & Edge	cross cluster replication	Compute	compute pool	capacity
622	Cross-Cluster, Multi-Region & Edge	edge offload	Compute	compute pool	capacity
623	Cross-Cluster, Multi-Region & Edge	global admission	Compute	compute pool	admission
AUDIT-0001	Kubernetes Dynamic Device Allocation	dra device class selection	Compute	compute pool	capacity
AUDIT-0002	Kubernetes Dynamic Device Allocation	dra claim capacity	Compute	compute pool	capacity
AUDIT-0003	Kubernetes Dynamic Device Allocation	dra claim sharing	Compute	compute pool	capacity
AUDIT-0004	Kubernetes Dynamic Device Allocation	dra device taint	Compute	compute pool	capacity
AUDIT-0005	Kubernetes Dynamic Device Allocation	dra device eviction	Compute	compute pool	admission
AUDIT-0006	Kubernetes Dynamic Device Allocation	dra binding readiness	Compute	compute pool	capacity
AUDIT-0007	Kubernetes Dynamic Device Allocation	dra binding failure response	Compute	compute pool	capacity
AUDIT-0008	Kubernetes Dynamic Device Allocation	dra device configuration	Compute	compute pool	setpoint
AUDIT-0009	Telecom RAN & Edge Radio	ran connection admission	Distribution	compute pool	admission
AUDIT-0010	Telecom RAN & Edge Radio	ran ue handover	Distribution	compute pool	capacity
AUDIT-0011	Telecom RAN & Edge Radio	ran cell traffic steering	Distribution	compute pool	capacity
AUDIT-0012	Telecom RAN & Edge Radio	ran slice resource budget	Distribution	compute pool	admission
AUDIT-0013	Telecom RAN & Edge Radio	ran prb allocation	Distribution	compute pool	capacity

#	Family	Muscle	Realms	Template	Knob
AUDIT-0014	Telecom RAN & Edge Radio	ran scheduler weight	Distribution	compute pool	setpoint
AUDIT-0015	Telecom RAN & Edge Radio	ran tx power	Distribution	compute pool	power
AUDIT-0016	Telecom RAN & Edge Radio	ran antenna tilt	Distribution	compute pool	capacity
AUDIT-0017	Telecom RAN & Edge Radio	ran carrier enable	Distribution	compute pool	capacity
AUDIT-0018	Telecom RAN & Edge Radio	ran cell sleep	Distribution	compute pool	capacity
AUDIT-0019	Telecom RAN & Edge Radio	ran du cu placement	Distribution	compute pool	capacity
AUDIT-0020	Telecom RAN & Edge Radio	ran fronthaul budget	Distribution	compute pool	admission
AUDIT-0021	DPU SmartNIC & Programmable IO	dpu pf tx rate	Compute	compute pool	capacity
AUDIT-0022	DPU SmartNIC & Programmable IO	dpu vf tx rate	Compute	compute pool	capacity
AUDIT-0023	DPU SmartNIC & Programmable IO	dpu sf tx rate	Compute	compute pool	capacity
AUDIT-0024	DPU SmartNIC & Programmable IO	dpu bandwidth share	Compute	compute pool	capacity
AUDIT-0025	DPU SmartNIC & Programmable IO	dpu qos group	Compute	compute pool	capacity
AUDIT-0026	DPU SmartNIC & Programmable IO	sriov vf count	Compute	compute pool	capacity
AUDIT-0027	DPU SmartNIC & Programmable IO	smartnic flow steering	Compute	compute pool	capacity
AUDIT-0028	DPU SmartNIC & Programmable IO	smartnic offload enable	Compute	compute pool	capacity
AUDIT-0029	DPU SmartNIC & Programmable IO	dpu cpu budget	Compute	compute pool	admission
AUDIT-0030	DPU SmartNIC & Programmable IO	dpu memory budget	Compute	compute pool	admission
AUDIT-0031	DPU SmartNIC & Programmable IO	dpu service placement	Compute	compute pool	capacity
AUDIT-0032	DPU SmartNIC & Programmable IO	dpu failover	Compute	compute pool	capacity
AUDIT-0033	Grid Transmission & Distribution	capacitor bank switch	Energy	process loop	capacity
AUDIT-0034	Grid Transmission & Distribution	voltage regulator tap	Energy	process loop	setpoint
AUDIT-0035	Grid Transmission & Distribution	transformer tap	Energy	process loop	setpoint
AUDIT-0036	Grid Transmission & Distribution	inverter real power	Energy	process loop	power
AUDIT-0037	Grid Transmission & Distribution	inverter reactive power	Energy	process loop	power
AUDIT-0038	Grid Transmission & Distribution	feeder voltage target	Energy	process loop	setpoint
AUDIT-0039	Grid Transmission & Distribution	feeder load transfer	Energy	process loop	capacity
AUDIT-0040	Grid Transmission & Distribution	distribution storage dispatch	Energy	process loop	admission
AUDIT-0041	Grid Transmission & Distribution	demand response dispatch	Energy	process loop	admission
AUDIT-0042	Grid Transmission & Distribution	frequency droop setpoint	Energy	process loop	setpoint
AUDIT-0043	Grid Transmission & Distribution	grid protection mode	Energy	process loop	admission
AUDIT-0044	Grid Transmission & Distribution	grid restoration sequence	Energy	process loop	admission
AUDIT-0045	Automotive EV & Mobile Powertrain	traction torque limit	Physics	motion axis	power
AUDIT-0046	Automotive EV & Mobile Powertrain	regen braking level	Physics	motion axis	power

#	Family	Muscle	Realms	Template	Knob
AUDIT-0047	Automotive EV & Mobile Powertrain	battery charge limit	Physics	motion axis	power
AUDIT-0048	Automotive EV & Mobile Powertrain	battery discharge limit	Physics	motion axis	power
AUDIT-0049	Automotive EV & Mobile Powertrain	battery thermal target	Physics	motion axis	power
AUDIT-0050	Automotive EV & Mobile Powertrain	motor thermal limit	Physics	motion axis	power
AUDIT-0051	Automotive EV & Mobile Powertrain	vehicle speed envelope	Physics	motion axis	capacity
AUDIT-0052	Automotive EV & Mobile Powertrain	energy recovery target	Physics	motion axis	power
AUDIT-0053	Automotive EV & Mobile Powertrain	auxiliary power budget	Physics	motion axis	power
AUDIT-0054	Automotive EV & Mobile Powertrain	fast charge current	Physics	motion axis	power
AUDIT-0055	Automotive EV & Mobile Powertrain	fast charge voltage	Physics	motion axis	power
AUDIT-0056	Automotive EV & Mobile Powertrain	vehicle safe state	Physics	motion axis	admission
AUDIT-0057	Semiconductor Fab & Precision Manufacturing	tool job dispatch	Energy	process loop	admission
AUDIT-0058	Semiconductor Fab & Precision Manufacturing	wafer route	Energy	process loop	capacity
AUDIT-0059	Semiconductor Fab & Precision Manufacturing	chamber recipe selection	Energy	process loop	admission
AUDIT-0060	Semiconductor Fab & Precision Manufacturing	chamber temperature	Energy	process loop	setpoint
AUDIT-0061	Semiconductor Fab & Precision Manufacturing	chamber pressure	Energy	process loop	setpoint
AUDIT-0062	Semiconductor Fab & Precision Manufacturing	gas flow	Energy	process loop	capacity
AUDIT-0063	Semiconductor Fab & Precision Manufacturing	rf power	Energy	process loop	power
AUDIT-0064	Semiconductor Fab & Precision Manufacturing	vacuum pump speed	Energy	process loop	capacity
AUDIT-0065	Semiconductor Fab & Precision Manufacturing	robot transfer rate	Energy	process loop	capacity
AUDIT-0066	Semiconductor Fab & Precision Manufacturing	lot priority	Energy	process loop	admission
AUDIT-0068	Semiconductor Fab & Precision Manufacturing	tool quarantine	Energy	process loop	admission
AUDIT-0069	Water Wastewater & Pumping	pump speed water	Energy	process loop	capacity
AUDIT-0070	Water Wastewater & Pumping	valve position water	Energy	process loop	setpoint
AUDIT-0071	Water Wastewater & Pumping	reservoir level target	Energy	process loop	setpoint
AUDIT-0072	Water Wastewater & Pumping	line pressure target	Energy	process loop	setpoint
AUDIT-0073	Water Wastewater & Pumping	flow target water	Energy	process loop	setpoint
AUDIT-0074	Water Wastewater & Pumping	aeration rate	Energy	process loop	power
AUDIT-0075	Water Wastewater & Pumping	chemical dose rate	Energy	process loop	power
AUDIT-0076	Water Wastewater & Pumping	filtration backwash	Energy	process loop	admission
AUDIT-0077	Water Wastewater & Pumping	lift station dispatch	Energy	process loop	admission
AUDIT-0078	Water Wastewater & Pumping	leak isolation	Energy	process loop	admission
AUDIT-0079	Water Wastewater & Pumping	water demand shed	Energy	process loop	admission
AUDIT-0080	Water Wastewater & Pumping	water emergency shutdown	Energy	process loop	admission

#	Family	Muscle	Realms	Template	Knob
AUDIT-0081	Building & Critical Environment HVAC	zone temperature target	Energy	thermal zone	setpoint
AUDIT-0082	Building & Critical Environment HVAC	zone airflow	Energy	thermal zone	capacity
AUDIT-0083	Building & Critical Environment HVAC	ahu fan speed	Energy	thermal zone	capacity
AUDIT-0084	Building & Critical Environment HVAC	damper position	Energy	thermal zone	capacity
AUDIT-0085	Building & Critical Environment HVAC	economizer position	Energy	thermal zone	capacity
AUDIT-0086	Building & Critical Environment HVAC	boiler setpoint	Energy	thermal zone	setpoint
AUDIT-0087	Building & Critical Environment HVAC	heat pump mode	Energy	thermal zone	admission
AUDIT-0088	Building & Critical Environment HVAC	humidity target	Energy	thermal zone	setpoint
AUDIT-0089	Building & Critical Environment HVAC	occupancy ventilation	Energy	thermal zone	admission
AUDIT-0090	Building & Critical Environment HVAC	building demand limit	Energy	thermal zone	power
AUDIT-0091	Building & Critical Environment HVAC	thermal storage dispatch	Energy	thermal zone	capacity
AUDIT-0092	Building & Critical Environment HVAC	hvac emergency mode	Energy	thermal zone	admission

APPENDIX

Appendix H. Source of the Engine

The engine, the bowl law, the bowl on every muscle, the two-wire GPU governor, the plug adapter and the nervous system, printed in full from the repository so that the book and the code can be read side by side. The engine file is sealed: its fingerprint is in `results/SEAL.json`, and `verify.py` fails if a byte of it changes.

`omnicompass/core.py`

240 lines.

```
"""Omni-Compass core: six-state ODE, RK4 integrator, bounded feedforward-plus-proportional controller.

State x = (E, U, I_U, S, B, B_dot).

(1) dE/dt = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u, |u| <= U_AUTHORITY
(3) dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S) = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
(6) dS/dt = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
(7) dB/dt = B_dot ; dB_dot/dt = gamma_c delta S - (omega_B/Q_B) B_dot - omega_B^2 B
(8) R_B[n] = finite-difference audit of (7); never fed back into the state.

Controller (one micro step, zero-order hold across all four RK4 stages):
    u = clip(-f_U(x, t) + KP (sigma - U), -U_AUTHORITY, +U_AUTHORITY)
where f_U is the uncontrolled U drift and sigma in {-1, +1} is the locked target.
"""
from __future__ import annotations

import math
from dataclasses import dataclass, asdict, fields
from typing import Dict, List, Optional, Tuple

EPS = 1e-12
KP = 12.0
U_AUTHORITY = 25.0
MACRO_STEPS = 20
MICRO_STEPS = 10
```

```

MACRO_DT = 0.1
MICRO_DT = MACRO_DT / MICRO_STEPS
BASIN_TOL = 0.10
CONVEY_WINDOW = 5
CERT_WINDOW = 10

PARAMETER_RANGES: Dict[str, Tuple[float, float]] = {
    "alpha": (4.2, 4.2), "beta_int": (0.0, 1.0), "beta_ext": (0.0, 1.0), "k": (1.7, 1.7),
    "sigma_1": (0.38, 0.38), "delta": (0.01, 1.0), "gamma_1": (1.35, 1.35), "gamma_c": (1.0, 1.0),
    "lambda_0": (-2.0, 2.0), "lambda_1": (0.1, 3.0), "lambda_2": (0.0, 2.0), "c": (0.5, 5.0),
    "E_max": (1.0, 10.0), "omega_B": (0.1, 5.0), "Q_B": (0.5, 10.0), "alpha_s": (0.05, 0.25),
    "beta_s": (0.05, 0.25), "mu": (2.0, 6.0), "alpha_E": (4.2, 4.2), "alpha_U": (4.2, 4.2),
    "lambda_I": (0.5, 2.0), "lambda_U": (0.0, 0.5),
}
# alpha, k, gamma_1, alpha_U do not enter equations (1)-(8).

@dataclass
class State:
    E: float
    U: float
    I_U: float
    S: float
    B: float
    B_dot: float

    def as_tuple(self) -> Tuple[float, ...]:
        return (self.E, self.U, self.I_U, self.S, self.B, self.B_dot)

@dataclass
class Params:
    alpha: float = 4.2
    beta_int: float = 0.0
    beta_ext: float = 0.0
    k: float = 1.7
    sigma_1: float = 0.38
    delta: float = 0.5
    gamma_1: float = 1.35
    gamma_c: float = 1.0
    lambda_0: float = 0.0
    lambda_1: float = 1.0
    lambda_2: float = 0.5
    c: float = 1.0
    E_max: float = 2.0
    omega_B: float = 1.0
    Q_B: float = 2.5
    alpha_s: float = 0.12
    beta_s: float = 0.10
    mu: float = 4.0
    alpha_E: float = 4.2
    alpha_U: float = 4.2
    lambda_I: float = 1.0
    lambda_U: float = 0.1

# ----- equations
def v_eff(x: State, p: Params, t: float = 0.0) -> float:
    z = p.lambda_0 + p.lambda_1 * (x.U - 0.5) + p.lambda_2 * x.S
    return math.cos(0.5 * p.omega_B * t) * (p.c * math.tanh(z))

def phi(S: float, p: Params) -> float:
    return 0.5 * p.alpha_s * S * S + 0.25 * p.beta_s * S ** 3 - p.delta * S

def s_flow(S: float, p: Params) -> float:
    return p.delta - p.alpha_s * S - 0.75 * p.beta_s * S * S

def derivatives(x: State, p: Params, t: float, u: float = 0.0) -> State:
    v = v_eff(x, p, t)
    dE = -p.alpha_E * x.E + p.beta_int + p.beta_ext + v
    dU = p.mu * x.U * (1.0 - x.U * x.U) - dE / max(p.E_max, 1e-6) - p.lambda_U * x.U + u
    dI = (1.0 - x.U) - p.sigma_1 * x.E - p.delta * x.S - p.lambda_I * x.I_U
    dS = s_flow(x.S, p)
    dBd = p.gamma_c * p.delta * x.S - (p.omega_B / max(p.Q_B, EPS)) * x.B_dot - p.omega_B ** 2 * x.B
    return State(dE, dU, dI, dS, x.B_dot, dBd)

def _axpy(x: State, k: State, h: float) -> State:
    return State(x.E + h * k.E, x.U + h * k.U, x.I_U + h * k.I_U, x.S + h * k.S, x.B + h * k.B, x.B_dot + h * k.B_dot)

```

```

def rk4_step(x: State, p: Params, t: float, h: float, u: float = 0.0) -> State:
    k1 = derivatives(x, p, t, u)
    k2 = derivatives(_axpy(x, k1, 0.5 * h), p, t + 0.5 * h, u)
    k3 = derivatives(_axpy(x, k2, 0.5 * h), p, t + 0.5 * h, u)
    k4 = derivatives(_axpy(x, k3, h), p, t + h, u)
    c = h / 6.0
    return State(*(a + c * (b1 + 2 * b2 + 2 * b3 + b4) for a, b1, b2, b3, b4 in
        zip(x.as_tuple(), k1.as_tuple(), k2.as_tuple(), k3.as_tuple(), k4.as_tuple()))))

def bath_residual(prev: State, cur: State, nxt: State, p: Params, h: float) -> float:
    d2 = (nxt.B - 2.0 * cur.B + prev.B) / (h * h)
    dm = (cur.B - prev.B) / h
    return d2 + (p.omega_B / max(p.Q_B, EPS)) * dm + p.omega_B ** 2 * cur.B - p.gamma_c * p.delta * cur.S

# ----- controller
def control_command(x: State, p: Params, t: float, target: int) -> Tuple[float, float]:
    """Return (u, raw) for one micro step. raw is the unclipped command."""
    drift = derivatives(x, p, t, 0.0).U
    raw = -drift + KP * (float(target) - x.U)
    return max(-U_AUTHORITY, min(U_AUTHORITY, raw)), raw

def basin_sign(U: float) -> int:
    if abs(U - 1.0) <= BASIN_TOL:
        return 1
    if abs(U + 1.0) <= BASIN_TOL:
        return -1
    return 0

def nearest_basin_sign(U: float) -> int:
    return 1 if abs(U - 1.0) <= abs(U + 1.0) else -1

def macro_step(x: State, p: Params, t0: float, target: Optional[int],
    micro_steps: int = MICRO_STEPS, macro_dt: float = MACRO_DT) -> Tuple[State, Dict[str, float]]:
    """Advance one macro interval. target=None runs the plant with u=0 (observe)."""
    h = macro_dt / micro_steps
    t = t0
    u_abs = u_sq = u_peak = 0.0
    sat = 0
    for _ in range(micro_steps):
        if target is None:
            u = raw = 0.0
        else:
            u, raw = control_command(x, p, t, target)
            u_abs += abs(u) * h
            u_sq += u * u * h
            u_peak = max(u_peak, abs(u))
            sat += int(abs(raw) > U_AUTHORITY + EPS)
            x = rk4_step(x, p, t, h, u)
            t += h
    return x, {"u_abs": u_abs, "u_sq": u_sq, "u_peak": u_peak, "saturated": sat}

```

```

# ----- trajectory
@dataclass
class Result:
    final: State
    U_hist: List[float]
    target_sign: int
    born: int
    convey: int
    convey_start: int
    convey_confirm: int
    cert: int
    cert_start: int
    cert_complete: int
    max_streak: int
    actuator_abs_integral: float
    actuator_sq_integral: float
    actuator_peak: float
    actuator_saturated_periods: int

def simulate(x0: State, p: Params, target: Optional[int] = None, controlled: bool = True) -> Result:
    """Canonical 20-macro-step trajectory with CONVEY-5 / CERT-10 predicates.

    target defaults to the nearest basin of U0. controlled=False runs u=0.
    """
    tgt = nearest_basin_sign(x0.U) if target is None else int(target)
    b0 = basin_sign(x0.U)

    born = int(b0 != 0)
    streak = 1 if born else 0
    ssign = b0 if born else 0
    best = streak
    convey = cert = 0
    cs = cc = ts = tc = -1
    ua = usq = upk = 0.0
    nsat = 0
    x = x0
    hist = [x0.U]
    for m in range(1, MACRO_STEPS + 1):
        x, a = macro_step(x, p, (m - 1) * MACRO_DT, tgt if controlled else None)
        ua += a["u_abs"]; usq += a["u_sq"]; upk = max(upk, a["u_peak"]); nsat += a["saturated"]
        hist.append(x.U)
        b = basin_sign(x.U)
        if b:
            if b == ssign:
                streak += 1
            else:
                ssign, streak = b, 1
        else:
            ssign, streak = 0, 0
        best = max(best, streak)
        if not convey and streak >= CONVEY_WINDOW:
            convey, cs, cc = 1, m - CONVEY_WINDOW + 1, m
        if not cert and streak >= CERT_WINDOW:
            cert, ts, tc = 1, m - CERT_WINDOW + 1, m
    return Result(x, hist, tgt, born, convey, cs, cc, cert, ts, tc, best, ua, usq, upk, nsat)

def sample_state(rng) -> State:
    return State(float(rng.uniform(0.0, 1.0)), float(rng.uniform(-1.0, 1.0)), float(rng.uniform(0.0, 1.0)),
        float(rng.uniform(-1.0, 1.0)), float(rng.uniform(-1.0, 1.0)), float(rng.uniform(-1.0, 1.0)))

def sample_params(rng) -> Params:
    return Params(**{k: float(rng.uniform(lo, hi)) for k, (lo, hi) in PARAMETER_RANGES.items()})

def s_roots(p: Params) -> Tuple[float, float]:
    """Roots of equation (6): S_minus < 0 < S_plus."""
    a = 0.75 * p.beta_s
    d = math.sqrt(p.alpha_s ** 2 + 4.0 * a * p.delta)
    return (-p.alpha_s - d) / (2.0 * a), (-p.alpha_s + d) / (2.0 * a)

```

omnicompass/bowl.py

174 lines.

```

# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE.
"""The bowl: one smooth law for every muscle, and the plug every muscle is wired through.

A muscle is a plug with one wire in and one wire out. The plug reads its meters and writes its lever; all the
thinking is here, in the brain, written once for every muscle.

The plug (Plug)
    read     the service reading (response time, temperature, queue) and the lever as it stands
    write    one value; the plug clips it to the cover, sends it, and reads back what the device took
    restore  the lever back to the snapshot, the value read once before the first write; it never moves after that
    one writer if the lever is found at a value the brain did not write, someone else owns it: the brain stops writing
            and leaves that value alone (restoring over it would fight the new owner)

Two bands
    cover    the outer band, the lever's hard range (the device's or the buyer's lowest and highest setting); every
            write is clipped to it, so nothing the math does can set the lever outside
    bowl     the service reading as a position in its band, 0 to 1 (0: calm, 1: the service line); the brain pulls it
            to the bottom of the bowl, the center (0.5 by default); the walls run to 0.05 and 0.95, and the last 5% on
            each side is cushion (10% in all) before the line

The force (Bowl.force), the physics of a ball in a bowl with the right friction
    pull     K_P x (position - center): gentle near the bottom, harder up the walls
    push     K_D x velocity: whatever is shoving the position (a load rising, heat building) is met by an equal and
            opposite push; it is also the friction that stops the ball sloshing past the bottom (with K_D chosen for
            critical damping it glides to the center and stops, no overshoot, no ringing)
    smooth   the force is A x tanh(raw / A): a spring near the center that bends over and flattens into its maximum A,
            never a corner, never a hammer; the same shape the engine already uses for v_eff (omnicompass/core.py)
    fail up  past the 0.95 wall the up side goes to its full force at once, and the down side may not act until the
            position is back inside the bowl

Two forces (antagonist pairs)
    up       a positive force adds capacity, power, cooling or speed (scale out, raise a clock, start a chiller)
    down     a negative force takes it back (scale in, lower a limit, warm a setpoint)
    each side has its own gain (lever units per unit of force), because adding and taking back do not cost the same;
    a muscle with two wires (a GPU's clock range and power limit) gives each side its own lever; a one-wire muscle
    carries both directions on its one lever

The frozen engine (omnicompass/core.py) and its proofs are untouched; this is a separate law, run as its own arm.
"""
from __future__ import annotations

import math
from dataclasses import dataclass, field
from typing import Optional

def clamp(v, lo, hi):
    return lo if v < lo else hi if v > hi else v

@dataclass
class Band:
    """A reading's band: lo is calm (position 0), hi is the line (position 1)."""
    lo: float

```

```

hi: float
center: float = 0.5
cushion: float = 0.05

def position(self, x: float) -> float:
    return (x - self.lo) / (self.hi - self.lo)

@property
def wall_low(self) -> float:
    return self.cushion

@property
def wall_high(self) -> float:
    return 1.0 - self.cushion

@dataclass
class Bowl:
    """The brain's law for one muscle. dt: seconds between decisions; tau: the muscle's response time in seconds
    (how long the service reading takes to follow the lever); kd defaults to critical damping for that response."""
    band: Band
    dt: float
    tau: float
    kp: float = 1.0
    kd: Optional[float] = None
    authority: float = 1.0
    smooth: float = 0.5 # the velocity estimate's smoothing (0: raw difference, near 1: heavily smoothed)
    p: Optional[float] = field(default=None, init=False)

    v: float = field(default=0.0, init=False)

    def __post_init__(self):
        if self.kd is None:
            # a first-order muscle with response tau under a P-D pull: critical damping when kd = 2 sqrt(kp tau) - 1
            # scaled to the decision period; never negative
            self.kd = max(0.0, 2.0 * math.sqrt(self.kp * self.tau / self.dt) - 1.0) * self.dt

    def sense(self, reading: float) -> float:
        p = self.band.position(reading)
        if self.p is not None:
            self.v = self.smooth * self.v + (1.0 - self.smooth) * (p - self.p) / self.dt
        self.p = p
        return p

    def force(self, reading: float) -> float:
        """Positive: the up side (add capacity); negative: the down side (take it back). Bounded by the authority."""
        p = self.sense(reading)
        if p >= self.band.wall_high:
            return self.authority # fail up: past the wall, full force up at once
        raw = self.kp * (p - self.band.center) + self.kd * self.v
        f = self.authority * math.tanh(raw / self.authority)
        if f < 0.0 and self.v > 0.0 and p > self.band.center:
            f = 0.0 # never take back while it climbs above the center
        return f

class ForeignWriter(Exception):

```

```

pass

class Plug:
    """One wire in, one wire out. Subclasses implement _read_service, _read_lever and _send; everything else is here."
    ↵ """

    def __init__(self, lo: float, hi: float, tolerance: float = 1e-6):
        self.lo, self.hi, self.tol = lo, hi, tolerance
        self.snapshot: Optional[float] = None
        self.last_written: Optional[float] = None
        self.owned = True
        self.writes = 0
        self.clipped = 0

    # the wires -----
    def _read_service(self) -> float:
        raise NotImplementedError

    def _read_lever(self) -> float:
        raise NotImplementedError

    def _send(self, value: float) -> None:
        raise NotImplementedError

    # the contract -----
    def attach(self) -> float:
        """Read the lever once, before any write: the restore point. It never moves after this."""

        self.snapshot = self._read_lever()
        return self.snapshot

    def read(self) -> float:
        return self._read_service()

    def lever(self) -> float:
        return self._read_lever()

    def write(self, value: float) -> Optional[float]:
        """Clip to the cover, check nobody else moved the lever, send, read back. None: not written (not owned)."""
        if not self.owned:
            return None
        now = self._read_lever()
        expected = self.last_written if self.last_written is not None else self.snapshot
        if expected is not None and abs(now - expected) > self.tol:
            self.owned = False
            # someone else moved it: it is theirs now
            raise ForeignWriter(f"lever at {now}, last written {expected}")
        v = clamp(value, self.lo, self.hi)
        self.clipped += int(v != value)
        if abs(v - now) <= self.tol:
            return now
        self._send(v)
        self.last_written = self._read_lever()
        self.writes += 1
        return self.last_written

    def restore(self) -> bool:
        """The lever back to the snapshot. Left alone when someone else owns it. True: at the snapshot (or not ours)."""
        ↵ """
        if not self.owned or self.snapshot is None:
            return True
        if abs(self._read_lever() - self.snapshot) > self.tol:
            self._send(self.snapshot)
            self.last_written = None
        return abs(self._read_lever() - self.snapshot) <= self.tol

```

realms/bowl_arm.py

111 lines.

```

# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE.
"""The bowl law on every realm muscle (omnicompass/bowl.py): each plant's knob held by its own bowl.

Each plant's service is read as one position in its bowl (0 calm, 1 the line): the worst of its queue or lateness, its
load above half, and, for a storage site, its draw on the grid connection above half. The last period in violation
reads as past the wall (fail up). The bowl's force moves the knob: a positive force adds capacity, power, cooling or
protection, a negative force gives it back, each knob inside its own cover. At the kill every knob returns to native.

Knob by plant (the override key the plant already obeys, its native value, its cover, and which way is "more"):
    compute_pool    power: limit share 0.4-1; replicas-type: the HPA target 0.5-0.95 (lower is more pods); machine pools:
                    one machine released while the force is clearly down; admission: native
    thermal_zone    setpoint between the band's stress (colder, more cooling) and calm ends; units: one released while th
                    ↪ e
                    force is clearly down; power: 0.5-1
    energy_storage  reserve between the band's ends (a lower reserve gives the battery to the site); power: native
    motion_axis     speed and effort: 0.4-1
    process_loop    setpoint from native toward the band's calm end; actuator range 0.3-1; power 0.4-1
"""
from __future__ import annotations

from omnicompass.bowl import Band, Bowl, clamp

UP, DOWN, RELEASE = 0.10, 0.02, -0.2

def position(plant) -> float:
    if plant.vhist and plant.vhist[-1]:
        return 1.0
    if plant.template == "thermal_zone":
        P = plant.P
        return (plant.T - min(P["stress"], P["calm"])) / (P["t_limit"] - min(P["stress"], P["calm"]))
    o = plant.observe()
    p = max(o["queue_ratio"], (o["load_ratio"] - 0.5) / 0.5)
    if plant.template == "energy_storage":
        p = max(p, (o["power_stress"] - 0.5) / 0.5)
    return p

def release_safe(plant) -> bool:
    """A discrete unit goes back only when what is left covers the recent peak (the plant's own release window)."""
    P = plant.P
    if plant.template == "compute_pool":
        from .plants import RELEASE_WINDOW, RELEASE_FRAC
        n = plant.n
        return (n > P["n_min"] and not plant.pending and plant.Q <= 0.0
                and len(plant.ahist) == RELEASE_WINDOW * P["startup_steps"]
                and max(plant.ahist) / ((n - 1) * plant.mu(plant.cap)) <= RELEASE_FRAC * 0.95)
    u = plant.units_on
    return u > 1 and plant.T <= plant.setpoint() and plant.qc_cmd_avg / ((u - 1) * P["q_unit_w"]) <= 0.8

def lever(plant, knob):
    """(override key, native value, low, high, sign): sign +1 when a higher value is more capacity."""
    P, t = plant.P, plant.template

```

```

if knob == "admission":
    return None
if t == "compute_pool":
    if knob == "power":
        return ("power", 1.0, 0.4, 1.0, 1)
    if P.get("ca"):
        return ("release", None, 0, 0, 0)
    return ("target", P["target"], P["target"], 0.95, -1)
if t == "thermal_zone":
    if knob == "setpoint":
        return ("setpoint", P["t_set"], min(P["stress"], P["calm"]), max(P["stress"], P["calm"]), -1)
    if knob == "capacity":
        return ("release", None, 0, 0, 0)
    return ("power", 1.0, 0.5, 1.0, 1)
if t == "energy_storage":
    if knob == "setpoint":
        return ("setpoint", P["reserve"], min(P["stress"], P["calm"]), max(P["stress"], P["calm"]), -1)
    return None
    # a battery's power limit gives nothing back: native
if t == "motion_axis":
    return ("power", 1.0, 0.4, 1.0, 1) if knob == "power" else ("capacity", 1.0, 0.4, 1.0, 1)
if t == "process_loop":
    if knob == "setpoint":
        lo, hi = min(P["sp"], P["calm"]), max(P["sp"], P["calm"])
        return ("setpoint", P["sp"], lo, hi, 1 if P["sp"] > P["calm"] else -1)
    if knob == "power":
        return ("power", 1.0, 0.4, 1.0, 1)
    return ("capacity", 1.0, 0.3, 1.0, 1)
return None

def bowl_apply(plant, knob, dt=1.0, tau=3.0):
    """One decision: the plant's bowl reads its position and moves its knob. Returns the override written."""
    lv = lever(plant, knob)
    if lv is None:
        plant.override = {}
        return {}
    if not hasattr(plant, "_bowl"):
        plant._bowl = Bowl(Band(0.0, 1.0), dt=dt, tau=tau, kp=1.0, smooth=0.3)
        plant._bowl.kd *= 3.0
        plant._bowl_x = lv[1]
    b = plant._bowl
    F = b.force(position(plant))
    key, native, lo, hi, sign = lv
    if key == "release":
        ok = F < RELEASE and b.p < b.band.center and release_safe(plant)
        plant.override = {"release": 1.0} if ok else {}
        return plant.override
    span = hi - lo
    if b.p >= b.band.wall_high:
        x = hi if sign > 0 else lo
        # fail up: full capacity at once
    else:
        g = UP if F > 0 else DOWN
        x = clamp(plant._bowl_x + sign * g * F * span, lo, hi)
    plant._bowl_x = x
    plant.override = {} if abs(x - native) < 1e-9 else {key: x}
    return plant.override

```

omni_controller/gpu_bowl.py

330 lines.

```

# SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
# Copyright (c) 2026 The Omni-Compass LLC. Evaluation and simulation use only; any other use requires a signed, paid
# Omni-Compass Enterprise License. See LICENSE.
"""Omni-Compass on one GPU through two wires: the bowl law (omnicompass/bowl.py) on a real card.

The card keeps its own control: NVIDIA's firmware still boosts and still protects the chip. Omni holds two settings th
↵ e
card already accepts, and nothing else:
up wire   the clock ceiling: how high boost may climb (nvidia-smi -lgc MIN,MAX; -rgc resets it)
down wire the power limit: the lid above what that ceiling draws (nvidia-smi -pl W)

Every decision (--interval seconds):
read      nvidia-smi: power.draw, temperature, utilization, power.limit, clocks.sm; the workload's response times
position  the service as one place in its bowl, 0 calm to 1 the line: response time only (the mean over the last
↵ at      --latency-window-s, between a tenth of the line, the bare service time, and the line; the 95th percentile
↵ y      or past the line, or a failed request, is past the wall). A card that is busy is doing its work; being bus
native    is not a breach and is not read as one
          what the card does on its own, learned from its own meter before the bowl may lower anything: while the
          ceiling is at the top and the card is busy, its clock and its draw (the clock its own power limit holds it
          at, and what that costs). Until --learn-samples busy readings are in, the ceiling stays at the top
race      while the card is saturated (utilization at or over --race-util: work is waiting), the ceiling goes to the
          top and the lid to the start limit, so a burst is served at full speed; the bowl paces only the slack
force     the bowl: pull to the center, push against what is rising, tanh-bounded; past the 0.95 wall: fail up
profile   service (default): down gain 0.01, the slow answers kept at native; batch: down gain 0.02, more saved,
          for work nobody waits on answer by answer
write     up wire: the ceiling moves by the force (fast up, gently down), inside its cover: from the card's own busy
          clock (never slower than native while there is work) to the top; down wire: the lid at the card's own busy
          draw plus --lid-headroom, never under it and never over the start limit, inside the declared envelope;
          fail up: ceiling to the top and the lid to the start limit at once
guards    blind (meters unreadable, response times stale): fail up; heat (the card reports a thermal or hardware
          slowdown): never tighten; one writer: a power limit neither mine nor the start means another writer, so
          observe only from then on and exit 5; read-back: every write is read back
restore   on exit, kill file or SIGTERM: clocks reset (-rgc) and the power limit back to the start, read back

The audit (--audit) carries the same records the bench reads from the one-wire governor (snapshot, decision with
telemetry and decided_by, write and actuator for each power-limit write, restored), plus clock_write records.
Exit: 0 clean, 3 restore failed, 4 a write refused, 5 another writer.
"""
from __future__ import annotations

import argparse
import json
import math
import os
import shlex
import signal
import subprocess
import sys
import time

from omnicompass.bowl import Band, Bowl, clamp
from omni_controller.gpu_governor import query, snapshot, throttle, slowed, WriteFailed, SNAPSHOT
from omni_controller.muscles import latency_sense, latency_window

```

```

PROFILES = {"service": 0.01, "batch": 0.02} # down gain per profile (docs/GPU_PREREGISTRATION.md, amendment 7)

def smi_run(smi, args):
    try:
        p = subprocess.run(shlex.split(smi) + args, capture_output=True, text=True, timeout=20)
        return p.returncode, p.stderr.strip()[:300]
    except (subprocess.SubprocessError, OSError) as e:
        return -1, f"type(e).__name__: {e}"[:300]

def query_top_clock(smi, g):
    try:
        out = subprocess.run(shlex.split(smi) + ["-i", str(g), "--query-gpu=clocks.max.sm", "--format=csv,noheader,nounits"],
                             capture_output=True, text=True, timeout=10, check=True).stdout
        return float(out.strip().splitlines()[0])
    except (subprocess.SubprocessError, OSError, ValueError, IndexError):
        return None

def query_min_clock(smi, g):
    """The lowest graphics clock the card supports (the floor of a -lgc range); 300 MHz if the driver will not list the
    em."""
    try:
        out = subprocess.run(shlex.split(smi) + ["-i", str(g), "--query-supported-clocks=graphics", "--format=csv,noheader,nounits"],
                             capture_output=True, text=True, timeout=10, check=True).stdout
        return int(min(float(x) for x in out.split() if x.strip()))
    except (subprocess.SubprocessError, OSError, ValueError):
        return 300

class GpuBowl:
    def __init__(self, a):
        self.a = a
        self.g = int(str(a.gpus).split(",")[0])
        self.log = open(a.audit, "a")
        self.writes = 0
        self.foreign = False
        self.clock_set = False
        self.enforced_ok = query(a.smi, [self.g], enforced=True) is not None
        s = query(a.smi, [self.g], self.enforced_ok)
        if s is None:
            raise SystemExit("nvidia-smi unreadable at start: no snapshot, so I take no authority")
        snap = snapshot(a.smi, [self.g])
        self.start = s[self.g]["limit"]
        self.expect = self.start
        self.top = query_top_clock(a.smi, self.g) or s[self.g]["clock_mhz"]
        self.floor_w = max(float(a.floor_w or 0.0), s[self.g]["min"])
        self.c_lo = a.clock_min_share * self.top
        self.c_floor = query_min_clock(a.smi, self.g)
        self.ceiling = self.top # where the bowl holds the ceiling (continuous)
        self.written_ceiling = self.top # what the card was last told
        self.busy_clk, self.busy_draw = [], [] # the card on its own: busy clock and busy draw, ceiling at the top

```

```

self.brain = Bowl(Band(0.0, 1.0), dt=a.interval, tau=2.0 * a.interval, kp=1.0, smooth=0.3)
self.brain.kd *= 3.0
self.audit({"snapshot": {str(self.g): {"limit_w": self.start, "min_limit_w": s[self.g]["min"],
                                     "enforced_w": s[self.g]["enforced"] if self.enforced_ok else None,
                                     "clock_top_mhz": self.top,
                                     **{f: snap[f][str(self.g)] for f in SNAPSHOT}}},
          "engine": "bowl, two wires", "mode": a.mode, "profile": a.profile,
          "down_gain": a.down_gain if a.down_gain is not None else PROFILES[a.profile],
          "covers": {"clock_mhz": [round(self.c_lo), round(self.top)], "power_w": [self.floor_w, self.start]
          })
    if a.mode == "cap" and snap["power.management"][str(self.g)].lower() != "enabled":
        self.audit({"refused": "power management not Enabled: a written limit would not bind"})
        raise SystemExit("power management not Enabled: cap mode refused, no authority taken")

def audit(self, rec):
    self.log.write(json.dumps({"time": time.time(), **rec}) + "\n"); self.log.flush()

def position(self, util):
    a = self.a
    ls = latency_sense(a.latency_file, a.latency_window_s) if a.latency_file and a.slo_ms else None
    if ls is not None and ls["blind"]:
        return None, ls
    resp = 0.0
    if ls is not None:
        # the position is the mean response time of the window between the bare service time and the line (what
        # the model reads); the 95th percentile at or past the line, or any failed request, is past the wall
        bare = a.slo_ms / 10.0
        w = latency_window(a.latency_file, a.latency_window_s)["ms"]

        mean = sum(w) / len(w) if w else ls["p95"]
        resp = (mean - bare) / (a.slo_ms - bare)
        if ls["fail"] or ls["p95"] >= a.slo_ms:
            resp = max(resp, 1.0)
    return resp, ls

def learn(self, r):
    """The card on its own: while the ceiling is at the top and the card is busy, its clock and draw are native's.
    """
    if r is not None and self.written_ceiling >= self.top and r["limit"] >= self.start - 1.0 and r["util"] >= 0.9:
        self.busy_clk = (self.busy_clk + [r["clock_mhz"]])[-200:]
        self.busy_draw = (self.busy_draw + [r["draw"]])[-200:]

def native(self):
    """(busy clock, busy draw) of the card on its own, or (None, None) until enough busy readings are in."""
    if len(self.busy_clk) < self.a.learn_samples:
        return None, None
    c, d = sorted(self.busy_clk), sorted(self.busy_draw)
    return c[len(c) // 2], d[int(0.9 * (len(d) - 1))]

def write_clock(self, mhz, why):
    mhz = int(round(mhz))
    cmd = ["-i", str(self.g), "-lgc", f"{self.c_floor},{mhz}"]
    if self.a.mode != "cap" or self.foreign:
        self.audit({"would_clock_write": cmd, "why": why}); return
    rc, err = smi_run(self.a.smi, cmd)
    self.audit({"clock_write": cmd, "why": why, "rc": rc, "stderr": err})
    if rc != 0:

```

```

        raise WriteFailed(f"nvidia-smi -lgc {self.c_floor},{mhz} returned {rc}: {err}")
    self.clock_set = True
    self.writes += 1

def write_limit(self, w, why):
    w = int(w)
    cmd = ["nvidia-smi", "-i", str(self.g), "-pl", str(w)]
    if self.a.mode != "cap" or self.foreign:
        self.audit({"would_write": cmd, "why": why}); return
    self.audit({"write": cmd, "why": why})
    t0 = time.time()
    rc, err = smi_run(self.a.smi, ["-i", str(self.g), "-pl", str(w)])
    act = {"gpu": self.g, "requested_w": w, "t_requested": t0, "rc": rc, "stderr": err}
    if rc != 0:
        self.audit({"actuator": act, "write_failed": cmd})
        raise WriteFailed(f"nvidia-smi -pl {w} returned {rc}: {err}")
    self.writes += 1
    s = query(self.a.smi, [self.g], self.enforced_ok)
    if s is not None:
        back = s[self.g]
        act.update({"readback_w": back["limit"], "enforced_w": back["enforced"], "t_readback": time.time(),
                    "realized": abs(back["limit"] - w) < 1.0, "override": back["enforced"] < back["limit"] - 1.0})
        if act["realized"]:
            act["delay_s"] = round(act["t_readback"] - t0, 3)
            self.expect = w
        self.audit({"actuator": act})

def step(self):
    a, g = self.a, self.g
    s = query(a.smi, [g], self.enforced_ok)
    if s is None:
        p, ls, r = None, None, None
    else:
        r = s[g]
        if abs(r["limit"] - self.expect) >= 1.0 and not self.foreign:
            self.foreign = True
            self.audit({"foreign_writer": {"gpu": g, "reads_w": r["limit"], "expected_w": self.expect},
                        "action": "observe only from now on; both wires left to the other writer"})
        p, ls = self.position(r["util"])
    self.learn(r)
    n_clk, n_draw = self.native()
    thr = throttle(a.smi, [g]) if r is not None else None
    heat = slowed((thr or {}).get(g))
    saturated = r is not None and r["util"] >= a.race_util
    if p is None or p >= self.brain.band.wall_high or saturated:
        # fail up past the wall or blind; and race while work waits (the card saturated: a queue is forming), so a
        # burst is always served at full speed and the bowl paces only the slack between bursts
        if p is not None:
            self.brain.force(p)
            ceiling, lid = self.top, self.start
            who = "blind_fail_up" if p is None else "fail_up" if p >= self.brain.band.wall_high else "race"
        else:
            F = self.brain.force(p)
            # the speed floor: never under the clock the card reaches on its own while busy (until that is learned,
            # the top), so work waiting on the card is never served slower than native
            c_floor = self.top if n_clk is None else max(self.c_lo, min(self.top, n_clk))

```

```

# the ceiling moves in whole clock steps, as the card's own clock does: the force times the gain (a share
# of the top clock per unit of force) rounded to whole --min-change-mhz steps; a pull under half a step
# moves nothing and is not stored up, so a calm card is paced only when the pull is clearly down
down = a.down_gain if a.down_gain is not None else PROFILES[a.profile]
step = (a.up_gain if F > 0 else down) * F * self.top
step = round(step / a.min_change_mhz) * a.min_change_mhz
ceiling = clamp(self.ceiling + step, c_floor, self.top)
if heat and ceiling < self.ceiling:
    ceiling, who = self.ceiling, "thermal_hold"
else:
    who = "bowl"
# the lid: the start limit scaled to what a fully busy card draws at this ceiling (a fifth of the draw doe
↪ s not
# scale with the clock; the rest goes as clock^2.5, clock times voltage squared), plus headroom; at the to
↪ p
# clock the lid is the start limit, so the lid never adds a hammer of its own
# the lid: never under what the card draws on its own while busy (its own meter, not a curve), plus
# headroom; the start limit until that is learned
lid = self.start if n_draw is None else clamp(math.ceil(n_draw * (1.0 + a.lid_headroom)), self.floor_w, se
↪ lf.start)
rec = {"decision": {str(g): {"telemetry": None if r is None else {
    "util": r["util"], "draw_w": r["draw"], "temp_c": r["temp"], "limit_w": r["limit"],
    "enforced_w": r["enforced"], "clock_mhz": r["clock_mhz"], "throttle": (thr or {}).get(g)},
    "position": None if p is None else round(p, 4), "velocity": round(self.brain.v, 4),
    "latency_p95_ms": None if not ls else ls.get("p95"),
    "ceiling_mhz": round(ceiling), "want_w": int(lid), "decided_by": who,
    "native_busy_clock_mhz": n_clk, "native_busy_draw_w": n_draw}}}
self.audit(rec)

self.ceiling = ceiling
if abs(ceiling - self.written_ceiling) >= a.min_change_mhz or (ceiling >= self.top and self.written_ceiling <
↪ self.top):
    if ceiling >= self.top and self.clock_set and a.mode == "cap" and not self.foreign:
        rc, err = smi_run(a.smi, ["-i", str(g), "-rgc"])
        self.audit({"clock_write": ["-i", str(g), "-rgc"], "why": who, "rc": rc, "stderr": err})
        if rc != 0:
            raise WriteFailed(f"nvidia-smi -rgc returned {rc}: {err}")
        self.clock_set = False
    elif ceiling < self.top:
        self.write_clock(ceiling, who)
        self.written_ceiling = ceiling
    cur = r["limit"] if r is not None else self.expect
    if abs(lid - cur) >= a.min_change_w or (lid == self.start and cur != lid):
        self.write_limit(lid, who)

def restore(self):
    failed = []
    if self.a.mode == "cap" and not self.foreign:
        rc, err = smi_run(self.a.smi, ["-i", str(self.g), "-rgc"])
        self.audit({"clock_write": ["-i", str(self.g), "-rgc"], "why": "restore", "rc": rc, "stderr": err})
        if rc != 0:
            failed.append(f"-rgc returned {rc}: {err}")
        s = query(self.a.smi, [self.g], self.enforced_ok)
        if s is None or abs(s[self.g]["limit"] - self.start) >= 1.0:
            try:
                self.write_limit(self.start, "kill switch: the limit read at start")
            except WriteFailed as e:

```

```

        failed.append(str(e))
        s = query(self.a.smi, [self.g], self.enforced_ok) or {}
        back = s[self.g]["limit"] if self.g in s else None
        ok = self.foreign or (not failed and back is not None and abs(back - self.start) < 1.0)
        self.audit({"restored": {str(self.g): back}, "start": {str(self.g): self.start}, "ok": ok,
                    "writes": self.writes, "foreign_writer": self.foreign, "restore_write_failed": failed})
        return ok

def parser():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--mode", choices=["watch", "cap"], default="watch")
    ap.add_argument("--gpus", default="0")
    ap.add_argument("--smi", default=os.environ.get("NVIDIA_SMI", "nvidia-smi"))
    ap.add_argument("--interval", type=float, default=2.0)
    ap.add_argument("--duration", type=float, default=0.0)
    ap.add_argument("--audit", default="gpu_bowl_audit.jsonl")
    ap.add_argument("--kill-file", default="/tmp/omni-gpu-kill")
    ap.add_argument("--latency-file", default="")
    ap.add_argument("--slo-ms", type=float, default=0.0)
    ap.add_argument("--latency-window-s", type=float, default=5.0, help="seconds of response times the position reads"
        "\n")
    ap.add_argument("--race-util", type=float, default=0.95, help="utilization at which work is waiting: race, never p
        "\nace")
    ap.add_argument("--learn-samples", type=int, default=15, help="busy readings of the card on its own before the bow
        "\nl may lower anything")
    ap.add_argument("--floor-w", type=float, default=0.0, help="the declared envelope's lowest watts")
    ap.add_argument("--clock-min-share", type=float, default=0.35, help="the clock ceiling's cover: lowest share of th
        "\ne top")
    ap.add_argument("--profile", choices=sorted(PROFILES), default="service",
        help="service (default): paces down gently enough that the slow answers are never slower than nati
        "\nve; "
        "\nbatch: paces harder, for work nobody waits on answer by answer (higher saving, p95 cost disc
        "\nlosed)")
    ap.add_argument("--up-gain", type=float, default=0.10, help="share of the top clock moved per unit of force, up")
    ap.add_argument("--down-gain", type=float, default=None, help="share of the top clock moved per unit of force, dow
        "\nn (default: the profile's)")
    ap.add_argument("--lid-headroom", type=float, default=0.10, help="the lid above the draw the ceiling takes")
    ap.add_argument("--min-change-mhz", type=float, default=15.0)
    ap.add_argument("--min-change-w", type=float, default=3.0)
    return ap

def main(argv=None):
    a = parser().parse_args(argv)
    gov = GpuBowl(a)
    stop = {"now": False}
    signal.signal(signal.SIGTERM, lambda *_: stop.__setitem__("now", True))
    t0 = time.time()
    failed = False
    try:
        while not stop["now"] and not os.path.exists(a.kill_file) and (a.duration <= 0 or time.time() - t0 < a.duratio
        "\nn):
            try:
                gov.step()
            except WriteFailed as e:
                gov.audit({"fatal": f"write failed: {e}"}); failed = True
                break
        except Exception as e: # noqa: BLE001 a failed decision is logged; the kill path still runs
            gov.audit({"error": f"{type(e).__name__}: {e}"})
            end = time.time() + a.interval
            while time.time() < end and not stop["now"] and not os.path.exists(a.kill_file):
                time.sleep(0.2)
    finally:
        ok = gov.restore()
        return 5 if gov.foreign else 3 if not ok else 4 if failed else 0

if __name__ == "__main__":
    sys.exit(main())

```

omnicompass/adapter.py

309 lines.

```

"""Omni-Compass stack governor.

sense      : stack telemetry -&gt; normalized observation vector
assimilate : observation blended into the six-state engine state
evolve     : one macro step of equations (1)-(7) with u = 0
allocate   : engine state -&gt; directive (capacity, change gating, power, rollback, routing, alerting, paging)
authority   : OBSERVE | AUTOPILOT; kill() revokes authority and returns control to the native managers
"""

from __future__ import annotations

import math
from collections import deque
from dataclasses import dataclass, asdict, field
from typing import Any, Dict, List, Optional

from .core import State, Params, macro_step, v_eff, control_command, U_AUTHORITY

OBSERVE = "observe"
AUTOPILOT = "autopilot"

# Engine parameters used for stack governance (fixed, not sampled).
STACK_PARAMS = Params(alpha=4.2, beta_int=0.0, beta_ext=0.0, k=1.7, sigma_1=0.38, delta=0.705, gamma_1=1.35,
                      gamma_c=1.0, lambda_0=-0.40, lambda_1=1.55, lambda_2=1.15, c=1.0, E_max=2.0,
                      omega_B=1.0, Q_B=2.5, alpha_s=0.12, beta_s=0.10, mu=3.782, alpha_E=3.487, alpha_U=4.2,
                      lambda_I=0.719, lambda_U=0.0)
INITIAL_STATE = State(E=0.12, U=0.82, I_U=0.0, S=0.0, B=0.0, B_dot=0.0)
ASSIMILATION = 0.339

def clamp(v: float, lo: float, hi: float) -&gt; float:
    return lo if v &lt; lo else hi if v &gt; hi else v

def observe_vector(obs: Dict[str, float], conflicts: int) -&gt; Dict[str, float]:
    """Normalize telemetry. Inputs are the stack's delayed observations."""
    return {
        "q": clamp(float(obs.get("queue_ratio", 0.0)), 0.0, 2.0),
        "load": clamp(float(obs.get("load_ratio", 0.0)), 0.0, 2.0),
        "power": clamp(float(obs.get("power_stress", 0.0)), 0.0, 1.5),
        "thermal": clamp(float(obs.get("thermal", 0.0)), 0.0, 1.5),
        "network": clamp(float(obs.get("network_stress", 0.0)), 0.0, 1.5),
        "drift": clamp(float(obs.get("drift_ratio", 0.0)), 0.0, 1.5),
        "stale": clamp(float(obs.get("stale", 0.0)), 0.0, 1.0),
        "security": clamp(float(obs.get("security_block", 0.0)), 0.0, 1.0),
        "conflict": clamp(conflicts / 4.0, 0.0, 1.0),
    }

def assimilate(x: State, o: Dict[str, float], p: Params) -&gt; State:
    """Blend the observation into the engine state and set the forcing terms."""
    q, load, pw, th, nw, dr, st, sec, cf = (o[k] for k in
        ("q", "load", "power", "thermal", "network", "drift", "stale", "security", "conflict"))
    e_obs = clamp(0.25 * q + 0.18 * max(0.0, load - 0.85) + 0.16 * pw + 0.13 * th + 0.10 * nw + 0.08 * dr
        + 0.06 * cf + 0.04 * st, 0.0, 1.5)
    u_obs = clamp(1.0 - (0.27 * q + 0.18 * pw + 0.16 * th + 0.12 * nw + 0.12 * dr + 0.10 * cf + 0.05 * st), 0.0, 1.0)
    s_obs = clamp(0.36 * th + 0.28 * pw + 0.18 * nw + 0.10 * sec + 0.08 * st - 0.20 * q, -1.0, 1.0)

```

```

b_obs = clamp(0.52 * pw + 0.26 * th + 0.22 * dr, -1.0, 1.5)
i_obs = clamp(q + dr + cf, 0.0, 2.0)
a = ASSIMILATION
p.beta_int = clamp(0.58 * q + 0.42 * max(0.0, load - 0.75), 0.0, 1.0)
p.beta_ext = clamp(0.33 * pw + 0.27 * th + 0.20 * nw + 0.12 * st + 0.08 * sec, 0.0, 1.0)
return State(E=(1 - a) * x.E + a * e_obs, U=(1 - a) * x.U + a * u_obs, I_U=(1 - a) * x.I_U + a * i_obs,
             S=(1 - a) * x.S + a * s_obs, B=(1 - a) * x.B + a * b_obs, B_dot=(1 - a) * x.B_dot + a * (b_obs - x.B)
             ~ )

@dataclass(frozen=True)
class AllocationLaw:
    """Engine state -> governing directive.

    release    capacity removed only while push <= push_release (the engine's controller reports convergence)
    add         without backlog, capacity added only while push >= push_add (the engine's controller reports need)
    push        equation (2) control law evaluated on the evolved state with target sigma = +1:
                u = clip(-f_U(x) + KP (1 - U), -U_AUTHORITY, U_AUTHORITY); push = u / U_AUTHORITY
                (f_U contains the double-well drift and -(dE/dt)/E_max from equation (1))
    headroom    rho* = clamp(rho0 - kI*I_U - kE*E, rho_min, rho0)
    sizing      load_s = load * cap_now if size_at_full_cap else load
    capacity    n_req = ceil(n * load / rho*) + ceil(kq * q * n); delta = clamp(n_req - n, -down_max, +up_max)
    guard       no capacity removal while q >= guard_queue
    band        capacity removed only when n_req <= n - down_band
    dwell       capacity removed only after surplus persists for down_dwell intervals and no earlier than
                down_after_add intervals after the last addition
    envelope    no capacity addition while observed power >= power_ceiling (power_ceiling_backlog while q >= guard_queue)
    ~ e)

    security    no capacity addition while a security block is observed
    change      Terraform plans only while U >= U_gate, only in the direction of the live capacity decision
    power       q <= guard_queue and U >= U_gate: cap = clamp(cap_now * load * (1 + margin), cap_min, 1) (trim to demand)
    ~ nd)

                q >= guard_queue at the power envelope: cap = max(cap_min, cap_now - spread_step) (spread)
                q >= guard_queue below the envelope: cap held (capacity added as nodes; 1 at max_nodes)
                otherwise cap = 1; always cap <= 1 - cap_gain * max(0, B - B_cap)
    rollback    a proposed rollback is authorised if S >= S_rollback, U is falling, or security blocks change
    replicas    direct actuation: desired = ceil(replicas * load / rho*) (the HPA formula with the engine-set target),
                increases unless capacity is being reduced, decreases only while reducing;
                otherwise autoscaler increases approved unless reducing, decreases only while reducing
    consistency a step that expands capacity never tightens the power cap
    paging      none: the governor holds every action a paged operator would take
    """
    rho0: float = 0.914
    rho_min: float = 0.627
    kI: float = 0.252
    kE: float = 0.297
    kq: float = 0.795
    up_max: int = 2
    down_max: int = 1
    guard_queue: float = 0.05
    power_ceiling: float = 0.95
    power_ceiling_backlog: float = 1.00
    spread_step: float = 0.05
    backlog_release: bool = False
    size_at_full_cap: bool = False
    push_release: float = 0.005

```

```

push_add: float = -9.0
U_gate: float = 0.481
margin: float = 0.006
cap_min: float = 0.706
B_cap: float = 0.40
cap_gain: float = 0.60
S_rollback: float = 0.55
route_network: float = 0.20
down_band: int = 4
down_dwell: int = 4
down_after_add: int = 4
min_nodes: int = 8
max_nodes: int = 40

@dataclass
class Governor:
    law: AllocationLaw = field(default_factory=AllocationLaw)
    mode: str = AUTOPILOT
    killed: bool = False
    x: State = field(default_factory=lambda: State(**asdict(INITIAL_STATE)))
    p: Params = field(default_factory=lambda: Params(**asdict(STACK_PARAMS)))
    t: float = 0.0
    current_cap: float = 1.0
    nodes: int = 20
    surplus_streak: int = 0
    since_add: int = 99
    last_push: float = 0.0

    log: deque = field(default_factory=lambda: deque(maxlen=4096))
    evolve: bool = True

    def kill(self) -> None:
        """Revoke authority; the engine continues to compute and log directives."""
        self.killed = True

    def set_mode(self, mode: str) -> None:
        if mode not in (OBSERVE, AUTOPILOT):
            raise ValueError(mode)
        self.mode = mode

    @property
    def has_authority(self) -> bool:
        return self.mode == AUTOPILOT and not self.killed

    def step(self, obs: Dict[str, float], conflicts: int) -> Dict[str, Any]:
        """Sense, assimilate, evolve, allocate."""
        o = observe_vector(obs, conflicts)
        U_before = self.x.U
        xa = assimilate(self.x, o, self.p)
        if self.evolve:
            xn, _ = macro_step(xa, self.p, self.t, target=None)
        else:
            xn = xa
        self.x = xn
        self.t += 0.1
        L = self.law

```

```

n = int(self.nodes)
u_push, _ = control_command(xn, self.p, self.t, +1)
push = u_push / U_AUTHORITY
self.last_push = push
rho = clamp(L.rho0 - L.kI * xn.I_U - L.kE * xn.E, L.rho_min, L.rho0)
converged = push <= L.push_release
load_sized = o["load"] * self.current_cap if L.size_at_full_cap else o["load"]
n_req = math.ceil(n * load_sized / rho) + math.ceil(L.kq * o["q"] * n)
delta = max(-L.down_max, min(L.up_max, n_req - n))
if delta < 0 and not converged:
    delta = 0
if delta > 0 and o["q"] < L.guard_queue and push < L.push_add:
    delta = 0
if delta < 0 and (o["q"] >= L.guard_queue or n_req > n - L.down_band):
    delta = 0
self.surplus_streak = self.surplus_streak + 1 if delta < 0 else 0
if delta < 0 and (self.surplus_streak < L.down_dwell or self.since_add < L.down_after_add):
    delta = 0
ceiling = L.power_ceiling_backlog if o["q"] >= L.guard_queue else L.power_ceiling
if delta > 0 and (o["power"] >= ceiling or n >= L.max_nodes):
    delta = 0
if o["security"] > 0.5:
    delta = min(delta, 0)
self.since_add = 0 if delta > 0 else self.since_add + 1
if delta < 0:
    self.surplus_streak = 0
at_envelope = o["power"] >= ceiling
if o["q"] < L.guard_queue and xn.U >= L.U_gate:
    cap = clamp(self.current_cap * o["load"] * (1.0 + L.margin), L.cap_min, 1.0)
elif o["q"] >= L.guard_queue and L.spread_step > 0.0:
    if at_envelope:
        cap = clamp(self.current_cap - L.spread_step, L.cap_min, 1.0)
    else:
        cap = 1.0 if L.backlog_release else (self.current_cap if n < L.max_nodes else 1.0)
elif o["q"] >= L.guard_queue and L.backlog_release:
    cap = 1.0
else:
    cap = 1.0
cap = min(cap, clamp(1.0 - L.cap_gain * max(0.0, xn.B - L.B_cap), L.cap_min, 1.0))
if delta > 0:
    cap = max(cap, self.current_cap)
command = clamp(0.5 * (v_eff(xn, self.p) / max(self.p.c, 1e-9) + 1.0), 0.0, 1.0)
d = {
    "node_delta": int(delta),
    "change_permitted": xn.U >= L.U_gate and o["security"] <= 0.5,
    "power_cap": cap,
    "rollback_authorized": xn.S >= L.S_rollback or xn.U < U_before or o["security"] > 0.5,
    "route_shift": o["network"] > L.route_network,
    "state": {"E": xn.E, "U": xn.U, "I_U": xn.I_U, "S": xn.S, "B": xn.B},
    "demand": rho,
    "load": o["load"],
    "command": command,
}
self.log.append({"mode": self.mode, "killed": self.killed, **{k: v for k, v in d.items() if k != "state"},
                **d["state"]})
return d

```

```

def directive_to_actions(d: Dict[str, Any], stack_state: Dict[str, Any], proposals: List[Dict[str, Any]],
                        cfg, direct_actuation: bool = False) -> List[Dict[str, Any]]:
    """Directive -> executable stack actions. direct_actuation: capacity is owned by the governor; no Terraform plans.
    """
    tag = "Omni-Compass"
    out: List[Dict[str, Any]] = []
    cur = int(stack_state["actual_nodes"])
    target = int(max(cfg.minimum_nodes, min(cfg.maximum_nodes, cur + int(d["node_delta"]))))
    if target != cur:
        out.append({"manager": tag, "action": "nodes", "target": target, "direction": 1 if target > cur else -1})
    if not direct_actuation and d["change_permitted"] and any(p["action"] == "terraform_plan" for p in proposals):
        tf_dir = 1 if target > stack_state["terraform_desired_nodes"] else -1
        live_dir = 1 if target > cur else -1 if target < cur else 0
        if abs(target - stack_state["terraform_desired_nodes"]) >= 2 and live_dir in (0, tf_dir):
            out.append({"manager": tag, "action": "terraform_plan", "target": target,
                        "direction": 1 if target > stack_state["terraform_desired_nodes"] else -1})
    reps = [int(p["target"]) for p in proposals if p["action"] == "replicas"]
    if direct_actuation:
        cur_r = int(stack_state["replicas"])
        rt = max(20, min(500, math.ceil(cur_r * float(d["load"]) / max(float(d["demand"]), 1e-9))))
        if (rt > cur_r and d["node_delta"] >= 0) or (rt < cur_r and d["node_delta"] < 0):
            out.append({"manager": tag, "action": "replicas", "target": rt, "direction": 1 if rt > cur_r else -1})
        reps = []
    if reps:
        cur_r = int(stack_state["replicas"])
        rt = None
        if d["node_delta"] >= 0 and max(reps) > cur_r:
            rt = max(reps)
        elif d["node_delta"] < 0 and min(reps) < cur_r:
            rt = min(reps)
        if rt is not None:
            out.append({"manager": tag, "action": "replicas", "target": rt,
                        "direction": 1 if rt > cur_r else -1})
    cap = float(d["power_cap"])
    if any(a["direction"] > 0 and a["action"] in ("nodes", "terraform_plan", "replicas") for a in out):
        cap = max(cap, float(stack_state["power_cap"]))
    if abs(cap - stack_state["power_cap"]) >= 1e-9:
        out.append({"manager": tag, "action": "power_cap", "target": cap,
                    "direction": -1 if cap < stack_state["power_cap"] else 1})
    if d["route_shift"]:
        out.append({"manager": tag, "action": "route_shift", "target": 1.0, "direction": 1})
    if any(p["action"] == "rollback" for p in proposals) and d["rollback_authorized"]:
        out.append({"manager": tag, "action": "rollback", "target": 1.0, "direction": -1})
    alerts = [p for p in proposals if p["action"] == "alert"]
    if alerts:
        out.append({"manager": tag, "action": "alert", "target": max(int(p["target"]) for p in alerts), "direction": 1
                    })
    return out

THROUGHPUT_MODE = dict(backlog_release=True, power_ceiling=9.0, power_ceiling_backlog=9.0, spread_step=0.0, B_cap=9.0,
                        size_at_full_cap=True, kI=0.05, kE=0.40, cap_min=0.65, down_band=1, down_dwell=10,
                        down_after_add=3, kq=0.30, U_gate=0.45, push_release=0.01)

```

```

FLEET_MODE = dict(THROUGHPUT_MODE, rho=0.829, rho_min=0.769, kI=0.072, kE=0.487, kq=0.635, down_band=3, down_dwell=1,
    ↳ down_after_add=5, push_release=0.053, cap_min=0.695, margin=0.131, U_gate=0.85, up_max=8)
FLEET_BALANCED_MODE = dict(FLEET_MODE, push_release=0.0098, push_add=0.0098)
FLEET_WEAR_MODE = dict(FLEET_MODE, push_release=0.0035, push_add=0.0035)
FLEET_DECISION_SECONDS = 60

def mode_law(mode: str, base: "AllocationLaw" = None) -> "AllocationLaw":
    """Operating modes.

    power_protect the site power limit is enforced (envelope, spread-and-throttle, bath cap; shield I4 active)
    fleet          throughput mode with constants selected on the 15-second fleet harness (fleet/), decisions every 60
    ↳ s;
    fleet_balanced energy-first: the equation (2) release gate is lenient and the add gate is off
    fleet_wear      fleet with the equation (2) gate active in both directions at 0.0098 (release and add)
    throughput      fleet with the equation (2) gate active in both directions at 0.0035 (release and add)
    ↳ ;
    the power envelope is not enforced (same power rules as the native stack); caps released on backlog

    capacity sized at full power (load * cap_now), so power trimming does not trigger machine starts;
    constants selected by constrained search on development seeds (minimum energy subject to backlog,
    health, recovery, reversal and invariant constraints)

    """
    from dataclasses import replace
    base = base or AllocationLaw()
    if mode == "power_protect":
        return base
    if mode == "throughput":
        return replace(base, **THROUGHPUT_MODE)
    if mode == "fleet":
        return replace(base, **FLEET_MODE)
    if mode == "fleet_balanced":
        return replace(base, **FLEET_BALANCED_MODE)
    if mode == "fleet_wear":
        return replace(base, **FLEET_WEAR_MODE)
    raise ValueError(mode)

```

omnicompass/nervous_system.py

158 lines.

```

"""The supervisory nervous system: one evolving engine state grants authority to every organ family.

Omni-Compass does not replace the specialist controllers. Each keeps its own mechanism:
- the HPA and VPA size pods;
- the Cluster Autoscaler and Karpenter size node pools;
- Linux schedutil picks CPU frequency;
- NVIDIA DCGM manages GPUs;
- rollout controllers and routers do their own work.

The nervous system decides, from one shared state, the envelope inside which each of them may act.

Inputs, all from the six-state engine (omnicompass/core.py, via the governor), and nothing tuned per organ:
kappa convergence = 1 if push <= push_release, else push_release / push. This is the equation-(2) gate made
continuous: the further the engine is from its basin, the less it may give back.
h basin health = clamp((U - U_gate) / (1 - U_gate), 0, 1)
sigma stress ratio = S / S*, where S* solves delta - alpha_s S - (3/4) beta_s S^2 = 0 (equation 6)
nu unmet need = max(0, I U) (equation 3)
calm = kappa * h * (1 - clamp(sigma - 1, 0, 1)) * (1 - clamp(nu, 0, 1))
in [0, 1]; 1 means fully settled

Authority per organ:
expand may the organ add capacity or performance. Always yes, except during a security hold.
contract may it give capacity or performance back. Only when calm >= the organ's reversibility threshold
(theta_pods 0.5, theta_nodes 0.7: a node costs a boot to reverse) and the SLO is clean.
step fraction of the organ's surplus it may shed this decision. Equals calm.
envelope for continuous organs, the allowed range:
    cpufreq ceiling (fraction of cpuinfo_max) in [1 - 0.35 calm, 1];
    GPU power limit (fraction of max) in [1 - 0.30 calm, 1];

```

```

        site power cap in [0.65, 1];
        traffic-shift fraction in [0, 0.5 calm];
        cooling supply-air setpoint (C) in [18, 18 + 9 calm].
        Heat above 0.96 forces the frequency and GPU ceilings down to 1 - 0.35 x excess.
batch      admit held work only if calm >= 0.5 and power stress < 0.9; pause pausable work if sigma > 1, power
           stress >= 0.95, or heat >= 0.96.
rollback   the engine's rollback authorisation (S high, U falling, or a security hold).

Global rules, applied last:
- observe: every authority is computed and logged; execute is False everywhere.
- kill: no authority at all.
- security hold: no capacity organ may expand. Cooling is the one protective organ: "expand" there means more
  cooling, which adds protection and no capacity, so it stays allowed.
The safety shield (omnicompass/shield.py) stays downstream and can still veto any action.
"""
from __future__ import annotations

from dataclasses import dataclass
from typing import Any, Dict

from omnicompass.closure import stress_equilibrium

ORGANS = ("pods", "nodes", "cpufreq", "gpu", "power", "batch", "routing", "rollback")
# The living band (the founder's rule): every level the nervous system hands out lives between 5% and 95% of its range
↳ .
# Nothing is driven to zero (a part with no work idles down to its floor, alive and ready) and nothing is driven to it
↳ S
# absolute top (the last 5% is never spent). The band is applied last, to every envelope, so no organ can leave it.

BAND = (0.05, 0.95)

def in_band(lo: float, hi: float) -> list:
    """Clip an envelope [lo, hi] (fractions of the organ's range) into the living band, keeping lo <= hi."""
    b0, b1 = BAND
    h = min(b1, max(b0, hi))
    return [min(h, max(b0, lo)), h]

THETA = {"pods": 0.5, "nodes": 0.7, "cpufreq": 0.3, "gpu": 0.3, "power": 0.3, "routing": 0.5}

def clamp(x, lo, hi):
    return lo if x < lo else hi if x > hi else x

@dataclass(frozen=True)
class NervousInputs:
    E: float; U: float; I_U: float; S: float
    push: float
    push_release: float = 0.2
    U_gate: float = 0.5
    s_eq: float = 1.0
    security_block: float = 0.0
    slo_clean: bool = True
    power_stress: float = 0.0
    thermal: float = 0.0

```

```

rollback: bool = False
stale: float = 0.0          # fraction of declared senses that are blind (dropout, frozen, unreadable)
mode: str = "autopilot"     # "observe" or "autopilot"
killed: bool = False

def scalars(i: NervousInputs) -> Dict[str, float]:
    kappa = 1.0 if i.push <= i.push_release else i.push_release / max(i.push, 1e-12)
    h = clamp((i.U - i.U_gate) / max(1.0 - i.U_gate, 1e-12), 0.0, 1.0)
    sigma = i.S / i.s_eq if i.s_eq > 0 else 0.0
    nu = max(0.0, i.I_U)
    calm = kappa * h * (1.0 - clamp(sigma - 1.0, 0.0, 1.0)) * (1.0 - clamp(nu, 0.0, 1.0))
    return {"kappa": kappa, "h": h, "sigma": sigma, "nu": nu, "calm": clamp(calm, 0.0, 1.0)}

def authority(i: NervousInputs) -> Dict[str, Any]:
    if i.killed:
        return {"execute": False, "killed": True, "organs": {}}
    sc = scalars(i); calm = sc["calm"]
    sec = i.security_block &gt; 0.5
    hot = i.thermal &gt; 0.96
    excess = clamp(i.thermal - 0.96, 0.0, 1.0)
    org: Dict[str, Dict[str, Any]] = {}
    seeing = i.stale <= 0.0          # never give capacity back on a blind sense; expanding stays allowed (the safe si
    ↪ de)
    for o, th in THETA.items():
        ok = calm &gt;= th and i.slo_clean and seeing
        org[o] = {"expand": not sec, "contract": ok, "step": calm if ok else 0.0}

    cf_lo = 1.0 - 0.35 * calm; gp_lo = 1.0 - 0.30 * calm
    cf_hi = 1.0 - 0.35 * excess if hot else 1.0; gp_hi = 1.0 - 0.35 * excess if hot else 1.0
    org["cpufreq"]["envelope"] = in_band(min(cf_lo, cf_hi), cf_hi)
    org["gpu"]["envelope"] = in_band(min(gp_lo, gp_hi), gp_hi)
    org["power"]["envelope"] = in_band(0.65, 1.0)
    # routing is an amount moved, not a level: it may move nothing, never more than the band's top
    org["routing"]["envelope"] = [0.0, min(BAND[1], 0.5 * calm) if not sec else 0.0]
    org["cooling"] = {"expand": True, "protective": True, "contract": calm &gt;= 0.3 and seeing, "step": calm, "envelope"
    ↪ : [18.0, 18.0 + 9.0 * calm]}
    org["batch"] = {"expand": (not sec) and seeing and calm &gt;= 0.5 and i.power_stress < 0.9, "contract": True, "protec
    ↪ tive_contract": True,
        "admit": (not sec) and seeing and calm &gt;= 0.5 and i.power_stress < 0.9,
        "pause": sc["sigma"] &gt; 1.0 or i.power_stress &gt;= 0.95 or hot, "step": calm}
    org["rollback"] = {"expand": False, "contract": False, "authorized": bool(i.rollback or sec), "step": 0.0}
    return {"execute": i.mode == "autopilot", "killed": False, "security_hold": sec, "scalars": sc, "organs": org}

def from_governor(g, obs: Dict[str, Any], d: Dict[str, Any] = None, mode: str = None) -> Dict[str, Any]:
    """Authority from a live omnicompass.adapter.Governor after its step (d = the step's directive)."""
    x = g.x; p = g.p; L = g.law
    return authority(NervousInputs(
        E=x.E, U=x.U, I_U=x.I_U, S=x.S, push=g.last_push, push_release=getattr(L, "push_release", 0.2),
        U_gate=getattr(L, "U_gate", 0.5), s_eq=stress_equilibrium(p.delta, p.alpha_s, p.beta_s),
        security_block=float(obs.get("security_block", 0.0)), slo_clean=bool(obs.get("slo_clean", True)),
        power_stress=float(obs.get("power_stress", 0.0)), thermal=float(obs.get("thermal", 0.0)),
        rollback=bool((d or {}).get("rollback_authorized", False)), stale=float(obs.get("stale", 0.0)),
        mode=mode or g.mode, killed=g.killed))

```

```

def node_release_gate(n: int, per_node_m: float, used_m: float, pending: int, pods_scaling_up: bool,
                    latency_breach_now: bool, rho: float, node_auth: Dict[str, Any],
                    senses_live: bool = True, last_command_landed: bool = True) -> Dict[str, Any]:
    """May the machine organ give one machine back now? Attribution: an organ is held back only by stress it can cause
    or cure. The node organ reads its own engine view (fed with machine-attributable pressure: pods waiting for a
    place), and the release must also pass:
        coordination pods are not scaling up and latency is not breached now (pods move first; machines never move
        against them, the rule of the benchmarked coordination)
        headroom      nothing is pending, and after the release the remaining machines run at or below the engine's own
        utilisation target rho: used / ((n - 1) x per_node) <= rho
        authority      the node organ's own calm, security and stress gates (authority() above) grant contraction
        senses          every declared sense is live (afferent integrity)
        proprioception the node organ's last command landed (efferent feedback: no new order to a muscle that did not
        carry out the previous one)
    Returns {"ok": bool, "reason": str, "util_after": float}."""
    util_after = used_m / max((n - 1) * per_node_m, 1e-9) if n > 1 else float("inf")
    checks = [("one machine left", n > 1), ("pods waiting", pending == 0), ("pods scaling up", not pods_scaling_up),
              ("latency breached now", not latency_breach_now), (f"util after {util_after:.2f} > rho {rho:.2f}", util_
    < after <= rho),
              ("node organ has no contraction authority", bool(node_auth.get("organs", {}).get("nodes", {})).get("contr
    < act", False))),
              ("a sense is blind", senses_live), ("last node command did not land", last_command_landed)]
    failed = [name for name, ok in checks if not ok]
    return {"ok": not failed, "reason": "; ".join(failed) or "release permitted", "util_after": util_after}

```

APPENDIX

Appendix I. The License

The license under which this book and the software are made available, as it stands in the file LICENSE at the root of the repository.

```
Omni-Compass Evaluation License, version 1.0
SPDX-License-Identifier: LicenseRef-OmniCompass-Evaluation-1.0
```

```
Copyright (c) 2026 The Omni-Compass LLC. All rights reserved.
```

```
This is NOT an open-source license. The software in this repository ("the Software") - including the Omni-Compass engine, the bowl law, the governors, the harnesses, the controllers, the plug and wiring, the documentation, the benchmarks and their results, and the methods and mechanism they embody - is the proprietary property of The Omni-Compass LLC and is protected by copyright and by patents and patent applications held by The Omni-Compass LLC.
The Software is made available publicly for EVALUATION AND SIMULATION PURPOSES ONLY.
```

```
INTELLECTUAL PROPERTY NOTICE. Patent applications, copyright registrations and trademark applications covering the Omni-Compass engine, its mathematics and its software have been filed in the United States by The Omni-Compass LLC. Every person who downloads, copies, forks, scans or runs the Software is on notice of these filings and of this license.
```

1. Permitted use (free of charge, no license fee). You may download, copy, read, run and modify the Software solely to
 - (a) evaluate it, including running its tests, benchmarks and simulations, and running it in shadow or test mode on systems you own or control for the purpose of evaluating it; and
 - (b) reproduce and check the results published in this repository.This permission is personal, non-exclusive, non-transferable, non-sublicensable and revocable.

2. Everything else requires a signed, paid license. Any use other than section 1 requires a written commercial license agreement signed by The Omni-Compass LLC and the licensee, and payment of the license fees it sets out (an "Omni-Compass Enterprise License"). This includes, without limitation:
 - (a) any commercial use, commercialization or monetization of the Software, its mechanism or its methods;
 - (b) use in production, or to operate, control or govern any system, fleet, cluster, facility, machine, vehicle or grid beyond evaluation;
 - (c) selling, licensing, sublicensing, leasing, renting, distributing or offering the Software, or any product or service that includes, is derived from or is operated by it, including as a hosted, managed or cloud service;
 - (d) incorporating the Software, or any part or derivative of it, into any product, service, platform or offering;
 - (e) using the Software, its results or its methods to build, train, benchmark or improve a competing product or service.
3. No other rights. No right, title, license or interest is granted except as expressly stated in section 1. In particular, no patent license is granted for any use other than section 1, and no trademark license is granted. "Omni-Compass" and its logos are trademarks of The Omni-Compass LLC.
4. Redistribution. You may not redistribute the Software or any modified version, in source or binary form, except as permitted by a signed Omni-Compass Enterprise License. Forks on a code-hosting service are permitted only for the purposes in section 1 and must keep this license and all notices unchanged.
5. Notices. You must keep this license, the copyright notice, the SPDX identifiers and all patent and trademark notices in every copy, including modified copies made under section 1.
6. Termination. Any use outside section 1 without a signed, paid Omni-Compass Enterprise License terminates the rights in section 1 automatically, and is infringement of The Omni-Compass LLC's copyright and patent rights.
7. No warranty; no liability. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT. IN NO EVENT SHALL THE OMNI-COMPASS LLC BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY ARISING FROM THE SOFTWARE OR ITS USE.
8. Third-party components. Files that carry a different license notice (for example `tests/third\_party/`) remain under their own terms.

Commercial licensing (Omni-Compass Enterprise License): The Omni-Compass LLC.

BACK MATTER

Contact

Licensing, pilots, evaluations and the Omni-Compass Enterprise License:

The Omni-Compass LLC

Owner and developer: AJ Dubra

www.omni-compass.com

Repository: github.com/The-Omni-Compass-LLC/The-Omni-Compass

Any use beyond evaluation and simulation requires a signed, paid Omni-Compass Enterprise License. Write to The Omni-Compass LLC before any production use.

OMNI-COMPASS

UNIFIED CONTROL AUTHORITY

EVALUATING ADMISSIBLE EVOLUTION ACROSS OPERATIONAL SYSTEMS



Unified, Transparent Decision Oversight Across Strategic Infrastructure

THE OMNI-COMPASS LLC • www.omni-compass.com

US patent, copyright and trademark applications filed. Evaluation and simulation use only; all other use requires a signed, paid Omni-Compass Enterprise License.